

# Behavioral Feature Models: Technical Report

JULIANE PÄSSLER, University of Oslo, Norway and TU Wien, Austria

SOPHIE FORTZ, King's College, UK and INRIA, Univ. Rennes, CNRS, IRISA, France

MAURICE H. TER BEEK, CNR-ISTI, Italy

FERRUCCIO DAMIANI, University of Turin, Italy

EINAR BROCH JOHNSEN, University of Oslo, Norway

MOHAMMAD REZA MOUSAVI, King's College, UK

SILVIA LIZETH TAPIA TARIFA, University of Oslo, Norway

Modeling Software Product Lines (SPLs) involves addressing two challenges: representing the structure of the SPL, called structural variability, and capturing its behavior, called behavioral variability. For structural variability, several established compositional modeling languages have been proposed, most notably feature models (FMs). For behavioral variability, there are no proposals that offer the same level of compositionality as offered by structural models. Moreover, these two types of models are traditionally developed in isolation. This separation often necessitates the maintenance of different models for structural and behavioral aspects, requiring synchronized updates to prevent discrepancies.

In this paper, we propose a novel formalism called *Behavioral Feature Models* (BFMs) that integrate structural and behavioral aspects within a single unified model. A BFM is an FM in which each feature has an associated behavioral model; constraints between the behavior of different features can be modeled as constraints between their associated events; and the behavior of each product of the SPL consists of the combined behavior of its features and the constraints between their behavior. Thus, the proposed formalism integrates both structural and behavioral variability in a compositional model, while preserving the separation of concerns typical of feature modeling. We illustrate the concept of a BFM through a running example of a cleaning robot SPL and provide a theoretical and an empirical evaluation where we compare the expressiveness and succinctness of BFMs with Featured Transition Systems (FTSs), the state-of-the-art formalism for behavioral modeling of SPLs. We show that BFMs are as expressive as acyclic FTSs, and that, in practice, BFMs can reduce the size of FTSs by up to 98.55%, with a median reduction of 70.81% and a standard deviation of 52.15%.

CCS Concepts: • **Software and its engineering** → **Software product lines**; **Formal methods**; • **Computer systems organization** → **Robotics**.

Additional Key Words and Phrases: software product lines, feature models, behavioral feature models, featured event structures, compositionality, robotics

## 1 INTRODUCTION

Software product line engineering [38, 88] is a software development paradigm concerned with developing a family of software products that share a common core and differ in some of their functionalities, called a *Software Product Line* (SPL) [14]. An important aspect of SPLs is modeling the variability of their products [29] in the form of *structural models* and *behavioral models*, where the former captures structural and the latter behavioral variability. This introduces a dichotomy with the consequence that the two types of models are traditionally kept apart. Furthermore, they are organized differently; while structural models offer compositional modeling techniques, current

---

Juliane Päßler completed the majority of the presented work at the University of Oslo and Sophie Fortz completed the majority of the presented work at King's College.

Authors' addresses: [Juliane Päßler](#), [juliane.paessler@tuwien.ac.at](mailto:juliane.paessler@tuwien.ac.at), University of Oslo, Oslo, Norway and TU Wien, Vienna, Austria; [Sophie Fortz](#), [sophie.fortz@kcl.ac.uk](mailto:sophie.fortz@kcl.ac.uk), King's College, London, UK and INRIA, Univ. Rennes, CNRS, IRISA, Rennes, France; [Maurice H. ter Beek](#), [maurice.terbeek@isti.cnr.it](mailto:maurice.terbeek@isti.cnr.it), CNR-ISTI, Pisa, Italy; [Ferruccio Damiani](#), [ferruccio.damiani@unito.it](mailto:ferruccio.damiani@unito.it), University of Turin, Turin, Italy; [Einar Broch Johnsen](#), [einari@ifi.uio.no](mailto:einari@ifi.uio.no), University of Oslo, Oslo, Norway; [Mohammad Reza Mousavi](#), [mohammad.mousavi@kcl.ac.uk](mailto:mohammad.mousavi@kcl.ac.uk), King's College, London, UK; [Silvia Lizeth Tapia Tarifa](#), [sltartifa@ifi.uio.no](mailto:sltartifa@ifi.uio.no), University of Oslo, Oslo, Norway.

behavioral models lack them. This makes the maintenance of SPL models challenging. The models evolve in isolation, and cumbersome, synchronized updates are needed to prevent discrepancies.

*Structural models* of SPLs, such as Feature Models (FMs) [6, 42, 71, 94], are used to organize the configuration space of the SPL in terms of *features*, where a feature is a name associated to a functionality. FMs contain composition operators to describe how different features are related and can be combined. Thereby, the FM describes the product variants of the SPL, which are identified by valid combinations of features, called *products*. *Behavioral models* of SPLs, such as Featured Transition Systems (FTSs) [36, 40], an extension of Transition Systems (TSs) [10], describe the behavior of all products of an SPL in one single model and allow answering questions about the behavior of single products. However, these behavioral models do not describe the behavior of their features. In fact, in many practical cases, the behavior of a single feature cannot be described as a TS or a state machine since the dependencies among its actions can cut across the FM structure. As a result, the behavioral models are often flattened, which means that the understanding of the SPL, gained from the structural models, is not reflected in the behavioral models and the separation of concerns expressed in terms of feature decomposition is lost.

To address the dichotomy in SPL models, this paper introduces *Behavioral Feature Models (BFMs)* that unify the modeling of structural and behavioral variability. BFM define the behavior of the products of an SPL *compositionally* from the behavior of its features. We propose to structure the behavioral models of products using the composition operators of FMs, which are well-understood operators to describe variability in a configuration space. As a result, we obtain a compositional model for the behavior of the products of an SPL, assembled from behavioral models of its individual features according to how these compose into products.

By integrating the structural and behavioral aspects of the configuration space within a single unified model, BFM facilitate reasoning over the behavior of partial feature models, including the behavior of its individual features, as well as over the reuse and composition of features. Ultimately, the compositionality inherent to BFM has the potential to make SPLs easier to develop and maintain: instead of modeling the behavior of the whole SPL monolithically, only the behavior of single features and their relationship to the behavior of other features need to be modeled. Furthermore, if a new feature is introduced into an SPL, the new behavior of the SPL can be obtained by just adding the behavior of this feature to the BFM of the old behavior of the SPL, easing model extension. In addition, the behavior of one feature can be reused across different SPLs, enabling the development of behavioral feature libraries: features with associated behavior define reusable patterns of behavior that can be combined into new SPLs or used to extend existing SPLs when needed.

*Contributions.* In summary, the main contributions of this report are as follows:

- (1) *Behavioral Feature Models (BFMs)*, a unifying and compositional model for structural and behavioral variability of SPLs, which combines FMs and Bundle Event Structures (BESs) [73], where the BESs are used to model both the behavior of individual features and the behavior of products;
- (2) *Featured Event Structures (FESs)*, an isomorphic view of BFM, that are more similar to FTSs;
- (3) trace-preserving algorithms for translating a BFM to an acyclic FTS and for translating an acyclic FTS to a BFM; allowing techniques and tools developed for FTSs to be applied to systems modeled using BFM; and
- (4) a theoretical and empirical evaluation showing that BFM are as expressive and at least as succinct as acyclic FTSs in theory, and more succinct than parallel compositions of acyclic FTSs in practice.

*Report organization.* The remainder of this paper is organized as follows. In Sect. 2, we provide the background definitions needed for the rest of the paper. In Sect. 3, we introduce BFM informally to give the reader a first overview of the used concepts, before defining them formally in Sect. 4 where we also define FESs and prove that BFMs and FESs are isomorphic. In Sect. 5, we provide translation algorithms between BFMs and acyclic FTSS and prove that they provide trace-equivalent structures. In Sect. 6, we prove that these translation algorithms can also be applied to parallel compositions of BFMs and acyclic FTSS while maintaining trace equivalence. Section 7 contains a theoretical and empirical evaluation of our approach. In Sect. 8, we position our contribution within the state of the art and review related work. In Sect. 9, we discuss our findings and possible shortcomings of BFMs and FTSS. Finally, in Sect. 10, we conclude the paper and present the directions of our ongoing research. We illustrate all concepts in Sects. 2 to 4 using a running example of a cleaning robot SPL presented in Example 2.1.

## How to read this report?

This report contains an extensive account of our newly introduced concept of BFMs. To help the reader navigate this article, we provide guidance on which parts to read for which purposes. Furthermore, concepts marked with ★ can safely be ignored when first reading the article, as they will only be needed later on.

*Getting an overview.* To get an overview of BFMs and their isomorphic structure FESs, we suggest first getting familiar with feature models and bundle event structures in Sects. 2.1 and 2.4 (while ignoring everything marked with ★) before reading the motivation and overview of BFMs in Sect. 3. This should provide the reader with enough information to get a feeling for what BFMs are, before reading their formal definition in Sect. 4.1 (again ignoring everything marked with ★) and getting to know FESs in Def. 4.8 and Example 4.10, an isomorphic structure to BFMs.

*Understanding how BFMs and FTSS compare.* To get a basis for understanding how BFMs and FTSS compare, we suggest getting familiar with TSs and FTSS in Sects. 2.2 and 2.3 first (ignoring everything marked with ★). The reader should then get acquainted with (behavioral) products, traces and configurations of BFMs in Sect. 4.3 (again ignoring everything marked with ★) to understand what we mean by a BFM and an FTSS being trace-equivalent in Sect. 4.6. In Sect. 5, the reader will then find the algorithms translating between BFMs and acyclic FTSS. The proofs for their trace-equivalence can be skipped by readers only interested in how the translation between these structures works. Section 6 then introduces the notion of parallel composition for FTSS and BFMs before proving that the translation algorithms can also be applied to parallel compositions of FTSS and BFMs while maintaining trace equivalence. A theoretical and empirical evaluation of BFMs, where we, among others, show how compositional models of FTSS and BFMs compare on existing benchmarks of FTSS, can be found in Sect. 7. For a more theoretical exposition of composition in BFMs and FTSS, we refer to Sects. 4.5 and 9.3, respectively.

*Understanding BFMs and FESs better.* Section 4.2 discusses why and how BFMs and FESs are isomorphic and Sect. 9.4 includes a discussion about the location of events in BFMs.

## 2 BACKGROUND

In this section, we present the necessary background to motivate our proposal. Namely, we use a running example (Example 2.1 below) to illustrate the concepts of a *feature model* (in Sect. 2.1), a *transition system* (in Sect. 2.2), a *featured transition system* (in Sect. 2.3) and a *bundle event structure* (in Sect. 2.4). The reader familiar with these concepts may want to read only the parts devoted to illustrating the running example (i.e., Examples 2.6, 2.8, 2.12, 2.15, 2.21, 2.25, 2.28, 2.30, 2.35 and 2.38)

and the peculiarities of our formalization (i.e., Remarks 2.4, 2.17 and 2.39 and Notations 2.14, 2.23 and 2.32). However, we suggest reading the entire text to become familiar with our notations.

*Example 2.1 (Cleaning robot SPL).* The running example is a simple SPL of cleaning robots, which we call the *cleaning robot SPL*. The behavior of a product of the SPL can be summarized as follows: the robot starts by optionally *mapping* the cleaning area, followed by a *cleaning* procedure consisting of *moving straight* two steps to the next position, if there is no obstacle, and *cleaning* there. In case the robot *detects an obstacle*, using either a *lidar* or a *camera*, it cannot move straight. Instead, the robot has to *bypass the obstacle*, move straight and then continue cleaning. After the cleaning procedure, the robot goes to *charge*.

## 2.1 Feature Models

A *Feature Model* (FM) allows a designer to organize functionalities of a system in a structured way by providing a means to specify (hierarchical) relations and constraints between functionalities, enabling the structured specification of products of an SPL. In an FM, a *feature* is a name representing some functionality, a *configuration* is a set of features, and each *valid configuration* (i.e., a configuration that satisfies the constraints in the FM) is called a *product*. There are many different definitions of what a feature is [37, 103], we use the definition of Apel et al. [6]: “a feature is a characteristic or end-user-visible behavior of a software system”.

Before defining FMs, we define feature expressions over a set of features, i.e., Boolean formulae which have feature names as their atomic propositions. We will use these feature expressions in feature models to denote constraints between features.

*Definition 2.2 (Feature expressions).* Let  $N$  be a set of feature names and let  $x$  range over  $N$ . The set of *feature expressions* over  $N$ , denoted  $\text{FExp}(N)$ , includes all Boolean expressions over elements of  $N$  (modulo logical equivalence) generated by the following grammar:

$$\phi ::= x \mid \phi \wedge \phi \mid \phi \vee \phi \mid \phi \rightarrow \phi \mid \phi \leftrightarrow \phi \mid \neg \phi \mid \text{ff} \mid \text{tt}.$$

In the sequel, let  $I^n$  denote a finite index set  $I$  with minimal cardinality  $n$  (so  $I^2$  denotes an index set  $I$  such that  $|I| \geq 2$ ).

*Definition 2.3 (Feature Model (FM)).* Let  $N$  be a finite set of feature names with  $f \in N$ . A *feature model* (FM)  $\mathcal{M}$  over  $N$  has the following syntax:

$$\begin{aligned} \mathcal{M} &::= (f, \{\mathcal{G}_i\}_{i \in I^0}, \text{FC}) & \text{Op} &::= \text{or} \mid \text{xor} \\ \mathcal{G} &::= \text{Uop}(\mathcal{M}) \mid \text{Op}(\mathcal{M}_j)_{j \in I^2} & \text{FC} &::= \phi \\ \text{Uop} &::= \text{opt} \mid \text{mnd} \end{aligned}$$

Here,  $f$  is the *root feature* of  $\mathcal{M}$ ; the  $\mathcal{G}_i, i \in I^0$  are *sub-FM groups*, each consisting of an indexed set of *child FMs* of  $\mathcal{M}$ ; a sub-FM group can be a unary operator (Uop) or an  $n$ -ary operator (Op) applied to FMs, indicating the *kind* of the sub-FM group; a unary operator can be of kind *optional* (opt) or of kind *mandatory* (mnd), while an  $n$ -ary operator is a choice between FMs, either of kind *inclusive or* (or) or of kind *exclusive or* (xor); *cross-tree feature constraints* FC can express constraints among features in the form of a feature expression  $\phi \in \text{FExp}(N)$ .

In Def. 2.3, FMs are defined compositionally, allowing decomposition. We will use this in Sect. 4.5 to define composition and decomposition of FMs and showing that BFs allow the same.

*Remark 2.4 (Key assumptions on FMs).*

- (1) Given an FM  $\mathcal{M}$  over  $N$  (see Def. 2.3) we assume (without loss of generality) that each feature in  $N$  is present exactly once in  $\mathcal{M}$ ; and it is the root feature of one of the sub-FMs of  $\mathcal{M}$ .

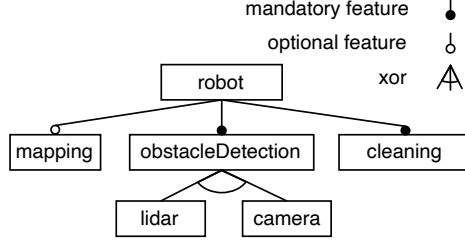


Fig. 1. A graphical representation of the FM of the cleaning robot SPL

- (2) Definition 2.3 allows cross-tree constraints FC to be situated inside sub-FMs;<sup>1</sup> without loss of generality, we assume that cross-tree constraints are situated in the smallest sub-FM that contains the mentioned features. This facilitates composition, decomposition and reusability.

The following notations are useful for working with FMs. We use the same notations for working with BFM (cf. Not. 4.3).

*Notation 2.5 (★ On FMs).* We use  $\text{features}(\mathcal{M})$  to denote the set of the features of the FM  $\mathcal{M}$ , we use  $\text{root}(\mathcal{M})$  to denote its root feature and we use  $\text{subTree}_f(\mathcal{M})$  to denote its sub-FM with root  $f$ . Moreover, given an FM  $\mathcal{M}$  and a feature  $f$  such that  $\text{subTree}_f(\mathcal{M}) = (f, \{\mathcal{G}_i\}_{i \in I^0}, \text{FC})$ , we use

- $\text{FC}_f(\mathcal{M})$  to denote the set of *cross-tree feature constraints* FC of  $f$ ;
- $\text{Uop}(f)$  and  $\text{Op}(f_1, \dots, f_m)$  (with  $m \geq 2$ ) to denote that  $\text{subTree}_f(\mathcal{M})$  and  $\text{subTree}_{f_j}(\mathcal{M})$  ( $1 \leq j \leq m$ ) are in sub-FM groups of  $\mathcal{M}$  of kind Uop and Op, respectively;
- $\text{gfc}_f(\mathcal{M}) \doteq \{\text{Uop}(\text{root}(\mathcal{M}_j)) \mid \text{Uop}(\mathcal{M}_j) = \mathcal{G}_i \text{ for some } i \in I^0\} \cup \{\text{Op}(\text{root}(\mathcal{M}_{j_1}), \dots, \text{root}(\mathcal{M}_{j_{m_i}})) \mid \text{Op}(\mathcal{M}_{j_1}, \dots, \mathcal{M}_{j_{m_i}}) = \mathcal{G}_i \text{ for some } i \in I^2\}$  to denote the set of *group feature constraints* of  $f$  which describe how the sub-FMs  $\mathcal{M}_j$  of  $\text{subTree}_f(\mathcal{M})$  are grouped into sub-FM groups.

To ease the understanding of FMs, an FM is often represented graphically as a tree in which the features are the nodes of the tree and the tree's edges are annotated with a graphical representation of the kinds of the sub-FM group represented by the sub-tree connected by the edge; cross-tree constraints are written below the FM.

To construct an FM for the cleaning robot SPL, we consider the capabilities of the cleaning robot, as described above, to be features.

*Example 2.6 (FM for the cleaning robot SPL).* Let  $N_r$  be a set of feature names containing the features *robot*, *mapping*, *cleaning*, *obstacleDetection*, *lidar* and *camera*, represented as  $r$ ,  $m$ ,  $c$ ,  $o$ ,  $li$  and  $ca$ , respectively. We define the FM  $\mathcal{M}_r$  of the cleaning robot SPL as follows:

$$\begin{aligned} \mathcal{M}_r &= (r, \{\text{opt}(\mathcal{M}_m), \text{mnd}(\mathcal{M}_c), \text{mnd}(\mathcal{M}_o)\}, \#) & \mathcal{M}_o &= (o, \{\text{xor}(\mathcal{M}_{li}, \mathcal{M}_{ca})\}, \#) \\ \mathcal{M}_m &= (m, \emptyset, \#) & \mathcal{M}_{li} &= (li, \emptyset, \#) \\ \mathcal{M}_c &= (c, \emptyset, \#) & \mathcal{M}_{ca} &= (ca, \emptyset, \#) \end{aligned}$$

The graphical representation of the feature model is shown in Fig. 1.

Another way of representing an FM is by providing a feature expression expressing the FM's structure and feature constraints. Below, we show how to translate the FM syntax (Def. 2.3) into a feature expression. This is sometimes called the logical semantics of an FM [6], which is why we denote it using  $\llbracket \cdot \rrbracket$ .

<sup>1</sup>In the literature on FMs, cross-tree constraints are often presented as a single set associated to the whole FM.

*Definition 2.7 (Propositional representation of the FM).* Let  $\mathcal{M}$  and  $\mathcal{M}_i$  be FMs over sets of feature names  $N$  and  $N_i$ , respectively, with  $N_i \subseteq N$ , and let  $f, f_i \in N$  be such that  $f = \text{root}(\mathcal{M})$  and  $f_i = \text{root}(\mathcal{M}_i)$ . The *propositional representation* of the FM  $\mathcal{M}$  is the pair  $\llbracket \mathcal{M} \rrbracket = (N, f \wedge \Phi(\mathcal{M}))$ , in which  $\Phi(\mathcal{M})$  is the feature expression over  $N$  defined inductively as:

$$\begin{aligned} \Phi((f, \{\mathcal{G}_i\}_{i \in I^0}, \text{FC})) &\doteq \bigwedge_{i \in I^0} \Phi((f, \{\mathcal{G}_i\}, \#)) \wedge \Phi(\text{FC}) \\ \Phi((f, \{\text{opt}(\mathcal{M}_1)\}, \#)) &\doteq (f_1 \rightarrow f) \wedge \Phi(\mathcal{M}_1) \\ \Phi((f, \{\text{mnd}(\mathcal{M}_1)\}, \#)) &\doteq (f_1 \leftrightarrow f) \wedge \Phi(\mathcal{M}_1) \\ \Phi((f, \{\text{or}(\mathcal{M}_i)_{i \in I^2}\}, \#)) &\doteq ((\bigvee_{i \in I^2} f_i) \leftrightarrow f) \wedge \bigwedge_{i \in I^2} \Phi(\mathcal{M}_i) \\ \Phi((f, \{\text{xor}(\mathcal{M}_i)_{i \in I^2}\}, \#)) &\doteq ((\bigvee_{i \in I^2} f_i) \leftrightarrow f) \wedge \bigwedge_{k < j \in I^2} \neg(f_j \wedge f_k) \wedge \bigwedge_{i \in I^2} \Phi(\mathcal{M}_i) \\ \Phi(\text{FC}) &\doteq \text{FC} \end{aligned}$$

*Example 2.8 (Propositional representation of the FM of the cleaning robot SPL).* The propositional representation of the FM  $\mathcal{M}_r$  of the cleaning robot defined in Example 2.6 is the pair  $(N_r, \phi_r)$ , with  $\phi_r = r \wedge \Phi(\mathcal{M}_r)$  and

$$\begin{aligned} \Phi(\mathcal{M}_r) &= ((m \rightarrow r) \wedge \Phi(\mathcal{M}_m) \wedge \Phi(\#)) \wedge ((c \leftrightarrow r) \wedge \Phi(\mathcal{M}_c) \wedge \Phi(\#)) \\ &\quad \wedge ((o \leftrightarrow r) \wedge \Phi(\mathcal{M}_o) \wedge \Phi(\#)) \wedge \Phi(\#) \\ &= (m \rightarrow r) \wedge (c \leftrightarrow r) \wedge (o \leftrightarrow r) \wedge ((li \vee ca) \leftrightarrow o) \wedge \neg(li \wedge ca) \\ \Phi(\mathcal{M}_m) &= \Phi(\#) = \# \\ \Phi(\mathcal{M}_c) &= \Phi(\#) = \# \\ \Phi(\mathcal{M}_o) &= ((li \vee ca) \leftrightarrow o) \wedge \neg(li \wedge ca) \wedge \Phi(\mathcal{M}_{li}) \wedge \Phi(\mathcal{M}_{ca}) \wedge \Phi(\#) \\ &= ((li \vee ca) \leftrightarrow o) \wedge \neg(li \wedge ca) \\ \Phi(\mathcal{M}_{li}) &= \Phi(\#) = \# \\ \Phi(\mathcal{M}_{ca}) &= \Phi(\#) = \# \end{aligned}$$

The products of an FM  $\mathcal{M}$  are defined in terms of its propositional representation. They are exactly the sets of features satisfying the propositional representation of a feature model.

*Definition 2.9 (Products of an FM).* Let  $N$  be a set of feature names, and let  $pr \subseteq N$ . An *evaluation over  $pr$*  is a function  $\text{eval}_{pr} : \text{FExp}(N) \rightarrow \mathbb{B}$ , which maps all variables  $x \in N$  of a feature expression  $\phi$  to  $\#$  if  $x \in pr$  and to  $\text{ff}$  otherwise. The *products described by a feature expression  $\phi \in \text{FExp}(N)$*  are the sets of feature names  $pr$  such that  $\text{eval}_{pr}$  satisfies  $\phi$ . Formally,  $\text{products}((N, \phi)) \doteq \{pr \mid \text{eval}_{pr}(\phi) \text{ and } pr \in \mathcal{P}(N)\}$ . The *products of an FM  $\mathcal{M}$*  are defined as  $\text{products}(\mathcal{M}) = \text{products}(\llbracket \mathcal{M} \rrbracket)$ .

We sometimes also write  $pr \models \phi$  for  $\text{eval}_{pr}(\phi) = \#$ .

*Notation 2.10 (★ Feature expression of a product).* For a set of features  $pr$ , we write  $\text{fexp}(pr)$  for the *feature expression of  $pr$* , i.e., for  $\bigwedge_{f \in pr} f \wedge \bigwedge_{g \notin pr} \neg g$ . In particular, we do so for products of an FM.

*Remark 2.11 (★ Features in products).* By the definitions of products (Def. 2.9) and propositional representation (Def. 2.7) of an FM, the following always holds: If a product  $pr$  contains a feature  $f$ , then it also contains its parent features. Inductively, this implies that  $pr$  also contains the parent of the parent of  $f$  and so on until the root, i.e., that for all  $g \in N$  with  $f \in \text{features}(\text{subTree}_g(\mathcal{M}))$  it holds that  $g \in pr$ .

*Example 2.12 (Products of the FM of the cleaning robot SPL).* Consider the FM  $\mathcal{M}_r$  of the cleaning robot SPL given in Example 2.6, graphically represented in Fig. 1, and its propositional representation  $\llbracket \mathcal{M}_r \rrbracket = (N_r, \phi_r)$  given in Example 2.8. We have  $\text{products}(\mathcal{M}_r) = \{pr_1, \dots, pr_4\}$  with  $pr_1 = \{r, c, o, li\}$ ,  $pr_2 = \{r, c, o, ca\}$ ,  $pr_3 = \{r, m, c, o, li\}$  and  $pr_4 = \{r, m, c, o, ca\}$ .



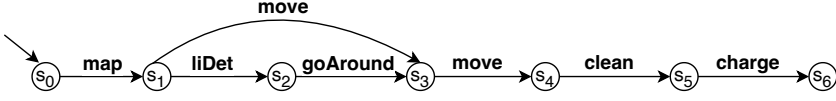


Fig. 2. A graphical representation of the TS for the product  $pr_3$  of the cleaning robot SPL.

## 2.2 Transition Systems

A Transition System (TS) [10] is a behavioral model consisting of a directed graph whose edges are labeled with actions. It can be used to model the behavior of an individual product of an SPL.

*Definition 2.13 (Transition system (TS)).* A Transition System (TS) over the actions  $A$  is a quadruple  $\mathcal{T} = (S, A, s_0, \delta)$ , where:

- $S$  is a finite (non-empty) set of *states*,
- $A$  is a finite set of *actions*,
- $s_0 \in S$  is the initial state, and
- $\delta \subseteq S \times A \times S$  is a transition relation.

*Notation 2.14 (On TSs).* We denote a transition  $t = (s, a, s') \in \delta$  as  $s \xrightarrow{a} s'$ . We use  $\text{actions}(\mathcal{T})$  to denote the set of actions  $A$  of the TS  $\mathcal{T}$ .

A TS is often represented graphically, with states represented by circles, the initial state indicated by an incoming arrow and the transition relation between states represented by arrows between these states, labeled with the action included in the transition relation between the states.

*Example 2.15 (TS for the product  $pr_3$  of the cleaning robot SPL).* The TS that describes the behavior of product  $pr_3$  in Example 2.12 is  $\mathcal{T}_3 = (S_3, A_3, s_0, \delta_3)$ , where

$$\begin{aligned} S_3 &= \{s_0, s_1, s_2, s_3, s_4, s_5, s_6\}, \\ A_3 &= \{\text{map}, \text{liDet}, \text{caDet}, \text{goAround}, \text{move}, \text{clean}, \text{charge}\}, \\ \delta_3 &= \{(s_0, \text{map}, s_1), (s_1, \text{liDet}, s_2), (s_1, \text{move}, s_3), (s_2, \text{goAround}, s_3), (s_3, \text{move}, s_4), (s_4, \text{clean}, s_5), \\ &\quad (s_5, \text{charge}, s_6)\}. \end{aligned}$$

It can be represented graphically as depicted in Fig. 2.

Before defining the traces of a TS, we define what it means for a state to be *reachable*.

*Definition 2.16 (Reachable states of a TS).* Let  $\mathcal{T} = (S, A, s_0, \delta)$  be a TS and  $s_1, s_2 \in S$ . We say that  $s_2$  is *reachable from*  $s_1$ , denoted  $\text{reachable}_{\mathcal{T}}(s_1, s_2)$  if there exists a sequence of transitions that leads from  $s_1$  to  $s_2$ , i.e., if

- $\exists a \in A. (s_1, a, s_2) \in \delta$ , or
- $\exists s \in S \exists a \in A. \text{reachable}_{\mathcal{T}}(s_1, s) \wedge (s, a, s_2) \in \delta$ .

When the TS  $\mathcal{T}$  is clear from the context, we simply write  $\text{reachable}(s_1, s)$  as a shorthand for  $\text{reachable}_{\mathcal{T}}(s_1, s)$ .

By a slight abuse of notation, we denote by  $\text{reachable}_{\mathcal{T}}(a_1, a_2)$  (or  $\text{reachable}(a_1, a_2)$ , for short) that an action  $a_2$  is reachable from an action  $a_1$  in a TS  $\mathcal{T}$  (defined similarly as in Def. 2.16).

*Remark 2.17 (Assumption on TS well-formedness).* We assume that all TSs are well-formed in the sense that all states are reachable from the initial state.

Note that some actions in a well-formed TS may not be associated with any transition. Therefore, we introduce the notion of *used actions*, i.e., actions that are associated with transitions of the TS. For instance, the used action of the TS  $\mathcal{T}_3$  in Example 2.15 are  $\text{usedAct}(\mathcal{T}_3) = A_3 \setminus \{\text{caDet}\}$ .

**Definition 2.18** (★ *Used actions of a TS*). Let  $\mathcal{T} = (S, A, s_0, \delta)$  be a TS. The set of *used actions* of  $\mathcal{T}$ , denoted  $\text{usedAct}(\mathcal{T})$ , is defined as  $\{a \in A \mid \exists s, s' \in S. (s, a, s') \in \delta\}$ .

The traces of a TS, i.e., the sequences of actions specified by it, are defined as follows.

**Definition 2.19** (*Traces of a TS*). A *trace* of a TS  $\mathcal{T} = (S, A, s_0, \delta)$  is a sequence  $\langle a_1, \dots, a_n \rangle$  ( $n \geq 1$ ) such that for all  $i \in \{1, \dots, n\}$  we have  $a_i \in A$  and there exists a transition  $s_{i-1} \xrightarrow{a_i} s_i \in \delta$ . We denote by  $\text{Tr}(\mathcal{T})$  the set of traces of a TS  $\mathcal{T}$ .

The following notations will make it easier to work with traces of a TS in Sect. 5.

**Notation 2.20** (★ *On traces of TSs*). For a trace  $tr = \langle a_1 \cdot \dots \cdot a_n \rangle$  with  $(s_{i-1}, a_i, s_i) \in \delta, i \in \{1, \dots, n\}$ , we write  $\text{actions}(tr)$  for the set  $\{a_1, \dots, a_n\}$  and  $s_0 \xrightarrow{tr} s_n$  for  $s_0 \xrightarrow{a_1} s_1 \xrightarrow{a_2} \dots \xrightarrow{a_n} s_n$ .

**Example 2.21** (*Traces of the TS for the product  $pr_3$  of the cleaning robot SPL*). The set of traces of  $\mathcal{T}_3$  of Example 2.15 consists of all prefixes of the traces  $\langle \text{map} \cdot \text{move} \cdot \text{move} \cdot \text{clean} \cdot \text{charge} \rangle$  and  $\langle \text{map} \cdot \text{liDet} \cdot \text{goAround} \cdot \text{move} \cdot \text{clean} \cdot \text{charge} \rangle$ , i.e.,

$\text{Tr}(\mathcal{T}_3) = \text{prefixClosure}(\{\langle \text{map} \cdot \text{move} \cdot \text{move} \cdot \text{clean} \cdot \text{charge} \rangle, \langle \text{map} \cdot \text{liDet} \cdot \text{goAround} \cdot \text{move} \cdot \text{clean} \cdot \text{charge} \rangle\})$ ,

where  $\text{prefixClosure}$  is the prefix-closure operator, which takes a set of traces and adds to it all prefixes of its traces (thus making it prefix-closed). Note that this also includes the empty trace.

### 2.3 Featured Transition Systems

The notion of a *featured transition system* (FTS) [36, 40] is built on top of the notion of a TS (see Def. 2.13). Namely, given a feature model  $\mathcal{M}$ , an FTS is a TS where, additionally to the action labels on transitions, transitions are also labeled with feature expressions over the features of  $\mathcal{M}$ . Thereby, an FTS describes the behavior of all products of an SPL in one model. The behavior of a product  $pr$  of  $\mathcal{M}$  is obtained by taking all transitions of the FTS that are labeled by feature expressions satisfied by  $pr$ , i.e., transitions labeled by feature expressions  $\phi$  such that  $\text{eval}_{pr}(\phi) = \#$  (defined in Def. 2.9). Modeling the behavior of all products in one model facilitates so called *family-based analysis* according to which all products of an SPL are analyzed in one run [105].

**Definition 2.22** (*Featured transition system (FTS)*). A *Featured Transition System (FTS)* over the features  $N$  and the actions  $A$  is a triple  $\mathcal{F} = (\mathcal{M}, \mathcal{T}, \mu)$ , where:

- $\mathcal{M}$  is an FM over  $N$ , called *the FM of  $\mathcal{F}$* ,
- $\mathcal{T} = (S, A, s_0, \delta)$  is a TS over  $A$ , called *the TS of  $\mathcal{F}$* ,
- $\mu : S \times A \times S \rightarrow \text{FExp}(N)$  is a mapping that associates each transition of  $\mathcal{T}$  with a feature expression over  $N$ , that we call the *product (configuration) knowledge of  $\mathcal{F}$*  in this paper.

**Notation 2.23** (*On FTSs*). The pair  $(\mathcal{T}, \mu)$  is called the *annotated TS of  $\mathcal{F}$* . We use  $\text{features}(\mathcal{F})$ ,  $\text{actions}(\mathcal{F})$  and  $\text{prodKnow}(\mathcal{F})$  to denote the set of features, the set of actions and product knowledge of the FTS  $\mathcal{F}$ , respectively.

A graphical representation of an annotated TS is obtained from the graphical representation of its TS by additionally labeling each transition with the propositional representation.

As for TSs (Def. 2.18), we define the notion of *used actions* for FTSs.

**Definition 2.24** (★ *Used actions of an FTS*). Let  $\mathcal{F} = (\mathcal{M}, \mathcal{T}, \mu)$  be an FTS. The set of *used actions* of  $\mathcal{F}$  is the set of used actions of  $\mathcal{T}$ , i.e.,  $\text{usedAct}(\mathcal{F}) \doteq \text{usedAct}(\mathcal{T})$ .



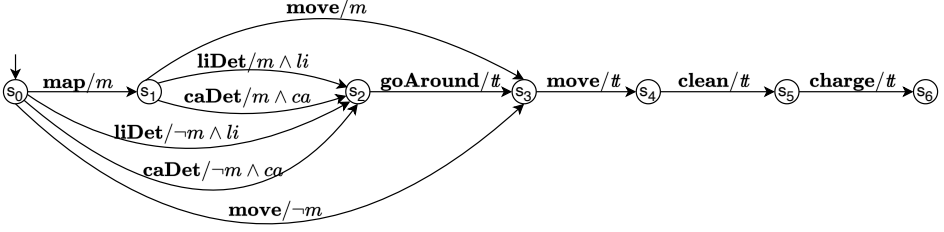


Fig. 3. A graphical representation of the annotated TS (i.e., the TS and product knowledge) of the FTS for the cleaning robot.

*Example 2.25 (FTS of the cleaning robot SPL).* The behavior of the cleaning robot SPL can be modeled by the FTS  $\mathcal{F}_r = (\mathcal{M}_r, (S_r, A_r, s_0, \delta_r), \mu_r)$ , where  $\mathcal{M}_r$  is the FM of Example 2.6 and

$$\begin{aligned}
 S_r &= \{s_0, s_1, s_2, s_3, s_4, s_5\}, \\
 A_r &= \{map, liDet, caDet, goAround, move, clean, charge\}, \\
 \delta_r &= \{(s_0, map, s_1), (s_0, liDet, s_2), (s_0, caDet, s_2), (s_0, move, s_3), (s_1, liDet, s_2), (s_1, caDet, s_2), \\
 &\quad (s_1, move, s_3), (s_2, goAround, s_3), (s_3, move, s_4), (s_4, clean, s_5), (s_5, charge, s_6)\}, \\
 \mu_r(t) &= \begin{cases} m & \text{if } t \in \{(s_0, map, s_1), (s_1, move, s_3), (s_2, goAround, s_3)\}, \\ \neg m \wedge li & \text{if } t = (s_0, liDet, s_2), \\ \neg m \wedge ca & \text{if } t = (s_0, caDet, s_2), \\ \neg m & \text{if } t = (s_0, move, s_3), \\ m \wedge li & \text{if } t = (s_1, liDet, s_2), \\ m \wedge ca & \text{if } t = (s_1, caDet, s_2), \\ \perp & \text{if } t \in \{(s_3, move, s_4), (s_4, clean, s_5), (s_5, charge, s_6)\}, \end{cases}
 \end{aligned}$$

where  $S_r = S_3$  and  $A_r = A_3$  for  $S_3, A_3$  in Example 2.15. It can be graphically represented as shown in Fig. 1 (which depicts the FM) and in Fig. 3 (which depicts its annotated TS). The used actions of  $\mathcal{F}_r$  are exactly its actions  $A_r$ .

The *products of an FTS* are the products of its FM, and the *behavioral products of an FTS* are the TSs associated with its products. Namely, a behavioral product of an FTS  $\mathcal{F}$  for a product  $pr$  is the so called *projection* of the TS of  $\mathcal{F}$  onto the sub-TS  $\mathcal{T}'$  such that for every transition in  $\mathcal{T}'$ , the feature expression associated to this transition in  $\mathcal{F}$  is satisfied by  $pr$ . Products and behavioral products are formally defined as follows.

*Definition 2.26 (Products and behavioral products of an FTS).* Given an FTS  $\mathcal{F} = (\mathcal{M}, \mathcal{T}, \mu)$ , the *products of  $\mathcal{F}$*  are the products of  $\mathcal{M}$ . For each of the FTS's products  $pr$ , the *behavioral product of  $\mathcal{F}$  for  $pr$*  is the TS  $\text{behPr}(\mathcal{F}, pr)$  obtained from the TS  $\mathcal{T} = (S, A, s_0, \delta)$  by:

- (1) first removing all the transitions  $t \in \delta$  such that  $\text{eval}_{pr}(\mu(t)) = \text{ff}$ ; and
- (2) then removing all the states and transitions that are no longer reachable from the initial state.

We write  $\text{products}(\mathcal{F})$  to denote the set of products of  $\mathcal{F}$ , and  $\text{behPr}(\mathcal{F})$  to denote the set of behavioral products of  $\mathcal{F}$ .

*Remark 2.27 (★ On used actions of behavioral products of an FTS).* Let  $\mathcal{F} = (\mathcal{M}, \mathcal{T}, \mu)$  be an FTS with  $\mathcal{T} = (S, A, s_0, \delta)$ , and let  $pr$  be one of its products. Then  $\text{usedAct}(\text{behPr}(\mathcal{F}, pr)) = \{a \in A \mid \exists s, s' \in S. \text{eval}_{pr}(\mu((s, a, s')))) = \text{tt} \wedge \text{reachable}_{\text{behPr}(\mathcal{F}, pr)}(s_0, s)\}$ .

*Example 2.28 (Behavioral product of the FTS  $\mathcal{F}_r$  for the product  $pr_3$ ).* Consider the FTS  $\mathcal{F}_r$  given in Example 2.25 (and depicted in Fig. 3) and the product  $pr_3 = \{r, m, c, o, li\}$  in Example 2.12. The behavioral product of  $\mathcal{F}_r$  for  $pr_3$  is  $\text{behPr}(\mathcal{F}_r, pr_3) = \mathcal{T}_3$ , where  $\mathcal{T}_3$  is the TS given in Example 2.15 (and depicted in Fig. 2).

The traces of an FTS for a product are obtained from its behavioral product as defined below.

*Definition 2.29 (Traces of an FTS for a product).* The traces of an FTS  $\mathcal{F}$  for a product  $pr$  are the traces of its behavioral product for  $pr$ , i.e., we define the set of traces  $\text{Tr}(\mathcal{F}, pr)$  of  $\mathcal{F}$  for  $pr$  as  $\text{Tr}(\mathcal{F}, pr) \doteq \text{Tr}(\text{behPr}(\mathcal{F}, pr))$ . More concretely, for an FTS  $\mathcal{F} = (\mathcal{M}, \mathcal{T}, \mu)$ , a product  $pr$  of  $\mathcal{F}$  with behavioral product  $\text{behPr}(\mathcal{F}, pr) = (S, A, s_0, \delta)$ , a trace in  $\text{Tr}(\mathcal{F}, pr)$  is a sequence  $\langle a_1, \dots, a_n \rangle$  ( $n > 1$ ) of actions  $a_i \in A$ ,  $i \in \{1, \dots, n\}$ , satisfying for all  $i \in \{1, \dots, n\}$ :

- (1)  $\exists s_{i-1}, s_i \in S. (s_{i-1}, a_i, s_i) \in \delta$ ; and
- (2)  $\forall (s_{i-1}, a_i, s_i) \in \delta. \text{eval}_{pr}(\mu((s_{i-1}, a_i, s_i))) = \#$ .

*Example 2.30 (Traces of the behavioral products of the FTS of the cleaning robot SPL).* Consider the FTS  $\mathcal{F}_r$  given in Example 2.25 (with annotated TS depicted in Fig. 3). The products of  $\mathcal{F}_r$  are  $pr_1, pr_2, pr_3$  and  $pr_4$  as defined in Example 2.12. The traces of its behavioral products  $\mathcal{T}_i = \text{behPr}(\mathcal{F}_r, pr_i)$  ( $1 \leq i \leq 4$ ) are the following:

$$\begin{aligned} \text{Tr}(\mathcal{T}_1) &= \text{prefixClosure}(\{\langle \text{move} \cdot \text{move} \cdot \text{clean} \cdot \text{charge} \rangle, \langle \text{liDet} \cdot \text{goAround} \cdot \text{move} \cdot \text{clean} \cdot \text{charge} \rangle\}) \\ \text{Tr}(\mathcal{T}_2) &= \text{prefixClosure}(\{\langle \text{move} \cdot \text{move} \cdot \text{clean} \cdot \text{charge} \rangle, \langle \text{caDet} \cdot \text{goAround} \cdot \text{move} \cdot \text{clean} \cdot \text{charge} \rangle\}) \\ \text{Tr}(\mathcal{T}_3) &= \text{prefixClosure}(\{\langle \text{map} \cdot \text{move} \cdot \text{move} \cdot \text{clean} \cdot \text{charge} \rangle, \\ &\quad \langle \text{map} \cdot \text{liDet} \cdot \text{goAround} \cdot \text{move} \cdot \text{clean} \cdot \text{charge} \rangle\}) \\ \text{Tr}(\mathcal{T}_4) &= \text{prefixClosure}(\{\langle \text{map} \cdot \text{move} \cdot \text{move} \cdot \text{clean} \cdot \text{charge} \rangle, \\ &\quad \langle \text{map} \cdot \text{caDet} \cdot \text{goAround} \cdot \text{move} \cdot \text{clean} \cdot \text{charge} \rangle\}) \end{aligned}$$

## 2.4 Bundle Event Structures

Event structures [83, 111] were first introduced to capture models of true concurrency: systems in which multiple events can happen “at the same time”. Event structures distinguish between *actions* and *events*, where the latter can be thought of as occurrences of the former. To model true concurrency, event structures use the concepts of *conflict* and *causality* to denote events that exclude each other and events that enable other events, respectively. Like a TS, an event structure specifies a set of traces (i.e., sequences of events); so it can be used to model the behavior of an individual product of an SPL. *Bundle event structures (BESs)* [73], defined next, are an extension of event structures in which the causality relation (a.k.a. bundle set or enabling relation) allows both disjunctive and conjunctive enabling relations to be specified. The flexible behavioral models provided by event structures, as well as the combination of disjunctive and conjunctive enabling constraints provided by bundle event structures, are exactly what we need to match the composition structure of FMs to address behavior.

*Definition 2.31 (Bundle event structures (BESs)).* A BES over a set of events  $E$  and a set of actions  $A$  is a tuple  $\mathcal{S} = (E, \#, \mapsto, A, l)$ , where

- $E$  is a finite set of events,
- $\# \subseteq E \times E$  is an irreflexive and symmetric relation, called the *conflict relation*, and
- $\mapsto \subseteq (\mathcal{P}(E) \setminus \{\emptyset\}) \times E$  is a relation, called the *causality relation*, satisfying:
  - (1)  $\forall (X, e) \in \mapsto \forall e_1, e_2 \in X. e_1 \neq e_2 \Rightarrow e_1 \# e_2$ ,
  - (2)  $\forall (X, e), (Y, e) \in \mapsto. X \neq Y \Rightarrow (X \not\subseteq Y \wedge Y \not\subseteq X)$ ,
- $A$  is a finite set of *actions*, called the *alphabet* of  $E$ , and
- $l : E \rightarrow A$  is the *labeling function*.

*Notation 2.32 (On BESs).* We use  $\text{events}(\mathcal{S})$  and  $\text{actions}(\mathcal{S})$  to denote the sets of events and actions of the BES  $\mathcal{S}$ , respectively. For events  $e, e_1, e_2$  and a set of events  $X$ , we write  $e_1 \# e_2$  for  $(e_1, e_2) \in \#$  and  $X \mapsto e$  for  $(X, e) \in \mapsto$ .

Similarly as for TSs and FTSs (Definitions 2.18 and 2.24), we define the set of *used actions* of a BES as the set of all actions for which an event occurrence exists.

*Definition 2.33 (★ Used actions of a BES).* Let  $\mathcal{S} = (E, \#, \mapsto, A, l)$  be a BES. The set of *used actions* of  $\mathcal{S}$ , denoted  $\text{usedAct}(\mathcal{S})$ , is defined as  $\{a \in A \mid \exists e \in E. l(e) = a\}$

The behavior of a BES can be specified by set of events, called *configurations*, conforming to the conflict and causality relation. A configuration  $C$  contains non-conflicting events such that if an event  $e \in E$  is included in  $C$  and  $(X, e) \in \mapsto$ , then  $C$  contains exactly one element of  $X$ . Configurations consist of events that constitute (partial) runs of the represented system. Therefore, a causality  $(X, e) \in \mapsto$  specifies *disjunctive causality* of events where exactly one event in  $X$  has to occur in a run of a system before  $e$  can occur. The condition enforcing  $X$  to contain only conflicting events ensures that an enabling event can be uniquely identified. *Conjunctive causality*, i.e., that a set of events (instead of a single event) has to occur in a run of the system before another event, can be specified by using several causality constraints. For example,  $\{e_1\} \mapsto e$  and  $\{e_2\} \mapsto e$  implies that both  $e_1$  and  $e_2$  are required for  $e$  to happen.

*Definition 2.34 (Configurations of a BES).* Let  $\mathcal{S} = (E, \#, \mapsto, A, l)$  be a BES. A *configuration*  $C \subseteq E$  of  $\mathcal{S}$  is a set of distinct events satisfying:

- (1)  $C$  is conflict free, i.e.,  $\forall e, e' \in C. (e, e') \notin \#$ ; and
- (2)  $\forall X \subseteq E \forall e \in C. (X, e) \in \mapsto \Rightarrow C \cap X \neq \emptyset$ .

*Example 2.35 (BES and configurations for the product  $pr_3$  of the cleaning robot SPL).* Consider the product  $pr_3$  of the cleaning robot SPL as described in Example 2.12. The BES  $\mathcal{S}_3 = (E_3, \#_3, \mapsto_3, A_3, l_3)$  describes the behavior of  $pr_3$ , where

$$\begin{aligned} E_3 &= \{\text{map}, \text{liDet}, \text{goAround}, \text{move}_1, \text{move}_2, \text{clean}, \text{charge}\}, \\ \#_3 &= \{(\text{liDet}, \text{move}_1), (\text{goAround}, \text{move}_1)\}, \\ \mapsto_3 &= \{(\{\text{map}\}, \text{liDet}), (\{\text{liDet}\}, \text{goAround}), (\{\text{map}\}, \text{move}_1), (\{\text{move}_1, \text{goAround}\}, \text{move}_2), \\ &\quad (\{\text{move}_2\}, \text{clean}), (\{\text{clean}\}, \text{charge})\}, \\ A_3 &= \{\text{map}, \text{liDet}, \text{caDet}, \text{goAround}, \text{move}, \text{clean}, \text{charge}\}, \text{ and} \\ l_3(e) &= \begin{cases} e & \text{if } e \in \{\text{map}, \text{liDet}, \text{goAround}, \text{clean}, \text{charge}\} \\ \text{move} & \text{if } e \in \{\text{move}_1, \text{move}_2\}. \end{cases} \end{aligned}$$

It can be represented graphically as depicted in Fig. 4. Its configurations are the sets  $C_0 = \emptyset$ ,  $C_1 = \{\text{map}\}$ ,  $C_2 = \{\text{map}, \text{liDet}\}$ ,  $C_3 = \{\text{map}, \text{liDet}, \text{goAround}\}$ ,  $C_4 = \{\text{map}, \text{liDet}, \text{goAround}, \text{move}_2\}$ ,  $C_5 = \{\text{map}, \text{liDet}, \text{goAround}, \text{move}_2, \text{clean}\}$ ,  $C_6 = \{\text{map}, \text{liDet}, \text{goAround}, \text{move}_2, \text{clean}, \text{charge}\}$ ,  $C_7 = \{\text{map}, \text{move}_1\}$ ,  $C_8 = \{\text{map}, \text{move}_1, \text{move}_2\}$ ,  $C_9 = \{\text{map}, \text{move}_1, \text{move}_2, \text{clean}\}$ , and finally  $C_{10} = \{\text{map}, \text{move}_1, \text{move}_2, \text{clean}, \text{charge}\}$  and its used actions are  $A_3 \setminus \{\text{caDet}\}$ .

A trace of events of a BES is a sequence of events that is conflict free and respects the causality relation, i.e., for every event  $e$  in a trace  $t$  and every  $X \subseteq E$  that causes  $e$ , an event of  $X$  is included in  $t$ ; more specifically, it comes before  $e$ . A corresponding trace of actions of a BES is defined using the labeling function.

*Definition 2.36 (Traces of a BES).* Let  $\mathcal{S} = (E, \#, \mapsto, A, l)$  be a BES.

- A *trace of events* of  $\mathcal{S}$  is a sequence  $\langle e_1 \cdot \dots \cdot e_n \rangle$  ( $n \geq 1$ ) of distinct events  $e_i \in E$ ,  $i \in \{1, \dots, n\}$ , satisfying:

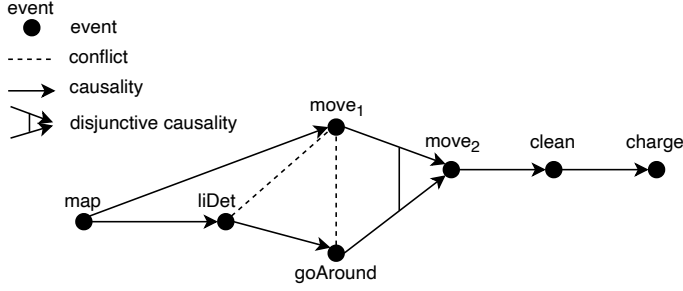


Fig. 4. The BES for the product  $pr_3$  of the cleaning robot

- (1)  $\{e_1, \dots, e_n\}$  is conflict free, i.e.,  $\forall i, j \in \{1, \dots, n\}. (e_i, e_j) \notin \#$ ;
  - (2)  $\forall X \subseteq E \forall i \in \{2, \dots, n\}. X \mapsto e_i \Rightarrow \{e_1, \dots, e_{i-1}\} \cap X \neq \emptyset$ ; and
  - (3)  $\nexists X \subseteq E. X \mapsto e_1$ .
- Let  $\langle e_1 \cdot \dots \cdot e_n \rangle$  ( $n \geq 1$ ) be a trace of events of  $\mathcal{S}$  with  $e_i \in E, i \in \{1, \dots, n\}$ . Then the corresponding *trace of actions of  $\mathcal{S}$*  is defined as the sequence  $\langle l(e_1) \cdot \dots \cdot l(e_n) \rangle$ .

We denote by  $\text{Tr}^{\text{ev}}(\mathcal{S})$  the set of traces of events of  $\mathcal{S}$  and by  $\text{Tr}(\mathcal{S})$  the set of traces of actions of  $\mathcal{S}$ .<sup>2</sup>

Note that for a configuration  $C$ , an element  $e \in C$  and  $(X, e) \in \mapsto$ , the set  $C \cap X$  contains exactly one element because all elements in  $X$  are in conflict and  $C$  is conflict-free. Therefore, the causality relation  $\mapsto$  introduces a partial order  $<_C$  on the events of  $C$ : for two events  $e, e' \in C$  it holds that  $e' <_C e$  if and only if there exists  $(X, e) \in \mapsto$  with  $e' \in C$  (and thus  $\{e'\} = C \cap X$ ). This partial order allows us to connect the notions of configuration and trace of a BES.

**LEMMA 2.37 (★ CONFIGURATIONS FORMING TRACES OF A BES).** *Let  $\mathcal{S} = (E, \#, \mapsto, A, l)$  be a BES and  $C \subseteq E$  be a set of events. Then  $C$  is a configuration of  $\mathcal{S}$  if and only if there exists a trace of events  $tr \in \text{Tr}^{\text{ev}}(\mathcal{S})$  such that  $C = \text{events}(tr)$ .*

**PROOF.** “ $\Rightarrow$ ” Let  $C$  be a configuration of  $\mathcal{S}$ , i.e.,  $C$  is conflict free and for all  $X \subseteq E$  and all  $e \in C$ , it holds that  $(X, e) \in \mapsto$  implies  $C \cap X \neq \emptyset$ . Now let  $\langle e_1, \dots, e_n \rangle$  be an ordering of the elements of  $C$  that respects the partial order  $<_C$  introduced by  $\mapsto$ . Then  $\langle e_1, \dots, e_n \rangle$  satisfies Condition (1) of Def. 2.36 because events  $e_1, \dots, e_n$  are conflict free since they are elements of  $C$ , and it satisfies Condition (2) of Def. 2.36 because the  $e_i, i \in \{1, \dots, n\}$ , are ordered according to the partial order induced by  $\mapsto$ . Thus,  $\langle e_1, \dots, e_n \rangle \in \text{Tr}(\mathcal{S})$ .

“ $\Leftarrow$ ” Let  $tr \in \text{Tr}(\mathcal{S})$  and  $C = \text{events}(tr)$ . Then  $C$  is conflict free by Condition (1) of Def. 2.36, and for all  $e \in C$  and all  $X \subseteq E$ , it holds that  $(X, e) \in \mapsto$  implies  $X \cap C = \emptyset$  by Conditions (2) and (3) of Def. 2.36.  $\square$

Note that the definition of traces of events of a BES (Def. 2.36) contains one more condition than the definition of configurations of a BES (Def. 2.34). A trace of events of a BES is a sequence of events, so we have to ensure that the events are ordered according to the causality relation and that the first event is not caused by another event, ensured by Conditions (2) and (3) of Def. 2.36, respectively. We do not need the second condition to define configurations because a configuration is a set.

<sup>2</sup>In Langerak [73, Def. 2.3], which is credited to Boudol and Castellani [28, page 10], a *trace of events* is called a *proving sequence*.

*Example 2.38 (Traces of the BES for the product  $pr_3$  of the cleaning robot SPL).* Consider product  $pr_3$  of the cleaning robot SPL as described in Example 2.12 and BES  $\mathcal{S}_3$  describing its behavior as shown in Example 2.35. The set of traces of events of  $\mathcal{S}_3$  consists of all prefixes of the traces  $\langle map \cdot move_1 \cdot move_2 \cdot clean \cdot charge \rangle$  and  $\langle map \cdot liDet \cdot goAround \cdot move_2 \cdot clean \cdot charge \rangle$ . The set of traces of actions of  $\mathcal{S}_3$  is exactly the set of traces of the TS for the product  $pr_3$  given in Example 2.21.

*Remark 2.39 (On the definition of BESs).* To simplify the presentation in Sect. 4, we slightly modified the definition of BESs given in [73, Def. 2.1] by: (i) excluding impossible events, i.e., events  $e$  with  $\emptyset \mapsto e$  (which, as pointed out in [73], can always be removed and were included because they are technically convenient in expressing some operations in a concise way); and (ii) not allowing (thanks to Condition (2)) causalities  $X \mapsto e$  such that there exists a causality  $Y \mapsto e$  such that  $X \subset Y$ , because the causality  $X \mapsto e$  would disallow traces using events in  $Y \setminus X$ .

In Sects. 5.1 and 5.2 we develop algorithms between BESs and acyclic TSs that preserve trace equivalence. We now define what it means for a TS and a BES to be trace equivalent.

*Definition 2.40 (★ Trace equivalence for TSs and BESs).* A TS  $\mathcal{T}$  and a BES  $\mathcal{S}$  are *trace equivalent* if

- (1) they have the same actions; and
- (2) it holds that  $\text{Tr}(\mathcal{T}) = \text{Tr}(\mathcal{S})$ .

### 3 BEHAVIORAL FEATURE MODELS: MOTIVATION AND OVERVIEW

*Behavioral Feature Models (BFMs)* facilitate compositional modeling of the structure and behavior of SPLs in one single model, similarly to the compositional modeling of the structure of an SPL in an FM. To do so, the structural model of an SPL in the form of an FM is extended by attaching behavior to every feature and specifying constraints between behaviors of different features. Thereby, BFMs are able to not only specify the behavior of all products of an SPL, but also the behavior of single features. We designed BFMs according to the following **Design Choices (DCs)**:

**DC1** Features are organized into an FM.

**DC2** The behavior of each product of the BFM is described in terms of events by a BES.

**DC3** The behavior of each feature is described in terms of events by a BES, and BESs associated with different features have disjoint sets of events (so that each event in the BFM is associated with exactly one feature).

**DC4** Constraints between behaviors associated with different features can be specified using conflicts and causalities.

**DC5** Dependencies between behavior and features can be specified using feature expressions.

A BFM can be understood as an FM (see DC1) in which each feature  $f$  is associated with a BES that models the behavior of  $f$ , and the BESs associated with different features contain disjoint sets of events (see DC3). So called *inter-feature event constraints* specify conflicts and causalities between events of BESs associated with different features (see DC4), while *product constraints* specify that certain behavior depends on the selection of features (see DC5). Thereby, a BFM contains *structural* cross-tree constraints in the form of cross-tree feature constraints (as an FM), *behavioral* cross-tree constraints in the form of inter-feature event constraints, and a mix of the two in the form of product constraints. The behavior of a product of the BFM is described by a BES (see DC2) that can be composed from the BESs associated to the product's features and the involved inter-feature event constraints.

*Example 3.1 (Creating a BFM for cleaning robot SPL).* As an example, we consider the cleaning robot SPL as defined in Example 2.1. We now describe how a BFM for the cleaning robot SPL can be created. The behavior of the cleaning robot SPL is determined by the actions

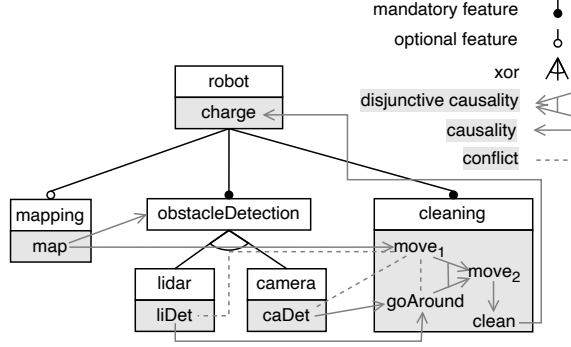


Fig. 5. A graphical representation of the BFM for the cleaning robot SPL

$A_r = \{map, liDet, goAround, move, clean, charge, caDet\}$ , as already defined in Example 2.25. The set of events of the BFM is determined by the behavior of the cleaning robot SPL. If an action is only used once in a run of the behavior of the SPL, then we include a copy of that action in the set of events. In this example, this is the case for every action apart from *move*. For *move*, we have to include two occurrences in  $E_r$ , called *move*<sub>1</sub> and *move*<sub>2</sub>, because the action appears twice in a trace of the cleaning robot SPL.

Now we decide which events should be associated with which feature of the FM of the cleaning robot SPL, where the FM is the same as the one in Example 2.6. The events should be associated with the lowest possible feature of the FM such that the behavior of a feature can be described using the events associated with it. Therefore, we decided to associate the event *map* with the feature *mapping*, the event *liDet* with the feature *lidar*, the event *caDet* with the feature *camera*, the events *move*<sub>1</sub>, *move*<sub>2</sub>, *goAround*, and *clean* with the feature *cleaning*, and the event *charge* with the feature *robot*. These are of course design choices and other assignments of events with features would be possible.

Next, we define the constraints among the events, both within the events associated with a feature as well as between events associated with different features. The information for how to define them is included in the textual description of the cleaning robot SPL in Example 2.1: The event *map* causes the events *liDet*, *caDet*, and *move*<sub>1</sub>; the events *liDet* and *caDet* are in conflict with the event *move*<sub>1</sub> and both cause the event *goAround*; the events *move*<sub>1</sub> and *goAround* are in conflict, and either of them causes the event *move*<sub>2</sub>; the event *move*<sub>2</sub> causes the event *clean*; and the event *clean* causes the event *charge*. Note that, as noted in Sect. 2.4, we used two different types of causality here: conjunctive causality (*liDet* and *caDet* both cause *move*<sub>1</sub>) and disjunctive causality (either one of *move*<sub>1</sub> and *goAround* causes *move*<sub>2</sub>). Combining the events associated with a feature and the conflicts and causalities between them gives the BES describing the behavior of a feature.

The final step is to specify dependencies of events on features. In our case, the only dependencies are that the events require the feature they are associated with. A graphical representation of the BFM  $\mathcal{B}_r$  for the cleaning robot SPL, which will be formally defined in Example 4.6, is shown in Fig. 5, where the behavioral part is highlighted with gray backgrounds and gray arrows.

Working with BFMs provides the additional advantage that its structure and behavior can easily be separated, providing an isomorphic view of BFMs called *featured event structures* (FESs) because they associate features to (bundle) event structures. Thus, BFMs enjoy the advantages of a combined model of the structure and the behavior of an SPL (such as, e.g., ease of maintenance), while keeping



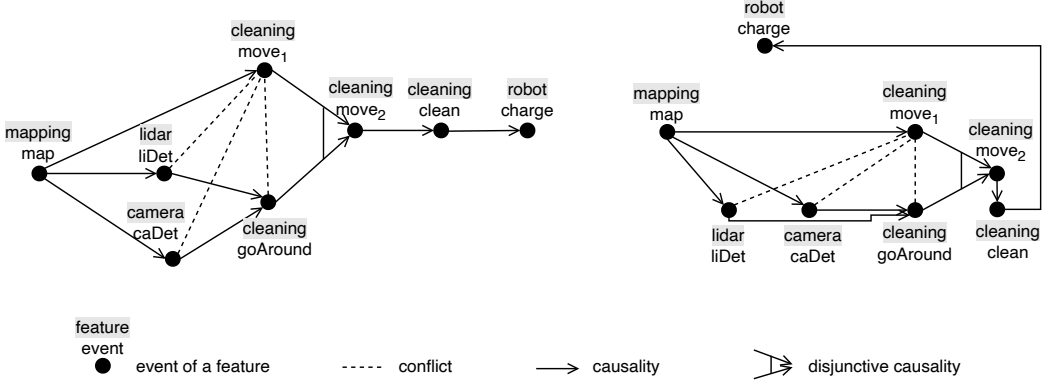


Fig. 6. Two graphical representations of the annotated BES (that is, the BES where each event is annotated by a feature, highlighted in gray) of the BFM in Fig. 5. *Left*: A graphical representation of the annotated BES where (to improve comprehensibility) events are positioned according to the topological ordering w.r.t. causality. *Right*: A graphical representation of the annotated BES where (to facilitate comparison with the graphical representation of the BFM in Fig. 5) events are positioned as in Fig. 5.

a separation of concerns by separating the model of the structure from the model of the behavior (as is the case for FTSs). We provide an intuition of FESs using the cleaning robot SPL.

*Example 3.2 (Isomorphic view of BFM for cleaning robot SPL).* It is worth observing that, in the graphical representation of the BFM  $\mathcal{B}_r$  in Fig. 5, we can clearly see four distinct entities. Namely:

- (1) an FM, identical to the FM  $\mathcal{M}_r$  depicted in Fig. 1, called *the FM of the BFM*;
- (2) the BES depicted in Fig. 6 (please ignore, for now, the feature names highlighted in gray), which is depicted in two different ways (to improve comprehensibility and to facilitate comparison with Fig. 5, respectively), called *the BES of the BFM*;
- (3) a mapping that associates each event with a feature, visualized by the feature names highlighted in gray that annotate each event in Fig. 6, called *event location knowledge of the BFM*; and
- (4) when including product constraints different from the event location knowledge, we would see them as an additional entity in Fig. 6, called *product knowledge of the BFM*.

Note that the focus of the representation of behavior in Fig. 6 is not on the behavior of single features. Instead, similar to FTSs, the focus is on the behavior of the whole SPL. Using the four different entities above provides an isomorphic view of BFMs, the FES  $\mathcal{E}_r$ .

As stated before, the behavior of a product of a BFM can be obtained by combining the behavior of its features and the necessary constraints between them. The FES that is isomorphic to a BFM provides a convenient structure for defining the behavior of products. We illustrate this using the cleaning robot SPL.

*Example 3.3 (Behavior of products of BFM for the cleaning robot SPL).* The products of BFM  $\mathcal{B}_r$  for the cleaning robot SPL are the products of its features model, as defined in Example 2.12. The behavior of each product  $pr$  can be obtained from the FES  $\mathcal{E}_r$ , depicted in Fig. 6, that is isomorphic to the BFM  $\mathcal{B}_r$  by:

- (1) removing all the events  $e$  that are labeled by a feature not included in  $pr$ ;
- (2) removing all the conflicts (depicted as dotted lines) that involve at least one removed event; and

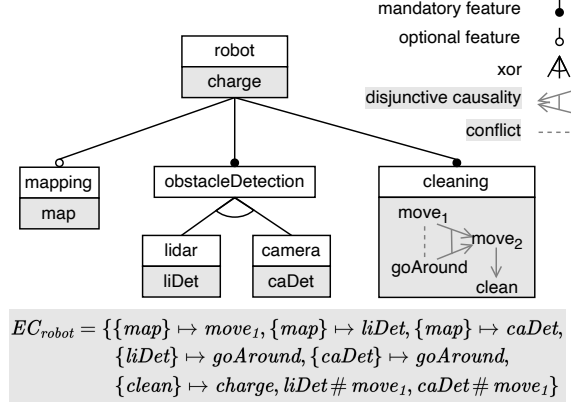


Fig. 7. A graphical representation of the BFM for the cleaning robot SPL where conflicts and causalities are displayed in textual form (cf. Fig. 5).

- (3) removing or transforming all the causalities (depicted as arrows) that involve at least one removed event, i.e., each causality  $X \mapsto e$  is either: (i) removed if  $e$  is removed or all the events in  $X$  are removed; or otherwise (ii) transformed into  $X' \mapsto e$ , where  $e$  is an event labeled by a feature in  $pr$  and  $X' \neq \emptyset$  is obtained from  $X$  by removing all the removed events.

For example, the behavior of the product  $pr_3 = \{r, m, c, o, li\}$  is the BES  $\mathcal{S}_3$  given in Example 2.35 (and depicted in Fig. 4).

A large portion of this paper is devoted to showing that BFMs and acyclic FTSs are trace equivalent. We shortly illustrate this using the cleaning robot SPL.

*Example 3.4 (Trace equivalence of FTS and BFM for the cleaning robot SPL).* The BFM  $\mathcal{B}_r$  (given in Fig. 5) and the FTS  $\mathcal{F}_r$  (given in Fig. 1, which depicts the FM of  $\mathcal{F}_r$ , and in Fig. 3, which depicts the annotated TS of  $\mathcal{F}_r$ ) are *trace equivalent*, i.e.:

- (1) they have the same features, actions and products; and
- (2) each of their products has the same behavior, i.e., for every product  $pr$ , the TS obtained from  $\mathcal{F}_r$  which describes the behavior of  $pr$  and the BES obtained from  $\mathcal{B}_r$  which describes the behavior of  $pr$  have the same traces.

However, unlike FTSs, both BFMs and FESs share the same kind of compositionality that is also present in FMs, namely:

- the behavior of a feature is described by a BES over events uniquely associated to the feature;
- the behavior of a product can be obtained by combining the behavior of its features and the involved constraints, completely ignoring behavior from other features not included in the product;
- a sub-FM describes the behavior of all included features (and only of these features); and
- behavior can easily be reused between several BFMs by copying a sub-BFM to another BFM and including cross-tree feature constraints and inter-feature event constraints.

*Remark 3.5 (On the graphical representation of BFMs).* To improve readability, we can remove the graphical representation of the inter-feature event constraints from the BFM's graphical representation and display them in textual form using, e.g., the syntax introduced in Def. 2.31 for the

conflicts and causalities of a BES. In the literature, this is sometimes done similarly for cross-tree feature constraints in the graphical representation of FMs. They are provided with the FM as a set of propositional formulae over feature names (to be understood as a conjunction between them).

For instance, a graphical representation of the BFM  $\mathcal{B}_r$  (depicted in Fig. 5) with conflicts and causalities displayed in textual form is given in Fig. 7, where  $EC_{robot}$  is the set of the intra-feature event constraints for the feature *robot*, which only contains conflicts and causalities between events associated with different features.

Product constraints always reflect which feature an event is associated with. If a product constraint of an event contains more information, then this information can be displayed in textual form (as a pair of the event and the associated feature expression) below the graphical representation.

## 4 BEHAVIORAL FEATURE MODELS: A FORMAL ACCOUNT

This section provides a formal account of BFMs and of the isomorphic concept of Featured Event Structures (FESSs). BFMs extend FMs with behavior, thus integrating structural and behavioral models of SPLs in one unified model. This unified model organizes behavior according to the structuring concepts of FMs, resulting in reusable and compositional building blocks for SPL development and maintenance.

### 4.1 Behavioral Feature Models

We start with introducing our main modeling concept, namely, Behavioral Feature Models. Throughout this section, we will use gray backgrounds to highlight extensions of FMs (introduced in Sect. 2.1, Def. 2.3) with behavior.

*Definition 4.1 (Behavioral Feature Model (BFM)).* Let  $N$  be a set of feature names with  $f \in N$ , let  $E$  be a set of events with  $e \in E$ , and let  $A$  be a set of actions. A *Behavioral Feature Model (BFM)*  $\mathcal{B}$  over  $N$ ,  $E$ , and  $A$  has the following syntax:

$$\begin{aligned}
 \mathcal{B} &::= (f : \mathcal{S}, \{C_i\}_{i \in I^0}, FC, EC, PC) \\
 C &::= \text{Uop}(\mathcal{B}) \mid \text{Op}(\mathcal{B}_i)_{i \in I^2} & FC &::= \phi \\
 \text{Uop} &::= \text{opt} \mid \text{mnd} & EC &::= \emptyset \mid e \# e \mid X \mapsto e \mid EC \ EC \\
 \text{Op} &::= \text{or} \mid \text{xor} & PC &::= \emptyset \mid (e, FC) \ PC
 \end{aligned}$$

Here,  $f$  is the *root feature* of  $\mathcal{B}$ ;  $\mathcal{S} = (E_f, \#_f, \mapsto_f, A_f, l_f)$  is the BES associated with the feature  $f$ , containing the events  $E_f \subseteq E$ , the *intra-feature event constraints*  $\#_f \cup \mapsto_f$  that describe conflicts ( $\#_f$ ) and causalities ( $\mapsto_f$ ) between events in  $E_f$ , the actions  $A_f \subseteq A$ , and the labeling function  $l_f : E_f \rightarrow A_f$ ; the  $C_i, i \in I^0$ , are *sub-BFM groups*, each consisting of an indexed set of *sub-BFMs* of  $\mathcal{B}$ ; a sub-BFM group can be a unary operator (Uop) or an  $n$ -ary operator (Op) of BFMs, indicating the *kind* of the sub-BFM group; a unary operator can be of kind *optional* (**opt**) or of kind *mandatory* (**mnd**), while an  $n$ -ary operator is a choice between BFMs, either of kind *inclusive or* (**or**) or of kind *exclusive or* (**xor**); *cross-tree feature constraints*  $FC$  can express constraints among features in the form of a feature expression  $\phi \in \text{FExp}(N)$ ; *inter-feature event constraints*  $EC$  associated with the feature  $f$  describe conflicts ( $\#$ ) and causalities ( $\mapsto$ ) between either events of distinct sub-BFMs or between events of  $E_f$  and events of a sub-BFM, where  $\#$  is an irreflexive and symmetric relation over  $E \times E$ , and  $\mapsto$  is a relation over  $(P(E) \setminus \{\emptyset\}) \times E$ , satisfying

- $\forall (X, e) \in \mapsto \forall e_1, e_2 \in X. e_1 \neq e_2 \Rightarrow e_1 \# e_2$ , and
- $\forall (X, e), (Y, e) \in \mapsto. X \neq Y \Rightarrow (X \not\subseteq Y \wedge Y \not\subseteq X)$ ;

and *product (configuration) constraints*  $PC$  associated with the feature  $f$  map events  $e \in E_f$  to feature constraints  $FC$ , where the feature constraints can relate to all features in  $N$ .

In Def. 4.1, BFM s are defined compositionally by mimicking the structure of FMs (cf. Sect. 2.1). We assume that each event is associated with exactly one feature, so that different features have disjoint sets of events. Thus, the set of events  $E$  of a BFM is the disjoint union of the sets of events associated with its features, and the set of actions  $A$  of a BFM is the (often not disjoint) union of the sets of actions associated with its features.

As for FMs in Not. 2.5, we define some notations that facilitate referring to different parts of BFM s (in Not. 4.3). First, we define what we mean by the union of two mappings with empty domain intersection.

*Notation 4.2 (Union of mappings).* Let  $f : F \rightarrow I$  and  $g : G \rightarrow J$  be mappings such that  $F \cap G = \emptyset$ . Then  $f \cup g : F \cup G \rightarrow I \cup J$  is defined as  $f \cup g(x) = f(x)$  if  $x \in F$  and  $f \cup g(x) = g(x)$  if  $x \in G$ .

*Notation 4.3 (★ On BFM s).* We use  $\text{features}(\mathcal{B})$  to denote the set of the features of the BFM  $\mathcal{B}$ ,  $\text{root}(\mathcal{B})$  to denote its root feature and  $\text{subTree}_f(\mathcal{B})$  to denote its sub-BFM with root  $f$ . Furthermore, given a BFM  $\mathcal{B}$  and a feature  $f$  such that  $\text{subTree}_f(\mathcal{B}) = (f : \mathcal{S}, \{C_i\}_{i \in I^0}, \text{FC}, \text{EC}, \text{PC})$ , with  $\mathcal{S} = (E_f, \#_f, \mapsto_f, A_f, l_f)$  and  $e \in E_f$ , we use

- $\text{FC}_f(\mathcal{B})$  to denote the set of *cross-tree feature constraints* FC of  $f$ ;
- $\text{Uop}(f)$  and  $\text{Op}(f_1, \dots, f_m)$  (with  $m \geq 2$ ) to denote that  $\text{subTree}_f(\mathcal{B})$  and  $\text{subTree}_{f_j}(\mathcal{B})$ ,  $j \in \{1, \dots, m\}$ , are in sub-BFM groups of  $\mathcal{B}$  of kind Uop and Op, respectively;
- $\text{gfc}_f(\mathcal{B}) \doteq \{\text{Uop}(\text{root}(\mathcal{B}_j)) \mid \text{Uop}(\mathcal{B}_j) = C_i \text{ for some } i \in I^0\}$   
 $\cup \{\text{Op}(\text{root}(\mathcal{B}_{j_1}), \dots, \text{root}(\mathcal{B}_{j_{m_i}})) \mid \text{Op}(\mathcal{B}_{j_1}, \dots, \mathcal{B}_{j_{m_i}}) = C_i \text{ for some } i \in I^2\}$   
 to denote the set of *group feature constraints* of  $f$  which describe how the sub-BFM s  $\mathcal{B}_j$  of  $\text{subTree}_f(\mathcal{B})$  are grouped into sub-BFM groups.
- $\text{events}_f(\mathcal{B}) \doteq E_f$  to denote the set of *events* of  $f$ ;
- $\text{events}(\mathcal{B}) \doteq \bigcup_{f \in \text{features}(\mathcal{B})} \text{events}_f(\mathcal{B})$  to denote the set of *events* of  $\mathcal{B}$ ;
- $\text{actions}_f(\mathcal{B}) \doteq A_f$  to denote the set of *actions* of  $f$ ;
- $\text{actions}(\mathcal{B}) \doteq \bigcup_{f \in \text{features}(\mathcal{B})} \text{actions}_f(\mathcal{B})$  to denote the set of *actions* of  $\mathcal{B}$ ;
- $\text{labelFun}_f(\mathcal{B}) \doteq l_f$  to denote the *labeling function* of  $f$ ;
- $\text{labelFun}(\mathcal{B}) \doteq \bigcup_{f \in \text{features}(\mathcal{B})} l_f$  to denote the *labeling function* of  $\mathcal{B}$ ;
- $\text{intraFtEc}_f(\mathcal{B}) \doteq \#_f \cup \mapsto_f$  to denote the set of *intra-feature event constraints* of  $f$ ;
- $\text{interFtEc}_f(\mathcal{B}) \doteq \text{EC}$  to denote the set of *inter-feature event constraints* of  $f$ ;
- $\text{confl}(\text{EC}) \doteq \{(e_1, e_2) \mid e_1 \# e_2 \in \text{EC}\}$  to denote the set of *inter-feature conflicts*;
- $\text{cause}(\text{EC}) \doteq \{(X, e) \mid X \mapsto e \in \text{EC}\}$  to denote the set of *inter-feature causalities*;
- $\text{ec}_f(\mathcal{B}) \doteq \text{intraFtEc}_f(\mathcal{B}) \cup \text{interFtEc}_f(\mathcal{B})$  to denote the set of *event constraints* of  $f$ ;
- $\text{behFt}_f(\mathcal{B}) \doteq (f, \mathcal{S})$  to denote the *behavioral feature* for  $f$ ;
- $\text{productC}_f(\mathcal{B})(e) \doteq \phi_e$ , where  $(e, \phi_e) \in \text{PC}$ , to denote the *product constraints* of  $f$  for  $e$ ; and
- $\text{productC}(\mathcal{B}) \doteq \bigcup_{f \in \text{features}(\mathcal{B})} \text{productC}_f(\mathcal{B})$  to denote the set of *product constraints* of  $\mathcal{B}$ .

*Remark 4.4 (Key assumptions on BFM s).* For every BFM  $\mathcal{B}$  and every feature  $f \in \text{features}(\mathcal{B})$  we assume the following:

- Inter-feature event constraints are situated in the smallest sub-BFM that contains the mentioned events. This assumption is in the same style as the second assumption on FMs in Rem. 2.4; we assume that information is situated as far down in the (B)FM as possible to facilitate composition, decomposition and reusability. In BFM s this means that each conflict and causality in  $\text{interFtEc}_f(\mathcal{B})$  involves only events of  $\text{subTree}_f(\mathcal{B})$ , and it either involves an event of  $f$  and (at least) one event of a feature in an immediate sub-BFM of  $f$ ; or it involves events in (at least) two immediate sub-BFM s of  $f$ . That is, conflict and causality in  $\text{interFtEc}_f(\mathcal{B})$  involve events from at least two of the following three disjoint sets: (i) events

of  $f$ , (ii) events of a feature in an immediate sub-BFM of  $f$ , and (iii) events of a feature in another immediate sub-BFM of  $f$ .

- Given the set  $\bigcup_{f \in \text{features}(\mathcal{B})} \text{ec}_f(\mathcal{B})$  of event constraints of  $\mathcal{B}$ , the included conflicts form a conflict relation over  $\text{events}_f(\mathcal{B})$  and the included causalities form a causality relation over  $\text{events}_f(\mathcal{B})$  (see Def. 2.31). This assumption is needed later to show that BFMs and the concept of featured event structures, introduced in Def. 4.8, are isomorphic.
- For every event associated with  $f$ , i.e., for every  $e \in \text{events}_f(\mathcal{B})$ , the product constraints of  $f$  for  $e$ , i.e.,  $\text{productC}_f(\mathcal{B})(e)$ , imply  $f$ . In particular, if a product satisfies  $\text{productC}_f(\mathcal{B})(e)$ , then  $f$  is included in the product.

The *used actions* of a BFM are defined through the used actions of the BESs associated with its features (see Def. 2.33).

**Definition 4.5** ( $\star$  *Used actions of a BFM*). Let  $\mathcal{B}$  be a BFM. The *used actions* of  $\mathcal{B}$ , denoted  $\text{usedAct}(\mathcal{B})$ , are defined as  $\bigcup_{f \in \text{features}(\mathcal{B})} \{\text{usedAct}(\mathcal{S}_f) \mid (f, \mathcal{S}_f) = \text{behFt}_f(\mathcal{B})\}$ .

**Example 4.6** (BFM for the cleaning robot SPL). Consider the example of the cleaning robot SPL over the set of features  $N_r$  from Example 2.6. Then the BFM of the cleaning robot SPL over  $N_r$ , the set of events  $E'_r = \{\text{charge}, \text{map}, \text{goAround}, \text{move}_1, \text{move}_2, \text{clean}, \text{liDet}, \text{caDet}\}$  and the set of actions  $A'_r = \{\text{charge}, \text{map}, \text{goAround}, \text{move}, \text{clean}, \text{liDet}, \text{caDet}\}$ , as depicted in Fig. 5, is defined as follows:

$$\begin{aligned}
 \mathcal{B}_r &\doteq (r : (E_r \doteq \{\text{charge}\}, \emptyset, \emptyset, A_r \doteq E_r, \text{id}_{E_r}), \{\text{opt}(\mathcal{B}_m), \text{mnd}(\mathcal{B}_c), \text{mnd}(\mathcal{B}_o)\}, \#, \text{EC}_r, \{(\text{charge}, r)\}) \\
 \mathcal{B}_m &\doteq (m : (E_m \doteq \{\text{map}\}, \emptyset, \emptyset, A_m \doteq E_m, \text{id}_{E_m}), \emptyset, \#, \emptyset, \{(\text{map}, m)\}) \\
 \mathcal{B}_c &\doteq (c : (E_c, \#_c, \mapsto_c, A_c, l_c), \emptyset, \#, \emptyset, \text{PC}_c) \\
 \mathcal{B}_o &\doteq (o : (E_o \doteq \emptyset, \emptyset, \emptyset, A_o \doteq E_o, \text{id}_{E_o}), \{\text{xor}(\mathcal{B}_{li}, \mathcal{B}_{ca})\}, \#, \emptyset, \emptyset) \\
 \mathcal{B}_{li} &\doteq (li : (E_{li} \doteq \{\text{liDet}\}, \emptyset, \emptyset, A_{li} \doteq E_{li}, \text{id}_{E_{li}}), \emptyset, \#, \emptyset, \{(\text{liDet}, li)\}) \\
 \mathcal{B}_{ca} &\doteq (ca : (E_{ca} \doteq \{\text{caDet}\}, \emptyset, \emptyset, A_{ca} \doteq E_{ca}, \text{id}_{E_{ca}}), \emptyset, \#, \emptyset, \{(\text{caDet}, ca)\}) \\
 \text{EC}_r &\doteq \{\{\text{map}\} \mapsto \text{move}_1, \{\text{map}\} \mapsto \text{liDet}, \{\text{map}\} \mapsto \text{caDet}, \{\text{liDet}\} \mapsto \text{goAround}, \{\text{caDet}\} \mapsto \text{goAround}, \\
 &\quad \{\text{clean}\} \mapsto \text{charge}, \text{liDet} \# \text{move}_1, \text{caDet} \# \text{move}_1\} \\
 E_c &\doteq \{\text{goAround}, \text{move}_1, \text{move}_2, \text{clean}\} \\
 A_c &\doteq \{\text{goAround}, \text{move}, \text{clean}\} \\
 l_c(e) &\doteq \begin{cases} e & \text{if } e \in \{\text{goAround}, \text{clean}\} \\ \text{move} & \text{if } e \in \{\text{move}_1, \text{move}_2\} \end{cases} \\
 \#_c &\doteq \{(\text{goAround}, \text{move}_1)\} \\
 \mapsto_c &\doteq \{(\{\text{goAround}, \text{move}_1\}, \text{move}_2), (\{\text{move}_2\}, \text{clean})\} \\
 \text{PC}_c &\doteq \{(\text{goAround}, c), (\text{move}_1, c), (\text{move}_2, c), (\text{clean}, c)\}
 \end{aligned}$$

where  $\text{id}_E : E \rightarrow E$  is the identity mapping and  $l_c : E_c \rightarrow A_c$ . The used actions of  $\mathcal{B}_r$  are all actions in  $A'_r$ .

**Notation 4.7** (On Example 4.6). In Example 4.6,  $E'_r$  denotes the set of all events of the sub-BFM with root *robot*. It is primed, i.e. denoted by  $E'_r$ , to distinguish it from the set of events  $E_r$  associated with the BES of the feature *robot*. We will use primes throughout the paper to mark these differences.

## 4.2 Featured Event Structures: an Isomorphic Representation of BFMs

In this section, we first (in Sect. 4.2.1) define Featured Event Structures (FESs) which provide an isomorphic representation of BFMs that has a similar structure as FTSSs in the following sense: it consists of an FM, a behavioral model and product knowledge (a mapping annotating the behavioral model with feature expressions) to define behavior of products, while it additionally includes event

location knowledge (a mapping associating behavior to features). Then (in Sect. 4.2.2) we define four mappings that, given a BFM  $\mathcal{B}$ , extract the FM, the BES, the product knowledge, and the event location knowledge of  $\mathcal{B}$ , respectively. Finally (in Sect. 4.2.3), we define an isomorphism that relates BFMs and FESs which contain the same information.

#### 4.2.1 Featured Event Structures.

*Definition 4.8 (Featured event structure (FES)).* A featured event structure (FES) over the features  $N$ , the events  $E$ , and the actions  $A$  is a quadruple  $\mathcal{E} = (\mathcal{M}, \mathcal{S}, \lambda, \nu)$ , where

- $\mathcal{M}$  is a FM over  $N$ , called *the FM of  $\mathcal{E}$* ,
- $\mathcal{S} = (E, \#, \mapsto, A, I)$  is a BES, called *the BES of  $\mathcal{E}$* ,
- $\lambda : E \rightarrow N$  is a mapping that associates each event of  $\mathcal{S}$  with a feature in  $N$ , which we call *event location knowledge of  $\mathcal{E}$* ,
- $\nu : E \rightarrow \text{FExp}(N)$  is a mapping that associates each event of  $\mathcal{E}$  with a feature expression over  $N$ , which we call the *product (configuration) knowledge of  $\mathcal{E}$* .

Without loss of generality, we assume that for every  $e \in E$  it holds  $\nu(e) \Rightarrow \lambda(e)$ .

The mapping  $\lambda$  shows the positioning of an event in a feature model while the mapping  $\nu$  expresses which products of the FM can include an event in their behavior. Assuming that for every event its product knowledge implies its event location knowledge reflects that an event can only occur in the behavior of a product if that product includes the feature the event is associated with. The triple  $(\mathcal{S}, \lambda, \nu)$  is called *the annotated BES of  $\mathcal{E}$* .

The *used actions* of an FES are defined through the used actions of its BES (see Def. 2.33).

*Definition 4.9 (★ Used actions of an FES).* Let  $\mathcal{E} = (\mathcal{M}, \mathcal{S}, \lambda, \nu)$  be an FES. The *used actions* of  $\mathcal{E}$ , denoted  $\text{usedAct}(\mathcal{E})$ , are defined as  $\text{usedAct}(\mathcal{E}) \doteq \text{usedAct}(\mathcal{S})$ .

*Example 4.10 (FES of the cleaning robot SPL).* Consider the example of the cleaning robot SPL. We can construct an FES  $\mathcal{E}'_r = (\mathcal{M}_r, \mathcal{S}'_r, \lambda'_r, \nu_r)$  over the features  $N_r$  (as defined in Example 2.6), the events  $E'_r$  (as defined in Example 4.6), and the actions  $A'_r$  (as defined in Example 4.6) for it, where the FM  $\mathcal{M}_r$  is as displayed in Fig. 1 and the annotated BES  $(\mathcal{S}'_r, \lambda'_r)$  is as displayed in Fig. 6. Formally, the FES of the cleaning robot is defined as:

- $\mathcal{M}_r$  as in Example 2.6;
- $\mathcal{S}'_r \doteq (E'_r, \#'_r, \mapsto'_r, A'_r, I'_r)$ , with  $\#'_r \doteq \#_c \cup \text{confl}(\text{EC}_r)$ ,  $\mapsto'_r \doteq \mapsto_c \cup \text{cause}(\text{EC}_r)$ , the sets  $E'_r$  and  $A'_r$  as defined in Example 4.6, and  $I'_r : E'_r \rightarrow A'_r$  with

$$I'_r(e) \doteq \begin{cases} e & \text{if } e \in \{\text{charge}, \text{map}, \text{goAround}, \text{clean}, \text{liDet}, \text{caDet}\}, \\ \text{move} & \text{if } e \in \{\text{move}_1, \text{move}_2\}; \end{cases}$$

- $\lambda'_r : E'_r \rightarrow N_r$ , with

$$\lambda'_r(e) \doteq \begin{cases} \text{robot} & \text{if } e \in E_r, \\ \text{mapping} & \text{if } e \in E_m, \\ \text{cleaning} & \text{if } e \in E_c, \\ \text{lidar} & \text{if } e \in E_{li}, \\ \text{camera} & \text{if } e \in E_{ca}; \text{ and} \end{cases}$$

- $\nu_r : E'_r \rightarrow \text{FExp}(N_r)$ , with  $\nu(e) \doteq \lambda(e)$  for all  $e \in E'_r$ ;

where  $\#_c, \mapsto_c$  and  $\text{EC}_r$  are as defined in Example 4.6. The used actions of  $\mathcal{E}_r$  are all actions in  $A'_r$ .



As for FMs and BFMs in Notations 2.5 and 4.3, respectively, we define some notations that will be helpful for working with FESs.

*Notation 4.11 (★ On FESs).* We use  $\text{features}(\mathcal{E})$ ,  $\text{events}(\mathcal{E})$ ,  $\text{actions}(\mathcal{E})$ ,  $\text{fm}(\mathcal{E})$ ,  $\text{bes}(\mathcal{E})$ ,  $\text{evLocKnow}(\mathcal{E})$  and  $\text{prodKnow}(\mathcal{E})$  to denote the set of features, the set of events, the set of actions, the FM, the BES, the event location knowledge, and the product knowledge of the FES  $\mathcal{E}$ , respectively. Moreover, given an FES  $\mathcal{E} = (\mathcal{M}, \mathcal{S}, \lambda, \nu)$  with  $\mathcal{S} = (E, \#, \mapsto, A, l)$  and one of its features  $f$ , we use

- $\text{FC}_f(\mathcal{E}) \doteq \text{FC}_f(\mathcal{M})$  to denote the set of *cross-tree feature constraints* of  $f$ ;
- $\text{gfc}_f(\mathcal{E}) \doteq \text{gfc}_f(\mathcal{M})$  to denote the set of *group-feature constraints* of  $f$ ;
- $\text{events}_f(\mathcal{E}) \doteq \{e \in E \mid \lambda(e) = f\}$  to denote the set of *events* of  $f$ ;
- $\text{actions}_f(\mathcal{E}) \doteq \{a \in A \mid \exists e \in \text{events}_f(\mathcal{E}). l(e) = a\}$  to denote the *actions* of  $f$ ;
- $\text{labelFun}(\mathcal{E}) \doteq l$  to denote the *labeling function* of  $\mathcal{E}$ ;
- $\text{labelFun}_f(\mathcal{E}) : \text{events}_f(\mathcal{E}) \rightarrow \text{actions}_f(\mathcal{E})$  where  $\text{labelFun}_f(\mathcal{E})(e) = \text{labelFun}(\mathcal{E})(e)$  for  $e \in \text{events}_f(\mathcal{E})$  to denote the *labeling function* of  $f$ ;
- $\text{ec}_f(\mathcal{E}) \doteq \{e_1 \# e_2 \mid e_1, e_2 \in E'\} \cup \{X \mapsto e \mid X \cup \{e\} \subseteq E'\}$ , where  $E' \doteq \{e \mid \lambda(e) \in \text{features}(\text{subTree}_f(\mathcal{M}))\}$  to denote the set of *event constraints* of  $f$ ;
- $\text{intraFtEc}_f(\mathcal{E}) \doteq \{e_1 \# e_2 \mid e_1, e_2 \in \text{events}_f(\mathcal{E})\} \cup \{X \mapsto e \mid X \cup \{e\} \subseteq \text{events}_f(\mathcal{E})\}$  to denote the set of *intra-feature event constraints* of  $f$ ;
- $\text{interFtEc}_f(\mathcal{E}) \doteq \text{ec}_f(\mathcal{E}) \setminus \text{intraFtEc}_f(\mathcal{E})$  to denote the set of *inter-feature event constraints* of  $f$ ;
- $\text{behFt}_f(\mathcal{E}) \doteq (f : (\text{events}_f(\mathcal{E}), \text{confl}(\text{intraFtEc}_f(\mathcal{E})), \text{cause}(\text{intraFtEc}_f(\mathcal{E})), \text{actions}_f(\mathcal{E}), \text{labelFun}_f(\mathcal{E})))$  to denote the *behavioral feature* for  $f$ ; and
- $\text{productC}_f(\mathcal{E}) \doteq \{(e, \nu(e)) \mid e \in \text{events}_f(\mathcal{E})\}$  to denote the *product constraints* of  $f$ .

To define the notion of a sub-FES with root  $f$  of an FES  $\mathcal{E}$  (in Def. 4.15), denoted by  $\text{subTree}_f(\mathcal{E})$ , we first recall a mathematical notation (in Not. 4.12), and then define how to restrict a conflict relation, a causality relation and a BES to a smaller set of events (in Def. 4.13).

*Notation 4.12 (Restriction of a mapping to a subdomain).* The restriction of a mapping  $\text{map} : \text{Domain} \rightarrow \text{Range}$  to the set  $\text{Subdomain} \subseteq \text{Domain}$ , denoted by  $\text{map}|_{\text{Subdomain}} : \text{Subdomain} \rightarrow \text{Range}$ , is defined as  $\text{map}|_{\text{Subdomain}}(e) = \text{map}(e)$  for  $e \in \text{Subdomain}$ .

*Definition 4.13 (Conflict, causality and BES event and action restriction).* Let  $E$  and  $E'$  be sets of events such that  $E' \subseteq E$ .

- (1) Let  $\# \subseteq E \times E$  be a conflict relation. The *restriction* of  $\#$  to  $E'$  is the conflict relation  $\rho_{E'}(\#) \doteq \# \cap (E' \times E')$ .
- (2) Let  $\mapsto \subseteq \mathcal{P}(E) \times E$  be a causality relation. The *restriction* of  $\mapsto$  to  $E'$  is the causality relation  $\rho_{E'}(\mapsto) \subseteq \mathcal{P}(E') \times E'$  such that

$$\rho_{E'}(\mapsto) \doteq \{(X \cap E', e) \mid (X \mapsto e) \wedge (e \in E') \wedge (X \cap E' \neq \emptyset) \wedge (\neg \exists Y \subseteq E. (Y \mapsto e \wedge (X \cap E') \subset (Y \cap E')))\}.$$

- (3) Let  $\mathcal{S} = (E, \#, \mapsto, A, l)$  be a BES. The *event restriction* of  $\mathcal{S}$  to  $E'$  is the BES  $\rho_{E'}(\mathcal{S}) \doteq (E', \rho_{E'}(\#), \rho_{E'}(\mapsto), A, l|_{E'})$ .

Since the definition of the restriction of a causality relation is not as straightforward as the one of a conflict relation, we explain the rationale in the following remark.

*Remark 4.14 (On Def. 4.13.(2)).* The condition

$$\neg \exists Y \subseteq E. (Y \mapsto e \wedge (X \cap E') \subset (Y \cap E')) \quad (*)$$

in the definition of  $\rho_{E'}(\mapsto)$  in Def. 4.13.(2) is needed to ensure that Point (2) of Def. 2.31 still holds for the resulting causality relation  $\mapsto' \doteq \rho_{E'}(\mapsto)$ : there could be causalities  $X \mapsto e$  and  $Y \mapsto e$ , with  $X \not\subseteq Y$  and  $X \cap E' \subset Y \cap E'$ , that would produce causalities  $(X \cap E') \mapsto' e$  and  $(Y \cap E') \mapsto' e$ . According to Point (2) of Def. 2.31, the causality  $(X \cap E') \mapsto' e$  must be excluded from  $\mapsto'$  for  $\mapsto'$  to be a causality relation. Therefore, Condition (\*) is needed in Def. 4.13.(2).

As an example, consider the BES  $\mathcal{S} = (E, \#, \mapsto, A, l)$  over the events  $E = \{e, e_1, e_2, e_3\}$  with  $\# = \{(e_1, e_2), (e_2, e_3)\}$  and  $\mapsto = \{(\{e_1, e_2\}, e), (\{e_2, e_3\}, e)\}$ . Furthermore, let  $X = \{e_2, e_3\}$ ,  $Y = \{e_1, e_2\}$  and  $E' = \{e, e_1, e_2\}$ . By using the definition of  $\rho_{E'}(\mapsto)$  without Condition (\*), we would get  $\rho_{E'}(\mapsto) = \{(\{e_1, e_2\}, e), (\{e_2\}, e)\}$ , which violates the condition in Point (2) of Def. 2.31. Condition (\*) ensures that  $\rho_{E'}(\mapsto) = \{(\{e_1, e_2\}, e)\}$  and, therefore, that  $\rho_{E'}(\mapsto)$  is a causality relation.

A sub-FES with root  $f$  of a FES  $\mathcal{E}$  is defined as the restriction of  $\mathcal{E}$  on the set of events associated with features in the feature model with root  $f$ .

*Definition 4.15 (Sub-FESs).* Given an FES  $\mathcal{E} = (\mathcal{M}, \mathcal{S}, \lambda, \nu)$  and a feature  $f \in \text{features}(\mathcal{M})$ , the sub-FES of  $\mathcal{E}$  with root  $f$  is

$$\text{subTree}_f(\mathcal{E}) \doteq (\text{subTree}_f(\mathcal{M}), \rho_{E'}(\mathcal{S}), \lambda|_{E'}, \nu|_{E'}),$$

where  $E' \doteq \{e \in E \mid \lambda(e) \in \text{features}(\text{subTree}_f(\mathcal{M}))\}$ .

*Example 4.16 (Sub-FES with root cleaning of the FES for the cleaning robot SPL).* Consider the FES  $\mathcal{E}'_r = (\mathcal{M}_r, \mathcal{S}'_r, \lambda'_r, \nu_r)$  of the cleaning robot SPL over the features  $N_r$ , the events  $E'_r$ , and the actions  $A'_r$ , as defined in Example 4.10, and the feature *cleaning*  $\in \text{features}(\mathcal{M}_r)$ . Then the sub-FES of  $\mathcal{E}'_r$  with root *cleaning* is  $\text{subTree}_c(\mathcal{E}'_r) = (\mathcal{M}_c, \mathcal{S}_c, \lambda_c, \nu_c)$ , in which

- $\mathcal{M}_c$  is as in Example 2.6;
- $\mathcal{S}_c \doteq (E_c, \#_c, \mapsto_c, A'_c, l_c)$ , where  $E_c, \#_c, \mapsto_c$ , and  $l_c$  are as in Example 4.6;
- $\lambda_c : E_c \rightarrow N_r$  maps all events in  $E_c$  to the feature *cleaning*; and
- $\nu_c : E_c \rightarrow \text{FExp}(N_r)$  maps all events in  $E_c$  to *cleaning*.

#### 4.2.2 FM, BES, Event Location Knowledge and Product Knowledge of a BFM.

To define the isomorphism between BFM and FESs, we first define the four extraction mappings that we will use in the definition. A BFM is an FM in which behavior is associated with the features. Thus, the FM of a BFM can be extracted by removing the behavioral part of the BFM.

*Definition 4.17 (FM of a BFM).* The FM of a BFM  $\mathcal{B}$ , written  $\text{fm}(\mathcal{B})$ , is recursively defined as follows (where, to improve readability, the behavioral parts are highlighted in gray).

$$\begin{aligned} \text{fm}((f : \mathcal{S}, \{C_i\}_{i \in I^0}, \text{FC}, \text{EC}, \text{PC})) &\doteq (f, \{\text{fm}(C_i)\}_{i \in I^0}, \text{FC}) \\ \text{fm}(\text{Uop}(\mathcal{B})) &\doteq \text{Uop}(\text{fm}(\mathcal{B})) \\ \text{fm}(\text{Op}(\mathcal{B}_i)_{i \in I^2}) &\doteq \text{Op}(\text{fm}(\mathcal{B}_i))_{i \in I^2} \end{aligned}$$

*Example 4.18 (FM of the BFM of the cleaning robot SPL).* Let  $\mathcal{B}_r$  be the BFM of the cleaning robot SPL as defined in Example 4.6 (and depicted in Fig. 5). Then the FM of  $\mathcal{B}_r$  is extracted as follows:

$$\begin{aligned} \text{fm}(\mathcal{B}_r) &= \mathcal{M}_r = (r, \{\text{opt}(\text{fm}(\mathcal{B}_m)), \text{mnd}(\text{fm}(\mathcal{B}_c)), \text{mnd}(\text{fm}(\mathcal{B}_o))\}, \#) \\ \text{fm}(\mathcal{B}_m) &= \mathcal{M}_m = (m, \emptyset, \#) \\ \text{fm}(\mathcal{B}_c) &= \mathcal{M}_c = (c, \emptyset, \#) \\ \text{fm}(\mathcal{B}_o) &= \mathcal{M}_o = (o, \{\text{xor}(\text{fm}(\mathcal{B}_{li}), \text{fm}(\mathcal{B}_{ca}))\}, \#) \\ \text{fm}(\mathcal{B}_{li}) &= \mathcal{M}_{li} = (li, \emptyset, \#) \\ \text{fm}(\mathcal{B}_{ca}) &= \mathcal{M}_{ca} = (ca, \emptyset, \#) \end{aligned}$$

where the FMs  $\mathcal{M}_f$ , for  $f \in \{r, m, c, o, li, ca\}$ , are the FMs obtained in Example 2.6. In particular,  $\mathcal{M}_r$  (which is the FM of  $\mathcal{B}_r$ ) is depicted in Fig. 1.

The following proposition states that, for every BFM  $\mathcal{B}$ , the underlying FM of the sub-BFM with root  $f$  of  $\mathcal{B}$  is the same as the sub-FM with root  $f$  of the underlying FM of  $\mathcal{B}$ .

**PROPOSITION 4.19 (ON FMS OF SUB-BFMS).** *Let  $\mathcal{B}$  be a BFM and  $f \in \text{features}(\mathcal{B})$ . Then it holds that  $\text{fm}(\text{subTree}_f(\mathcal{B})) = \text{subTree}_f(\text{fm}(\mathcal{B}))$ .*

**PROOF.** Straightforward using the definition of the sub-BFM with root  $f$  (Def. 4.1) and the definition of an FM of a BFM (Def. 4.17).  $\square$

The BES of a BFM  $\mathcal{B}$  can be obtained by collecting all events associated with the features of  $\mathcal{B}$  as well as all conflicts and causalities associated with the features of  $\mathcal{B}$  and all inter-feature conflicts and causalities.

**Definition 4.20 (BES of a BFM).** The BES of a BFM  $\mathcal{B}$ , denoted by  $\text{bes}(\mathcal{B})$ , is defined inductively as follows:

$$\begin{aligned}
 \text{bes}((f : (E_f, \#_f, \mapsto_f, A_f, l_f), \emptyset, \text{FC}, \text{EC}, \text{PC})) &\doteq (E_f, \#_f \cup \text{confl}(\text{EC}), \mapsto_f \cup \text{cause}(\text{EC}), A_f, l_f) \\
 \text{bes}((f : (E_f, \#_f, \mapsto_f, A_f, l_f), \{\text{Uop}(\mathcal{B}_1)\}, \text{FC}, \text{EC}, \text{PC})) \\
 &\doteq (E_1 \cup E_f, \#_1 \cup \#_f \cup \text{confl}(\text{EC}), \mapsto_1 \cup \mapsto_f \cup \text{cause}(\text{EC}), A_1 \cup A_f, l_1 \cup l_f), \\
 &\quad \text{where } (E_1, \#_1, \mapsto_1, A_1, l_1) = \text{bes}(\mathcal{B}_1) \\
 \text{bes}((f : (E_f, \#_f, \mapsto_f, A_f, l_f), \{\text{Op}(\mathcal{B}_i)_{i \in I^2}\}, \text{FC}, \text{EC}, \text{PC})) \\
 &\doteq (\bigcup_{i \in I^2} E_i \cup E_f, \bigcup_{i \in I^2} \#_i \cup \#_f \cup \text{confl}(\text{EC}), \bigcup_{i \in I^2} \mapsto_i \cup \mapsto_f \cup \text{cause}(\text{EC}), \bigcup_{i \in I^2} A_i \cup A_f, \bigcup_{i \in I^2} l_i \cup l_f), \\
 &\quad \text{where } (E_i, \#_i, \mapsto_i, A_i, l_i) = \text{bes}(\mathcal{B}_i) \\
 \text{bes}((f : (E_f, \#_f, \mapsto_f, A_f, l_f), \{\text{C}_i\}_{i \in I^2}, \text{FC}, \text{EC}, \text{PC})) \\
 &\doteq (\bigcup_{i \in I^0} E_i \cup E_f, \bigcup_{i \in I^0} \#_i \cup \#_f \cup \text{confl}(\text{EC}), \bigcup_{i \in I^0} \mapsto_i \cup \mapsto_f \cup \text{cause}(\text{EC}), \bigcup_{i \in I^0} A_i \cup A_f, \bigcup_{i \in I^0} l_i \cup l_f), \\
 &\quad \text{where } (E_i, \#_i, \mapsto_i, A_i, l_i) = \text{bes}((f : \emptyset, \{\text{C}_i\}, \emptyset, \emptyset))
 \end{aligned}$$

**Example 4.21 (BES of the BFM of the cleaning robot SPL).** Let  $\mathcal{B}_r$  be the BFM of the cleaning robot SPL as defined in Example 4.6 (and depicted in Fig. 5). Then the BES of  $\mathcal{B}_r$  is extracted as follows:

$$\begin{aligned}
 S'_r &\doteq \text{bes}(\mathcal{B}_r) = (E'_r \doteq \{\text{charge}\} \cup E_m \cup E_c \cup E'_o, \\
 &\quad \#'_r \doteq \{(\text{liDet}, \text{move}_1), (\text{caDet}, \text{move}_1)\} \cup \#_c, \\
 &\quad \mapsto'_r \doteq \{(\{\text{map}\}, \text{move}_1), (\{\text{map}\}, E'_o), (\{\text{liDet}\}, \text{goAround}), \\
 &\quad \quad (\{\text{caDet}\}, \text{goAround}), (\{\text{clean}\}, \text{charge})\} \cup \mapsto_c, \\
 &\quad A'_r \doteq \{\text{charge}\} \cup A_m \cup A_c \cup A'_o, \\
 &\quad l'_r \doteq \text{id}_{E_r} \cup l_m \cup l_c \cup l'_o) \\
 S_m &\doteq \text{bes}(\mathcal{B}_m) = (E_m \doteq \{\text{map}\}, \emptyset, \emptyset, A_m \doteq E_m, l_m \doteq \text{id}_{E_m}) \\
 S_c &\doteq \text{bes}(\mathcal{B}_c) = (E_c \doteq \{\text{goAround}, \text{move}_1, \text{move}_2, \text{clean}\}, \\
 &\quad \#_c \doteq \{(\text{goAround}, \text{move}_1)\}, \\
 &\quad \mapsto_c \doteq \{(\{\text{goAround}, \text{move}_1\}, \text{move}_2), (\{\text{move}_2\}, \text{clean})\}, \\
 &\quad A_c \doteq \{\text{goAround}, \text{move}, \text{clean}\}, \\
 &\quad l_c(e) \doteq \begin{cases} e & \text{if } e \in \{\text{goAround}, \text{clean}\}, \\ \text{move} & \text{if } e \in \{\text{move}_1, \text{move}_2\} \end{cases} \\
 S'_o &\doteq \text{bes}(\mathcal{B}_o) = (E'_o \doteq E_{li} \cup E_{ca}, \emptyset, \emptyset, A'_o \doteq A_{li} \cup A_{ca}, l'_o \doteq l_{li} \cup l_{ca}) \\
 S_{li} &\doteq \text{bes}(\mathcal{B}_{li}) = (E_{li} \doteq \{\text{liDet}\}, \emptyset, \emptyset, A_{li} \doteq \{\text{liDet}\}, l_{li} \doteq \text{id}_{E_{li}}) \\
 S_{ca} &\doteq \text{bes}(\mathcal{B}_{ca}) = (E_{ca} \doteq \{\text{caDet}\}, \emptyset, \emptyset, A_{ca} \doteq \{\text{caDet}\}, l_{ca} \doteq \text{id}_{E_{ca}})
 \end{aligned}$$

A graphical representation of  $\text{bes}(\mathcal{B}_r)$  can be found in Fig. 6 by ignoring the feature annotations highlighted in gray (which represent event location knowledge). Note that the BES  $\text{bes}(\mathcal{B}_r)$  of the

BFM  $\mathcal{B}_r$  of the cleaning robot SPL is equal to the BES  $\mathcal{S}'_r$  of the FES  $\mathcal{E}'_r$  of the cleaning robot SPL in Example 4.10.

The first behavioral mapping we need is the one extracting event location knowledge of a BFM, which maps an event to the feature it is associated with in the BFM.

*Definition 4.22 (Event location knowledge of a BFM).* Let  $N$  be a set of features and  $E$  be a set of events. *Event location knowledge of a BFM  $\mathcal{B}$*  over features  $N$ , events  $E$  and actions  $A$  is a mapping  $\text{evLocKnow}(\mathcal{B}) : E \rightarrow N$  defined by  $\text{evLocKnow}(\mathcal{B})(e) = f$ , where  $\text{behFt}_f(\mathcal{B}) = (f : (E_f, \#_f, \mapsto_f, A_f, l_f))$  with  $e \in E_f$ .

*Example 4.23 (Event location knowledge of the BFM of the cleaning robot SPL).* Let  $\mathcal{B}_r$  be the BFM of the cleaning robot SPL over  $N_r$ ,  $E'_r$  and  $A'_r$  as defined in Example 4.6 (and depicted in Fig. 5). Then the event location knowledge of  $\mathcal{B}_r$  is the mapping  $\text{evLocKnow}(\mathcal{B}_r) : E'_r \rightarrow N_r$  defined by:

$$\text{evLocKnow}(\mathcal{B}_r)(e) = \begin{cases} \text{mapping} & \text{if } e = \text{map}, \\ \text{cleaning} & \text{if } e \in \{\text{goAround}, \text{clean}, \text{move}_1, \text{move}_2\}, \\ \text{lidar} & \text{if } e = \text{liDet}, \\ \text{camera} & \text{if } e = \text{caDet}, \\ \text{robot} & \text{if } e = \text{charge}. \end{cases}$$

Note that the event location knowledge  $\text{evLocKnow}(\mathcal{B}_r)$  of the BFM  $\mathcal{B}_r$  of the cleaning robot SPL is equal to the event location knowledge  $\lambda'_r$  of the FES  $\mathcal{E}'_r$  of the cleaning robot SPL in Example 4.10.

The second behavioral mapping is the one extracting product (configuration) knowledge from a BFM, representing the feature restrictions of events.

*Definition 4.24 (Product (configuration) knowledge of a BFM).* Let  $N$  be a set of features and  $E$  be a set of events. *Product (configuration) knowledge of a BFM  $\mathcal{B}$*  over features  $N$ , events  $E$ , and actions  $A$ , denoted by  $\text{prodKnow}(\mathcal{B})$ , is a mapping  $\text{prodKnow}(\mathcal{B}) : E \rightarrow \text{FExp}(N)$  defined by  $\text{prodKnow}(\mathcal{B})(e) = \phi_e$  where  $(e, \phi_e) \in \text{productC}(\mathcal{B})$ .

*Example 4.25 (Product knowledge of the BFM of the cleaning robot SPL).* Consider the BFM  $\mathcal{B}_r$  of the cleaning robot SPL over  $N_r$ ,  $E'_r$  and  $A'_r$  as in Example 4.6. Then  $\text{prodKnow}(\mathcal{B}_r) : E'_r \rightarrow \text{FExp}(N_r)$  is exactly the same mapping as  $\text{evLocKnow}(\mathcal{B}_r)$ .

#### 4.2.3 FES View of a BFM and BFM View of an FES.

Now we have all the ingredients in place to define the mapping between BFMs and FESs that maps a BFM to the tuple containing its extracted FM, its extracted BES, its extracted event location knowledge and its extracted product knowledge.

*Definition 4.26 (FES view of a BFM).* Let  $\mathbb{B}$  and  $\mathbb{E}$  be the sets of BFMs and FESs, respectively, over a set of features  $N$ , a set of events  $E$ , and a set of actions  $A$ . The mapping

$$\text{fes} : \mathbb{B} \rightarrow \mathbb{E}, \mathcal{B} \mapsto (\text{fm}(\mathcal{B}), \text{bes}(\mathcal{B}), \text{evLocKnow}(\mathcal{B}), \text{prodKnow}(\mathcal{B}))$$

takes a BFM  $\mathcal{B}$  over  $N$ ,  $E$ , and  $A$  and returns the *FES view of  $\mathcal{B}$* , which is the FES over  $N$ ,  $E$ , and  $A$  consisting of the extracted FM, BES, event location knowledge and product knowledge of  $\mathcal{B}$ .

*Example 4.27 (FES view of the BFM of the cleaning robot SPL).* The FES view of the BFM  $\mathcal{B}_r$  of the cleaning robot SPL as defined in Example 4.6 (and depicted in Fig. 5) is the FES  $\mathcal{E}'_r = (\mathcal{M}_r, \mathcal{S}'_r, \lambda'_r, \nu_r)$ , where  $\mathcal{M}_r$ ,  $\mathcal{S}'_r$ ,  $\lambda'_r$  and  $\nu_r$  are given in Examples 4.18, 4.21, 4.23 and 4.25, respectively. Thus, it is the FES defined in Example 4.10. In particular, the FM  $\mathcal{M}_r$  is depicted in Fig. 1 and the annotated BES  $(\mathcal{S}'_r, \lambda'_r)$  is depicted in Fig. 6.

We furthermore define a mapping that takes an FES and returns a BFM that is inductively constructed from the structure of the FES's FM using the notations defined in Not. 4.11, i.e., using the events associated with features via the event location knowledge, the product knowledge, and the different event and feature constraints obtained from the BES and the FM, respectively.

*Definition 4.28 (BFM view of FES).* Let  $\mathbb{B}$  and  $\mathbb{E}$  be the sets of BFMs and FESs, respectively, over a set of features  $N$ , a set of events  $E$ , and a set of actions  $A$ . The mapping  $\text{bfm} : \mathbb{E} \rightarrow \mathbb{B}$  takes an FES  $\mathcal{E}$  over  $N$ ,  $E$ , and  $A$  and returns the *BFM view of  $\mathcal{E}$* , which is the BFM over  $N$ ,  $E$ , and  $A$

$$\text{bfm}(\mathcal{E}) \doteq (f : \mathcal{S}_f, \text{GFC}_f, \text{FC}_f(\mathcal{E}), \text{interFtEc}_f(\mathcal{E}), \text{productC}_f(\mathcal{E})),$$

where

$$\begin{aligned} f &= \text{root}(\text{fm}(\mathcal{E})), \\ \mathcal{S}_f &= (\text{events}_f(\mathcal{E}), \text{confl}(\text{intraFtEc}_f(\mathcal{E})), \text{cause}(\text{intraFtEc}_f(\mathcal{E})), \\ &\quad \text{actions}_f(\mathcal{E}), \text{labelFun}_f(\mathcal{E})), \text{ and} \\ \text{GFC}_f &= \{ \text{Uop}(\text{bfm}(\text{subTree}_{f_i}(\mathcal{E}))) \mid \text{Uop}(f_i) \in \text{gfc}_f(\mathcal{E}) \} \\ &\quad \cup \{ \text{Op}(\text{bfm}(\text{subTree}_{f_{j_1}}(\mathcal{E})), \dots, \text{bfm}(\text{subTree}_{f_{j_m}}(\mathcal{E}))) \mid \text{Op}(f_{j_1}, \dots, f_{j_m}) \in \text{gfc}_f(\mathcal{E}) \}. \end{aligned}$$

*Example 4.29 (BFM view of the FES of the cleaning robot SPL).* The BFM view of the FES  $\mathcal{E}'_r$  of the cleaning robot SPL as defined in Example 4.10 (and depicted in Figs. 1 and 6) is the BFM  $\mathcal{B}_r$  as defined in Example 4.6 (and depicted in Fig. 5).

The following proposition states that the mappings  $\text{fes}$  and  $\text{bfm}$  are isomorphisms.

**PROPOSITION 4.30 (MAPPINGS  $\text{fes}$  AND  $\text{bfm}$  ARE ISOMORPHISMS).** *Let  $\mathbb{B}$  and  $\mathbb{E}$  be the sets of BFMs and FESs, respectively, over a set of features  $N$ , a set of events  $E$ , and a set of actions  $A$ . Let  $\mathcal{B} \in \mathbb{B}$ ,  $\mathcal{E} \in \mathbb{E}$  and  $f \in N$ . Then it holds that:*

- (1)  $\text{fm}(\mathcal{B}) = \text{fm}(\text{fes}(\mathcal{B}))$  and  $\text{fm}(\mathcal{E}) = \text{fm}(\text{bfm}(\mathcal{E}))$ ;
- (2)  $\text{bes}(\mathcal{B}) = \text{bes}(\text{fes}(\mathcal{B}))$  and  $\text{bes}(\mathcal{E}) = \text{bes}(\text{bfm}(\mathcal{E}))$ ;
- (3)  $\text{evLocKnow}(\mathcal{B}) = \text{evLocKnow}(\text{fes}(\mathcal{B}))$  and  $\text{evLocKnow}(\mathcal{E}) = \text{evLocKnow}(\text{bfm}(\mathcal{E}))$ ;
- (4)  $\text{prodKnow}(\mathcal{B}) = \text{prodKnow}(\text{fes}(\mathcal{B}))$  and  $\text{prodKnow}(\mathcal{E}) = \text{prodKnow}(\text{bfm}(\mathcal{E}))$ ;
- (5)  $\text{behFt}_f(\mathcal{B}) = \text{behFt}_f(\text{fes}(\mathcal{B}))$  and  $\text{behFt}_f(\mathcal{E}) = \text{behFt}_f(\text{bfm}(\mathcal{E}))$ ;
- (6)  $\text{events}_f(\mathcal{B}) = \text{events}_f(\text{fes}(\mathcal{B}))$  and  $\text{events}_f(\mathcal{E}) = \text{events}_f(\text{bfm}(\mathcal{E}))$ ;
- (7)  $\text{actions}_f(\mathcal{B}) = \text{actions}_f(\text{fes}(\mathcal{B}))$  and  $\text{actions}_f(\mathcal{E}) = \text{actions}_f(\text{bfm}(\mathcal{E}))$ ;
- (8)  $\text{labelFun}_f(\mathcal{B}) = \text{labelFun}_f(\text{fes}(\mathcal{B}))$  and  $\text{labelFun}_f(\mathcal{E}) = \text{labelFun}_f(\text{bfm}(\mathcal{E}))$ ;
- (9)  $\text{intraFtEc}_f(\mathcal{B}) = \text{intraFtEc}_f(\text{fes}(\mathcal{B}))$  and  $\text{intraFtEc}_f(\mathcal{E}) = \text{intraFtEc}_f(\text{bfm}(\mathcal{E}))$ ;
- (10)  $\text{interFtEc}_f(\mathcal{B}) = \text{interFtEc}_f(\text{fes}(\mathcal{B}))$  and  $\text{interFtEc}_f(\mathcal{E}) = \text{interFtEc}_f(\text{bfm}(\mathcal{E}))$ ;
- (11)  $\text{FC}_f(\mathcal{B}) = \text{FC}_f(\text{fes}(\mathcal{B}))$  and  $\text{FC}_f(\mathcal{E}) = \text{FC}_f(\text{bfm}(\mathcal{E}))$ ;
- (12)  $\text{gfc}_f(\mathcal{B}) = \text{gfc}_f(\text{fes}(\mathcal{B}))$  and  $\text{gfc}_f(\mathcal{E}) = \text{gfc}_f(\text{bfm}(\mathcal{E}))$ ;
- (13)  $\text{ec}_f(\mathcal{B}) = \text{ec}_f(\text{fes}(\mathcal{B}))$  and  $\text{ec}_f(\mathcal{E}) = \text{ec}_f(\text{bfm}(\mathcal{E}))$ ;
- (14)  $\text{productC}_f(\mathcal{B}) = \text{productC}_f(\text{fes}(\mathcal{B}))$  and  $\text{productC}_f(\mathcal{E}) = \text{productC}_f(\text{bfm}(\mathcal{E}))$ ;
- (15)  $\text{subTree}_f(\mathcal{B}) = \text{bfm}(\text{subTree}_f(\text{fes}(\mathcal{B})))$  and  $\text{subTree}_f(\mathcal{E}) = \text{fes}(\text{subTree}_f(\text{bfm}(\mathcal{E})))$ ; and
- (16)  $\text{fes} \circ \text{bfm} = \text{id}_{\mathbb{B}}$  and  $\text{bfm} \circ \text{fes} = \text{id}_{\mathbb{E}}$ , where  $\text{id}_{\mathbb{B}}$  and  $\text{id}_{\mathbb{E}}$  are the identity mappings on  $\mathbb{B}$  and  $\mathbb{E}$ , respectively.

Therefore, the mappings  $\text{fes}$  and  $\text{bfm}$  are isomorphisms.

**PROOF.** Straightforward from Definitions 4.26 and 4.28 and Notations 4.3 and 4.11.  $\square$

### 4.3 Products and Behavioral Products of a BFM/FES

As for FMs, we also define a notion of products for BFMs and FESs. Furthermore, since (different from FMs), BFMs and FESs contain behavior, we also define the notion of a behavioral product of a BFM and an FES, corresponding to a product and its behavior.

*Definition 4.31 (Products and behavioral products of a BFM/FES).* Given an FES  $\mathcal{E} = (\mathcal{M}, \mathcal{S}, \lambda, \nu)$ , the *products* of  $\mathcal{E}$  are the products of  $\mathcal{M}$ , and for each of its products  $pr$ , the *behavioral product* of  $\mathcal{E}$  for  $pr$  is the BES  $\text{behPr}(\mathcal{E}, pr) = \rho_{E'}(\mathcal{S})$ , where  $E' = \{e \mid pr \models \nu(e)\}$ . We write  $\text{products}(\mathcal{E})$  to denote the set of products of  $\mathcal{E}$  and  $\text{behPr}(\mathcal{E})$  to denote the set of behavioral products of  $\mathcal{E}$ .

The products and behavioral products of a BFM are defined using its FES view: For a BFM  $\mathcal{B}$  we define its products as  $\text{products}(\mathcal{B}) = \text{products}(\text{fes}(\mathcal{B}))$  and a behavioral product with respect to  $pr \in \text{products}(\mathcal{B})$  as  $\text{behPr}(\mathcal{B}, pr) = \text{behPr}(\text{fes}(\mathcal{B}), pr)$ . We denote by  $\text{behPr}(\mathcal{B})$  the set of all behavioral products of  $\mathcal{B}$ .

Note that in the above definition, the set  $E'$  includes all events  $e$  associated with features included in the product (i.e.,  $\lambda(e) \in pr$ ) such that the product also satisfies the product knowledge (i.e.,  $pr \models \nu(e)$ ): By our assumption (cf. Def. 4.8) it holds for every event  $e$  that  $\nu(e) \Rightarrow \lambda(e)$ . Thus, if a product satisfies  $\nu(e)$ , then it also satisfies  $\lambda(e)$ , implying  $\lambda(e) \in pr$ .

*Remark 4.32 (★ On used actions of behavioral products of a BFM/FES).* Let  $\mathcal{E} = (\mathcal{M}, \mathcal{S}, \lambda, \nu)$  be an FES with  $\mathcal{S} = (E, \#, \mapsto, A, I)$ , and let  $pr$  be one of its products. Then  $\text{usedAct}(\text{behPr}(\mathcal{E}, pr)) = \{a \in A \mid \exists e \in E. (I(e) = a \wedge pr \models \nu(e))\}$ . The same holds for the behavioral products of a BFM, using its FES-view.

*Example 4.33 (Behavioral products of the cleaning robot SPL).* Consider the BFM  $\mathcal{B}_r$  of the cleaning robot SPL over the actions  $A'_r$  defined in Example 4.6 (and depicted in Fig. 5) and its FES view  $\mathcal{E}'_r$  defined in Example 4.10 (its FM is depicted in Fig. 1 and its annotated BES is depicted in Fig. 6). The products of  $\mathcal{B}_r$  are the products  $pr_1 = \{r, c, o, li\}$ ,  $pr_2 = \{r, c, o, ca\}$ ,  $pr_3 = \{r, m, c, o, li\}$  and  $pr_4 = \{r, m, c, o, ca\}$  as defined in Example 2.12. Its behavioral products are the following:

$$\begin{aligned}
\mathcal{S}_1 &\doteq \text{behPr}(\mathcal{E}'_r, pr_1) = (E_1, \#, \mapsto_1, A'_r, I_1), \text{ with} \\
&\quad E_1 \doteq \{\text{charge}, \text{goAround}, \text{clean}, \text{move}_1, \text{move}_2, \text{liDet}\}, \\
&\quad \#_1 \doteq \{(\text{goAround}, \text{move}_1), (\text{liDet}, \text{move}_1)\}, \\
&\quad \mapsto_1 \doteq \{(\{\text{liDet}\}, \text{goAround}), (\{\text{goAround}, \text{move}_1\}, \text{move}_2), (\{\text{move}_2\}, \text{clean}), \\
&\quad \quad (\{\text{clean}\}, \text{charge})\} \\
&\quad I_1(e) \doteq \begin{cases} e & \text{if } e \in \{\text{charge}, \text{goAround}, \text{clean}, \text{liDet}\}, \\ \text{move} & \text{if } e \in \{\text{move}_1, \text{move}_2\}; \end{cases} \\
\mathcal{S}_2 &\doteq \text{behPr}(\mathcal{E}'_r, pr_2) = (E_2, \#, \mapsto_2, A'_r, I_2), \text{ with} \\
&\quad E_2 \doteq \{\text{charge}, \text{goAround}, \text{clean}, \text{move}_1, \text{move}_2, \text{caDet}\}, \\
&\quad \#_2 \doteq \{(\text{goAround}, \text{move}_1), (\text{caDet}, \text{move}_1)\}, \\
&\quad \mapsto_2 \doteq \{(\{\text{caDet}\}, \text{goAround}), (\{\text{goAround}, \text{move}_1\}, \text{move}_2), (\{\text{move}_2\}, \text{clean}), \\
&\quad \quad (\{\text{clean}\}, \text{charge})\} \\
&\quad I_2(e) \doteq \begin{cases} e & \text{if } e \in \{\text{charge}, \text{goAround}, \text{clean}, \text{caDet}\}, \\ \text{move} & \text{if } e \in \{\text{move}_1, \text{move}_2\}; \end{cases} \\
\mathcal{S}_3 &\doteq \text{behPr}(\mathcal{E}'_r, pr_3) = (E_3 \doteq E_1 \cup \{\text{map}\}, \#_3 \doteq \#_1, \mapsto_3 \doteq \mapsto_1 \cup \{(\{\text{map}\}, \text{move}_1), (\{\text{map}\}, \text{liDet})\}, \\
&\quad A'_r, I_3 \doteq I_1 \cup \text{id}_{\{\text{map}\}}); \\
\mathcal{S}_4 &\doteq \text{behPr}(\mathcal{E}'_r, pr_4) = (E_4 \doteq E_2 \cup \{\text{map}\}, \#_4 \doteq \#_2, \mapsto_4 \doteq \mapsto_2 \cup \{(\{\text{map}\}, \text{move}_1), (\{\text{map}\}, \text{caDet})\}, \\
&\quad A'_r, I_4 \doteq I_2 \cup \text{id}_{\{\text{map}\}}).
\end{aligned}$$

Note that  $\mathcal{S}_3$  is as in Example 2.35.

#### 4.4 Configurations and Traces of a BFM/FES

The configurations of a BFM/FES are defined in terms of its behavioral products, using the definition of a configuration of a BES (Def. 2.34).



*Definition 4.34 (Configuration of a BFM/FES).* Let  $\mathcal{E} = (\mathcal{M}, \mathcal{S}, \lambda, \nu)$  be an FES with  $\mathcal{S} = (E, \#, \mapsto, A, I)$ . A set  $C \subseteq E$  is called a *configuration* of  $\mathcal{E}$  if there exists a product  $pr$  of  $\mathcal{E}$  such that  $C$  is a configuration of  $\text{behPr}(\mathcal{E}, pr)$ . More concretely, a set  $C \subseteq E$  is a configuration of  $\mathcal{E}$  if

- (0) there exists a product  $pr$  of  $\mathcal{E}$  such that  $pr \models \nu(e)$  for all  $e \in C$ ;
- (1)  $C$  is conflict free, i.e.,  $\forall e, e' \in C. (e, e') \notin \#$ ; and
- (2)  $\forall X \subseteq E' \forall e \in C. X \mapsto e \Rightarrow C \cap X \neq \emptyset$  for  $\text{behPr}(\mathcal{E}, pr) = (E', \#, \mapsto', A, I')$ .

Analogously, a configuration of a BFM  $\mathcal{B}$  is a configuration of one of the behavioral products of  $\mathcal{B}$ . We write  $\text{Config}(\mathcal{X}, pr)$  for the set of configurations of a BFM/FES  $\mathcal{X}$  for the product  $pr$  and  $\text{Config}(\mathcal{X})$  for  $\bigcup_{pr \in \text{products}(\mathcal{X})} \text{Config}(\mathcal{X}, pr)$ .

Note that Condition (2) is slightly different from Condition (2) in Def. 2.34 because we only consider the events and causality relation of the behavioral product for  $pr$ , not all events and the causality relation of  $\mathcal{E}$ .

*Example 4.35 (Configurations of the product  $pr_3$  of the cleaning robot SPL).* Consider the BFM  $\mathcal{B}_r$  of the cleaning robot SPL defined in Example 4.6 (and depicted in Fig. 5) and its FES view  $\mathcal{E}_r$  defined in Example 4.27 (its FM is depicted in Fig. 1 and its annotated BES is depicted in Fig. 6). Then the configurations of the behavioral product  $\mathcal{S}_3 = \text{behPr}(\mathcal{E}_r, pr_3)$  as defined in Example 4.33 are the sets  $\emptyset$ ,  $\{map\}$ ,  $\{map, move_1\}$ ,  $\{map, move_1, move_2\}$ ,  $\{map, move_1, move_2, clean\}$ ,  $\{map, move_1, move_2, clean, charge\}$ ,  $\{map, liDet\}$ ,  $\{map, liDet, goAround\}$ ,  $\{map, liDet, goAround, move_2\}$ ,  $\{map, liDet, goAround, move_2, clean\}$ ,  $\{map, liDet, goAround, move_2, clean, charge\}$ , which are the same as in Example 2.35.

Similarly, the traces of BFMs and FESs are defined via the traces of their behavioral products, i.e., using the definition of an event or action trace of a BES (Def. 2.36). As for BESs, we distinguish between traces of events and traces of actions for FESs and BFMs.

*Definition 4.36 (Traces of a BFM/FES for a product).* Let  $\mathcal{E} = (\mathcal{M}, \mathcal{S}, \lambda, \nu)$  be an FES, and let  $pr$  be one of its products. A *trace of events* of  $\mathcal{E}$  for  $pr$  is a trace of events of  $\text{behPr}(\mathcal{E}, pr)$ , and a *trace of actions* of  $\mathcal{E}$  for  $pr$  is a trace of actions of  $\text{behPr}(\mathcal{E}, pr)$ .

More concretely, if  $\text{behPr}(\mathcal{E}, pr) = (E, \#, \mapsto, A, I)$ , then a trace of events of  $\mathcal{E}$  for  $pr$  is a sequence  $\langle e_1 \dots e_n \rangle$  ( $n \geq 1$ ) of distinct events  $e_i \in E, i \in \{1, \dots, n\}$ , satisfying

- (0)  $\forall i \in \{1, \dots, n\}. pr \models \nu(e_i)$ ;
- (1)  $\{e_1, \dots, e_n\}$  is conflict-free;
- (2)  $\forall X \subseteq E = \{e \mid pr \models \nu(e)\} \forall i \in \{2, \dots, n\}. X \mapsto e_i \Rightarrow \{e_1, \dots, e_{i-1}\} \cap X \neq \emptyset$ ; and
- (3)  $\nexists X \subseteq E. X \mapsto e_1$ .

We denote the set of traces of events of  $\mathcal{E}$  for  $pr$  by  $\text{Tr}^{\text{ev}}(\mathcal{E}, pr)$ , and the set of traces of actions of  $\mathcal{E}$  for  $pr$  by  $\text{Tr}(\mathcal{E}, pr)$ .

The traces of a BFM  $\mathcal{B}$  for one of its products  $pr$  are defined using its FES view:  $\text{Tr}^{\text{ev}}(\mathcal{B}, pr) \doteq \text{Tr}^{\text{ev}}(\text{fes}(\mathcal{B}), pr)$  and  $\text{Tr}(\mathcal{B}, pr) \doteq \text{Tr}(\text{fes}(\mathcal{B}), pr)$ .

Note that Conditions (1)–(3) are as in Def. 2.36, slightly modified for BFMs/FESs.

*Example 4.37 (Traces of the behavioral products of the BFM of the cleaning robot SPL).* Consider the BFM  $\mathcal{B}_r$  of the cleaning robot SPL defined in Example 4.6 (and depicted in Fig. 5) and its FES view  $\mathcal{E}_r$  defined in Example 4.10 (its FM is depicted in Fig. 1 and its annotated BES is depicted in Fig. 6). The products of  $\mathcal{B}_r$  are  $pr_1, pr_2, pr_3$  and  $pr_4$  as defined in Example 2.12 and its behavioral products are  $\mathcal{S}_i = \text{behPr}(\mathcal{E}_r, pr_i)$  ( $1 \leq i \leq 4$ ) as defined in Example 4.33. Their event traces are the

following:

$$\begin{aligned} \text{Tr}(\mathcal{S}_1) &= \text{prefixClosure}(\{\langle \text{move}_1 \cdot \text{move}_2 \cdot \text{clean} \cdot \text{charge} \rangle, \langle \text{liDet} \cdot \text{goAround} \cdot \text{move}_2 \cdot \text{clean} \cdot \text{charge} \rangle\}), \\ \text{Tr}(\mathcal{S}_2) &= \text{prefixClosure}(\{\langle \text{move}_1 \cdot \text{move}_2 \cdot \text{clean} \cdot \text{charge} \rangle, \langle \text{caDet} \cdot \text{goAround} \cdot \text{move}_2 \cdot \text{clean} \cdot \text{charge} \rangle\}), \\ \text{Tr}(\mathcal{S}_3) &= \text{prefixClosure}(\{\langle \text{map} \cdot \text{move}_1 \cdot \text{move}_2 \cdot \text{clean} \cdot \text{charge} \rangle, \\ &\quad \langle \text{map} \cdot \text{liDet} \cdot \text{goAround} \cdot \text{move}_2 \cdot \text{clean} \cdot \text{charge} \rangle\}), \text{ and} \\ \text{Tr}(\mathcal{S}_4) &= \text{prefixClosure}(\{\langle \text{map} \cdot \text{move}_1 \cdot \text{move}_2 \cdot \text{clean} \cdot \text{charge} \rangle, \\ &\quad \langle \text{map} \cdot \text{caDet} \cdot \text{goAround} \cdot \text{move}_2 \cdot \text{clean} \cdot \text{charge} \rangle\}). \end{aligned}$$

Note that the corresponding action traces are the same as the traces of the FTS of the cleaning robot SPL for  $p_1, \dots, p_4$  (see Example 2.30)

The following lemma is the equivalent of Lemma 2.37 for BFM/FESs, showing that configurations form traces of a BFM/FES.

**LEMMA 4.38 (★ CONFIGURATIONS FORMING TRACES OF A BFM/FES).** *Let  $\mathcal{X}$  be a BFM/FES over the events  $E$ , let  $pr \in \text{products}(\mathcal{X})$ , and let  $e_1, \dots, e_n \in E$  for  $n > 1$ . Then  $\exists tr \in \text{Tr}^{\text{ev}}(\mathcal{X}, pr)$  such that  $\{e_1, \dots, e_n\} = \text{events}(tr)$  if and only if  $\{e_1, \dots, e_n\} \in \text{Config}(\mathcal{X}, pr)$ .*

**PROOF.** We provide the proof for an FES  $\mathcal{E} = (\mathcal{M}, \mathcal{S}, \lambda, \nu)$ ; for a BFM  $\mathcal{B}$  the proof works exactly the same, using  $\text{fes}(\mathcal{B})$ . Let us first recall the definition of the behavioral product of  $\mathcal{E}$  for  $pr$  (Def. 4.31). It is the BES  $\text{behPr}(\mathcal{E}, pr) = \rho_{E'}(\mathcal{S}) \doteq (E', \#, \mapsto', A', I')$  with events  $E' = \{e \in E \mid pr \models \nu(e)\}$ .

“ $\Rightarrow$ ” Let  $tr \in \text{Tr}(\mathcal{E}, pr)$  with  $\{e_1, \dots, e_n\} = \text{events}(tr)$ . Then Conditions (0)–(3) of Def. 4.36 hold for  $tr$  and imply Conditions (0')–(2') of Def. 4.34 for  $\{e_1, \dots, e_n\}$ , i.e., they imply that  $\{e_1, \dots, e_n\}$  is a configuration of  $\mathcal{E}$  for  $pr$ : Condition (0) implies Condition (0'), Condition (1) implies Condition (1'), and Conditions (2) and (3) imply Condition (2').

“ $\Leftarrow$ ” Let  $\{e_1, \dots, e_n\} \in \text{Config}(\mathcal{E}, pr)$ , and let  $\langle e'_1, \dots, e'_n \rangle$  be an ordering of  $\{e_1, \dots, e_n\}$  according to the partial order  $<_{\{e_1, \dots, e_n\}}$  introduced by  $\mapsto'$  (for an explanation see the text above Lemma 2.37). Then the ordering ensures that  $\langle e'_1, \dots, e'_n \rangle$  satisfies Conditions (2) and (3) of being a trace of  $\mathcal{X}$  for  $pr$  (see Def. 4.36), and  $\langle e'_1, \dots, e'_n \rangle$  satisfies Conditions (0) and (1) because  $\{e'_1, \dots, e'_n\} = \{e_1, \dots, e_n\}$  is a configuration of  $\mathcal{X}$  for  $pr$  (see Def. 4.34). Thus,  $\langle e'_1, \dots, e'_n \rangle$  is a trace of events of  $\mathcal{X}$  for  $pr$ .  $\square$

The following definition will be needed to define an algorithm from an FES to an FTS in Sect. 5.4.

**Definition 4.39 (★ Feature expression associated with a configuration of a BFM/FES).** Let  $\mathcal{X}$  be a BFM/FES over the features  $N$  and let  $C \in \text{Config}(\mathcal{X})$  be a configuration of  $\mathcal{X}$ . Then the *feature expression associated with  $C$*  is the feature expression  $\phi_C \in \text{FExp}(N)$  with

$$\phi_C \doteq \bigvee_{\{pr \in \text{products}(\mathcal{X}) \mid C \in \text{Config}(\mathcal{X}, pr)\}} \text{fexp}(pr)$$

That is, the feature expression associated with  $C$  is the disjunction of the feature expressions for all products that have  $C$  as a configuration.

**Example 4.40 (★ Feature expressions associated with configurations of the product  $pr_3$  of the cleaning robot SPL).** Consider the BFM  $\mathcal{B}_r$  of the cleaning robot SPL defined in Example 4.6 (and depicted in Fig. 5) with products  $pr_1, pr_2, pr_3$  and  $pr_4$  as defined in Example 2.12 and the configurations of its product  $pr_3$  as defined in Example 4.35. Then the feature expressions associated with the elements of  $\text{Config}(\mathcal{B}_r, pr_3)$  are the following:

$$\begin{aligned} \phi_\emptyset &= \text{fexp}(pr_1) \vee \text{fexp}(pr_2) \vee \text{fexp}(pr_3) \vee \text{fexp}(pr_4) \\ \phi_{\{\text{map}\}} &= \phi_{\{\text{map}, \text{move}_1\}} = \phi_{\{\text{map}, \text{move}_1, \text{move}_2\}} = \phi_{\{\text{map}, \text{move}_1, \text{move}_2, \text{clean}\}} = \phi_{\{\text{map}, \text{move}_1, \text{move}_2, \text{clean}, \text{charge}\}} \\ &= \text{fexp}(pr_3) \vee \text{fexp}(pr_4) \\ \phi_{\{\text{map}, \text{liDet}\}} &= \phi_{\{\text{map}, \text{liDet}, \text{goAround}\}} = \phi_{\{\text{map}, \text{liDet}, \text{goAround}, \text{move}_2\}} = \phi_{\{\text{map}, \text{liDet}, \text{goAround}, \text{move}_2, \text{clean}\}} \\ &= \phi_{\{\text{map}, \text{liDet}, \text{goAround}, \text{move}_2, \text{clean}, \text{charge}\}} = \text{fexp}(pr_3) \end{aligned}$$

The following lemma will be needed to prove trace equivalence of an FES and an FTS in Sect. 5.4.

**LEMMA 4.41 (★ ON FEATURE EXPRESSIONS ASSOCIATED WITH CONFIGURATIONS).** *Let  $\mathcal{X}$  be a BFM/FES over the features  $N$ , let  $C \in \text{Config}(\mathcal{X})$  be a configuration of  $\mathcal{X}$ , and let  $pr \in \text{products}(\mathcal{X})$  be a product of  $\mathcal{X}$ . Then  $\text{eval}_{pr}(\phi_C) = \#$  if and only if  $C \in \text{Config}(\mathcal{X}, pr)$ .*

**PROOF.** Recall that  $\phi_C = \bigvee_{\{pr' \in \text{products}(\mathcal{X}) \mid C \in \text{Config}(\mathcal{X}, pr')\}} \text{fexp}(pr')$  (Def. 4.39), where  $\text{fexp}(pr') = \bigwedge_{f \in pr'} f \wedge \bigwedge_{g \notin pr'} \neg g$  for a product  $pr' \in \text{products}(\mathcal{X})$  (Not. 2.10). That is, for every product  $pr' \in \text{products}(\mathcal{X})$ , the feature expression  $\text{fexp}(pr')$  is a feature expression over all features of  $N$ ; more concretely, it is a conjunction over all (negated or non-negated) features of  $N$ . Since  $\phi_C$  is a disjunction of all these feature expressions, it evaluates to true if and only if at least one of its conjuncts evaluates to true.

“ $\Rightarrow$ ” Let  $\text{eval}_{pr}(\phi_C) = \#$ . As we observed above, this means that there exists  $pr' \in \text{products}(\mathcal{X})$  with  $C \in \text{Config}(\mathcal{X}, pr')$  such that  $\text{eval}_{pr}(\text{fexp}(pr')) = \#$ . This implies  $pr = pr'$  because a product does not satisfy another product’s feature expression. Therefore,  $C \in \text{Config}(\mathcal{X}, pr)$ .

“ $\Leftarrow$ ” Let  $C \in \text{Config}(\mathcal{X}, pr)$ . Then by definition  $\text{fexp}(pr)$  (Not. 2.10) is included in  $\phi_C$ , so  $\text{eval}_{pr}(\phi_C) = \#$ .  $\square$

#### 4.5 Composing and Decomposing BFM and FESs

In Sect. 3, we motivated the design of BFMs (introducing behavior in FMs), among others, by the claim that BFMs feature the same kind of compositionality as FMs. In this section, we shortly give an overview of how FMs can be composed and decomposed, and then show that these concepts are also reflected in the structure of BFMs and FESs, thereby supporting the motivation given in Sect. 3.

FMs come with a natural notion of composition: the FMs of (sub)systems can be *composed* into a larger FM by making them immediate sub-FMs of a new feature (the root of the FM of the composed system) and adding group feature constraints and cross-tree constraints. Thereby, a (sub-)FM can be reused across different (composed) FMs.

*Definition 4.42 (FM composition).* Given

- a feature  $f$ ;
- a number of  $m \geq 1$  FMs  $\mathcal{M}_i$ , for  $i \in \{1, \dots, m\}$ , such that the  $m+1$  sets  $\{f\}$ ,  $\text{features}(\mathcal{B}_1)$ ,  $\dots$ ,  $\text{features}(\mathcal{B}_m)$  are all pairwise disjoint;
- a set of group feature constraints which describe how to group the FMs  $\mathcal{M}_i$  into sub-FM groups  $\{\mathcal{G}_j\}_{j \in I'}$ ; and
- cross-tree feature constraints  $\text{FC}$ , where the features included in the feature expression belong to at least two distinct  $\mathcal{M}_i$ ’s;

their *composition* is the FM  $\mathcal{M} = (f, \{\mathcal{G}_i\}_{i \in I'}, \text{FC})$  over the set of features  $\{f\} \cup (\bigcup_{i=1, \dots, m} \text{features}(\mathcal{M}_i))$ .

Since this composition is reversible, FMs can also be decomposed: sub-FMs can be extracted from a larger FM and analyzed or refactored in isolation, or be reused in other FMs. By interleaving composition and decomposition, a sub-FM of an FM can be replaced by other sub-FMs.

*Definition 4.43 (FM decomposition).* An FM  $\mathcal{M}$  in which the root feature  $f = \text{root}(\mathcal{M})$  has  $m \geq 1$  child features  $f_1, \dots, f_m$  can be decomposed into

- the root feature  $f$ ;
- its immediate sub-FMs  $\mathcal{M}_i = \text{subTree}_{f_i}(\mathcal{M})$ , for  $i \in \{1, \dots, m\}$ ;
- the set  $\text{gfc}_f(\mathcal{M})$  of group feature constraints of the root feature  $f$ , which describe how the  $\mathcal{M}_i$ ’s are grouped into sub-FM groups of  $\mathcal{M}$ ; and
- the cross-tree feature constraints  $\text{FC}_f(\mathcal{M})$  of the root feature  $f$ .

The following two definitions show how the notion of FM composition can be lifted to BFM and FESs, respectively. Extensions to FMs with behavior are again highlighted with gray background in the definition of composition for BFM.

*Definition 4.44 (BFM composition).* Given

- (1) a behavioral feature  $(f, \mathcal{S})$ ;
- (2) a number of  $m \geq 1$  BFMs  $\mathcal{B}_i$ , for  $i \in \{1, \dots, m\}$ , such that the  $m+1$  sets  $\{f\}$ ,  $\text{features}(\mathcal{B}_1), \dots, \text{features}(\mathcal{B}_m)$  are all pairwise disjoint and likewise the  $m+1$  sets  $\text{events}(\mathcal{S})$ ,  $\text{events}(\mathcal{B}_1), \dots, \text{events}(\mathcal{B}_m)$  are all pairwise disjoint;
- (3) a set of group feature constraints which describe how to group the BFMs  $\mathcal{B}_i$  into sub-BFM groups  $\{C_j\}_{j \in I^1}$ ;
- (4) cross-tree feature constraints FC, where the features included in the feature expression belong to at least two distinct  $\mathcal{M}_i$ 's;
- (5) a set of event constraints EC, where each conflict and causality in EC either involves at least one event in  $\text{events}(\mathcal{S})$  and at least one event associated with a feature belonging to one of the  $\mathcal{B}_i$ 's, or involves events associated with features belonging to at least two distinct  $\mathcal{B}_i$ 's; and
- (6) a set of product constraints PC that has an element  $(e, \phi_e)$  for each  $e \in \text{events}(\mathcal{S})$ .

their *composition* is the BFM  $\mathcal{B} = (f : \mathcal{S}, \{C_j\}_{j \in I^1}, \text{FC}_f, \text{EC}, \text{PC})$  over the set of events  $\text{events}(\mathcal{S}) \cup (\bigcup_{i=1, \dots, m} \text{events}(\mathcal{B}_i))$ , the set of actions  $\text{actions}(\mathcal{S}) \cup (\bigcup_{i=1, \dots, m} \text{actions}(\mathcal{B}_i))$  and the set of features  $f \cup (\bigcup_{i=1, \dots, m} \text{features}(\mathcal{B}_i))$ .

*Definition 4.45 (FES composition).* Given

- (a) a behavioral feature  $(f, (E_f, \#_f, \mapsto_f, A_f, l_f))$ ;
- (b) a number of  $m \geq 1$  FESs  $\mathcal{E}_i = (\mathcal{M}_i, (E_i, \#_i, \mapsto_i, A_i, l_i), \lambda_i, \nu_i)$ , for  $i \in \{1, \dots, m\}$ , with  $\text{root}(\mathcal{M}_i) = f_i$  such that the sets  $\{f\}$ ,  $\text{features}(\mathcal{M}_1), \dots, \text{features}(\mathcal{M}_m)$  are all pairwise disjoint and likewise the sets  $E_f, E_1, \dots, E_m$  are all pairwise disjoint;
- (c) a set of group feature constraints which describes how to group the FMs  $\mathcal{M}_i$  into sub-FM groups  $\{\mathcal{G}_j\}_{j \in I^1}$ ;
- (d) cross-tree feature constraints FC, where the features included in the feature expression belong to at least two distinct  $\mathcal{M}_i$ 's;
- (e) a set of event constraints EC, where each conflict and causality in EC involves at least one event in  $E_f$  and one event in one of the  $E_i$ 's, or involves events of at least two distinct  $E_i$ 's; and
- (f) a feature expression  $\phi_e \in \text{FExp}(N)$ , for each  $e \in E_f$ , where  $N$  is defined as below;

their *composition* is the FES  $\mathcal{E} = (\mathcal{M}, \mathcal{S}, \lambda, \nu)$  over the set of features  $N = \{f\} \cup (\bigcup_{i=1, \dots, m} \text{features}(\mathcal{M}_i))$ , the set of events  $E = E_f \cup (\bigcup_{i=1, \dots, m} E_i)$ , and the set of actions  $A = A_f \cup (\bigcup_{i=1, \dots, m} A_i)$  where

- $\mathcal{M} = (f, \{\mathcal{G}_j\}_{j \in I^1}, \text{FC})$ ;
- $\mathcal{S} = (E, \text{confl}(\text{EC}) \cup \#_f \cup (\bigcup_{i=1, \dots, m} \#_i), \text{cause}(\text{EC}) \cup \mapsto_f \cup (\bigcup_{i=1, \dots, m} \mapsto_i), A, l_f \cup (\bigcup_{i=1, \dots, m} l_i))$ ;
- $\lambda : E \rightarrow N$  with  $\lambda(e) = \begin{cases} \lambda_i(e) & \text{if } e \in E_i, i \in \{1, \dots, m\}, \\ f & \text{if } e \in E_f; \text{ and} \end{cases}$
- $\nu : E \rightarrow \text{FExp}(N)$  with  $\nu(e) = \begin{cases} \nu_i(e) & \text{if } e \in E_i, i \in \{1, \dots, m\}, \\ \phi_e & \text{if } e \in E_f. \end{cases}$

The following definition shows how the notion of FM decomposition can be lifted to BFMs and FESs. The extensions to FMs with behavior are again highlighted with gray background for BFMs.

**Definition 4.46 (BFM/FES decomposition).** A BFM or FES  $\mathcal{X}$ , with  $\mathcal{M} = \text{fm}(\mathcal{X})$ , in which the root feature  $f = \text{root}(\mathcal{M})$  has  $m \geq 1$  child features  $f_1, \dots, f_m$  can be decomposed into

- (1) the behavioral feature  $\text{behFt}_f(\mathcal{X})$  of the root feature  $f$ ;
- (2) its immediate sub-BFMs or sub-FESs  $\mathcal{X}_i = \text{subTree}_{f_i}(\mathcal{X})$ , for  $i \in \{1, \dots, m\}$ ;
- (3) the set  $\text{gfc}_f(\mathcal{X})$  of group feature constraints of the root feature  $f$ , which describe how the  $\mathcal{X}_i$ 's are grouped into sub-BFM groups of  $\mathcal{X}$  in case of a BFM, and how the  $\text{fm}(\mathcal{X}_i)$ 's are grouped into sub-FM groups of  $\mathcal{M}$  in case of an FES;
- (4) the cross-tree feature constraints  $\text{FC}_f(\mathcal{X})$  of the root feature  $f$ ;
- (5) the inter-feature event constraints  $\text{interFtEc}_f(\mathcal{X})$  of the root feature  $f$ ; and
- (6) the product constraints  $\text{productC}_f(\mathcal{X})$  of the root feature  $f$ .

It is worth observing that, by design, decomposing a BFM or FES as described in Def. 4.46 can be reversed by re-composing it as described in Definitions 4.44 and 4.45, respectively.

**Example 4.47 (Decomposing and composing the BFM of the cleaning robot SPL).** Consider the BFM  $\mathcal{B}_r = (r : (E_r, \emptyset, \emptyset, A_r, \text{id}_{E_r}), \{\text{opt}(\mathcal{B}_m), \text{mnd}(\mathcal{B}_c), \text{mnd}(\mathcal{B}_o)\}, t, \text{EC}_r, \text{PC}_r)$  of the cleaning robot SPL as defined in Example 4.6. Then, using Def. 4.46, it can be decomposed into

- the behavioral feature  $(r, (E_r, \emptyset, \emptyset, A_r, \text{id}_{E_r}))$ ;
- its immediate sub-BFMs  $\mathcal{B}_m, \mathcal{B}_c$  and  $\mathcal{B}_o$ ;
- the set  $\text{gfc}_r(\mathcal{B}_r) = \{\text{opt}(m), \text{mnd}(c), \text{mnd}(o)\}$  of group feature constraints;
- the cross-tree feature constraints  $\text{FC}_r(\mathcal{B}_r) = t$ ;
- the cross-tree event constraints  $\text{interFtEc}_r(\mathcal{B}_r) = \text{EC}_r$ ; and
- the product constraints  $\text{productC}_r(\mathcal{B}_r) = \text{PC}_r$ .

Now, given the above, we can re-compose the BFM  $\mathcal{B}_r$ , by using Def. 4.44.

#### 4.6 Trace Equivalence for BFMs, FESs and FTSs

**Definition 4.48 (Trace equivalence for BFMs/FESs and FTSs).** A BFM or FES  $\mathcal{X}$  and an FTS  $\mathcal{F}$  are *trace equivalent* if

- they have the same features, actions, and products; and
- for each of their products  $pr$ , it holds that  $\text{Tr}(\text{behPr}(\mathcal{X}, pr)) = \text{Tr}(\text{behPr}(\mathcal{F}, pr))$ .

**Definition 4.49 (Trace equivalence for BFMs and FESs).** A BFM  $\mathcal{B}$  and an FES  $\mathcal{E}$  are *trace equivalent* if

- they have the same features, events, actions, and products; and
- for each of their products  $pr$ , it holds that  $\text{Tr}^{\text{ev}}(\text{behPr}(\mathcal{B}, pr)) = \text{Tr}^{\text{ev}}(\text{behPr}(\mathcal{E}, pr))$  and  $\text{Tr}(\text{behPr}(\mathcal{B}, pr)) = \text{Tr}(\text{behPr}(\mathcal{E}, pr))$ .

Note that in both Definitions 4.48 and 4.49, the first condition talks about the actions of an FTS, FES, or BFM, not about its *used* actions. The first condition of these definitions imposes an additional requirement to trace equivalence: to simplify the presentation, according to our definition, two FTSs, FESs, or BFMs can be trace equivalent (regardless of the traces of their behavioral products) only if they share the same features, actions, products and (if neither is an FTS) events. Thus, if the first condition does not hold, there is no need to check the second.

**Example 4.50 (Trace equivalence of FTS and BFM/FES for cleaning robot SPL).** Consider the FTS  $\mathcal{F}_r$  given in Example 2.25 (and depicted in Fig. 3) and the BFM  $\mathcal{B}_r$  given in Example 4.6 (and depicted in Fig. 5) and its FES view  $\mathcal{E}'_r$  given in Example 4.10 (its FM is depicted in Fig. 1 and its annotated BES is depicted in Fig. 6). Then  $\mathcal{F}_r$  and  $\mathcal{B}_r$  are trace-equivalent, as can be seen from the above mentioned examples and Examples 2.12, 2.30, 4.33 and 4.37.

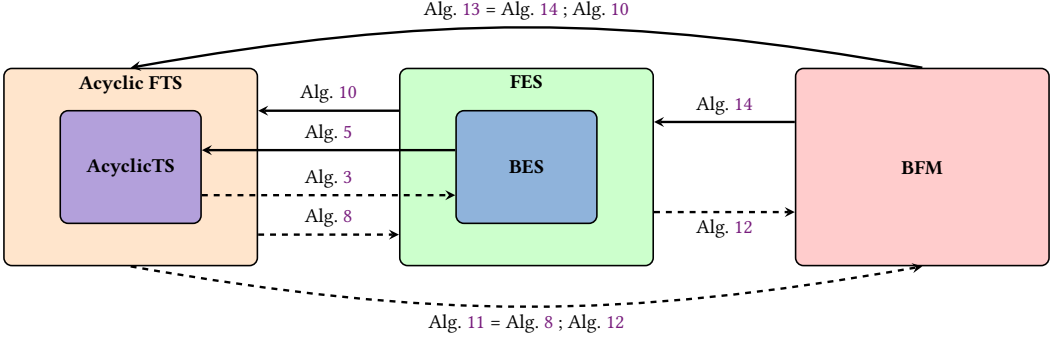


Fig. 8. A schematic representation relating the algorithms included in Sect. 5

## 5 FROM BFMs TO FTSs AND BACK BY PRESERVING TRACE EQUIVALENCE

Now that we have defined BFMs and their semantics, we show how to translate acyclic FTSs to BFMs and BFMs to acyclic FTSs in Sect. 5.5 and Sect. 5.6, respectively, and prove their trace equivalence.

As a first step towards these translations, we provide algorithms to translate acyclic TSs to BESs, BESs to acyclic TSs, acyclic FTSs to FESs, and FESs to acyclic FTSs in Sects. 5.1 to 5.4, and show their trace equivalence.

A schematic representation relating the different algorithms can be found in Fig. 8. It shows that

- the algorithm from an acyclic FTS to an FES uses the algorithm from an acyclic TS to a BES;
- the algorithm from an FES to an acyclic FTS uses the algorithm from a BES to an acyclic TS;
- the algorithms between acyclic FTSs and BFMs use the algorithms between acyclic FTSs and FESs.

### 5.1 From Acyclic TSs to BESs by Preserving Traces

Let us first define formally what we mean by an acyclic TS.

**Definition 5.1 (Acyclic TS).** Let  $\mathcal{T} = (S, A, s_0, \delta)$  be a TS. Then  $\mathcal{T}$  is called *acyclic* if it does not contain cycles, i.e., if  $\forall s_1, s_2 \in S. \neg \text{reachable}(s_1, s_2) \vee \neg \text{reachable}(s_2, s_1)$ .

When translating an acyclic TS to a BES, we first translate it to what we call a *linear* TS and then to a BES.

**Definition 5.2 (Linear TS).** Let  $\mathcal{T} = (S, A, s_0, \delta)$  be a TS. Then  $\mathcal{T}$  is called *linear* if it is acyclic and each action is associated with at most one transition, i.e., if

- $\forall s_1, s_2 \in S. \neg \text{reachable}(s_1, s_2) \vee \neg \text{reachable}(s_2, s_1)$ ; and
- $\forall a \in A \exists!(s, s') \in S \times S. (s, a, s') \in \delta$ .

The idea of translating an acyclic TS to a BES (Alg. 3) is to (1) translate the acyclic TS to a linear TS and a labeling function (Alg. 1), and (2) translate the linear TS to a BES (Alg. 2).

The idea of Alg. 1 is to rename the actions of an acyclic TS  $\mathcal{T}$  such that every transition is labeled with a unique action; these actions will form the set of events of the BES. In addition, it creates a labeling function that maps the new, unique actions to the actions of  $\mathcal{T}$ . More concretely, Alg. 1 works as follows. For every action  $a$  of the acyclic TS  $\mathcal{T}$  (Line 2) and every transition in  $\mathcal{T}$  labeled with this action (Line 4) introduce a new occurrence  $a_i$  of  $a$  (Line 5) in a set  $E$  (Line 6); this will be the set of actions of the linear TS  $\mathcal{T}^{lin}$  and the set of events of the BES. Instead of the transition



**Algorithm 1** linTS: from acyclic TS to linear TS and labeling function**Input:** acyclic TS  $\mathcal{T} = (S, A, s_0, \delta)$ **Output:** linear TS  $\mathcal{T}^{lin} = (S, E, s_0, \delta^{lin})$  and labeling function  $l : E \rightarrow A$ 

```

1: let  $E := \emptyset, \delta^{lin} := \emptyset, l := \emptyset$   $\triangleright \delta^{lin} \subseteq S \times E \times S, l : E \rightarrow A$ 
2: for  $a \in A$  do
3:    $i := 0$ 
4:   for  $(s, a, s') \in \delta$  do  $\triangleright$  Go through all transitions labeled with  $a$ 
5:     let  $a_i := a$   $\triangleright$  Introduce new occurrence of  $a$ 
6:      $E := E \cup \{a_i\}$   $\triangleright$  Include  $a_i$  in set of events  $E$ 
7:      $\delta^{lin} := \delta^{lin} \cup \{(s, a_i, s')\}$   $\triangleright$  Include relabeled transition in new transition relation
8:      $l := l \cup \{(a_i, a)\}$   $\triangleright$  Include labeling for  $a_i$ 
9:      $i := i + 1$ 
10: return  $(S, E, s_0, \delta^{lin}), l$ 

```

labeled with  $a$  in  $\mathcal{T}$ , include a transition with the same states but label  $a_i$  in the transition relation  $\delta^{lin}$  of  $\mathcal{T}^{lin}$  (Line 7) and store the information that  $a_i$  is an occurrence of  $a$  in the labeling function (Line 8). This yields a linear TS  $\mathcal{T}^{lin}$  in which every transition is labeled with an element of  $E$  that can be mapped to an action of  $\mathcal{T}$  using the labeling function.

The idea of Alg. 2 is to use reachability and previous transitions in a linear TS to construct conflict and causality relations of a BES. More concretely, Alg. 2 works as follows. The sets of actions and events of the BES equal the set of actions of the linear TS  $\mathcal{T}$ , and the labeling function is the identity function  $\text{id}_A$  that maps every action to itself (Line 1). The conflict relation consists of all pairs of distinct actions of  $\mathcal{T}$  that cannot reach one another (Line 2). The intuition behind this is that two actions that are not reachable from one another in a TS cannot occur in the same trace of the TS, which is exactly what the conflict relation of a BES expresses. To define a causality relation, we include a pair  $(X, a)$  for every action  $a$  of  $\mathcal{T}$ , where  $X$  contains all “incoming actions to  $a$ ”, i.e., all actions that happen directly before  $a$  in a trace of  $\mathcal{T}$  (Lines 4 and 5). Note that this step is only possible since every transition of  $\mathcal{T}$  contains a unique action, i.e., because  $\mathcal{T}$  is linear. In this way, the causalities are inductively build up such that the order of actions in a trace of  $\mathcal{T}$  is also preserved in a trace of its translation. Since this construction also includes a pair  $(X, a)$  in the causality relation for actions  $a$  “exiting from the initial state”, the causality relation includes  $(\emptyset, a)$  for these  $a$ , which need to be excluded (Line 6).

Lastly, Alg. 3 combines the previous two algorithms. Using Alg. 1, the acyclic TS  $\mathcal{T}$  is translated to a linear TS  $\mathcal{T}^{lin}$  and a labeling function. Then Alg. 2 is used to translate  $\mathcal{T}^{lin}$  to a BES. The final BES  $\mathcal{S}$  is composed as follows. The actions of  $\mathcal{T}^{lin}$  are the events of  $\mathcal{S}$ , the conflict and causality relations of  $\mathcal{S}$  are the ones created with Alg. 2, the actions of  $\mathcal{S}$  are the actions of  $\mathcal{T}$ , and the labeling function of  $\mathcal{S}$  is the labeling function created with Alg. 1.

We first prove that Alg. 1 produces a linear TS whose traces are related to the traces of the input TS.

**PROPOSITION 5.3 (LINEARIZATION OF ACYCLIC TS).** *Let  $\mathcal{T} = (S, A, s_0, \delta)$  be an acyclic TS, and let  $\mathcal{T}^{lin} = (S, E, s_0, \delta^{lin})$  and  $l$  be its translation via Alg. 1. Then  $\mathcal{T}^{lin}$  is a linear TS and*

- (1) *for all  $\langle a_1 \dots a_n \rangle \in \text{Tr}(\mathcal{T})$  with  $a_1, \dots, a_n \in A$  there exist  $e_1, \dots, e_n \in E$  with  $l(e_i) = a_i$  for all  $i \in \{1, \dots, n\}$  such that  $\langle e_1 \dots e_n \rangle \in \text{Tr}(\mathcal{T}^{lin})$ ; and*
- (2) *for all  $\langle e_1 \dots e_n \rangle \in \text{Tr}(\mathcal{T}^{lin})$  with  $e_1, \dots, e_n \in E$ , it holds that  $\langle l(e_1), \dots, l(e_n) \rangle \in \text{Tr}(\mathcal{T})$ .*

**PROOF.** To show that  $\mathcal{T}^{lin}$  is a linear TS, we have to show that each  $e \in E$  is associated with at most one transition and that  $\mathcal{T}^{lin}$  does not contain cycles, see Def. 5.2. The first condition holds by construction and the second condition holds since  $\mathcal{T}$  is acyclic. Thus,  $\mathcal{T}^{lin}$  is a linear TS.

---

**Algorithm 2** LinTS2BES: from linear TS to BES
 

---

**Input:** linear TS  $\mathcal{T} = (S, A, s_0, \delta)$ 
**Output:** BES  $(A, \#, \mapsto, A, \text{id}_A)$ 

- 1:  $\text{id}_A := (a, a)$  for all  $a \in A$  ▷ Labeling function is the identity
  - 2:  $\# := \{ (a_1, a_2) \mid a_1, a_2 \in A \wedge a_1 \neq a_2 \wedge \neg \text{reachable}(a_1, a_2) \wedge \neg \text{reachable}(a_2, a_1) \}$
  - 3:  $\mapsto := \emptyset$
  - 4: **for**  $a \in A$  **do**
  - 5:    $\mapsto := \mapsto \cup \{ (X, a) \mid X = \{ a' \in A \mid \exists s_1, s_2, s_3 \in S. (s_1, a', s_2), (s_2, a, s_3) \in \delta \} \}$
  - 6:    $\mapsto := \mapsto \setminus \{ (\emptyset, a) \}$
  - 7: **return**  $(A, \#, \mapsto, A, \text{id}_A)$
- 

---

**Algorithm 3** AcyTS2BES: from acyclic TS to BES
 

---

**Input:** acyclic TS  $\mathcal{T} = (S, A, s_0, \delta)$ 
**Output:** BES  $(E, \#, \mapsto, A, l)$ 

- 1: let  $(\mathcal{T}^{lin}, l) := \text{linTS}(\mathcal{T})$  ▷ Get linearized TS & labeling function
  - 2:  $(E, \#, \mapsto, E, \text{id}_E) := \text{LinTS2BES}(\mathcal{T}^{lin})$  ▷ Translate linearized TS with LinTS2BES
  - 3: **return**  $(E, \#, \mapsto, A, l)$
- 

Ad (i): Let  $\langle a_1 \cdot \dots \cdot a_n \rangle \in \text{Tr}(\mathcal{T})$  with  $a_1, \dots, a_n \in A$ , and let  $s_0, s_1, \dots, s_n \in S$  such that  $(s_{i-1}, a_i, s_i) \in \delta$  for all  $i \in \{1, \dots, n\}$ . By Lines 4 to 8 of Alg. 1, there exist  $e_1, \dots, e_n \in E$  such that  $(s_{i-1}, e_i, s_i) \in \delta^{lin}$  and  $l(e_i) = a_i$  for all  $i \in \{1, \dots, n\}$ . Then  $\langle e_1 \cdot \dots \cdot e_n \rangle \in \text{Tr}(\mathcal{T}^{lin})$  by definition of traces of a TS (Def. 2.19).

Ad (ii): Let  $\langle e_1 \cdot \dots \cdot e_n \rangle \in \text{Tr}(\mathcal{T}^{lin})$  with  $e_1, \dots, e_n \in E$ , and let  $s_0, s_1, \dots, s_n \in S$  such that  $(s_{i-1}, e_i, s_i) \in \delta^{lin}$  for all  $i \in \{1, \dots, n\}$ . By Lines 4 to 8 of Alg. 1, there exists  $a_i \in A$  for all  $i \in \{1, \dots, n\}$  such that  $l(e_i) = a_i$ . The same lines imply that  $(s_{i-1}, a_i, s_i) \in \delta$  for all  $i \in \{1, \dots, n\}$ . Thus,  $\langle a_1 \cdot \dots \cdot a_n \rangle \in \text{Tr}(\mathcal{T})$ . □

Next, we prove that Alg. 2 translates a linear TS to a BES.

PROPOSITION 5.4 (LinTS2BES( $\mathcal{T}$ ) IS A BES). *Let  $\mathcal{T}$  be a linear TS. Then LinTS2BES( $\mathcal{T}$ ) is a BES.*

PROOF. Let  $\mathcal{T} = (S, A, s_0, \delta)$  and  $\text{LinTS2BES}(\mathcal{T}) = (A, \#, \mapsto, A, \text{id}_A)$ . We have to show that:

- (1)  $A$  is a finite set,
- (2)  $\# \subseteq A \times A$  is an irreflexive and symmetric relation, and
- (3)  $\mapsto \subseteq (\mathcal{P}(A) \setminus \{\emptyset\}) \times A$  is a relation, satisfying:
  - (a)  $\forall (X, a) \in \mapsto \forall a_1, a_2 \in X. a_1 \neq a_2 \Rightarrow a_1 \# a_2$ ;
  - (b)  $\forall (X, a), (Y, a) \in \mapsto. X \neq Y \Rightarrow (X \not\subseteq Y \wedge Y \not\subseteq X)$ ; and
- (4)  $\text{id}_A : A \rightarrow A$  is a function.

Condition 1 is immediate, Condition 4 holds by Line 1 of Alg. 2, and Condition 2 holds by Line 2 of Alg. 2 because only pairs of distinct elements are included in  $\#$  (implying that  $\#$  is irreflexive) that are not reachable from one another (implying that  $\#$  is symmetric).

Ad 3: The causality relation  $\mapsto$  is a subset of  $\mathcal{P}(A) \times A$  because of Line 5 of Alg. 2, and it is a subset of  $(\mathcal{P}(A) \setminus \{\emptyset\}) \times A$  because of Line 6 of Alg. 2. Now consider Condition 3a. If  $(X, a) \in \mapsto$ , then all  $a' \in X$  are immediate predecessors of  $a$ , i.e., if  $(s, a, s') \in \delta$  and  $X = \{a_1, \dots, a_n\}$ , then there exist  $s_1, \dots, s_n \in S$  with  $(s_i, a_i, s) \in \delta$  for all  $i \in \{1, \dots, n\}$ . Since  $\mathcal{T}$  is linear, this also implies that  $\neg \text{reachable}(a_i, a_j)$  for all  $i, j \in \{1, \dots, n\}$ , with  $i \neq j$ , thus  $(a_i, a_j) \in \#$  for all  $i, j \in \{1, \dots, n\}$ , with  $i \neq j$ , by Line 2 of Alg. 2. Condition 3b holds because there is at most one  $(X, a) \in \mapsto$  for every  $a \in A$  by the construction of  $\mapsto$  in Lines 4–6 of Alg. 2. □

We now prove that Alg. 3 translates an acyclic TS to a BES.

**PROPOSITION 5.5** (AcyTS2BES( $\mathcal{T}$ ) IS A BES). *Let  $\mathcal{T}$  be an acyclic TS. Then AcyTS2BES( $\mathcal{T}$ ) is a BES.*

**PROOF.** Let  $\mathcal{T} = (S, A, s_0, \delta)$  be an acyclic TS, and let  $\text{AcyTS2BES}(\mathcal{T}) = (E, \#, \mapsto, A, l)$ . By Prop. 5.3, the TS  $(S, E, s_0, \delta^{lin})$  constructed in Alg. 3 is linear. Proposition 5.4 implies that  $(E, \#, \mapsto, E, \text{id}_E)$  is a BES. By definition,  $A$  is a set of actions and  $l : E \rightarrow A$  is a labeling function, so  $(E, \#, \mapsto, A, l)$  is a BES.  $\square$

Next we show that the linear TS and its translation to a BES via Alg. 2 are trace equivalent.

**PROPOSITION 5.6** (TRACE EQUIVALENCE  $\mathcal{T}$  AND  $\text{LinTS2BES}(\mathcal{T})$ ). *Let  $\mathcal{T} = (S, A, s_0, \delta)$  be a linear TS. Then  $\mathcal{T}$  and  $\text{LinTS2BES}(\mathcal{T})$  are trace equivalent.*

**PROOF.** By Def. 2.40, we have to show that  $\mathcal{T}$  and  $\text{LinTS2BES}(\mathcal{T})$  have the same actions and that  $\text{Tr}(\mathcal{T}) = \text{Tr}(\text{LinTS2BES}(\mathcal{T}))$ . By construction, they have the same actions. We now show that  $\text{Tr}(\mathcal{T}) = \text{Tr}(\text{LinTS2BES}(\mathcal{T}))$ . To do so, let  $\text{LinTS2BES}(\mathcal{T}) = (A, \#, \mapsto, A, \text{id}_A)$ . Since the sets of actions and events of  $\text{LinTS2BES}(\mathcal{T})$  are equal and the labeling function is the identity, it holds that  $\text{Tr}(\text{LinTS2BES}(\mathcal{T})) = \text{Tr}^{\text{ev}}(\text{LinTS2BES}(\mathcal{T}))$ .

“ $\subseteq$ ” We first show that for all  $tr \in \text{Tr}(\mathcal{T})$ , it holds that  $tr \in \text{Tr}(\text{LinTS2BES}(\mathcal{T}))$ . We do this by induction on the length of  $tr$  for  $tr \in \text{Tr}(\mathcal{T})$ .

Base cases: If  $tr$  is the empty trace, then  $tr \in \text{Tr}(\text{LinTS2BES}(\mathcal{T}))$  by the definition of traces of a BES (Def. 2.36). If  $tr = \langle a \rangle$  for  $a \in A$ , then there exists  $s \in S$  such that  $(s_0, a, s) \in \delta$ . By Line 5 of Alg. 2, there does not exist  $X \subseteq A$  with  $X \mapsto a$ , which implies that  $tr \in \text{Tr}(\text{LinTS2BES}(\mathcal{T}))$ .

Induction step: Assume that for all traces  $tr \in \text{Tr}(\mathcal{T})$  of length  $n$ , it holds that  $tr \in \text{Tr}(\text{LinTS2BES}(\mathcal{T}))$ . Let  $tr' \in \text{Tr}(\mathcal{T})$  be a trace of length  $n + 1$  with  $tr' = \langle a_1 \dots a_n \cdot a \rangle$ , for  $a_i, a \in A, i \in \{1, \dots, n + 1\}$ . By the definition of the traces of a TS (Def. 2.19), the sequence  $tr = \langle a_1 \dots a_n \rangle$  is a trace of  $\mathcal{T}$  of length  $n$  and there exist states  $s, s' \in S$  with  $s_0 \xrightarrow{tr} s \xrightarrow{a} s'$ . The induction hypothesis implies that  $tr \in \text{Tr}(\text{LinTS2BES}(\mathcal{T}))$ . We have to show that  $tr' \in \text{Tr}(\text{LinTS2BES}(\mathcal{T}))$ .

By Def. 2.36, we need to show (1)  $(a_i, a_j) \notin \#$  for all  $i \neq j \in \{1, \dots, n + 1\}$ , (2) for all  $i \in \{2, \dots, n + 1\}$  and for all  $X \mapsto a_i$ , it holds  $\{a_1, \dots, a_{i-1}\} \cap X \neq \emptyset$ , and (3) there does not exist  $X \subseteq E$  with  $X \mapsto a_1$ . Since  $tr$ , which is  $tr'$  without  $a$ , is a trace of  $\text{LinTS2BES}(\mathcal{T})$ , it suffices to show that (1')  $(a_i, a) \notin \#$ , for all  $i \in \{1, \dots, n\}$ , and (2') for all  $X \subseteq A$  with  $X \mapsto a$  it holds  $\{a_1, \dots, a_n\} \cap X \neq \emptyset$ .

Ad (1'): Since  $tr'$  is a trace in  $\mathcal{T}$ , the action  $a$  is reachable from all  $a_i, i \in \{1, \dots, n\}$ , i.e.,  $\text{reachable}(a_i, a)$  for all  $i \in \{1, \dots, n\}$ . Thus  $(a_i, a) \notin \#$  by Line 2 of Alg. 2.

Ad (2'): Let  $X \subseteq A$  with  $(X, a) \in \mapsto$ . By Line 5 of Alg. 2, it holds that  $X = \{a' \in A \mid \exists s_1 \in S. (s_1, a', s) \in \delta\}$  (using the linearity of  $\mathcal{T}$  and the fact that  $(s, a, s') \in \delta$ ). Thus, by the definition of  $tr'$ , it holds that  $a_n \in X$ , implying  $a_n \in X \cap \{a_1, \dots, a_n\}$ .

“ $\supseteq$ ” Now we show that for all  $tr \in \text{Tr}(\text{LinTS2BES}(\mathcal{T}))$ , it holds that  $tr \in \text{Tr}(\mathcal{T})$ . We do this by induction on the length of  $tr$  for  $tr \in \text{Tr}(\text{LinTS2BES}(\mathcal{T}))$ .

Base case: If  $tr$  is the empty trace, then  $tr \in \text{Tr}(\mathcal{T})$ . If  $tr = \langle a \rangle$  for  $a \in A$ , then there does not exist  $X \subseteq A$  with  $X \mapsto a$  by the definition of traces of a BES (Def. 2.36). Thus, Lines 5 and 6 of Alg. 2 imply that there do not exist  $s_1, s_2, s_3 \in S$  and  $a' \in A$  with  $(s_1, a', s_2), (s_2, a, s_3) \in \delta$ . Since we assume that all actions in  $\mathcal{T}$  are reachable from the initial state (Rem. 2.17), it has to hold that there exists  $s \in S$  with  $(s_0, a, s) \in \delta$ , which implies  $tr \in \text{Tr}(\mathcal{T})$ .

Induction step: Assume that for all traces  $tr \in \text{Tr}(\text{LinTS2BES}(\mathcal{T}))$  of length  $n$ , it holds that  $tr \in \text{Tr}(\mathcal{T})$ . Let  $tr' \in \text{Tr}(\text{LinTS2BES}(\mathcal{T}))$  be a trace of length  $n + 1$  with  $tr' = \langle a_1 \dots a_n \cdot a \rangle$  for  $a_i, a \in A, i \in \{1, \dots, n\}$ . By the definition of traces of a BES (Def. 2.36), the sequence  $tr = \langle a_1 \dots a_n \rangle$  is a trace of  $\text{LinTS2BES}(\mathcal{T})$  of length  $n$ . From the induction hypothesis we obtain that  $tr \in \text{Tr}(\mathcal{T})$ ,

i.e., there exists  $s \in S$  with  $s_0 \xrightarrow{tr} s$ . We have to show that  $tr' \in \text{Tr}(\mathcal{T})$ , i.e., there exists a state  $s' \in S$  with  $(s, a, s') \in \delta$ .

We first show that there exists  $X \subseteq A$  with  $(X, a) \in \mapsto$ . Assume (towards contradiction) that there does not exist  $X \subseteq A$  with  $(X, a) \in \mapsto$ . Then there does not exist  $a' \in A$  and  $s_1, s_2, s_3 \in S$  such that  $(s_1, a', s_2), (s_2, a, s_3) \in \delta$  by Line 5 of Alg. 2. In particular, it must hold that there exists  $\hat{s} \in S$  with  $(s_0, a, \hat{s}) \in \delta$ . Since  $tr'$  is a trace of  $\text{LinTS2BES}(\mathcal{T})$ , none of its elements are in conflict, i.e., by Line 2 of Alg. 2 it holds that  $\text{reachable}(a_i, a)$  or  $\text{reachable}(a, a_i)$ , for all  $i \in \{1, \dots, n\}$ . From the facts that  $(s_0, a, \hat{s}) \in \delta$  and  $\mathcal{T}$  is linear, we obtain that  $\text{reachable}(a, a_i)$  for all  $i \in \{1, \dots, n\}$ . In particular,  $\text{reachable}(a, a_1)$ . However,  $tr$  is a trace in  $\mathcal{T}$ , so there exists  $s_1 \in S$  with  $(s_0, a_1, s_1) \in \delta$ , which is a contradiction to  $\text{reachable}(a, a_1)$  by the linearity of  $\mathcal{T}$ . Thus, there exists  $X \subseteq A$  with  $(X, a) \in \mapsto$ .

Let  $X \subseteq A$  with  $(X, a) \in \mapsto$ . Then there exists  $k \in \{1, \dots, n\}$  such that  $a_k \in X \cap \{a_1, \dots, a_n\}$ , because  $tr'$  is a trace of  $\text{LinTS2BES}(\mathcal{T})$ . That is, by the definition of  $\mapsto$  in Line 5 of Alg. 2, there exist  $s_1, s_2, s_3 \in S$  such that  $(s_1, a_k, s_2), (s_2, a, s_3) \in \delta$ . We have to show  $k = n$ , since this implies the lemma.

Since  $tr'$  is a trace of  $\text{LinTS2BES}(\mathcal{T})$ , its actions are not in conflict, in particular  $(a_i, a) \notin \#$ , for all  $i \in \{1, \dots, n\}$ . By Line 2 of Alg. 2 this implies that either  $\text{reachable}(a_i, a)$  or  $\text{reachable}(a, a_i)$ , for all  $i \in \{1, \dots, n\}$ . We want to show  $\text{reachable}(a_i, a)$ , for all  $i \in \{1, \dots, n\}$ . Since  $tr$  is a trace in  $\mathcal{T}$ , this implies  $k = n$  above, and thus implies the lemma.

Assume (towards contradiction) that there exists an  $i \in \{1, \dots, n\}$  with  $\text{reachable}(a, a_i)$ , and let  $\ell \in \{1, \dots, n\}$  be the smallest integer with this property, i.e., for all  $j < \ell$  it holds that  $\text{reachable}(a_j, a)$ . Case 1:  $\ell = 1$ . Since  $tr$  is a trace in  $\mathcal{T}$ , there exists  $s_1 \in S$  with  $(s_0, a_1, s_1) \in \delta$ . Since  $\mathcal{T}$  is linear, this is a contradiction to  $\text{reachable}(a, a_1)$ . Case 2:  $\ell > 1$ . Since  $tr$  is a trace in  $\mathcal{T}$ , there exist  $s_{\ell-1}, s_\ell, s_{\ell+1} \in S$  with  $(s_{\ell-1}, a_{\ell-1}, s_\ell), (s_\ell, a_\ell, s_{\ell+1}) \in \delta$ . By the choice of  $\ell$ , it must hold that  $\text{reachable}(a_{\ell-1}, a)$  and  $\text{reachable}(a, a_\ell)$ . Thus, there exist  $\hat{s}, \tilde{s} \in S$  with  $(\hat{s}, a, \tilde{s}) \in \delta$  and  $\text{reachable}(s_\ell, \hat{s})$  and  $\text{reachable}(\tilde{s}, s_\ell)$ , resulting in a loop in  $\mathcal{T}$ , which is a contradiction to  $\mathcal{T}$  being linear. Therefore, it must hold, for all  $i \in \{1, \dots, n\}$ , that  $\text{reachable}(a_i, a)$ , implying that  $k = n$ , which concludes the proof.  $\square$

Next we show that an acyclic TS and its translation to a BES via Alg. 3 are trace equivalent.

**PROPOSITION 5.7 (TRACE EQUIVALENCE  $\mathcal{T}$  AND  $\text{AcyTS2BES}(\mathcal{T})$ ).** *Let  $\mathcal{T} = (S, A, s_0, \delta)$  be an acyclic TS. Then  $\mathcal{T}$  and  $\text{AcyTS2BES}(\mathcal{T})$  are trace equivalent.*

**PROOF.** By Def. 2.40, we have to show that  $\mathcal{T}$  and  $\text{AcyTS2BES}(\mathcal{T})$  have the same actions and that  $\text{Tr}(\mathcal{T}) = \text{Tr}(\text{AcyTS2BES}(\mathcal{T}))$ . By construction, they have the same actions. We now show that  $\text{Tr}(\mathcal{T}) = \text{Tr}(\text{AcyTS2BES}(\mathcal{T}))$ . To do so, let  $\text{AcyTS2BES}(\mathcal{T}) = (E, \#, \mapsto, A, I)$ , and let  $\mathcal{T}^{lin} = (S, E, s_0, \delta^{lin})$  be the linear TS produced in Line 1 of Alg. 3.

“ $\subseteq$ ” Let  $\langle a_1 \cdot \dots \cdot a_n \rangle \in \text{Tr}(\mathcal{T})$  for  $a_1, \dots, a_n \in A$ . By Prop. 5.3 there exist  $e_1, \dots, e_n \in E$  with  $l(e_i) = a_i$  for  $i \in \{1, \dots, n\}$  such that  $\langle e_1 \cdot \dots \cdot e_n \rangle \in \text{Tr}(\mathcal{T}^{lin}) \stackrel{\text{Prop. 5.6}}{=} \text{Tr}(\text{LinTS2BES}(\mathcal{T}^{lin})) \stackrel{\text{Line 2}}{=} \text{Tr}((E, \#, \mapsto, E, \text{id}_E)) \stackrel{\text{Def. 2.36}}{=} \text{Tr}^{ev}((E, \#, \mapsto, E, \text{id}_E)) = \text{Tr}^{ev}(\text{AcyTS2BES}(\mathcal{T}))$ . By definition of action traces (Def. 2.36), it holds that  $\langle a_1 \cdot \dots \cdot a_n \rangle = \langle l(e_1) \cdot \dots \cdot l(e_n) \rangle \in \text{Tr}(\text{AcyTS2BES}(\mathcal{T}))$ .

“ $\supseteq$ ” Let  $\langle a_1 \cdot \dots \cdot a_n \rangle \in \text{Tr}(\text{AcyTS2BES}(\mathcal{T}))$  for  $a_1, \dots, a_n \in A$ . By Def. 2.36, there exist  $e_1, \dots, e_n \in E$  with  $l(e_i) = a_i$  for all  $i \in \{1, \dots, n\}$  such that  $\langle e_1 \cdot \dots \cdot e_n \rangle \in \text{Tr}^{ev}(\text{AcyTS2BES}(\mathcal{T})) \stackrel{\text{Def. 2.36}}{=} \text{Tr}((E, \#, \mapsto, E, \text{id}_E)) \stackrel{\text{Line 2}}{=} \text{Tr}(\text{LinTS2BES}(\mathcal{T}^{lin})) \stackrel{\text{Prop. 5.6}}{=} \text{Tr}(\mathcal{T}^{lin})$ . By Prop. 5.3, it holds that  $\langle a_1 \cdot \dots \cdot a_n \rangle = \langle l(e_1), \dots, l(e_n) \rangle \in \text{Tr}(\mathcal{T})$ .  $\square$

## 5.2 From BESs to Acyclic TSs by Preserving Traces

To translate a BES to an acyclic TS (Alg. 5), the idea is to first translate a BES with identity labeling function to a TS (Alg. 4) and then relabel the transitions of the TS.

**Algorithm 4** BESid2TS: from BES with identity labeling function to TS**Input:** BES  $\mathcal{S} = (E, \#, \mapsto, E, \text{id}_E)$ **Output:** TS  $(S, E, s_0, \delta)$ 

- 1:  $S := \text{Config}(\mathcal{S}), s_0 := \emptyset, \delta := \emptyset$
- 2: **for**  $s, s' \in S, e \in E$  **such that**  $s' \setminus s = \{e\}$  **do**
- 3:      $\delta := \delta \cup \{(s, e, s')\}$
- 4: **return**  $(S, E, s_0, \delta)$

**Algorithm 5** BES2AcyTS: from BES to acyclic TS**Input:** BES  $(E, \#, \mapsto, A, l)$ **Output:** acyclic TS  $(S, A, s_0, \delta)$ 

- 1: let  $(S, E, s_0, \delta') := \text{BESid2TS}((E, \#, \mapsto, E, \text{id}_E)), \delta := \emptyset$   $\triangleright \delta \subseteq S \times A \times S$
- 2: **for**  $(s, e, s') \in \delta'$  **do**
- 3:      $\delta := \delta \cup \{(s, l(e), s')\}$   $\triangleright$  Label transitions with actions, not events
- 4: **return**  $(S, A, s_0, \delta)$

Algorithm 4 defines a translation from a BES with identity labeling function to a TS. The algorithm works as follows. The states of the translated BES are defined as the BES's configurations (Line 1), since the configurations of a BES correspond to its traces, i.e., they contain all events of a trace. By defining the states of the TS in this way, the initial state can be defined as the empty configuration, and the transition relation by including a transition  $(s, e, s')$  if  $s'$  can be obtained from  $s$  by adding  $e$  (Lines 1 to 3). Thereby, the configurations (which are the states of the translated BES) determine the transition relation of the translated BES, ensuring that the BES and its translation are trace equivalent.

Algorithm 5 defines a translation from a BES to an acyclic TS, as proven in Prop. 5.8. The algorithm works as follows. First, the BES with equal sets of actions and events and identity labeling function is translated with Alg. 4 to a TS with transition relation  $\delta'$  (Line 1). This gives the set of states and the initial state of the output TS. To obtain the transition relation of the output TS, a transition is included for every transition in  $\delta'$ . This transition is labeled with the corresponding action according to the labeling function (Lines 2 and 3).

We first prove that the translation of a BES via Alg. 5 is an acyclic TS.

**PROPOSITION 5.8** ( $\text{BES2AcyTS}(\mathcal{S})$  IS AN ACYCLIC TS). *Let  $\mathcal{S}$  be a BES. Then  $\text{BES2AcyTS}(\mathcal{S})$  is an acyclic TS.*

**PROOF.** By construction,  $\text{BES2AcyTS}(\mathcal{S})$  is a TS. To show that this TS cannot contain cycles, let  $\mathcal{S} = (E, \#, \mapsto, A, l)$ , and let  $\text{BES2AcyTS}(\mathcal{S}) = (S, A, s_0, \delta)$ . Assume that  $\text{BES2AcyTS}(\mathcal{S})$  contains a cycle of length  $n$ . Then this cycle is of the form  $s_1 \xrightarrow{a_1} s_2 \xrightarrow{a_2} \dots \xrightarrow{a_{n-1}} s_n \xrightarrow{a_n} s_1$  for  $s_i \in S$ ,  $a_i \in A$ ,  $i \in \{1, \dots, n\}$ . By Lines 2 and 3 of Alg. 5, there exist  $e_i \in E$ ,  $i \in \{1, \dots, n\}$ , with  $a_i = l(e_i)$  and  $(s_i, e_i, s_{i+1}) \in \delta'$  for  $(S, E, s_0, \delta') := \text{BESid2TS}((E, \#, \mapsto, E, \text{id}_E))$ . By Lines 2 and 3 of Alg. 4, it holds that (i)  $s_{i+1} \setminus s_i = \{e_i\}$  for all  $i \in \{1, \dots, n-1\}$  and (ii)  $s_1 \setminus s_n = \{e_n\}$ . The point (i) implies that  $s_n = s_1 \cup \bigcup_{i=1}^{n-1} \{e_i\}$ . Together with point (ii) this implies that  $\{e_n\} = s_1 \setminus (s_1 \cup \bigcup_{i=1}^{n-1} \{e_i\})$ , a contradiction. Therefore,  $\text{BES2AcyTS}(\mathcal{S})$  is an acyclic TS.  $\square$

Next we prove that a BES with identity labeling function and its translation via Alg. 4 are trace equivalent.

**PROPOSITION 5.9** (TRACE EQUIVALENCE OF  $\mathcal{S}$  AND  $\text{BESid2TS}(\mathcal{S})$ ). *Let  $\mathcal{S} = (E, \#, \mapsto, E, \text{id}_E)$  be a BES with identity labeling function. Then  $\text{BESid2TS}(\mathcal{S})$  and  $\mathcal{S}$  are trace equivalent.*

PROOF. By Def. 2.40, we have to show that  $\mathcal{S}$  and  $\text{BESid2TS}(\mathcal{S})$  have the same actions, and that  $\text{Tr}(\mathcal{S}) = \text{Tr}(\text{BESid2TS}(\mathcal{S}))$ . By construction, they have the same actions. We now show that  $\text{Tr}(\mathcal{S}) = \text{Tr}(\text{BESid2TS}(\mathcal{S}))$ . Since the sets of actions and events are the same in  $\mathcal{S}$  and the labeling function is the identity, it holds that  $\text{Tr}^{\text{ev}}(\mathcal{S}) = \text{Tr}(\mathcal{S})$ . Let  $\text{BESid2TS}(\mathcal{S}) = (\mathcal{S}, E, s_0, \delta)$ .

“ $\subseteq$ ” We first show that for all  $tr \in \text{Tr}(\mathcal{S})$ , it holds that  $tr \in \text{Tr}(\text{BESid2TS}(\mathcal{S}))$ . We do this by induction on the length of  $tr$ , for  $tr \in \text{Tr}(\mathcal{S})$ .

Base case: If  $tr$  is the empty trace, then  $tr \in \text{Tr}(\text{BESid2TS}(\mathcal{S}))$  because  $s_0 = \emptyset$ . If  $tr = \langle e \rangle$  for  $e \in E$ , then  $\{e\}$  is a configuration of  $\mathcal{S}$  by Lemma 2.37, so  $tr \in \text{Tr}(\text{BESid2TS}(\mathcal{S}))$  by Lines 2 and 3 of Alg. 4.

Induction step: Assume that for all traces  $tr \in \text{Tr}(\mathcal{S})$  of length  $n$ , it holds that  $tr \in \text{Tr}(\text{BESid2TS}(\mathcal{S}))$ . Let  $tr' \in \text{Tr}(\mathcal{S})$  be a trace of length  $n + 1$  with  $tr' = \langle e_1 \cdot \dots \cdot e_n \cdot e \rangle$ , for  $e_i, e \in E$ , with  $i \in \{1, \dots, n\}$ . By the definition of traces of a BES (Def. 2.36),  $tr = \langle e_1 \cdot \dots \cdot e_n \rangle$  is a trace in  $\mathcal{S}$ . By the induction hypothesis,  $tr$  is also a trace in  $\text{BESid2TS}(\mathcal{S})$ , i.e., there exists  $s \in S$  with  $s_0 \xrightarrow{tr} s$ . We have to show that  $tr' \in \text{Tr}(\text{BESid2TS}(\mathcal{S}))$ , i.e., that there exists  $s' \in S$  with  $(s, e, s') \in \delta$ .

By Lemma 2.37, the sets  $C = \{e_1, \dots, e_n\}$  and  $C' = C \cup \{e\}$  are configurations of  $\mathcal{S}$ . Line 1 of Alg. 4 implies that there exist states  $s_C, s_{C'} \in S$  such that  $s_C = C$  and  $s_{C'} = C'$ . Because  $C' = C \cup \{e\}$ , it follows from Lines 2–3 of Alg. 4 that  $(s_C, e, s_{C'}) \in \delta$ . Furthermore, since  $C = \text{events}(tr)$ , where  $tr$  is a trace from  $s_0$  to  $s$ , Lemma 5.10 implies that  $s = C$ . Thus,  $(s = s_C, e, s_{C'}) \in \delta$ , which implies that  $tr' \in \text{Tr}(\text{BESid2TS}(\mathcal{S}))$ .

“ $\supseteq$ ” Now we show that for all  $tr \in \text{Tr}(\text{BESid2TS}(\mathcal{S}))$ , it holds that  $tr \in \text{Tr}(\mathcal{S})$ . We do this by induction on the length of  $tr$  for  $tr \in \text{Tr}(\text{BESid2TS}(\mathcal{S}))$ .

Base case: If  $tr$  is the empty trace, then  $tr \in \text{Tr}(\mathcal{S})$  by definition (Def. 2.36). If  $tr = \langle e \rangle$ , then there exists a state  $s \in S$  with  $(s_0, e, s) \in \delta$ . By Lines 2 and 3 of Alg. 4 it holds that  $\{e\} = s \setminus s_0 = s \setminus \emptyset = s$ , which implies by Line 1 that  $\{e\}$  is a configuration of  $\mathcal{S}$  which is a trace of  $\mathcal{S}$  by the definition of traces (Def. 2.36).

Induction step: Assume that for all traces  $tr \in \text{Tr}(\text{BESid2TS}(\mathcal{S}))$  of length  $n$ , it holds that  $tr \in \text{Tr}(\mathcal{S})$ . Let  $tr' \in \text{Tr}(\text{BESid2TS}(\mathcal{S}))$  be a trace of length  $n + 1$  with  $tr' = \langle e_1 \cdot \dots \cdot e_n \cdot e \rangle$  for  $e_i, e \in E$ ,  $i \in \{1, \dots, n\}$ , and let  $s \in S$  with  $s_0 \xrightarrow{tr} s$ . By the definition of traces of a TS (Def. 2.19), the sequence  $tr = \langle e_1 \cdot \dots \cdot e_n \rangle$  is a trace of  $\text{BESid2TS}(\mathcal{S})$  of length  $n$ . From the induction hypothesis we obtain that  $tr \in \text{Tr}(\mathcal{S})$ . We have to show that  $tr' \in \text{Tr}(\mathcal{S})$ .

Since  $tr$  is a trace in  $\mathcal{S}$ , it is enough to show (1')  $(e_i, e) \notin \#$  for all  $i \in \{1, \dots, n\}$  and (2')  $\forall X \subseteq E$  with  $X \mapsto e$  it holds  $X \cap \{e_1, \dots, e_n\} \neq \emptyset$  (see Def. 2.36). By Lemma 5.10 it holds that  $s = \text{events}(tr')$  and by Line 1 of Alg. 4 it holds that  $s \in \text{Config}(\mathcal{S})$ . Thus, the definition of a configuration of a BES (Def. 2.34) implies (1') and (2').

□

In the proof of Prop. 5.9, we need a lemma that determines, for a state of a BES translated via Alg. 4, the corresponding configuration of the BES.

LEMMA 5.10 (ON ALG. 4). *Let  $\mathcal{S}$  be a BES with identity labeling function and  $\text{BESid2TS}(\mathcal{S}) = (\mathcal{S}, E, s_0, \delta)$  its translation via Alg. 4. Let  $s \in S$  and  $tr \in \text{Tr}(\text{BESid2TS}(\mathcal{S}))$  with  $s_0 \xrightarrow{tr} s$ . Then  $s = \text{events}(tr)$ .*

PROOF. We do the proof by induction on the length of  $tr$ .

Base case:  $tr$  is the empty trace. Then  $\text{events}(tr) = \emptyset = s_0$ .

Induction step: Assume that for all traces  $tr \in \text{BESid2TS}(\mathcal{S})$  of length  $n$  the lemma holds. Let  $tr' \in \text{Tr}(\text{BESid2TS}(\mathcal{S}))$  be a trace of length  $n + 1$  with  $tr' = \langle e_1 \cdot \dots \cdot e_n \cdot e \rangle$ , for  $e_i, e \in E$ , with  $i \in \{1, \dots, n\}$ , and  $s_0 \xrightarrow{tr'} s'$ , for  $s' \in S$ . We have to show that  $s' = \text{events}(tr')$ .



**Algorithm 6** linFTS: from acyclic FTS to linear FTS**Input:** acyclic FTS  $\mathcal{F} = (\mathcal{M}, \mathcal{T}, \mu)$  where  $\mathcal{T} = (S, A, s_0, \delta)$ **Output:** linear FTS  $\mathcal{F}^{lin} = (\mathcal{M}, \mathcal{T}^{lin}, \mu^{lin})$  where  $\mathcal{T}^{lin} = (S, E, s_0, \delta^{lin})$ 

- 1:  $(\mathcal{T}^{lin}, l) = ((S, E, s_0, \delta^{lin}), l) := \text{linTS}(\mathcal{T})$  ▷ Get linearized TS & labeling function
- 2: **for**  $e \in E, (s, e, s') \in \delta^{lin}$  **do** ▷ Only one transition is labeled with  $e$  since  $\mathcal{T}^{lin}$  is linear
- 3:      $\mu^{lin}((s, e, s')) := \mu((s, l(e), s'))$
- 4: **return**  $(\mathcal{M}, \mathcal{T}^{lin}, \mu^{lin})$

By the definition of traces of a TS (Def. 2.19), the sequence  $tr = \langle e_1 \cdot \dots \cdot e_n \rangle$  is a trace of  $\text{BESid2TS}(\mathcal{S})$  of length  $n$  and there exists  $s \in S$  with  $s_0 \xrightarrow{tr} s \xrightarrow{e} s'$ . By the induction hypothesis, it holds that  $s = \text{events}(tr)$ . Since  $(s, e, s') \in \delta$ , Lines 2–3 of Alg. 4 imply that  $s' = s \cup \{e\}$ . Thus,  $s' = \text{events}(tr) \cup \{e\} = \{e_1, \dots, e_n, e\} = \text{events}(tr')$ .  $\square$

Now we prove that a BES and its translation via Alg. 5 are trace equivalent.

**PROPOSITION 5.11 (TRACE EQUIVALENCE OF  $\mathcal{S}$  AND  $\text{BES2AcyTS}(\mathcal{S})$ ).** *Let  $\mathcal{S} = (E, \#, \mapsto, A, l)$  be a BES. Then  $\text{BES2AcyTS}(\mathcal{S})$  and  $\mathcal{S}$  are trace equivalent.*

**PROOF.** By Def. 2.40, we have to show that  $\mathcal{S}$  and  $\text{BES2AcyTS}(\mathcal{S})$  have the same actions, and that  $\text{Tr}(\mathcal{S}) = \text{Tr}(\text{BES2AcyTS}(\mathcal{S}))$ . By construction, they have the same actions. We now show that  $\text{Tr}(\mathcal{S}) = \text{Tr}(\text{BES2AcyTS}(\mathcal{S}))$ . Let  $\text{BES2AcyTS}(\mathcal{S}) = (S, A, s_0, \delta)$ , and let  $(S, E, s_0, \delta') = \text{BESid2TS}((E, \#, \mapsto, E, \text{id}_E))$ .

“ $\subseteq$ ” Let  $\langle a_1 \cdot \dots \cdot a_n \rangle \in \text{Tr}(\mathcal{S})$  for  $a_1, \dots, a_n \in A$ . By definition of action traces (Def. 2.36), there exist  $e_1, \dots, e_n \in E$  with  $l(e_i) = a_i$  for all  $i \in \{1, \dots, n\}$  such that  $\langle e_1 \cdot \dots \cdot e_n \rangle \in \text{Tr}^{\text{ev}}(\mathcal{S}) = \text{Tr}^{\text{ev}}(E, \#, \mapsto, E, \text{id}_E) \stackrel{\text{Def. 2.36}}{=} \text{Tr}(E, \#, \mapsto, E, \text{id}_E) \stackrel{\text{Prop. 5.9}}{=} \text{Tr}(\text{BESid2TS}((E, \#, \mapsto, E, \text{id}_E)))$ . Thus, there exist states  $s_1, \dots, s_{n+1} \in S$  with  $(s_i, e_i, s_{i+1}) \in \delta'$  for all  $i \in \{1, \dots, n\}$ . By Lines 2 and 3 of Alg. 5 it holds that  $(s_i, a_i, s_{i+1}) = (s_i, l(e_i), s_{i+1}) \in \delta$  for all  $i \in \{1, \dots, n\}$ . Therefore,  $\langle a_1 \cdot \dots \cdot a_n \rangle \in \text{Tr}(\text{BES2AcyTS}(\mathcal{S}))$ .

“ $\supseteq$ ” Let  $\langle a_1 \cdot \dots \cdot a_n \rangle \in \text{Tr}(\text{BES2AcyTS}(\mathcal{S}))$  for  $a_1, \dots, a_n \in A$ . Then there exist states  $s_1, \dots, s_{n+1} \in S$  with  $(s_i, a_i, s_{i+1}) \in \delta$  for all  $i \in \{1, \dots, n\}$ . By Lines 2 and 3 of Alg. 5, there exist  $e_1, \dots, e_n \in E$  with  $l(e_i) = a_i$  and  $(s_i, e_i, s_{i+1}) \in \delta'$  for all  $i \in \{1, \dots, n\}$ . Thus,  $\langle e_1 \cdot \dots \cdot e_n \rangle \in \text{Tr}((S, E, s_0, \delta')) \stackrel{\text{Def.}}{=} \text{Tr}(\text{BESid2TS}((E, \#, \mapsto, E, \text{id}_E))) \stackrel{\text{Prop. 5.9}}{=} \text{Tr}((E, \#, \mapsto, E, \text{id}_E)) \stackrel{\text{Def. 2.36}}{=} \text{Tr}^{\text{ev}}(\mathcal{S})$ . By definition of action traces (Def. 2.36), it holds that  $\langle a_1 \cdot \dots \cdot a_n \rangle = \langle l(e_1) \cdot \dots \cdot l(e_n) \rangle \in \text{Tr}(\mathcal{S})$ .  $\square$

### 5.3 From Acyclic FTSs to FESs by Preserving Traces

Now that we have defined an algorithm for translating an acyclic TS to a BES, we can define an algorithm from acyclic FTSs to FESs based on the algorithm from acyclic TSs to BES. First, we define what an acyclic and linear FTS is.

**Definition 5.12 (Acyclic and linear FTS).** Let  $\mathcal{F} = (\mathcal{F}, \mathcal{T}, \mu)$  be an FTS. Then  $\mathcal{F}$  is called *acyclic* or *linear* if  $\mathcal{T}$  is acyclic or linear, respectively.

To translate an acyclic FTS to an FES (Alg. 8), we proceed similarly as in Alg. 5: (1) translate the acyclic FTS to a linear FTS (Alg. 6), and (2) translate the linear FTS to an FES (Alg. 7).

Algorithm 6 defines a translation from an acyclic FTS  $\mathcal{F}$  to a linear FTS  $\mathcal{F}^{lin}$ . The algorithm works as follows. First, the TS of  $\mathcal{F}$  is translated with Alg. 1 to a linear TS  $\mathcal{T}^{lin}$ , which will be the TS of  $\mathcal{F}^{lin}$ , and a labeling function  $l$  (Line 1). For every action  $e$  of  $\mathcal{T}^{lin}$  and the transition in  $\mathcal{T}^{lin}$  labeled with  $e$ , which is unique because  $\mathcal{T}^{lin}$  is linear, (Line 2), the product knowledge of  $\mathcal{F}^{lin}$



**Algorithm 7** LinFTS2FES: from linear FTS to FES**Input:** linear FTS  $\mathcal{F} = (\mathcal{M}, \mathcal{T}, \mu)$  where  $\mathcal{T} = (S, E, s_0, \delta)$ **Output:** FES  $\mathcal{E} = (\mathcal{M}, \mathcal{S}, \lambda, \nu)$  with  $\mathcal{S} = (E, \#, \mapsto, E, \text{id}_E)$ 

- 1:  $\mathcal{S} := \text{LinTS2BES}(\mathcal{T})$
- 2: let  $s_0, \dots, s_n (n \geq 0)$  be a topological ordering of  $S$
- 3: **for**  $i = 0, \dots, n - 1$  **do** ▷ Go through the states in topological order
- 4:     **for**  $(s_i, e, s) \in \delta$  **do** ▷ Take all transitions starting with  $s_i$
- 5:          $\lambda(e) := \text{root}(\mathcal{M})$
- 6:          $\nu(e) := \mu((s_i, e, s)) \wedge \left( \bigvee_{(s', e', s_i) \in \delta} \nu(e') \right)$  ▷ Recursively construct  $\nu$  from  $\nu$  of previous transitions
- 7: **return**  $(\mathcal{M}, \mathcal{S}, \lambda, \nu)$

**Algorithm 8** AcyFTS2FES: from acyclic FTS to FES**Input:** acyclic FTS  $\mathcal{F} = (\mathcal{M}, \mathcal{T}, \mu)$  where  $\mathcal{T} = (S, A, s_0, \delta)$ **Output:** FES  $\mathcal{E} = (\mathcal{M}, \mathcal{S}, \lambda, \nu)$ 

- 1:  $\mathcal{S} := \text{AcyTS2BES}(\mathcal{T})$
- 2:  $\mathcal{F}^{\text{lin}} := \text{linFTS}(\mathcal{F})$
- 3:  $(\mathcal{M}, \mathcal{S}', \lambda, \nu) := \text{LinFTS2FES}(\mathcal{F}^{\text{lin}})$
- 4: **return**  $(\mathcal{M}, \mathcal{S}, \lambda, \nu)$

is defined as the product knowledge of the corresponding transition in  $\mathcal{T}$ , obtained by using the labeling function  $l$  on  $e$  (Line 3).

Algorithm 7 defines a translation from a linear FTS  $\mathcal{F}$  to an FES  $\mathcal{E}$ . The algorithm works as follows. The BES of  $\mathcal{E}$  is obtained using Alg. 2 with the underlying (linear) TS of  $\mathcal{F}$  (Line 1). Since an FTS does not contain information on which behavior is associated with which feature, all events are associated with the root of the FM (Line 5). The product knowledge  $\nu$  is defined recursively: We take a topological ordering [41] of  $S$  (Line 2), i.e., a total order  $\leq$  of  $S$  such that if  $s \leq s'$ , then there exists no path in  $\mathcal{T}$  from  $s'$  to  $s$  (and reversely, if there exists a path from  $s$  to  $s'$ , then  $s \leq s'$ ). Then we go through the states according to their topological ordering, starting with the initial state (Line 3). For each state, we consider all transitions starting from this state (Line 4). Since we go through the states in topological order, the product knowledge  $\nu$  of all “incoming actions” has already been defined. Therefore, we can define the product knowledge  $\nu$  of an action associated with transition  $t$  as the conjunction of (i) the feature expression associated with  $t$ , and (ii) the disjunction of the product knowledge of all “incoming actions” (Line 6). Thus, the product knowledge of an action reflects which products of  $\mathcal{F}$  “can reach this action”, i.e., a product satisfies the product knowledge of an action if and only if this action is reachable in its behavioral product (see Lemma 5.16).

Algorithm 8 defines a translation from an acyclic FTS  $\mathcal{F}$  to an FES  $\mathcal{E}$ . The BES of  $\mathcal{E}$  is obtained using Alg. 3 (Line 1). The acyclic FTS  $\mathcal{F}$  is then translated to a linear FTS (Line 2), which is translated with Alg. 7 to an FES with identity labeling function. This provides the event location knowledge and the product knowledge of  $\mathcal{E}$ .

We first prove that Alg. 6 translates an acyclic FTS to a linear FTS. This proposition is the equivalent of Prop. 5.3 for FTSs.

**PROPOSITION 5.13 (LINEARIZATION OF ACYCLIC FTS).** *Let  $\mathcal{F} = (\mathcal{M}, \mathcal{T}, \mu)$  be an acyclic FTS with  $\mathcal{T} = (S, A, s_0, \delta)$ , and let  $\mathcal{F}^{\text{lin}} = (\mathcal{M}, \mathcal{T}^{\text{lin}}, \mu^{\text{lin}})$  be its translation via Alg. 6 with  $\mathcal{T}^{\text{lin}} = (S, E, s_0, \delta^{\text{lin}})$  and labeling function  $l$ . Then  $\mathcal{F}^{\text{lin}}$  is a linear FTS and for every product  $pr \in \text{products}(\mathcal{F}) = \text{products}(\mathcal{F}^{\text{lin}})$  the following holds:*

- (1) for all  $\langle a_1 \dots a_n \rangle \in \text{Tr}(\mathcal{F}, pr)$  with  $a_1, \dots, a_n \in \text{actions}(\text{behPr}(\mathcal{F}, pr))$  there exist  $e_1, \dots, e_n \in \text{actions}(\text{behPr}(\mathcal{F}^{lin}, pr))$  with  $l(e_i) = a_i$  for all  $i \in \{1, \dots, n\}$  such that  $\langle e_1 \dots e_n \rangle \in \text{Tr}(\mathcal{T}^{lin}, pr)$ ; and
- (2) for all  $\langle e_1 \dots e_n \rangle \in \text{Tr}(\mathcal{F}^{lin}, pr)$  with  $e_1, \dots, e_n \in \text{actions}(\text{behPr}(\mathcal{F}^{lin}, pr))$ , it holds that  $\langle l(e_1), \dots, l(e_n) \rangle \in \text{Tr}(\mathcal{T}, pr)$ .

PROOF. To show that  $\mathcal{F}^{lin}$  is a linear FTS, we have to show that each  $e \in E$  is associated with at most one transition and that  $\mathcal{F}^{lin}$  does not contain cycles, see Def. 5.12. The first condition holds by construction and the second condition holds since  $\mathcal{F}$  is acyclic. Thus,  $\mathcal{T}^{lin}$  is a linear FTS.

Ad (i): Let  $\langle a_1 \dots a_n \rangle \in \text{Tr}(\mathcal{F}, pr)$  with  $a_1, \dots, a_n \in \text{actions}(\text{behPr}(\mathcal{F}, pr))$ , and let  $s_0, s_1, \dots, s_n \in S$  such that  $(s_{i-1}, a_i, s_i) \in \delta$  for all  $i \in \{1, \dots, n\}$ . By the definition of traces of an FTS (Def. 2.29), it holds for every  $i \in \{1, \dots, n\}$  that  $\text{eval}_{pr}(\mu((s_{i-1}, a_i, s_i))) = \#$ . In the same way as in the proof of Prop. 5.3, it can be shown that there exist  $e_1, \dots, e_n \in E$  such that  $(s_{i-1}, e_i, s_i) \in \delta^{lin}$  and  $l(e_i) = a_i$  for all  $i \in \{1, \dots, n\}$ . Furthermore, by the definition of  $\mu^{lin}$  in Line 3 of Alg. 6 it holds that  $\text{eval}_{pr}(\mu^{lin}((s_{i-1}, e_i, s_i))) = \#$  for every  $i \in \{1, \dots, n\}$ . Therefore,  $e_1, \dots, e_n \in \text{actions}(\text{behPr}(\mathcal{F}^{lin}, pr))$ , which implies that  $\langle e_1 \dots e_n \rangle \in \text{Tr}(\mathcal{F}^{lin}, pr)$ .

Ad (ii): Let  $\langle e_1 \dots e_n \rangle \in \text{Tr}(\mathcal{F}^{lin}, pr)$  with  $e_1, \dots, e_n \in \text{actions}(\text{behPr}(\mathcal{F}^{lin}, pr))$ , and let  $s_0, s_1, \dots, s_n \in S$  such that  $(s_{i-1}, e_i, s_i) \in \delta^{lin}$  for all  $i \in \{1, \dots, n\}$ . By the definition of traces of an FTS (Def. 2.29), it holds for every  $i \in \{1, \dots, n\}$  that  $\text{eval}_{pr}(\mu^{lin}((s_{i-1}, e_i, s_i))) = \#$ . In the same way as in the proof of Prop. 5.3, it can be shown that for every  $i \in \{1, \dots, n\}$  there exist  $a_i \in A$  such that  $l(e_i) = a_i$  and  $(s_{i-1}, a_i, s_i) \in \delta$ . Furthermore, by the definition of  $\mu^{lin}$  in Line 3 of Alg. 6 it holds that  $\text{eval}_{pr}(\mu((s_{i-1}, a_i, s_i))) = \#$  for every  $i \in \{1, \dots, n\}$ . Therefore,  $a_1, \dots, a_n \in \text{actions}(\text{behPr}(\mathcal{F}, pr))$ , which implies that  $\langle a_1 \dots a_n \rangle \in \text{Tr}(\mathcal{F}, pr)$ .  $\square$

Now we prove that Alg. 8 translates an acyclic FTS to an FES.

PROPOSITION 5.14 (ACYFES2FES( $\mathcal{F}$ ) IS AN FES). *Let  $\mathcal{F}$  be an acyclic FTS. Then  $\text{LinFES2FES}(\mathcal{F})$  is an FES.*

PROOF. The proof is immediate since for every acyclic TS  $\mathcal{T}$ , its translation  $\text{AcyTS2BES}(\mathcal{T})$  is a BES by Prop. 5.5.  $\square$

Now we prove that a linear FTS and its translation via Alg. 7 are trace equivalent.

PROPOSITION 5.15 (TRACE EQUIVALENCE OF  $\mathcal{F}$  AND  $\text{LinFES2FES}(\mathcal{F})$ ). *Let  $\mathcal{F}$  be a linear FTS over the set of features  $N$ . Then  $\text{LinFES2FES}(\mathcal{F})$  and  $\mathcal{F}$  are trace equivalent.*

PROOF. For  $\mathcal{F}$  and  $\text{LinFES2FES}(\mathcal{F})$  to be trace equivalent, they have to have the same features, actions and products, and for every product  $pr$  it has to hold that  $\text{Tr}(\mathcal{F}, pr) = \text{Tr}(\text{LinFES2FES}(\mathcal{F}), pr)$ . By construction,  $\mathcal{F}$  and  $\text{LinFES2FES}(\mathcal{F})$  have the same features, actions and products.

Let  $pr \in \text{products}(\mathcal{F}) = \text{products}(\text{LinFES2FES}(\mathcal{F}))$ , let  $\mathcal{F} = (\mathcal{M}, \mathcal{T}, \mu)$  with  $\mathcal{T} = (S, E, s_0, \delta)$ , let  $\text{behPr}(\mathcal{F}, pr) = (S', E, s'_0, \delta')$ , let  $\text{LinFES2FES}(\mathcal{F}) = (\mathcal{M}, \mathcal{S}, \lambda, \nu)$  with  $\mathcal{S} = (E, \#, \mapsto, E, \text{id}_E)$ , and let  $\text{behPr}(\text{LinFES2FES}(\mathcal{F}), pr) = (E', \#, \mapsto', E, \text{id}_{E'})$  with  $E' = \{e \in E \mid pr \models \nu(e)\}$ . It remains to show that the sets of traces of  $\mathcal{F}$  for  $pr$  and of  $\text{LinFES2FES}(\mathcal{F})$  for  $pr$  are the same, i.e.,  $\text{Tr}(\mathcal{F}, pr) = \text{Tr}(\text{LinFES2FES}(\mathcal{F}), pr)$ . Since the labeling function of  $\text{behPr}(\text{LinFES2FES}(\mathcal{F}), pr)$  is the identity on  $E'$ , it holds that  $\text{Tr}(\text{LinFES2FES}(\mathcal{F}), pr) = \text{Tr}^{\text{ev}}(\text{LinFES2FES}(\mathcal{F}), pr)$ . This proof is similar to the one of Prop. 5.6.

“ $\subseteq$ ” We first show that for all  $tr \in \text{Tr}(\mathcal{F}, pr)$ , it holds that  $tr \in \text{Tr}(\text{LinFES2FES}(\mathcal{F}), pr)$ , i.e., that  $tr \in \text{Tr}(\text{behPr}(\text{LinFES2FES}(\mathcal{F}), pr))$ . We do this by induction on the length of  $tr$  for  $tr \in \text{Tr}(\mathcal{F}, pr)$ .

Base case: If  $tr$  is the empty trace, then  $tr \in \text{Tr}(\text{LinFES2FES}(\mathcal{F}), pr)$  by the definition of traces of a BES (Def. 2.36). If  $tr = \langle e \rangle$  for  $e \in E$ , then Conditions (1)–(3) of being a trace of  $\text{LinFES2FES}(\mathcal{F})$

for  $pr$  (see Def. 4.36) follow in the same way as in the proof of Prop. 5.6. For Condition (0), note that there exists a state  $s_1 \in S$  such that  $(s_0, e, s_1)$  is a transition in  $\text{behPr}(\mathcal{F}, pr)$  because  $tr$  is a trace of  $\mathcal{F}$  for  $pr$ . In particular,  $\text{eval}_{pr}(\mu((s_0, e, s_1))) = \#$  (cf. Def. 2.26), i.e.,  $pr \models \mu((s_0, e, s_1))$ , and  $\mu((s_0, e, s_1)) = v(e)$  by Line 6 of Alg. 7. Thus,  $pr \models v(e)$ .

Induction step: Assume that for all traces  $tr \in \text{Tr}(\mathcal{F}, pr)$  of length  $n$ , it holds that  $tr$  is also a trace in  $\text{LinFTS2FES}(\mathcal{F})$  for  $pr$ . Let  $tr' \in \text{Tr}(\mathcal{F}, pr)$  be a trace of length  $n+1$  with  $tr' = \langle e_1 \dots e_n \cdot e \rangle$ , for  $e_i, e \in E$ , with  $i \in \{1, \dots, n\}$ . For all transitions  $t$  in  $tr'$ , we have  $\text{eval}_{pr}(\mu(t)) = \#$  by the definition of the traces of an FTS for a product (Def. 2.29), i.e.,  $pr \models \mu(t)$ . Furthermore,  $tr = \langle e_1 \dots e_n \rangle$  is a trace of  $\mathcal{F}$  for  $pr$  of length  $n$  and there exist states  $s, s' \in S$  with  $s_0 \xrightarrow{tr} s \xrightarrow{e} s'$ . From the induction hypothesis we obtain that  $tr \in \text{Tr}(\text{LinFTS2FES}(\mathcal{F}), pr)$ . We have to show that  $tr' \in \text{Tr}(\text{LinFTS2FES}(\mathcal{F}), pr)$ .

Since  $tr$  is a trace of  $\text{LinFTS2FES}(\mathcal{F})$  for  $pr$ , it is enough to show that (0')  $pr \models v(e)$ , (1')  $(e, e_i) \notin \#$ , for all  $i \in \{1, \dots, n\}$ , and (2') for all  $X \subseteq E'$ , it holds that  $X \mapsto' e$  implies  $X \cap \{e_1, \dots, e_n\} \neq \emptyset$ , see Def. 4.36. (The proofs of (1') and (2') are very similar to the proofs of (1') and (2') in Prop. 5.6.)

Ad (0'): Since  $(s, e, s')$  is a transition in the  $\text{behPr}(\mathcal{F}, pr)$ , it holds  $\text{eval}_{pr}(\mu((s, e, s'))) = \#$  (cf. Def. 2.26), i.e.,  $pr \models \mu((s, e, s'))$ . By Line 6 of Alg. 7, it holds  $v(e) = \mu((s, e, s')) \wedge (\bigvee_{(\tilde{s}, \tilde{e}, \tilde{s}) \in \delta} v(\tilde{e}))$ . In particular, the disjunction contains the last transition of  $tr$ , i.e., the transition of  $tr$  ending in  $s$ . Since  $tr \in \text{Tr}(\text{LinFTS2FES}(\mathcal{F}), pr)$ , the product  $pr$  satisfies the product knowledge  $v$  of all actions in  $tr$ . In particular,  $pr \models v((\tilde{s}, \tilde{e}, s))$  for  $(\tilde{s}, \tilde{e}, s) \in \delta$  with  $\tilde{e} \in \text{actions}(tr)$ . Therefore,  $pr \models v(e)$ .

Ad (1'): Since  $tr'$  is a trace of  $\mathcal{F}$  for  $pr$ , the action  $e$  is reachable from all  $e_i$ , with  $i \in \{1, \dots, n\}$ , i.e.,  $\text{reachable}(e_i, e)$ , for all  $i \in \{1, \dots, n\}$ . Thus,  $(e_i, e) \notin \#$  by Line 2 of Alg. 2, which implies that  $(e_i, e) \notin \#$ , because  $\# = \# \cap (E' \times E')$ .

Ad (2'): We need to show that for all  $(X, e) \in \mapsto'$ , there exists an  $i \in \{1, \dots, n\}$  such that  $e_i \in X \cap \{e_1, \dots, e_n\}$ . Let  $(X, e) \in \mapsto'$ . By Line 5 of Alg. 2 and by the definition of the behavioral product (Def. 4.31), it holds that  $X = \{e' \in E' \mid \exists s_1 \in S. (s_1, e', s) \in \delta\}$  (using the linearity of  $\mathcal{F}$  and the fact that  $(s, e, s') \in \delta$ ). Thus, by the definition of  $tr'$ , it holds that  $e_n \in X$ , which implies that  $e_n \in X \cap \{e_1, \dots, e_n\}$ .

“ $\supseteq$ ” Now we show that for all  $tr \in \text{Tr}(\text{LinFTS2FES}(\mathcal{F}), pr)$ , it holds that  $tr \in \text{Tr}(\mathcal{F}, pr) = \text{Tr}(\text{behPr}(\mathcal{F}, pr))$ . We do this by induction on the length of  $tr \in \text{Tr}(\text{LinFTS2FES}(\mathcal{F}), pr)$ .

Base case: If  $tr$  is the empty trace, then  $tr \in \text{Tr}(\mathcal{F}, pr)$  by the definition of traces of an FTS for a product (Def. 2.29). If  $tr = \langle e \rangle$  for  $e \in E$ , then it follows in the same way as in the proof of Prop. 5.6 that there exists  $s \in S$  with  $(s_0, e, s) \in \delta$ . Furthermore, since  $tr$  is a trace in  $\text{behPr}(\text{LinFTS2FES}(\mathcal{F}), pr)$ , it holds  $pr \models v(e)$  (see Def. 4.36), and by Line 6 of Alg. 7 it holds  $v(e) = \mu((s_0, e, s))$ . Thus,  $\text{eval}_{pr}(\mu((s_0, e, s))) = \#$ , implying  $tr \in \text{Tr}(\mathcal{F}, pr)$ .

Induction step: Assume that for all traces  $tr \in \text{Tr}(\text{LinFTS2FES}(\mathcal{F}), pr)$  of length  $n$ , it holds that  $tr \in \text{Tr}(\mathcal{F}, pr)$ . Let  $tr' \in \text{Tr}(\text{LinFTS2FES}(\mathcal{F}), pr)$  be a trace of length  $n+1$  with  $tr' = \langle e_1 \dots e_n \cdot e \rangle$  for  $e_i, e \in E'$  with  $i \in \{1, \dots, n\}$ . By the definition of traces of an FES for a product (Def. 4.36), the sequence  $tr = \langle e_1 \dots e_n \rangle$  is a trace of  $\text{LinFTS2FES}(\mathcal{F})$  for  $pr$ , and by the induction hypothesis it holds that  $tr \in \text{Tr}(\mathcal{F}, pr)$ . That is, there exists a state  $s \in S$  with  $s_0 \xrightarrow{tr} s$  and  $\text{eval}_{pr}(\mu(t)) = \#$ , for all transitions  $t$  in  $tr$ . We have to show that  $tr' \in \text{Tr}(\mathcal{F}, pr)$ . Since  $tr$  is a trace of  $\mathcal{F}$  for  $pr$ , it is enough to show that there exists a state  $s' \in S$  with (1)  $(s, e, s') \in \delta$  and (2)  $\text{eval}_{pr}(\mu((s, e, s'))) = \#$ .

Ad (1): The proof works in the same way as in Prop. 5.6, using traces in  $\mathcal{F}$  for  $pr$  and configurations of  $\text{LinFTS2FES}(\mathcal{F})$  for  $pr$ .

We first show that there exists  $X \subseteq E'$  with  $(X, e) \in \mapsto'$ . Assume (towards contradiction) that there exists no  $X \subseteq E'$  with  $(X, e) \in \mapsto'$ . Then either (i) there exists no  $X \subseteq E$  with  $(X, e) \in \mapsto$  (in which case we proceed in the same way as in the proof of Prop. 5.6) or (ii) for all  $X \subseteq E$  with  $(X, e) \in \mapsto$ , it holds that  $X \cap E' = \emptyset$ . In case (ii), there does not exist  $e' \in E'$  and  $s_1, s_2, s_3 \in S$  with

$(s_1, e', s_2), (s_2, e, s_3) \in \delta$  by Line 5 of Alg. 2 (called in Line 1 of Alg. 9). However, since  $e$  is a used action of  $\text{behPr}(\text{LinFts2Fes}(\mathcal{F}), pr)$ , it is also a used action of  $\text{behPr}(\mathcal{F}, pr)$ , by Lemma 5.17. Thus, the action  $e$  must be reachable in  $\text{behPr}(\mathcal{F}, pr)$  (see Rem. 2.27). This is a contradiction, because there exist no  $e' \in E' = \text{usedAct}(\text{behPr}(\text{LinFts2Fes}(\mathcal{F}), pr)) = \text{usedAct}(\text{behPr}(\mathcal{F}, pr))$  and  $s_1, s_2, s_3 \in S$  with  $(s_1, e', s_2), (s_2, e, s_3) \in \delta$ . Thus, it must hold that there exists  $X \subseteq E'$  with  $(X, e) \in \mapsto'$ .

Let  $X \subseteq E'$  with  $(X, e) \in \mapsto'$ . Then there exists  $k \in \{1, \dots, n\}$  such that  $e_k \in X \cap \{e_1, \dots, e_n\}$ , because  $tr'$  is a trace of  $\text{LinFts2Fes}(\mathcal{F})$ . That is, by the definition of  $\mapsto$  in Line 5 of Alg. 2, there exist  $s_1, s_2, s_3 \in S$  such that  $(s_1, e_k, s_2), (s_2, e, s_3) \in \delta$ . We have to show  $k = n$ .

Since  $tr'$  is a trace of  $\text{LinFts2Fes}(\mathcal{F})$ , its events (which are equal to the actions) are not in conflict; in particular,  $(e_i, e) \notin \# \subseteq \#$ , for all  $i \in \{1, \dots, n\}$ . By Line 2 of Alg. 2, this implies that either  $\text{reachable}(e_i, e)$  or  $\text{reachable}(e, e_i)$ , for all  $i \in \{1, \dots, n\}$ . It is possible to show that  $\text{reachable}(e_i, e)$  holds for all  $i \in \{1, \dots, n\}$  in the same way as in the proof of Prop. 5.6. Since  $tr$  is a trace in  $\mathcal{F}$  for  $pr$ , this implies  $k = n$  above, which finishes the proof of (1).

Ad (2): By the definition of traces of an FES for a product (Def. 4.36), it holds  $pr \models v(e)$  because  $tr'$  is a trace of  $\text{LinFts2Fes}(\mathcal{F})$  for  $pr$  and  $e \in \text{actions}(tr')$ . Since  $v(e) = \mu((s, e, s')) \wedge (\bigvee_{(\tilde{s}, \tilde{e}, \tilde{s}) \in \delta} v(\tilde{e}))$  by Line 6 of Alg. 7, it holds that  $pr \models \mu((s, e, s'))$ , i.e.,  $\text{eval}_{pr}(\mu((s, e, s'))) = \#$ .  $\square$

In the proof of Alg. 7, we need that the behavioral products of a linear FTS and its translation via Alg. 7 have the same used actions (cf. Lemma 5.17). To prove this, we first prove the following result.

**LEMMA 5.16 (PRODUCT KNOWLEDGE OF  $\text{LinFts2Fes}(\mathcal{F})$ ).** *Let  $\mathcal{F}$  be a linear FTS, let  $\text{LinFts2Fes}(\mathcal{F})$  be its translation via Alg. 7, let  $pr \in \text{products}(\mathcal{F}) = \text{products}(\text{LinFts2Fes}(\mathcal{F}))$  be a product, and  $e \in \text{actions}(\mathcal{F})$ . Then  $pr \models v(e)$  if and only if there exists a trace in  $\mathcal{F}$  for  $pr$  that has  $e$  as last element.*

**PROOF.** Let  $\mathcal{F} = (\mathcal{M}, \mathcal{T}, \mu)$  with  $\mathcal{T} = (S, E, s_0, \delta)$ . Let  $s_0, \dots, s_n (n \geq 0)$  be a topological ordering of  $S$ . We do the proof by induction on the topological ordering.

“ $\Rightarrow$ ” Base case: Let  $e \in \text{actions}(\mathcal{F})$  with  $(s_0, e, s) \in \delta$ , and let  $pr \models v(e)$ . By Line 6 of Alg. 7 it holds  $v(e) = \mu((s_0, e, s))$ . Thus,  $pr \models \mu((s_0, e, s))$ , which means that  $\langle e \rangle$  is a trace in  $\mathcal{F}$  for  $pr$ .

Induction step: Assume for all  $i \leq m$  and actions  $e$  with  $(s_i, e, s) \in \delta$  it holds  $pr \models v(e)$  implies that there exists a trace in  $\mathcal{F}$  for  $pr$  with  $e$  as a last element. Let  $e \in \text{actions}(\mathcal{F})$  with  $(s_{m+1}, e, s) \in \delta$  and  $pr \models v(e)$ . By Line 6 of Alg. 7 it holds  $v(e) = \mu((s_{m+1}, e, s)) \wedge (\bigvee_{(s', e', s_{m+1}) \in \delta} v(e'))$ . Since  $pr \models v(e)$ , it holds  $pr \models \mu((s_{m+1}, e, s))$  and there exists  $(s', e', s_{m+1}) \in \delta$  with  $pr \models v(se')$ . By construction it must hold that  $s' = s_i$  for some  $i < m + 1$ . Therefore, the induction hypothesis implies that there exists a trace  $tr$  in  $\mathcal{F}$  for  $pr$  with  $e'$  as last element. Thus, the trace  $tr$  with additionally included element  $e$  at the end is a trace in  $\mathcal{F}$  for  $pr$  (see Def. 2.29).

“ $\Leftarrow$ ” Base case: Let  $e \in \text{actions}(\mathcal{F})$  with  $(s_0, e, s) \in \delta$ , and let  $tr$  be a trace in  $\mathcal{F}$  for  $pr$  with  $e$  as last element. Then  $pr \models \mu((s_0, e, s))$  (see Def. 2.29). By Line 6 of Alg. 7 it holds  $v(e) = \mu((s_0, e, s))$ , implying  $pr \models v(e)$ .

Induction step: Assume for all  $i \leq m$  and actions  $e$  with  $(s_i, e, s) \in \delta$  it holds that if there exists a trace in  $\mathcal{F}$  for  $pr$  that has  $e$  as a last element, then  $pr \models v(e)$ . Let  $e \in \text{actions}(\mathcal{F})$  with  $(s_{m+1}, e, s) \in \delta$  and there exists a trace  $tr \doteq \langle e_1 \dots e_k \cdot e \rangle$  in  $\mathcal{F}$  for  $pr$  that has  $e$  as a last element. Then  $pr \models \mu((s_{m+1}, e, s))$  (Def. 2.29). Let  $(s_{i-1}, e_i, s_i) \in \delta$  for all  $i \in \{1, \dots, k\}$ . Consider  $(s_{k-1}, e_k, s_k)$ . Then  $s_k = s_{m+1}$  because  $tr$  is a trace in  $\mathcal{F}$  for  $pr$  (see Def. 2.29). The induction hypothesis implies  $pr \models v(e_k)$ . Furthermore, by Line 6 of Alg. 7 it holds  $v(e) = \mu((s_{m+1}, e, s)) \wedge (\bigvee_{(s', e', s_{m+1}) \in \delta} v(e'))$ . In particular,  $v(e_m)$  is a disjunct in  $\bigvee_{(s', e', s_{m+1}) \in \delta} v(e')$ . Since  $pr \models \mu((s_{m+1}, e, s)) \wedge v(e_m)$ , it holds  $pr \models v(e)$ .  $\square$

**LEMMA 5.17 ( $\text{LinFts2Fes}(\mathcal{F})$  PRESERVES USED ACTIONS OF BEHAVIORAL PRODUCTS).** *Let  $\mathcal{F}$  be a linear FTS, let  $\text{LinFts2Fes}(\mathcal{F})$  be its translation via Alg. 7, and let  $pr \in \text{products}(\mathcal{F}) =$*

---

**Algorithm 9** FESid2FTS: from FES with identity labeling function to FTS
 

---

**Input:** FES  $\mathcal{E} = (\mathcal{M}, \mathcal{S}, \lambda, \nu)$  with  $\mathcal{S} = (E, \#, \mapsto, E, \text{id}_E)$

**Output:** FTS  $\mathcal{F} = (\mathcal{M}, \mathcal{T}, \mu)$  with  $\mathcal{T} = (S, E, s_0, \delta)$

- 1:  $S := \text{Config}(\mathcal{E}), s_0 := \emptyset, \delta := \emptyset$
  - 2: **for**  $s, s' \in S, e \in E$  **such that**  $s' \setminus s = \{e\}$  **do**
  - 3:      $\delta := \delta \cup \{(s, e, s')\}$
  - 4:      $\mu((s, e, s')) := \phi_s \wedge \phi_{s'}$
  - 5: **return**  $(\mathcal{M}, \mathcal{T}, \mu)$
- 

products(LinFTS2FES( $\mathcal{F}$ )) be a product. Then  $\text{behPr}(\mathcal{F}, pr)$  and  $\text{behPr}(\text{LinFTS2FES}(\mathcal{F}), pr)$  have the same used actions.

PROOF. Let  $\mathcal{F} = (\mathcal{M}, \mathcal{T}, \mu)$  with  $\mathcal{T} = (S, E, s_0, \delta)$  and let  $pr$  be a product of  $\mathcal{F}$ .

“ $\subseteq$ ” Let  $e \in \text{usedAct}(\text{behPr}(\mathcal{F}, pr))$ , and let  $s, s' \in S$  with  $(s, e, s') \in \delta$ . Since  $e$  is a used action of  $\text{behPr}(\mathcal{F}, pr)$ , it holds that  $\text{eval}_{pr}(\mu((s, e, s'))) = \#$  and  $s$  is reachable from  $s_0$  in  $\text{behPr}(\mathcal{F}, pr)$  (see Rem. 2.27). In particular, there exists a trace in  $\mathcal{F}$  for  $pr$  with  $e$  as last element (see Def. 2.29). Thus, Lemma 5.16 implies that  $pr \models \nu(e)$ . Since  $\text{behPr}(\text{LinFTS2FES}(\mathcal{F}), pr)$  has the identity labeling function, this is enough to show that  $e \in \text{usedAct}(\text{behPr}(\text{LinFTS2FES}(\mathcal{F}), pr))$  (see Rem. 4.32).

“ $\supseteq$ ” Let  $e \in \text{usedAct}(\text{behPr}(\text{LinFTS2FES}(\mathcal{F}), pr))$ . Then  $pr \models \nu(e)$  (see Rem. 4.32). We have to show that there exist  $s, s' \in S$  such that  $\text{eval}_{pr}(\mu((s, e, s'))) = \#$  and  $s$  is reachable from  $s_0$  in  $\text{behPr}(\mathcal{F}, pr)$ . Line 6 of Alg. 7 implies that  $\nu(e) = \mu((s, e, s')) \wedge (\bigvee_{(\tilde{s}, \tilde{e}, \tilde{s}) \in \delta} \nu(\tilde{e}))$  for  $(s, e, s') \in \delta$ . Thus,  $pr \models \mu((s, e, s'))$  and therefore  $\text{eval}_{pr}(\mu((s, e, s'))) = \#$ . Since  $pr \models \nu(e)$ , Lemma 5.16 implies that there exists a trace in  $\mathcal{F}$  for  $pr$  that has  $e$  as last element. In particular,  $s$  is reachable from  $s_0$  in  $\text{behPr}(\mathcal{F}, pr)$  (see Def. 2.29).  $\square$

Now we can prove that an acyclic FTS and its translation via Alg. 8 are trace equivalent.

PROPOSITION 5.18 (TRACE EQUIVALENCE OF  $\mathcal{F}$  AND  $\text{AcyFTS2FES}(\mathcal{F})$ ). *Let  $\mathcal{F}$  be an acyclic FTS. Then  $\text{AcyFTS2FES}(\mathcal{F})$  and  $\mathcal{F}$  are trace equivalent.*

PROOF. For  $\mathcal{F}$  and  $\text{AcyFTS2FES}(\mathcal{F})$  to be trace equivalent, they have to have the same features, actions and products, and for every product  $pr$  it has to hold that  $\text{Tr}(\mathcal{F}, pr) = \text{Tr}(\text{AcyFTS2FES}(\mathcal{F}), pr)$  (see Def. 4.48). By construction,  $\mathcal{F}$  and  $\text{AcyFTS2FES}(\mathcal{F})$  have the same features, actions and products. The remainder of the proof works in a similar way as the proof of Prop. 5.7, using Prop. 5.13.

Let  $pr \in \text{products}(\mathcal{F}) = \text{products}(\text{AcyFTS2FES}(\mathcal{F}))$ , and let  $\mathcal{F}^{\text{lin}}$  be the linearized FTS as in Line 2 of Alg. 8. By Prop. 5.15, it holds that  $\text{Tr}(\mathcal{F}^{\text{lin}}, pr) = \text{Tr}(\text{LinFTS2FES}(\mathcal{F}^{\text{lin}}), pr)$ . By construction, the set of traces of actions of  $\text{LinFTS2FES}(\mathcal{F}^{\text{lin}})$  for  $pr$  is equal to the set of traces of events of  $\text{AcyFTS2FES}(\mathcal{F})$  for  $pr$ . Furthermore, the same labeling function is applied to translate a trace of  $\mathcal{F}^{\text{lin}}$  for  $pr$  to a trace of  $\mathcal{F}$  for  $pr$ , and to translate a trace of events of  $\text{AcyFTS2FES}(\mathcal{F})$  for  $pr$  to a trace of actions of  $\text{AcyFTS2FES}(\mathcal{F})$  for  $pr$ . Thus, it holds that  $\text{Tr}(\mathcal{F}, pr) = \text{Tr}(\text{AcyFTS2FES}(\mathcal{F}), pr)$ .  $\square$

## 5.4 From FESs to Acyclic FTSs by Preserving Traces

To translate an FES to an acyclic FTS (Alg. 10), we proceed similarly as in Alg. 5: First translate an FES, whose underlying BES has an identity labeling function, to a TS and get the product knowledge (Alg. 9). Then relabel the transitions of the TS.

Algorithm 9 defines a translation from an FES to an FTS, as proven in Prop. 5.19. The algorithm works as follows. As in Alg. 4, the set of states of the translated FES is defined as the set of configurations, where we consider the configurations of the FES here, not the configurations of the underlying BES (Line 1). Intuitively, this ensures that the traces of the FES are preserved in

**Algorithm 10** FES2AcyFTS: from FES to acyclic FTS**Input:** FES  $\mathcal{E} = (\mathcal{M}, \mathcal{S}, \lambda, \nu)$  with  $\mathcal{S} = (E, \#, \mapsto, A, l)$ **Output:** acyclic FTS  $\mathcal{F} = (\mathcal{M}, \mathcal{T}, \mu)$  with  $\mathcal{T} = (S, A, s_0, \delta)$ 

- 1: let  $\mathcal{S}' := (E, \#, \mapsto, E, \text{id}_E)$ ,  $\delta := \emptyset$  ▷ BES with identity labeling function,  $\delta \subseteq S \times A \times S$
- 2:  $(\mathcal{M}, (S, E, s_0, \delta'), \mu) := \text{FESid2FTS}((\mathcal{M}, \mathcal{S}', \lambda, \nu))$
- 3: **for**  $(s, e, s') \in \delta'$  **do**
- 4:      $\delta := \delta \cup (s, l(e), s')$
- 5: **return**  $(\mathcal{M}, \mathcal{T}, \mu)$  with  $\mathcal{T} := (S, A, s_0, \delta)$

the FTS. To understand why we need to consider the configurations of the FES, recall that the traces of an FES are the traces of its behavioral products, which correspond to the configurations of its behavioral products. Every configuration of an FES is a configuration of a BES, obtained by restricting the BES of the FES to a set of events. To obtain an FTS that is trace equivalent to the FES, we need to ensure that all configurations of the FES's behavioral products, i.e., all configurations of the FES, are included. The transition relation is constructed in the same way as in Alg. 4 (Lines 2 and 3 of Alg. 9), and the feature expression associated with a transition is obtained from the feature expressions associated with the configurations from the source and target states (see Def. 4.39) of the transition (Line 4).

Algorithm 10 defines a translation from an FES to an acyclic FTS, as proven in Prop. 5.20. The algorithm works as follows. Analogously to the procedure in Alg. 5, we first use Alg. 9 to translate the FES with equal sets of actions and events and identity labeling function in the BES to an FTS (Line 2) with transition relation  $\delta'$ . This gives the set of states, the initial state, and the product knowledge of the output FTS. To obtain the transition relation of the output FTS, a transition is included for every transition in  $\delta'$  (as in Alg. 5). This transition is labeled with the corresponding action according to the labeling function (Lines 3 and 4).

We first prove that Alg. 9 translates an FES with identity labeling function to an FTS.

**PROPOSITION 5.19** (FESid2FTS( $\mathcal{E}$ ) IS AN FTS). *Let  $\mathcal{E}$  be an FES with identity labeling function. Then FESid2FTS( $\mathcal{E}$ ) is an FTS.*

**PROOF.** Let  $\mathcal{E}$  be an FES over the features  $N$  with identity labeling function, and let  $\text{FESid2FTS}(\mathcal{E}) = (\mathcal{M}, \mathcal{T}, \mu)$ , with  $\mathcal{T} = (S, E, s_0, \delta)$ . We have to show that:

- $\mathcal{M}$  is an FM over  $N$ ,
- $\mathcal{T} = (S, E, s_0, \delta)$  is a TS over  $E$ ,
- $\mu : S \times E \times S \rightarrow \text{FExp}(N)$  is a mapping.

By construction,  $\mathcal{M}$  is an FM over  $N$ . The only difference in Lines 1–3 of Alg. 9 compared to Alg. 4 is that the set of states is defined as the set of configurations of the FES instead of the configurations of the underlying BES. Thus, we can proceed as in the proof of Prop. 5.8 to see that  $\mathcal{T}$  is a TS over  $E$ . Since  $S$  is defined as the set of configurations of  $\mathcal{E}$ , we know that  $\mu : S \times E \times S \rightarrow \text{FExp}(N)$  is a mapping because  $\phi_s, \phi_{s'} \in \text{FExp}(N)$ , for  $s, s' \in S$ , see Def. 4.39.  $\square$

Next, we prove that Alg. 10 translates an FES to an acyclic FTS.

**PROPOSITION 5.20** (FES2AcyFTS( $\mathcal{E}$ ) IS AN ACYCLIC FTS). *Let  $\mathcal{E}$  be an FES. Then FES2AcyFTS( $\mathcal{E}$ ) is an acyclic FTS.*

**PROOF.** Similarly as in Prop. 5.19, it is possible to show that FES2AcyFTS( $\mathcal{E}$ ) is an FTS. The proof to show that FES2AcyFTS( $\mathcal{E}$ ) is acyclic works in the same way as in Prop. 5.8.  $\square$

Now we can prove that an FES and its translation via Alg. 10 are trace equivalent.



**PROPOSITION 5.21 (TRACE EQUIVALENCE OF  $\mathcal{E}$  AND  $\text{FESid2FTS}(\mathcal{E})$ ).** *Let  $\mathcal{E} = (\mathcal{M}, \mathcal{S}, \lambda, \nu)$  be an FES with identity labeling function. Then  $\text{FESid2FTS}(\mathcal{E})$  and  $\mathcal{E}$  are trace equivalent.*

**PROOF.** For  $\mathcal{E}$  and  $\text{FESid2FTS}(\mathcal{E})$  to be trace equivalent, they have to have the same features, actions and products, and for every product  $pr$  it has to hold that  $\text{Tr}(\mathcal{E}, pr) = \text{Tr}(\text{FESid2FTS}(\mathcal{E}), pr)$ . By construction,  $\mathcal{E}$  and  $\text{FESid2FTS}(\mathcal{E})$  have the same features, actions and products. Let  $pr \in \text{products}(\mathcal{E}) = \text{products}(\text{FESid2FTS}(\mathcal{E}))$ , let  $\text{FESid2FTS}(\mathcal{E}) = (\mathcal{M}, \mathcal{T}, \mu)$  with  $\mathcal{T} = (S, E, s_0, \delta)$ , let  $\text{behPr}(\text{FESid2FTS}(\mathcal{E}), pr) = (S', E, s_0, \delta')$ , and let  $\text{behPr}(\mathcal{E}, pr) = (E', \#, \mapsto', E, l')$ , where  $E' \doteq \{e \in E \mid pr \models \nu(e)\}$ . We have to show that  $\text{Tr}(\mathcal{E}, pr) = \text{Tr}(\text{FESid2FTS}(\mathcal{E}), pr)$ . Since the sets of actions and events of  $\mathcal{E}$  are equal and the labeling function is the identity function, it holds that  $\text{Tr}^{\text{ev}}(\mathcal{E}, pr) = \text{Tr}(\mathcal{E}, pr)$ . The proof uses the proof of Prop. 5.9.

“ $\subseteq$ ” We first show that for all  $tr \in \text{Tr}(\mathcal{E}, pr)$ , it holds that  $tr \in \text{Tr}(\text{FESid2FTS}(\mathcal{E}), pr)$ . We do this by induction on the length of  $tr$  for  $tr \in \text{Tr}(\mathcal{E}, pr)$ .

Base case: If  $tr$  is the empty trace, then  $tr \in \text{Tr}(\text{FESid2FTS}(\mathcal{E}), pr)$ , because the empty trace is a trace for every product of an FTS. If  $tr = \langle e \rangle$  for  $e \in E$ , then  $\{e\}$  is a configuration of  $\text{behPr}(\mathcal{E}, pr)$  by Lemma 4.38, so  $tr \in \text{Tr}(\text{FESid2FTS}(\mathcal{E}), pr)$  by Alg. 9.

Induction step: Assume that for all traces  $tr \in \text{Tr}(\mathcal{E}, pr)$  of length  $n$ , it holds that  $tr$  is also a trace of  $\text{FESid2FTS}(\mathcal{E})$  for  $pr$ . Let  $tr' \in \text{Tr}(\mathcal{E}, pr)$  be a trace of length  $n + 1$  with  $tr' = \langle e_1 \dots e_n \cdot e \rangle$  for  $e_i, e \in E$  with  $i \in \{1, \dots, n\}$ . By the definition of traces of actions of an FES (Def. 4.36),  $tr = \langle e_1 \dots e_n \rangle$  is a trace in  $\mathcal{E}$  for  $pr$ . By the induction hypothesis,  $tr$  is also a trace in  $\text{FESid2FTS}(\mathcal{E})$  for  $pr$ , i.e., there exists  $s \in S$  with  $s_0 \xrightarrow{tr} s$  and  $\text{eval}_{pr}(\mu(t)) = \#$ , for all transitions  $t$  of  $tr$ . We have to show that  $tr' \in \text{Tr}(\text{FESid2FTS}(\mathcal{E}), pr)$ . Since  $tr$  is a trace of actions in  $\text{FESid2FTS}(\mathcal{E})$  for  $pr$ , it suffices to show that (1) there exists  $s' \in S$  with  $(s, e, s') \in \delta$  and (2)  $\text{eval}_{pr}(\mu((s, e, s'))) = \#$ , i.e.,  $pr \models \mu((s, e, s'))$ .

Ad (1): By Lemma 4.38, the sets  $C = \{e_1, \dots, e_n\}$  and  $C' = C \cup \{e\}$  are configurations of  $\mathcal{E}$  for  $pr$ . Line 1 of Alg. 9 implies that there exist states  $s_C, s_{C'} \in S$  such that  $s_C = C$  and  $s_{C'} = C'$ . Since  $C' = C \cup \{e\}$ , it follows from Lines 2–3 of Alg. 9 that  $(s_C, e, s_{C'}) \in \delta$ . Furthermore, since  $C = \text{events}(tr)$ , where  $tr$  is a trace from  $s_0$  to  $s$ , Lemma 5.22 implies that  $s = C$ . Thus,  $(s = s_C, e, s_{C'}) \in \delta$ .

Ad (2): Since  $C$  and  $C'$  are configurations of  $\mathcal{E}$  for  $pr$ , it holds that  $pr \models \phi_C$  and  $pr \models \phi_{C'}$ , where  $\phi_C$  and  $\phi_{C'}$  are the feature expressions associated with  $C$  and  $C'$ , respectively, see Def. 4.39. Furthermore, by (1) we obtain  $(s, e, s') \in \delta$  with  $s = C$  and  $s' = C'$ . Thus,  $pr \models \phi_C \wedge \phi_{C'} = \mu((s, e, s'))$  (see Line 4 of Alg. 9).

“ $\supseteq$ ” Now we show that for all  $tr \in \text{Tr}(\text{FESid2FTS}(\mathcal{E}), pr)$ , it holds that  $tr \in \text{Tr}(\mathcal{E}, pr)$ .

Base case: If  $tr$  is the empty trace, then  $tr \in \text{Tr}(\mathcal{E}, pr)$  by definition (Def. 4.36). If  $tr = \langle e \rangle$ , then there exists a state  $s \in S'$  with  $(s_0, e, s) \in \delta'$ , i.e.,  $\text{eval}_{pr}(\mu((s_0, e, s))) = \#$ . By Lines 2 and 3 of Alg. 9 it holds that  $\{e\} = s \setminus s_0 = s \setminus \emptyset = s$ , which implies by Line 1 (and since  $\text{eval}_{pr}(\mu((s_0, e, s))) = \#$ ) that  $\{e\}$  is a configuration of  $\mathcal{E}$  for  $pr$  which is a trace of  $\mathcal{E}$  for  $pr$  by the definition of traces of actions (Def. 4.36).

Induction step: Assume that for all traces  $tr \in \text{Tr}(\text{FESid2FTS}(\mathcal{E}), pr)$  of length  $n$ , it holds that  $tr \in \text{Tr}(\mathcal{E}, pr)$ . Let  $tr' \in \text{Tr}(\text{FESid2FTS}(\mathcal{E}), pr)$  be a trace of length  $n + 1$  with  $tr' = \langle e_1 \dots e_n \cdot e \rangle$  for  $e_i, e \in E$  with  $i \in \{1, \dots, n\}$ , and let  $s \in S$  with  $s_0 \xrightarrow{tr'} s$ . It holds  $\text{eval}_{pr}(t) = \#$  for all transitions  $t$  in  $tr'$ . By the definition of traces of an FTS for a product (Def. 2.29), the sequence  $tr = \langle e_1 \dots e_n \rangle$  is a trace of  $\text{FESid2FTS}(\mathcal{E})$  for  $pr$  of length  $n$ . From the induction hypothesis we obtain that  $tr \in \text{Tr}(\mathcal{E}, pr)$ . We have to show that  $tr' \in \text{Tr}(\mathcal{E}, pr)$ .

Since  $tr$  is a trace of actions in  $\mathcal{E}$  for  $pr$ , it is enough to show (0')  $pr \models \nu(e)$ , (1')  $(e_i, e) \notin \#$  for all  $i \in \{1, \dots, n\}$  and (2')  $\forall X \subseteq E'$  with  $X \mapsto e$  it holds  $X \cap \{e_1, \dots, e_n\} \neq \emptyset$  (see Def. 4.36). By Lemma 5.22 it holds that  $s = \text{events}(tr')$ , and by Line 1 of Alg. 9 it holds that  $s \in \text{Config}(\mathcal{E})$ . Thus, the definition of a configuration of an FES (Def. 4.34) implies (1') and (2'). Since  $tr'$  is a trace in  $\text{FESid2FTS}(\mathcal{E})$  for  $pr$ , it holds that  $pr \models \mu((s', e, s)) = \phi_s \wedge \phi_{s'}$ , for  $(s, e, s') \in \delta$ , implying  $pr \models \phi_s$

**Algorithm 11** AcyFts2Bfm: from acyclic FTS to BFM**Input:** acyclic FTS  $\mathcal{F}$ **Output:** BFM  $\mathcal{B}$ 1:  $\mathcal{E} := \text{AcyFts2Fes}(\mathcal{F})$ 

▷ Get trace equivalent FES

2:  $\mathcal{B} := \text{bfm}(\mathcal{E})$ 

▷ Get isomorphic BFM

3: **return**  $\mathcal{B}$ 

and  $pr \models \phi_{s'}$ , see Def. 4.39. By Lemma 4.41, this implies  $s \in \text{Config}(\mathcal{E}, pr)$ , thus (0') holds (see Def. 4.34). □

In the proof of Alg. 9, we need a lemma that determines, for a state of a translated FES (with identity labeling function) via Alg. 9, the corresponding configuration of the FES.

LEMMA 5.22 (ON ALG. 9). *Let  $\mathcal{E}$  be an FES with identity labeling function, and let  $\text{FESid2Fts}(\mathcal{E})$  be its translation via Alg. 9. Let  $s \in S$  and  $tr \in \text{Tr}(\text{FESid2Fts}(\mathcal{E}))$ , with  $s_0 \xrightarrow{tr} s$ . Then  $s = \text{events}(tr)$ .*

PROOF. The proof is the same as the proof of Lemma 5.10, using a trace with respect to a product of  $\mathcal{E}$ . □

Now we can prove that an FES and its translation via Alg. 10 are trace equivalent.

PROPOSITION 5.23 (TRACE EQUIVALENCE OF  $\mathcal{E}$  AND  $\text{FES2AcyFts}(\mathcal{E})$ ). *Let  $\mathcal{E}$  be an FES. Then  $\text{FES2AcyFts}(\mathcal{E})$  and  $\mathcal{E}$  are trace equivalent.*

PROOF. For  $\mathcal{E}$  and  $\text{FES2AcyFts}(\mathcal{E})$  to be trace equivalent, they have to have the same features, actions and products, and for every product  $pr$  it has to hold that  $\text{Tr}(\mathcal{E}, pr) = \text{Tr}(\text{FES2AcyFts}(\mathcal{E}), pr)$ . By construction,  $\mathcal{E}$  and  $\text{FES2AcyFts}(\mathcal{E})$  have the same features, actions and products. Let  $pr \in \text{products}(\mathcal{E}) = \text{products}(\text{FES2AcyFts}(\mathcal{E}))$ , let  $\mathcal{E} = (\mathcal{M}, \mathcal{S}, \lambda, \nu)$  with  $\mathcal{S} = (E, \#, \mapsto, A, l)$ , and let  $\mathcal{S}' = (E, \#, \mapsto, E, \text{id}_E)$ .

By Prop. 5.21, it holds that  $\text{Tr}((\mathcal{M}, \mathcal{S}', \lambda, \nu)) = \text{Tr}(\text{FESid2Fts}(\mathcal{M}, \mathcal{S}', \lambda, \nu))$ . Furthermore, the labeling function  $l$  is used both to translate a trace of actions of  $(\mathcal{M}, \mathcal{S}', \lambda, \nu)$  for  $pr$  to a trace of actions of  $\mathcal{E}$  for  $pr$ , and to translate a trace of  $\text{FESid2Fts}(\mathcal{M}, \mathcal{S}', \lambda, \nu)$  to a trace of  $\text{FES2AcyFts}(\mathcal{E})$  for  $pr$ . □

## 5.5 From Linear FTSs to BFM by Preserving Traces

Algorithm 11 defines a translation from an acyclic FTS to a BFM, as proven in Prop. 5.24. The algorithm works as follows. First, the acyclic FTS is translated to a trace-equivalent FES using Alg. 8 (see Line 1 of Alg. 11). This FES is then translated to its isomorphic BFM using Alg. 12 (see Line 2 of Alg. 11), which is the algorithmic version of Def. 4.28. Getting the BFM view of an FES (Alg. 12) consists of splitting the knowledge aggregated in the FES to knowledge associated with single features. To determine which knowledge to associate to which feature, the event location knowledge (mostly written as  $\lambda$ ) of an FES can be used as defined in Not. 4.11. The structure of the isomorphic BFM is the same as the structure of the FM of the FES (see Lines 3–11 of Alg. 12). For every feature  $f$  of the FM, the inter-feature event constraints of  $f$ , the product constraints of  $f$ , and the BES modeling the behavior of  $f$  are extracted (see Lines 1 and 2 of Alg. 12). The mappings used in Alg. 12 are defined as in Not. 4.11.

We first prove that Alg. 11 translates an acyclic FTS to a BFM.

PROPOSITION 5.24 ( $\text{AcyFts2Bfm}(\mathcal{F})$  IS A BFM). *Let  $\mathcal{F}$  be an acyclic FTS. Then  $\text{AcyFts2Bfm}(\mathcal{F})$  is a BFM.*

---

**Algorithm 12** bfm: BFM view of FES (see Def. 4.28)
 

---

**Input:** FES  $\mathcal{E} = (\mathcal{M}, \mathcal{S}, \lambda, \nu)$  with  $\mathcal{M} = (f, \{\mathcal{G}_i\}_{i \in I^0}, \text{FC})$  and  $\mathcal{S} = (E, \#, \mapsto, A, l)$

**Output:** BFM  $\mathcal{B} = (f : \mathcal{S}', \{C_i\}_{i \in I^0}, \text{FC}, \text{EC}, \text{PC})$

```

1:  $\text{EC} = \text{interFtEc}_f(\mathcal{E})$ ,  $\text{PC} := \text{productC}_f(\mathcal{E})$  ▷ Extract knowledge related to  $f$ 
2:  $\mathcal{S}' := (\text{events}_f(\mathcal{E}), \text{confl}(\text{intraFtEc}_f(\mathcal{E})), \text{cause}(\text{intraFtEc}_f(\mathcal{E})), \text{actions}_f(\mathcal{E}), \text{labelFun}_f(\mathcal{E}))$  ▷ Extract BES of  $f$ 
3: switch  $\mathcal{G}_i$  do ▷ Do the procedure for every child of  $f$ 
4:   case  $\text{mnd}(\hat{\mathcal{M}})$ 
5:      $C_i := \text{mnd}(\text{bfm}(\hat{\mathcal{M}}))$  ▷ Call algorithm on  $\hat{\mathcal{M}}$  and include as mandatory sub-FM group
6:   case  $\text{opt}(\hat{\mathcal{M}})$ 
7:      $C_i := \text{opt}(\text{bfm}(\hat{\mathcal{M}}))$  ▷ Call algorithm on  $\hat{\mathcal{M}}$  and include as optional sub-FM group
8:   case  $\text{or}(\hat{\mathcal{M}}_j)_{j \in I^2}$ 
9:      $C_i := \text{or}(\text{bfm}(\hat{\mathcal{M}}_j)_{j \in I^2})$  ▷ Call algorithm on all  $\hat{\mathcal{M}}_j$  and include as or sub-FM group
10:  case  $\text{xor}(\hat{\mathcal{M}}_j)_{j \in I^2}$ 
11:     $C_i := \text{xor}(\text{bfm}(\hat{\mathcal{M}}_j)_{j \in I^2})$  ▷ Call algorithm on all  $\hat{\mathcal{M}}_j$ , and include as xor sub-FM group
12: return  $(f : \mathcal{S}', \{C_i\}_{i \in I^0}, \text{FC}, \text{EC}, \text{PC})$ 

```

---



---

**Algorithm 13** BFM2AcyFTS: from BFM to acyclic FTS
 

---

**Input:** BFM  $\mathcal{B}$

**Output:** acyclic FTS  $\mathcal{F}$

```

1:  $\mathcal{E} := \text{fes}(\mathcal{B})$  ▷ Get isomorphic FES
2:  $\mathcal{F} := \text{FES2AcyFTS}(\mathcal{E})$  ▷ Get trace equivalent acyclic FTS
3: return  $\mathcal{F}$ 

```

---



---

**Algorithm 14** fes: FES view of BFM (see Def. 4.26)
 

---

**Input:** BFM  $\mathcal{B}$

**Output:** FES  $\mathcal{E} = (\mathcal{M}, \mathcal{S}, \lambda, \nu)$

```

1:  $\mathcal{E} := (\text{fm}(\mathcal{B}), \text{bes}(\mathcal{B}), \text{eLocKnow}(\mathcal{B}), \text{prodKnow}(\mathcal{B}))$  ▷ Use Algorithms 15 to 18
2: return  $\mathcal{E}$ 

```

---

**PROOF.** In Prop. 5.14 we proved that Alg. 8 produces an FES, and Alg. 12 produces a BFM by its definition. Therefore, for an acyclic FTS  $\mathcal{F}$ , the translation  $\text{AcyFTS2BFM}(\mathcal{F}) = \text{bfm}(\text{AcyFTS2FES}(\mathcal{F}))$  is a BFM. □

Now we can prove that an acyclic FTS and its translation via Alg. 11 are trace equivalent.

**THEOREM 5.25 (TRACE EQUIVALENCE OF  $\mathcal{F}$  AND  $\text{AcyFTS2BFM}(\mathcal{F})$ ).** *Let  $\mathcal{F}$  be an acyclic FTS. Then  $\text{AcyFTS2BFM}(\mathcal{F})$  and  $\mathcal{F}$  are trace equivalent.*

**PROOF.** By Prop. 5.18,  $\mathcal{F}$  is trace equivalent to  $\text{AcyFTS2FES}(\mathcal{F})$ , and by Prop. 4.30 it holds that  $\text{bfm}(\text{AcyFTS2FES}(\mathcal{F}))$  and  $\text{AcyFTS2FES}(\mathcal{F})$  are isomorphic. Therefore, it holds that  $\text{AcyFTS2BFM}(\mathcal{F}) = \text{bfm}(\text{AcyFTS2FES}(\mathcal{F}))$  and  $\mathcal{F}$  are trace equivalent. □

## 5.6 From BFMs to FTSs by Preserving Traces

Algorithm 13 defines a translation from a BFM to an acyclic FTS, as proven in Prop. 5.26. The algorithm works as follows. First, we use the isomorphism in Def. 4.26 to translate a BFM to its isomorphic FES view (Line 1), which we then translate via Alg. 9 to an FTS (Line 2). To define

**Algorithm 15** fm: extract FM from a BFM (see Def. 4.17)**Input:** BFM  $\mathcal{B} = (f : \mathcal{S}, \{C_i\}_{i \in I^0}, \text{FC}, \text{EC}, \text{PC})$ **Output:** FM  $\mathcal{M} = (f', \{\mathcal{G}_i\}_{i \in I^0}, \text{FC}')$ 

```

1:  $f' := f, \text{FC}' := \text{FC}$ 
2: switch  $C_i$  do                                 $\triangleright$  Do the procedure for every child of  $f$ 
3:   case  $\text{mnd}(\hat{\mathcal{B}})$ 
4:      $\mathcal{G}_i := \text{mnd}(\text{fm}(\hat{\mathcal{B}}))$                  $\triangleright$  Call algorithm on  $\hat{\mathcal{B}}$  and include as mandatory sub-BFM group
5:   case  $\text{opt}(\hat{\mathcal{B}})$ 
6:      $\mathcal{G}_i := \text{opt}(\text{fm}(\hat{\mathcal{B}}))$                  $\triangleright$  Call algorithm on  $\hat{\mathcal{B}}$  and include as optional sub-BFM group
7:   case  $\text{or}(\mathcal{B}_j)_{j \in I^2}$ 
8:      $\mathcal{G}_i := \text{or}(\text{fm}(\mathcal{B}_j)_{j \in I^2})$          $\triangleright$  Call algorithm on all  $\hat{\mathcal{B}}_j$  and include as or sub-BFM group
9:   case  $\text{xor}(\mathcal{B}_j)_{j \in I^2}$ 
10:     $\mathcal{G}_i := \text{xor}(\text{fm}(\mathcal{B}_j)_{j \in I^2})$        $\triangleright$  Call algorithm on all  $\hat{\mathcal{B}}_j$  and include as xor sub-BFM group
11: return  $(f', \{\mathcal{G}_i\}_{i \in I^0}, \text{FC}')$ 

```

**Algorithm 16** bes: extract BES from a BFM (see Def. 4.20)**Input:** BFM  $\mathcal{B} = (f : \mathcal{S}, \{C_i\}_{i \in I^0}, \text{FC}, \text{EC}, \text{PC})$  with  $\mathcal{S} = (E, \#, \mapsto, A, l)$ **Output:** BES  $\mathcal{S}' = (E', \#, \mapsto', A', l')$ 

```

1:  $E' := E, \# := \# \cup \text{confl}(\text{EC}), \mapsto' := \mapsto \cup \text{cause}(\text{EC}), A' := A, l' := l$ 
2: switch  $C_i$  do                                 $\triangleright$  Do the procedure for every child of  $f$ 
3:   case  $\text{Uop}(\hat{\mathcal{B}})$ 
4:      $(\hat{E}, \hat{\#}, \hat{\mapsto}, \hat{A}, \hat{l}) := \text{bes}(\hat{\mathcal{B}})$          $\triangleright$  Get BES of sub-BFM
5:      $E' := E' \cup \hat{E}, \# := \# \cup \hat{\#}, \mapsto' := \mapsto' \cup \hat{\mapsto}, A' := A' \cup \hat{A}, l' := l' \cup \hat{l}$ 
6:   case  $\text{Op}(\mathcal{B}_i)_{i \in I^2}$ 
7:     for  $i \in I^2$  do
8:        $(\hat{E}_i, \hat{\#}_i, \hat{\mapsto}_i, \hat{A}_i, \hat{l}_i) := \text{bes}(\mathcal{B}_i)$      $\triangleright$  Get BES of sub-BFM
9:        $E' := E' \cup \hat{E}_i, \# := \# \cup \hat{\#}_i, \mapsto' := \mapsto' \cup \hat{\mapsto}_i, A' := A' \cup \hat{A}_i, l' := l' \cup \hat{l}_i$ 
10: return  $(E', \#, \mapsto', A', l')$ 

```

**Algorithm 17** eLocKnow: extract event location knowledge from a BFM (see Def. 4.22)**Input:** BFM  $\mathcal{B} = (f : \mathcal{S}, \{C_i\}_{i \in I^0}, \text{FC}, \text{EC}, \text{PC})$  with  $\mathcal{S} = (E, \#, \mapsto, A, l)$  over features  $N$ **Output:**  $\lambda \subseteq E \times N$ 

```

1:  $\lambda := \{(e, f) \mid e \in E\}$ 
2: switch  $C_i$  do                                 $\triangleright$  Do the procedure for every child of  $f$ 
3:   case  $\text{Uop}(\hat{\mathcal{B}})$ 
4:      $\lambda := \lambda \cup \text{eLocKnow}(\hat{\mathcal{B}})$                  $\triangleright$  Get event location knowledge of sub-BFM
5:   case  $\text{Op}(\mathcal{B}_i)_{i \in I^2}$ 
6:      $\lambda := \lambda \cup_{i \in I^2} \text{eLocKnow}(\mathcal{B}_i)$          $\triangleright$  Get event location knowledge of sub-BFMs
7: return  $\lambda$ 

```

the FES view of a BFM (Alg. 14) we provide algorithms for extracting the FM, the BES, the event location knowledge, and the product knowledge of a BFM in Algorithms 15 to 18, respectively. These algorithms reflect the definitions of these mappings provided in Definitions 4.17, 4.20, 4.22 and 4.24. To extract the FM of a BFM (Alg. 15), it is enough to delete all elements of the BFM that are not needed for the FM. The BES of a BFM can be extracted by collecting all events, conflicts

---

**Algorithm 18** prodKnow: extract product (configuration) knowledge from a BFM (see Def. 4.24)
 

---

**Input:** BFM  $\mathcal{B} = (f : \mathcal{S}, \{C_i\}_{i \in I^0}, \text{FC}, \text{EC}, \text{PC})$  with  $\mathcal{S} = (E, \#, \mapsto, A, l)$  over features  $N$

**Output:**  $\nu \subseteq E \times \text{FExp}(N)$

```

1:  $\nu := \text{PC}$ 
2: switch  $C_i$  do                                      $\triangleright$  Do the procedure for every child of  $f$ 
3:   case  $\text{Uop}(\hat{\mathcal{B}})$ 
4:      $\nu := \nu \cup \text{prodKnow}(\hat{\mathcal{B}})$                         $\triangleright$  Get product knowledge of sub-BFM
5:   case  $\text{Op}(\mathcal{B}_i)_{i \in I^2}$ 
6:      $\nu := \nu \cup \bigcup_{i \in I^2} \text{prodKnow}(\mathcal{B}_i)$           $\triangleright$  Get product knowledge of sub-BFMs
7: return  $\nu$ 

```

---

and causalities associated with all features of the BFM (Alg. 16). The event location knowledge of a BFM consists of all pairs of events and features such that the event is associated with the feature (Alg. 17), and the product knowledge consists of all product constraints associated with all features of the BFM (Alg. 18).

We first prove that Alg. 13 translates a BFM to an acyclic FTS.

**PROPOSITION 5.26** (BFM2AcyFTS( $\mathcal{B}$ ) IS AN FTS). *Let  $\mathcal{B}$  be a BFM. Then BFM2AcyFTS( $\mathcal{B}$ ) is an acyclic FTS.*

**PROOF.** By definition, the FES view  $\text{fes}(\mathcal{B})$  of a BFM  $\mathcal{B}$  is an FES. In Prop. 5.20 we proved that Alg. 10 produces an acyclic FTS. Therefore,  $\text{BFM2AcyFTS}(\mathcal{B}) = \text{FES2AcyFTS}(\text{fes}(\mathcal{B}))$  is an acyclic FTS.  $\square$

Now we can prove that a BFM and its translation via Alg. 13 are trace equivalent.

**THEOREM 5.27** (TRACE EQUIVALENCE OF  $\mathcal{B}$  AND BFM2AcyFTS( $\mathcal{B}$ )). *Let  $\mathcal{B}$  be a BFM. Then BFM2AcyFTS( $\mathcal{B}$ ) and  $\mathcal{B}$  are trace equivalent.*

**PROOF.** By Prop. 4.30, a BFM  $\mathcal{B}$  and its FES view  $\text{fes}(\mathcal{B})$  are isomorphic, and by Prop. 5.23 it holds that  $\text{fes}(\mathcal{B})$  and  $\text{FES2AcyFTS}(\text{fes}(\mathcal{B}))$  are trace equivalent. Therefore,  $\text{BFM2AcyFTS}(\mathcal{B}) = \text{FES2AcyFTS}(\text{fes}(\mathcal{B}))$  and  $\mathcal{B}$  are trace equivalent.  $\square$

## 6 PARALLEL COMPOSITION

In our empirical evaluation, we do not only compare BFMs and acyclic FTSs, but parallel compositions of acyclic FTSs to parallel compositions of BFMs. In this section, we establish the definitions of parallel compositions of BESs, FESs, BFMs, TSs, and FTSs in Sects. 6.1 to 6.5 and prove some properties that allow us to use the translation algorithms of Sect. 5 also for parallel compositions (Sect. 6.6).

### 6.1 Parallel Composition of BESs

First, we define synchronizing and free events of two BESs: these two categories of events are treated differently in a composition of BESs.

**Definition 6.1** (Synchronizing and free events). Let  $\mathcal{S}_1, \mathcal{S}_2$  be BESs with  $\mathcal{S}_i = (E_i, \#_i, \mapsto_i, A_i, l_i)$ ,  $i \in \{1, 2\}$ , and  $E_1 \cap E_2 = \emptyset$ .

- The *synchronizing events* of  $\mathcal{S}_i$  are  $E_i^s \doteq \{e \in E_i \mid l_i(e) \in A_1 \cap A_2\}$  for  $i \in \{1, 2\}$ .
- The *free (non-synchronizing) events* of  $\mathcal{S}_i$  are  $E_i^f \doteq E_i \setminus E_i^s$  for  $i \in \{1, 2\}$ .

Our definition of parallel composition of BESs is a slight variant of the one proposed by Langerak [73].

**Definition 6.2 (Parallel composition of BESs).** Let  $\mathcal{S}_1, \mathcal{S}_2$  be BESs with  $\mathcal{S}_i = (E_i, \#_i, \mapsto_i, A_i, l_i), i \in \{1, 2\}$ , and  $E_1 \cap E_2 = \emptyset$ . The *parallel composition*  $\mathcal{S}_1 \parallel \mathcal{S}_2$  of  $\mathcal{S}_1$  and  $\mathcal{S}_2$  is the BES  $(E, \#, \mapsto, A, l)$  with

- $E \doteq (E_1^f \times \{*\}) \cup (\{*\} \times E_2^f) \cup \{(e, f) \in E_1^s \times E_2^s \mid l_1(e) = l_2(f)\}$ ;
- $\# \doteq \{(e, f), (e', f') \in E \mid (e \#_1 e') \vee (f \#_2 f') \vee (e = e' \neq * \wedge f \neq f') \vee (f = f' \neq * \wedge e \neq e')\}$ ;
- $\mapsto^{pre} \doteq \{(X, (e, f)) \in \mathcal{P}(E) \times E \mid (\exists X_1 \subseteq E_1. (X_1 \mapsto_1 e \wedge X = \{(e_i, f_j) \in E \mid e_i \in X_1\})) \vee (\exists X_2 \subseteq E_2. (X_2 \mapsto_2 f \wedge X = \{(e_i, f_j) \in E \mid f_j \in X_2\}))\}$ ;
- $\mapsto \doteq \mapsto^{pre} \setminus \{X \mapsto^{pre}(e, f) \mid \exists Y \subseteq E. Y \mapsto^{pre}(e, f) \wedge X \subset Y\}$ ;
- $A \doteq A_1 \cup A_2$ ; and
- $l : E \rightarrow A$  with  $l((e, f)) \doteq \begin{cases} l_2(f) & \text{if } e = * \\ l_1(e) & \text{otherwise} \end{cases}$

The set of events  $E$  is composed of (i) all free events of  $E_1$ , paired with an auxiliary symbol  $*$ , (ii) all free events of  $E_2$ , paired with an auxiliary symbol  $*$ , and (iii) pairs of events in  $E_1$  and  $E_2$  that have the same label; in particular, every event in  $E_1$  with label  $a \in A$  is paired with every event in  $E_2$  with label  $a$  and the other way around. Two events are in conflict in  $\mathcal{S}_1 \parallel \mathcal{S}_2$  if and only if (i) the first components are in conflict in  $E_1$ , or (ii) the second components are in conflict in  $E_2$ , or (iii) they share a common component unequal to  $*$ . A set  $X \subseteq E$  is causal for an event  $(e, f) \in E$  if and only if either (i) the projection of  $X$  on  $E_1$  is causal for  $e$  in  $\mathcal{S}_1$ , or (ii) the projection of  $X$  on  $E_2$  is causal for  $f$  in  $\mathcal{S}_2$ , and in both cases, there exists no  $Y \subseteq E$  with this property such that  $X \subset Y$ . The set of actions  $A$  is composed of the sets of actions of the components. The labeling function is equal to  $l_2$  if the first component is  $*$  and equal to  $l_1$  otherwise.

**Example 6.3 (Parallel composition with the BES of the feature cleaning of the cleaning robot SPL).** Let  $\mathcal{S}_c \doteq (E_c, \#_c, \mapsto_c, A_c, l_c)$  be the BES of the feature cleaning of the cleaning robot SPL from Example 4.6. That is,  $E_c = \{\text{goAround}, \text{move}_1, \text{move}_2, \text{clean}\}$ ,  $\#_c = \{(\text{goAround}, \text{move}_1)\}$ ,  $\mapsto_c = \{(\{\text{goAround}, \text{move}_1\}, \text{move}_2), (\{\text{move}_2\}, \text{clean})\}$ ,  $A_c = \{\text{goAround}, \text{move}, \text{clean}\}$ ,  $l_c(e) = e$  if  $e \in \{\text{goAround}, \text{clean}\}$ , and  $l_c(e) = \text{move}$  if  $e \in \{\text{move}_1, \text{move}_2\}$ . Furthermore, consider the new BES  $\mathcal{S}_{new} = (E_n, \emptyset, \mapsto_n, A_n, l_n)$  with which  $\mathcal{S}_c$  should be composed, where  $E_n \doteq \{\text{move}_3, \text{survey}\}$ ,  $\mapsto_n \doteq \{(\{\text{move}_3\}, \text{survey})\}$ ,  $A_n \doteq \{\text{move}, \text{survey}\}$ ,  $l_n(\text{move}_3) = \text{move}$  and  $l_n(\text{survey}) = \text{survey}$ .

Then  $\mathcal{S}_c \parallel \mathcal{S}_{new} = (E, \#, \mapsto, A, l)$  with

- $E \doteq \{(\text{goAround}, *), (\text{clean}, *), (*, \text{survey}), (\text{move}_1, \text{move}_3), (\text{move}_2, \text{move}_3)\}$ ;
- $(\text{goAround}, *) \# (\text{move}_1, \text{move}_3), (\text{move}_1, \text{move}_3) \# (\text{move}_2, \text{move}_3)$ ;
- $\{(\text{move}_1, \text{move}_3), (\text{goAround}, *)\} \mapsto (\text{move}_2, \text{move}_3), \{(\text{move}_2, \text{move}_3)\} \mapsto (\text{clean}, *), \{(\text{move}_1, \text{move}_3), (\text{move}_2, \text{move}_3)\} \mapsto (*, \text{survey})$ ;
- $A \doteq \{\text{move}, \text{goAround}, \text{clean}, \text{survey}\}$  and;
- $l((e, f)) \doteq \begin{cases} \text{move} & \text{if } (e, f) \in \{(\text{move}_1, \text{move}_3), (\text{move}_2, \text{move}_3)\} \\ \text{goAround} & \text{if } (e, f) = (\text{goAround}, *) \\ \text{clean} & \text{if } (e, f) = (\text{clean}, *) \\ \text{survey} & \text{if } (e, f) = (*, \text{survey}) \end{cases}$

Note that there does not exist a trace with both  $(\text{move}_1, \text{move}_3)$  and  $(\text{move}_2, \text{move}_3)$  (because they are in conflict) even though there exists a causality  $\{(\text{move}_1, \text{move}_3), (\text{goAround}, *)\} \mapsto (\text{move}_2, \text{move}_3)$ .

**Remark 6.4 (On the parallel composition of BESs).**

- Our definition of parallel composition of BESs slightly differs from the one of Langerak [73]. As in the definition of BESs (Def. 2.31), we exclude all causalities  $X \mapsto^{pre}(e, f)$  for which



there exists  $Y \mapsto^{pre}(e, f)$  with  $X \subset Y$  because the causality  $X \mapsto^{pre}(e, f)$  would disallow traces using events in  $Y \setminus X$  (see also Rem. 2.39).

- The definition of  $\mapsto$  in Def. 6.2 produces a causality relation (Def. 2.31): The second condition of Def. 2.31 holds by definition. Let  $X \subseteq E$  and  $(e, f) \in E$  with  $X \mapsto (e, f)$  such that there exists  $X_1 \subseteq E_1$  with  $X_1 \mapsto_1 e$  and  $X = \{(e_i, f_j) \in E \mid e_i \in X_1\}$  (the other case follows similarly). For  $(e_i, f_j), (e_k, f_m) \in X$ , with  $(e_i, f_j) \neq (e_k, f_m)$  either  $e_i \neq e_k$  or  $e_i = e_k \neq * \wedge f_j \neq f_m$ . In the first case, the fact that  $X_1 \mapsto_1 e$  and  $e_i, e_k \in X_1$  implies  $e_i \#_1 e_k$ , which in turn implies  $(e_i, f_j) \# (e_k, f_m)$ . In the second case, the definition of  $\#$  implies  $(e_i, f_j) \# (e_k, f_m)$ .
- The labeling function  $l$  is the union of the component labeling functions: For an event  $(*, f) \in E$  it is equal to  $l_2$ , for an event  $(e, *)$  it is equal to  $l_1$ , and for an event  $(e, f) \in E$  with  $e \neq * \neq f$  it holds  $l_1(e) = l_2(f)$  by the definition of  $E$ , so  $l$  is equal to  $l_1$  and  $l_2$  in this case.

LEMMA 6.5 (ON CONFLICT AND CAUSALITY OF THE PARALLEL COMPOSITION OF BESs). *Let  $\mathcal{S}_1, \mathcal{S}_2$  be BESs with  $\mathcal{S}_i = (E_i, \#_i, \mapsto_i, A_i, l_i)$ ,  $i \in \{1, 2\}$ , and  $E_1 \cap E_2 = \emptyset$ , and let  $\mathcal{S}_1 \parallel \mathcal{S}_2$  be their parallel composition. Then we make the following observations about the conflict and causality relation of  $\mathcal{S}_1 \parallel \mathcal{S}_2$ :*

- (1) *All conflicts of  $\mathcal{S}_1$  and  $\mathcal{S}_2$  are preserved in  $\mathcal{S}_1 \parallel \mathcal{S}_2$ . That is, if  $e_1, e_2 \in E_1$  are in conflict in  $E_1$ , then all events in  $E$  containing  $e_1$  are in conflict with all events in  $E$  containing  $e_2$  (and similarly for events in  $E_2$ ).*
- (2) *The parallel composition does not introduce new conflicts in  $\mathcal{S}_1$  and  $\mathcal{S}_2$  in the following sense: If  $(e, f) \in E$  and  $(e', f') \in E$  are not in conflict in  $\mathcal{S}_1 \parallel \mathcal{S}_2$ , then  $(e, e') \notin \#_1$  and  $(f, f') \notin \#_2$ .*
- (3) *For every  $X_1 \mapsto_1 e$ , there exists an  $X \mapsto (e, f)$  for every  $f \in E_2$  such that  $(e, f) \in E$ . Similarly, for every  $X_2 \mapsto_2 f$ , there exists an  $X \mapsto (e, f)$  for every  $e \in E_1$  such that  $(e, f) \in E$ .*
- (4) *The parallel composition does not introduce new initial elements in the following sense: For every  $(e, f) \in E$  that is the first event of a trace in  $\mathcal{S}_1 \parallel \mathcal{S}_2$  (i.e.,  $\nexists X \subseteq E. X \mapsto (e, f)$ ), either  $e$  (resp.  $f$ ) is the first event of a trace in  $\mathcal{S}_1$  (resp.  $\mathcal{S}_2$ ), or  $e = *$  (resp.  $f = *$ ).*

PROOF. (1), (2), and (3) follow immediately from the definition of parallel composition of BESs (Def. 6.2).

For (4), consider the case where  $e \neq *$  ( $f \neq *$  follows similarly). We have to show that there does not exist  $X_1 \subseteq E_1$  with  $X_1 \mapsto_1 e$ . Assume (towards contradiction) that there exists  $X_1 \subseteq E_1$  with  $X_1 \mapsto_1 e$  and let  $\tilde{X} \doteq \{(e_i, f_j) \in E \mid e_i \in X_1\}$ . By definition of  $\mapsto$ , it holds  $\tilde{X} \mapsto (e, f_j)$  for all  $f_j \in E_2$  with  $(e, f_j) \in E$  (see (3)). In particular,  $\tilde{X} \mapsto (e, f)$ , a contradiction to  $(e, f)$  is the first element of a trace in  $\mathcal{S}_1 \parallel \mathcal{S}_2$ .  $\square$

Notation 6.6 (On BES traces).

- Let  $A$  be a set and let  $tr = \langle a_1 \cdot \dots \cdot a_n \rangle$  and  $tr' = \langle a'_1 \cdot \dots \cdot a'_m \rangle$  be sequences of elements  $a_i, a'_j \in A$  for  $i \in \{1, \dots, n\}$  and  $j \in \{1, \dots, m\}$ . Then  $tr \cdot tr'$  denotes the concatenation  $\langle a_1 \cdot \dots \cdot a_n \cdot a'_1 \cdot \dots \cdot a'_m \rangle$  of the sequences  $tr$  and  $tr'$ .
- Let  $tr = \langle e_1 \cdot \dots \cdot e_n \rangle$  be an event trace in a BES with labeling function  $l$ . Then we denote by  $l(tr)$  the action trace  $\langle l(e_1) \cdot \dots \cdot l(e_n) \rangle$ .
- Let  $tr$  be a sequence of actions or events of a BES. Then we denote by  $tr[i]$  the  $i$ -th action (resp. event) of  $tr$ . The operation  $length(tr)$  returns the number of actions (resp. events) of  $tr$ . The operation  $pop(tr)$  gives the first action (resp. event) of  $tr$  and deletes it from  $tr$ . The operation  $append(tr, a)$  is defined as  $append(tr, a) \doteq tr \cdot \langle a \rangle$  for an action (resp. event)  $a$ .

Definition 6.7 (Projection of traces).

- Let  $X$  be a TS, FTS, FES, or BFM over the set of actions  $A$ , let  $A' \subseteq A$ , and let  $tr$  be a trace (of actions) of  $X$ . The *projection* of  $tr$  on  $A'$ , written  $tr|_{A'}$ , is inductively defined as follows:

$$\begin{aligned} \langle \rangle|_{A'} &\doteq \langle \rangle \\ (tr' \cdot \langle a \rangle)|_{A'} &\doteq tr'|_{A'} \text{ if } a \notin A' \\ (tr' \cdot \langle a \rangle)|_{A'} &\doteq tr'|_{A'} \cdot \langle a \rangle \text{ if } a \in A' \end{aligned}$$

where  $tr'$  is a trace (of actions) of  $X$  and  $a \in A$ .

- Let  $\mathcal{S}_1, \mathcal{S}_2$  be BESs with  $\mathcal{S}_i = (E_i, \#_i, \mapsto_i, A_i, l_i)$ ,  $i \in \{1, 2\}$ , and  $E_1 \cap E_2 = \emptyset$ . Let  $\mathcal{S}_1 \parallel \mathcal{S}_2 = (E, \#, \mapsto, A, l)$ , and let  $tr$  be an event trace of  $\mathcal{S}_1 \parallel \mathcal{S}_2$ . The *projection* of  $tr$  on  $E_i$ ,  $i \in \{1, 2\}$ , written  $tr|_{E_i}$ , is defined inductively as follows:

$$\begin{aligned} \langle \rangle|_{E_i} &\doteq \langle \rangle \text{ for } i \in \{1, 2\} \\ (tr' \cdot \langle (e, f) \rangle)|_{E_1} &\doteq tr'|_{E_1} \text{ if } e \notin E_1 \\ (tr' \cdot \langle (e, f) \rangle)|_{E_1} &\doteq tr'|_{E_1} \cdot \langle e \rangle \text{ if } e \in E_1 \\ (tr' \cdot \langle (e, f) \rangle)|_{E_2} &\doteq tr'|_{E_2} \text{ if } f \notin E_2 \\ (tr' \cdot \langle (e, f) \rangle)|_{E_2} &\doteq tr'|_{E_2} \cdot \langle f \rangle \text{ if } f \in E_2 \end{aligned}$$

where  $tr'$  is an event trace of  $\mathcal{S}_1 \parallel \mathcal{S}_2$  and  $(e, f) \in E$ .

In the following proposition, we prove the following: The projection of a trace of actions (resp. events) of a parallel composition of BESs is a trace of actions (resp. events) in each component BES. On the other hand, give a sequence of actions (resp. events)  $tr$  of a parallel composition of BESs such that the projection of  $tr$  on each component BES is a trace in this component, the sequence  $tr$  is also a trace in the parallel composition.

**PROPOSITION 6.8 (PROJECTION OF BES TRACES).** *Let  $\mathcal{S}_1, \mathcal{S}_2$  be BESs with  $\mathcal{S}_i = (E_i, \#_i, \mapsto_i, A_i, l_i)$ ,  $i \in \{1, 2\}$ , and  $E_1 \cap E_2 = \emptyset$ . Let  $\mathcal{S}_1 \parallel \mathcal{S}_2 = (E, \#, \mapsto, A, l)$ .*

- (1) *Let  $tr$  be a sequence of events in  $E$ . Then  $tr \in \text{Tr}^{\text{ev}}(\mathcal{S}_1 \parallel \mathcal{S}_2) \Leftrightarrow \forall i \in \{1, 2\}. tr|_{E_i} \in \text{Tr}^{\text{ev}}(\mathcal{S}_i)$ .*
- (2) *Let  $tr$  be a sequence of actions in  $A$ . Then  $tr \in \text{Tr}(\mathcal{S}_1 \parallel \mathcal{S}_2) \Leftrightarrow \forall i \in \{1, 2\}. tr|_{A_i} \in \text{Tr}(\mathcal{S}_i)$ .*

**PROOF.** (1) “ $\Rightarrow$ ” Let  $tr$  be a sequence of events in  $E$ . We show  $tr \in \text{Tr}^{\text{ev}}(\mathcal{S}_1 \parallel \mathcal{S}_2) \Rightarrow tr|_{E_1} \in \text{Tr}^{\text{ev}}(\mathcal{S}_1)$  by induction over the length of  $tr$ ; the proof for  $E_2$  works similarly.

Base case: If  $tr$  is the empty trace, the projection  $tr|_{E_1} = \langle \rangle$  is a trace in  $\text{Tr}^{\text{ev}}(\mathcal{S}_1)$ . Let  $tr = \langle (e, f) \rangle$ . Then  $tr|_{E_1}$  is the empty trace if  $e = *$  and  $\langle e \rangle$  if  $e \neq *$ . In the first case,  $tr|_{E_1}$  is an event trace of  $\mathcal{S}_1$ . In the second case, observation (4) of Lemma 6.5 implies that  $tr|_{E_1} \in \text{Tr}^{\text{ev}}(\mathcal{S}_1)$ .

Induction step: Assume  $tr \in \text{Tr}^{\text{ev}}(\mathcal{S}_1 \parallel \mathcal{S}_2) \Rightarrow tr|_{E_1} \in \text{Tr}^{\text{ev}}(\mathcal{S}_1)$  holds for all traces of length  $n$ . Let  $tr' \in \text{Tr}^{\text{ev}}(\mathcal{S}_1 \parallel \mathcal{S}_2)$  be a trace of length  $n+1$  with  $tr' = \langle (e_1, f_1) \rangle \dots \langle (e_n, f_n) \rangle \cdot \langle (e_{n+1}, f_{n+1}) \rangle$ , and let  $tr \doteq \langle (e_1, f_1) \rangle \dots \langle (e_n, f_n) \rangle$ . Then  $tr|_{E_1} \in \text{Tr}^{\text{ev}}(\mathcal{S}_1)$  by the induction hypothesis.

By the definition of projection (Def. 6.7), it holds  $tr'|_{E_1} = tr|_{E_1}$  if  $e_{n+1} = *$  and  $tr'|_{E_1} = tr|_{E_1} \cdot \langle e_{n+1} \rangle$  otherwise. In the former case, it holds  $tr'|_{E_1} = tr|_{E_1} \in \text{Tr}^{\text{ev}}(\mathcal{S}_1)$ . In the latter case, since  $tr|_{E_1} \in \text{Tr}^{\text{ev}}(\mathcal{S}_1)$ , it is enough to show (1)  $(e, e_{n+1}) \notin \#_1$  for all  $e \in \text{events}(tr|_{E_1})$ , and (2) for all  $X_1 \subseteq E_1$  with  $X_1 \mapsto_1 e_{n+1}$  it holds  $\text{events}(tr|_{E_1}) \cap X_1 \neq \emptyset$ . The condition in (1) follows immediately from (2) of Lemma 6.5 since  $tr'$  is a trace in  $\mathcal{S}_1 \parallel \mathcal{S}_2$ .

Ad (2): Since  $tr' \in \text{Tr}^{\text{ev}}(\mathcal{S}_1 \parallel \mathcal{S}_2)$ , it holds for all  $X \subseteq E$  with  $X \mapsto (e_{n+1}, f_{n+1})$  that  $\text{events}(tr) \cap X \neq \emptyset$ . Let  $X_1 \subseteq E_1$  with  $X_1 \mapsto_1 e_{n+1}$ , and let  $X \doteq \{(e_i, f_j) \in E \mid e_i \in X_1\}$ . Then  $X \mapsto (e_{n+1}, f)$  for all  $(e_{n+1}, f) \in E$ , see (3) of Lemma 6.5. In particular, it holds  $X \mapsto (e_{n+1}, f_{n+1})$  and thus  $\text{events}(tr) \cap X \neq \emptyset$ . Let  $i \in \{1, \dots, n\}$  with  $(e_i, f_i) \in X \cap \text{events}(tr)$ . Then  $e_i \in X_1$  by the definition of  $X$  (in particular,  $e_i \neq *$ ) and  $e_i \in \text{events}(tr|_{E_1})$  by the definition of projection (Def. 6.7) since  $e_i \neq *$ . That is,  $e_i \in X_1 \cap \text{events}(tr|_{E_1})$ .

(1) “ $\Leftarrow$ ” We show  $\forall i \in \{1, 2\}. tr|_{E_i} \in \text{Tr}^{\text{ev}}(\mathcal{S}_i) \Rightarrow tr \in \text{Tr}^{\text{ev}}(\mathcal{S}_1 \parallel \mathcal{S}_2)$  by induction on the length of  $tr$ .

Base case: If  $tr$  is empty, then  $tr \in \text{Tr}^{\text{ev}}(\mathcal{S}_1 \parallel \mathcal{S}_2)$ . Let  $tr = \langle (e, f) \rangle$  with  $tr|_{E_i} \in \text{Tr}^{\text{ev}}(\mathcal{S}_i)$  for  $i \in \{1, 2\}$ . We have to show that there does not exist  $X \subseteq E$  with  $X \mapsto (e, f)$  for the three cases (i)  $e = *$  and  $f \neq *$ , (ii)  $e \neq *$  and  $f = *$ , and (iii)  $e \neq *$  and  $f \neq *$ . All three follow directly from the construction of  $\mapsto$  (Def. 6.7) because  $tr|_{E_i} \in \text{Tr}^{\text{ev}}(\mathcal{S}_i)$  for  $i \in \{1, 2\}$ .

Induction step: Assume  $\forall i \in \{1, 2\}. tr|_{E_i} \in \text{Tr}^{\text{ev}}(\mathcal{S}_i) \Rightarrow tr \in \text{Tr}^{\text{ev}}(\mathcal{S}_1 \parallel \mathcal{S}_2)$  holds for all  $tr$  of length  $n$ . Let  $tr' = \langle (e_1, f_1) \cdot \dots \cdot (e_n, f_n) \cdot (e_{n+1}, f_{n+1}) \rangle$  be a sequence of events in  $E$  of length  $n+1$  such that  $tr'|_{E_i} \in \text{Tr}^{\text{ev}}(\mathcal{S}_i)$  for  $i \in \{1, 2\}$ , and let  $tr \doteq \langle (e_1, f_1) \cdot \dots \cdot (e_n, f_n) \rangle$ . Then trivially  $tr|_{E_i} \in \text{Tr}^{\text{ev}}(\mathcal{S}_i)$  for  $i \in \{1, 2\}$ , and  $tr \in \text{Tr}^{\text{ev}}(\mathcal{S}_1 \parallel \mathcal{S}_2)$  by the induction hypothesis. Since  $tr \in \text{Tr}^{\text{ev}}(\mathcal{S}_1 \parallel \mathcal{S}_2)$ , it is enough to show (1)  $((e_j, f_j), (e_{n+1}, f_{n+1})) \notin \#$  for all  $j \in \{1, \dots, n\}$ , and (2) for all  $X \subseteq E$  with  $X \mapsto (e_{n+1}, f_{n+1})$  it holds  $\text{events}(tr) \cap X \neq \emptyset$ .

Ad (1): Let  $I = \{i \in \{1, \dots, n\} \mid f_i = *\}$ ,  $J = \{j \in \{1, \dots, n\} \mid e_j = *\}$ , and  $K = \{1, \dots, n\} \setminus (I \cup J)$ . Consider  $(e_i, f_i)$  with  $i \in I$ , i.e.,  $f_i = *$ . By the definition of  $\#$  (Def. 6.2), the event  $(e_i, *)$  can only be in conflict with  $(e_{n+1}, f_{n+1})$  if  $e_{n+1} \neq *$  and  $e_i \# e_{n+1}$ . Assume (towards contradiction) that  $e_{n+1} \neq *$  and  $e_i \# e_{n+1}$ . Since  $e_i \neq * \neq e_{n+1}$ , it holds  $e_i, e_{n+1} \in \text{events}(tr'|_{E_1})$  by the definition of projection (Def. 6.7). However,  $tr'|_{E_1} \in \text{Tr}^{\text{ev}}(\mathcal{S}_1)$ , so its set of events is conflict free, a contradiction. Therefore,  $((e_i, f_i), (e_{n+1}, f_{n+1})) \notin \#$  for all  $i \in I$ . Similarly, it follows  $((e_j, f_j), (e_{n+1}, f_{n+1})) \notin \#$  for all  $j \in J$ .

Consider  $(e_k, f_k)$  with  $k \in K$ , i.e.,  $e_k \neq * \neq f_k$ . Then  $e_k \in \text{events}(tr'|_{E_1})$  and  $f_k \in \text{events}(tr'|_{E_2})$ . Assume (towards contradiction) that  $(e_k, f_k) \# (e_{n+1}, f_{n+1})$ . Then one of the following four cases holds: (i)  $e_k \#_1 e_{n+1}$ , (ii)  $f_k \#_2 f_{n+1}$ , (iii)  $e_k = e_{n+1}$  and  $f_k \neq f_{n+1}$ , or (iv)  $f_k = f_{n+1}$  and  $e_k \neq e_{n+1}$ . In case (i) it follows in particular that  $e_{n+1} \neq *$ , so  $e_{n+1} \in \text{events}(tr'|_{E_1})$ . But  $tr'|_{E_1} \in \text{Tr}^{\text{ev}}(\mathcal{S}_1)$ , so its set of events is conflict free, a contradiction to  $e_k \#_1 e_{n+1}$ . In case (iii) it holds  $e_k, e_{n+1} \in \text{events}(tr'|_{E_1})$  with  $e_k = e_{n+1}$ , a contradiction to  $tr'|_{E_1} \in \text{Tr}^{\text{ev}}(\mathcal{S}_1)$  since events are unique in event traces (Def. 2.36). In the same way, we get a contradiction in cases (ii) and (iv). Therefore,  $((e_k, f_k), (e_{n+1}, f_{n+1})) \notin \#$  for all  $k \in K$ .

Ad (2): We have to show that for all  $X \subseteq E$  with  $X \mapsto (e_{n+1}, f_{n+1})$  it holds  $\text{events}(tr) \cap X \neq \emptyset$ . Let  $X \subseteq E$  with  $X \mapsto (e_{n+1}, f_{n+1})$ . Then either (i) there exists  $X_1 \subseteq E_1$  with  $X_1 \mapsto_1 e_{n+1}$  and  $X = \{(e, f) \in E \mid e \in X_1\}$ , or (ii) there exists  $X_2 \subseteq E_2$  with  $X_2 \mapsto_2 f_{n+1}$  and  $X = \{(e, f) \in E \mid f \in X_2\}$ . We consider case (i), the other case follows similarly. Let  $X_1 \subseteq E_1$  with  $X_1 \mapsto_1 e_{n+1}$  and  $X = \{(e, f) \in E \mid e \in X_1\}$ . Then  $e_{n+1} \neq *$  and thus  $e_{n+1} \in \text{events}(tr'|_{E_1})$ . Since  $tr'|_{E_1} \in \text{Tr}^{\text{ev}}(\mathcal{S}_1)$ , it holds  $X_1 \cap \text{events}(tr|_{E_1}) \neq \emptyset$ . Let  $j \in \{1, \dots, n\}$  with  $e_j \in X_1 \cap \text{events}(tr|_{E_1})$ . By definition of  $X$ , it holds  $(e_j, f_j) \in X$  and trivially,  $(e_j, f_j) \in \text{events}(tr)$ . Thus,  $(e_j, f_j) \in X \cap \text{events}(tr)$ .

(2) “ $\Rightarrow$ ” For a sequence  $tr$  of actions in  $A$ , we have to show  $tr \in \text{Tr}(\mathcal{S}_1 \parallel \mathcal{S}_2) \Rightarrow \forall i \in \{1, 2\}. tr|_{A_i} \in \text{Tr}(\mathcal{S}_i)$ . Let  $tr \in \text{Tr}(\mathcal{S}_1 \parallel \mathcal{S}_2)$ , and let  $tr^{\text{ev}} \in \text{Tr}^{\text{ev}}(\mathcal{S}_1 \parallel \mathcal{S}_2)$  with  $l(tr^{\text{ev}}) = tr$ . By item (1) of Prop. 6.8 it holds  $tr^{\text{ev}}|_{E_i} \in \text{Tr}^{\text{ev}}(\mathcal{S}_i)$  for  $i \in \{1, 2\}$ . Then  $l(tr^{\text{ev}}|_{E_i}) \in \text{Tr}(\mathcal{S}_i)$  for  $i \in \{1, 2\}$  (see Def. 2.36). Furthermore, by the definition of projection (Def. 6.7) and event and action traces (Def. 2.36), it holds  $l(tr^{\text{ev}}|_{E_i}) = tr|_{A_i}$  for  $i \in \{1, 2\}$ . Therefore,  $tr|_{A_i} \in \text{Tr}(\mathcal{S}_i)$  for  $i \in \{1, 2\}$ .

(2) “ $\Leftarrow$ ” For a sequence  $tr$  of actions in  $A$ , we show  $\forall i \in \{1, 2\}. tr|_{A_i} \in \text{Tr}(\mathcal{S}_i) \Rightarrow tr \in \text{Tr}(\mathcal{S}_1 \parallel \mathcal{S}_2)$ . Let  $tr$  be a sequence of actions in  $A$  such that  $tr|_{A_i} \in \text{Tr}(\mathcal{S}_i)$  for  $i \in \{1, 2\}$ , and let  $tr_i \in \text{Tr}^{\text{ev}}(\mathcal{S}_i)$  such that  $l_i(tr_i) = tr|_{A_i}$  for  $i \in \{1, 2\}$ . Then Lemma 6.9.(1) implies that there exists a sequence of events in  $\mathcal{S}_1 \parallel \mathcal{S}_2$ , denoted  $tr^{\text{ev}}$ , such that  $tr^{\text{ev}}|_{E_i} = tr_i \in \text{Tr}^{\text{ev}}(\mathcal{S}_i)$  for  $i \in \{1, 2\}$ . Prop. 6.8.(1) implies that  $tr^{\text{ev}} \in \text{Tr}^{\text{ev}}(\mathcal{S}_1 \parallel \mathcal{S}_2)$ . By Lemma 6.9.(2) it also holds  $l(tr^{\text{ev}}) = tr$ , so  $tr \in \text{Tr}(\mathcal{S}_1 \parallel \mathcal{S}_2)$  (see Def. 2.36).  $\square$

For the proof of Prop. 6.8.(2) we need to combine two event traces in two different BESs to an event trace in their parallel composition, where these two event traces are constructed from a sequence of actions in the parallel composition. This is done using Alg. 19. Algorithm 19 works as follows: Given a sequence  $tr$  of actions in  $\mathcal{S}_1 \parallel \mathcal{S}_2$  and traces of events  $tr_i$  of  $\mathcal{S}_i$ , go through all actions of  $tr$  (Line 2). If  $tr_1$  is not empty but  $tr_2$  is empty, append the first element of  $tr_1$  paired with

**Algorithm 19** combEvTr: from event traces in components to event trace in the parallel composition**Input:** sequence  $tr$  of actions in  $S_1 \parallel S_2$ , labeling functions  $l_i$  of  $S_i$ , and traces  $tr_i$  of events in  $S_i$  for  $i \in \{1, 2\}$ **Output:** sequence  $tr^{ev}$  of events in  $S_1 \parallel S_2$ 

```

1: let  $tr^{ev} := \langle \rangle$ ,  $i := 0$ 
2: for  $i \leq \text{length}(tr)$  do ▷ go through all actions in  $tr$ 
3:   if  $tr_1 \neq \langle \rangle$  then
4:     if  $tr_2 \neq \langle \rangle$  then ▷  $tr_1$  and  $tr_2$  are nonempty
5:       if  $l_1(tr_1[1]) = l_2(tr_2[1])$  then ▷ first elements have the same label
6:          $tr^{ev} := \text{append}(tr^{ev}, (\text{pop}(tr_1), \text{pop}(tr_2)))$  ▷ append pair of first events to  $tr^{ev}$ 
7:       else ▷ first elements do not have the same label
8:         if  $tr[i] = l_1(tr_1[1])$  then ▷ current action in  $tr$  comes from  $tr_1$ 
9:            $tr^{ev} := \text{append}(tr^{ev}, (\text{pop}(tr_1), *))$ 
10:        else ▷ current action in  $tr$  comes from  $tr_2$ 
11:           $tr^{ev} := \text{append}(tr^{ev}, (*, \text{pop}(tr_2)))$ 
12:        else ▷  $tr_2$  is empty but  $tr_1$  is not
13:           $tr^{ev} := \text{append}(tr^{ev}, (\text{pop}(tr_1), *))$ 
14:        else if  $tr_2 \neq \langle \rangle$  then ▷  $tr_1$  is empty but  $tr_2$  is not
15:           $tr^{ev} := \text{append}(tr^{ev}, (*, \text{pop}(tr_2)))$ 
16:         $i := i + 1$ 
17: return  $tr^{ev}$ 

```

$*$  to  $tr^{ev}$  and delete it from  $tr_1$  (Lines 12 and 13). If  $tr_2$  is not empty but  $tr_1$  is empty, append the first element of  $tr_2$  paired with  $*$  to  $tr^{ev}$  and delete it from  $tr_2$  (Lines 14 and 15).

If both  $tr_1$  and  $tr_2$  are nonempty, first check whether the labels of their first events match (Line 5). If they do, append  $(tr_1[1], tr_2[1])$  to  $tr^{ev}$  and delete the first elements from  $tr_1$  and  $tr_2$  (Line 6). If the first elements of  $tr_1$  and  $tr_2$  do not have the same label, append the event paired with  $*$  to  $tr^{ev}$  that has the same label as the currently considered action of  $tr$  (Lines 7 to 11).

We will only apply this algorithm to traces of events  $tr_i$  of  $S_i$  such that  $tr|_{A_i} = l_i(tr_i)$  for  $i \in \{1, 2\}$ , where  $A_i$  and  $l_i$  are the set of actions and the labeling function of  $S_i$ , respectively, for  $i \in \{1, 2\}$ .

**LEMMA 6.9 (COMBINING EVENT TRACES).** *Let  $S_1, S_2$  be BESs with  $S_i = (E_i, \#, \mapsto_i, A_i, l_i)$ ,  $i \in \{1, 2\}$ , and  $E_1 \cap E_2 = \emptyset$ . Let  $S_1 \parallel S_2 = (E, \#, \mapsto, A, l)$ . For all sequences  $tr$  of actions in  $A$  such that  $tr|_{A_i} \in \text{Tr}(S_i)$  for  $i \in \{1, 2\}$  and for all  $tr_i \in \text{Tr}^{ev}(S_i)$  such that  $l_i(tr_i) = tr|_{A_i}$ ,  $i \in \{1, 2\}$ , the following hold:*

- (1)  $\text{combEvTr}(tr_1, tr_2)$  is a sequence of events in  $S_1 \parallel S_2$  such that for all  $i \in \{1, 2\}$  it holds  $\text{combEvTr}(tr_1, tr_2)|_{E_i} = tr_i$ , and
- (2)  $l(\text{combEvTr}(tr_1, tr_2)) = tr$ .

**PROOF.** We do the proof by induction on the length of  $tr$ .

**Base case:** The lemma holds for the empty trace. Let  $tr = \langle a \rangle$  for  $a \in A$  such that  $tr|_{A_i} \in \text{Tr}(S_i)$  for  $i \in \{1, 2\}$ . If  $a \in A_1 \cap A_2$ , then  $tr|_{A_i} = \langle a \rangle$  for  $i \in \{1, 2\}$ . Let  $tr_i = \langle e_i \rangle \in \text{Tr}^{ev}(S_i)$  such that  $l_i(tr_i) = tr|_{A_i}$  for  $i \in \{1, 2\}$ . Since  $l_1(e_1) = l_2(e_2) = a$ , Line 5 of Alg. 19 implies that  $\text{combEvTr}(tr_1, tr_2) = \langle (e_1, e_2) \rangle$ . Thus, (1) and (2) hold. If  $a \in A \setminus A_2$ , then  $tr|_{A_1} = \langle a \rangle$  and  $tr|_{A_2} = \langle \rangle$ . Let  $tr_1 = \langle e \rangle \in \text{Tr}^{ev}(S_1)$  and  $tr_2 = \langle \rangle \in \text{Tr}^{ev}(S_2)$  such that  $l_i(tr_i) = tr|_{A_i}$  for  $i \in \{1, 2\}$ . Since  $tr_2 = \langle \rangle$ , Line 13 of Alg. 19 implies  $\text{combEvTr}(tr_1, tr_2) = \langle (e, *) \rangle$ . Thus, (1) and (2) hold. Similarly for the case  $a \in A \setminus A_1$ .

**Induction step:** Assume that the lemma holds for all sequences of actions of length  $n$ . Let  $tr' \doteq \langle a_1 \cdot \dots \cdot a_n \cdot a \rangle$  be a sequence of actions of  $A$  (of length  $n + 1$ ) such that  $tr'|_{A_i} \in \text{Tr}(S_i)$  for  $i \in \{1, 2\}$ , and let  $tr \doteq \langle a_1 \cdot \dots \cdot a_n \rangle$ .

If  $a \in A_1 \cap A_2$ , then  $tr'|_{A_i} = tr|_{A_i} \cdot \langle a \rangle$  for  $i \in \{1, 2\}$ . Let  $tr'_1 \doteq \langle e_1 \dots e_i \cdot e \rangle \in \text{Tr}^{\text{ev}}(\mathcal{S}_1)$  and  $tr'_2 \doteq \langle f_1 \dots f_j \cdot f \rangle \in \text{Tr}^{\text{ev}}(\mathcal{S}_2)$  such that  $l_i(tr'_i) = tr'|_{A_i}$  for  $i \in \{1, 2\}$ . For  $tr_1 \doteq \langle e_1 \dots e_i \rangle \in \text{Tr}^{\text{ev}}(\mathcal{S}_1)$  and  $tr_2 \doteq \langle f_1 \dots f_j \rangle \in \text{Tr}^{\text{ev}}(\mathcal{S}_2)$  it holds  $l_i(tr_i) = tr|_{A_i}$ . Therefore, the induction hypothesis implies that  $tr^{\text{ev}} \doteq \text{combEvTr}(tr_1, tr_2)$  is a sequence of events in  $\mathcal{S}_1 \parallel \mathcal{S}_2$  such that  $tr^{\text{ev}}|_{E_i} = tr_i$  for  $i \in \{1, 2\}$ , and  $l(tr^{\text{ev}}) = tr$ . By construction and Line 5 of Alg. 19, it holds  $(tr^{\text{ev}})' \doteq \text{combEvTr}(tr'_1, tr'_2) = tr^{\text{ev}} \cdot \langle (e, f) \rangle$  with  $(e, f) \in E$ . Thus,  $(tr^{\text{ev}})'$  is a sequence of events in  $E$  such that  $(tr^{\text{ev}})'|_{E_i} = tr^{\text{ev}}|_{E_i} \cdot \langle (e, f) \rangle|_{E_i} = tr_i \cdot \langle (e, f) \rangle|_{E_i} = tr'_i$  for  $i \in \{1, 2\}$  (which proves (1)). Furthermore,  $l((tr^{\text{ev}})') = l(tr^{\text{ev}} \cdot \langle (e, f) \rangle) = l(tr^{\text{ev}}) \cdot l(\langle (e, f) \rangle) = tr \cdot \langle a \rangle = tr'$  (which proves (2)).

If  $a \in A \setminus A_2$ , the proof works similarly: Then  $tr'|_{A_1} = tr|_{A_1} \cdot \langle a \rangle$  and  $tr'|_{A_2} = tr|_{A_2}$ . Let  $tr'_1 \doteq \langle e_1 \dots e_i \cdot e \rangle \in \text{Tr}^{\text{ev}}(\mathcal{S}_1)$  and  $tr'_2 \doteq \langle f_1 \dots f_j \rangle \in \text{Tr}^{\text{ev}}(\mathcal{S}_2)$  such that  $l_i(tr'_i) = tr'|_{A_i}$  for  $i \in \{1, 2\}$ . For  $tr_1 \doteq \langle e_1 \dots e_i \rangle \in \text{Tr}^{\text{ev}}(\mathcal{S}_1)$  and  $tr_2 \doteq \langle f_1 \dots f_j \rangle \in \text{Tr}^{\text{ev}}(\mathcal{S}_2)$  it holds  $l_i(tr_i) = tr|_{A_i}$ . Therefore, the induction hypothesis implies that  $tr^{\text{ev}} \doteq \text{combEvTr}(tr_1, tr_2)$  is a sequence of events in  $\mathcal{S}_1 \parallel \mathcal{S}_2$  such that  $tr^{\text{ev}}|_{E_i} = tr_i$  for  $i \in \{1, 2\}$ , and  $l(tr^{\text{ev}}) = tr$ . By construction and Line 13 of Alg. 19, it holds  $(tr^{\text{ev}})' \doteq \text{combEvTr}(tr'_1, tr'_2) = tr^{\text{ev}} \cdot \langle (e, *) \rangle$  with  $(e, *) \in E$ . Thus,  $(tr^{\text{ev}})'$  is a sequence of events in  $E$  such that  $(tr^{\text{ev}})'|_{E_i} = tr^{\text{ev}}|_{E_i} \cdot \langle (e, *) \rangle|_{E_i} = tr_i \cdot \langle (e, *) \rangle|_{E_i} = tr'_i$  for  $i \in \{1, 2\}$  (which proves (1)). Furthermore,  $l((tr^{\text{ev}})') = l(tr^{\text{ev}} \cdot \langle (e, *) \rangle) = l(tr^{\text{ev}}) \cdot l(\langle (e, *) \rangle) = tr \cdot \langle a \rangle = tr'$  (which proves (2)).

If  $a \in A \setminus A_1$ , the proof works in the same way.  $\square$

## 6.2 Parallel Composition of FESs

The definition of parallel composition of FESs builds upon the definition of parallel composition of BESs (Def. 6.2), where the location knowledge uses the least common ancestor as defined next.

*Definition 6.10 (Least common ancestor).* Let  $\mathcal{M}$  be an FM over the set of features  $N$ . Then the *least common ancestor* of a subset of features  $Q \subseteq N$ , denoted  $\text{lca}(\mathcal{M}, Q)$ , is the lowest feature in  $\mathcal{M}$  which is an ancestor of all features in  $Q$ . Formally,  $\text{lca}(\mathcal{M}, Q) \doteq f \in N$  such that:

- (1)  $f$  is the root of a subtree  $\mathcal{G}$  of  $\mathcal{M}$  (i.e.,  $\mathcal{G} = \text{subTree}_f(\mathcal{M})$ );
- (2) all  $q \in Q$  are features in  $\mathcal{G}$  (i.e.,  $\forall q \in Q. q \in \text{features}(\mathcal{G})$ );
- (3)  $\mathcal{G}$  is the minimal subtree containing all  $q \in Q$  (i.e.,  $(\exists f' \in N \exists \mathcal{G}' = \text{subTree}_{f'}(\mathcal{M}) \forall q \in Q. q \in \mathcal{G}') \Leftrightarrow \mathcal{G}' = \mathcal{G}$ ).

*Example 6.11 (Least common ancestor in FM of the cleaning robot SPL).* Consider the FM  $\mathcal{M}_r$  of the cleaning robot SPL as defined in Example 2.6. Then examples for least common ancestors are  $\text{lca}(\mathcal{M}_r, \{o, c\}) = r$  and  $\text{lca}(\mathcal{M}_r, \{li, ca\}) = o$ .

*Remark 6.12 (Least Common Ancestor).* Let  $\mathcal{M}$  be an FM over the set of features  $N$ , let  $Q \subseteq N$ , and let  $pr$  be a product of  $\mathcal{M}$ . Then Rem. 2.11 implies that if  $q \in pr$  for all  $q \in Q$ , then  $\text{lca}(\mathcal{M}, Q) \in pr$ .

*Definition 6.13 (Parallel composition of FESs).* Let  $\mathcal{E}_1, \mathcal{E}_2$  be FESs with  $\mathcal{E}_i = (\mathcal{M}, \mathcal{S}_i, \lambda_i, \nu_i)$ ,  $i \in \{1, 2\}$ , and  $\text{events}(\mathcal{S}_1) \cap \text{events}(\mathcal{S}_2) = \emptyset$ . The *parallel composition*  $\mathcal{E}_1 \parallel \mathcal{E}_2$  of  $\mathcal{E}_1$  and  $\mathcal{E}_2$  is the FES  $(\mathcal{M}, \mathcal{S}, \lambda, \nu)$  with

- $\mathcal{S} \doteq \mathcal{S}_1 \parallel \mathcal{S}_2 \doteq (E, \#, \mapsto, A, l)$ ,
- $\lambda : E \rightarrow \text{features}(\mathcal{M})$  with  $\lambda((e, f)) \doteq \begin{cases} \lambda_1(e) & \text{if } f = * \\ \lambda_2(f) & \text{if } e = * \\ \text{lca}(\mathcal{M}, \{\lambda_1(e), \lambda_2(f)\}) & \text{otherwise} \end{cases}$
- $\nu : E \rightarrow \text{FExp}(\text{features}(\mathcal{M}))$  with  $\nu((e, f)) \doteq \begin{cases} \nu_1(e) & \text{if } f = * \\ \nu_2(f) & \text{if } e = * \\ \lambda((e, f)) \wedge \nu_1(e) \wedge \nu_2(f) & \text{otherwise} \end{cases}$

The underlying BES of a parallel composition of FESs is the parallel composition of the component BESs. The event location knowledge  $\lambda$  agrees with a component event location knowledge  $\lambda_i, i \in \{1, 2\}$ , if the event includes  $*$ . Otherwise, the event location knowledge associates the event to the least common ancestor of the features its components are associated with. The product knowledge  $\nu$  agrees with a component product knowledge  $\nu_i, i \in \{1, 2\}$ , if the event includes  $*$ . Otherwise, the product knowledge maps an event to the conjunction of the event location knowledge of this event and the feature expressions its components are associated with. That is, an event  $(e, f) \in E$  with  $e \neq * \neq f$  can occur in a trace of a product  $pr$  if  $pr$  includes the least common ancestor of  $\lambda_1(e)$  and  $\lambda_2(f)$  and  $pr$  satisfies  $\nu_1(e)$  and  $\nu_2(f)$ .

Note that the parallel composition of FESs satisfies our assumptions on FESs (cf. Def. 4.8) and has the same products as its components.

*Example 6.14 (Parallel composition with the sub-FES with root cleaning of the cleaning robot SPL).* Let  $\mathcal{E}_c \doteq (\mathcal{M}_r, \mathcal{S}_c, \lambda_c, \nu_c)$  be the sub-FES of the FES of the cleaning robot SPL with root *cleaning* from Example 4.16 with the whole feature model  $\mathcal{M}_r$  of the cleaning robot SPL. Let  $\mathcal{E}_{new} \doteq (\mathcal{M}_r, \mathcal{S}_{new}, \lambda_{new}, \nu_{new})$  be an FES where  $\mathcal{S}_{new}$  is as defined in Example 6.3 and  $\lambda_{new}(e) \doteq o$ ,  $\nu_{new}(e) \doteq o \wedge \neg c$  for all  $e \in \text{events}(\mathcal{S}_{new})$ .

Then  $\mathcal{E}_c \parallel \mathcal{E}_{new} = (\mathcal{M}_r, \mathcal{S}_c \parallel \mathcal{S}_{new}, \lambda, \nu)$  with

- $\mathcal{S}_c \parallel \mathcal{S}_{new}$  as in Example 6.3;
- $\lambda((e, f)) = \begin{cases} c & \text{if } (e, f) \in \{(goAround, *), (clean, *)\} \\ o & \text{if } (e, f) = (*, survey) \\ r & \text{if } (e, f) \in \{(move_1, move_3), (move_2, move_3)\} \end{cases}$
- $\nu((e, f)) = \begin{cases} c & \text{if } (e, f) \in \{(goAround, *), (clean, *)\} \\ o \wedge \neg c & \text{if } (e, f) = (*, survey) \\ r \wedge c \wedge o \wedge \neg c & \text{if } (e, f) \in \{(move_1, move_3), (move_2, move_3)\} \end{cases}$

Thus, the events  $(move_1, move_3)$  and  $(move_2, move_3)$  will never be included in a trace of  $\mathcal{E}_c \parallel \mathcal{E}_{new}$  because a product cannot satisfy  $r \wedge c \wedge o \wedge \neg c$ .

**PROPOSITION 6.15 (TRACES OF PARALLEL COMPOSITIONS OF FESs).** *Let  $\mathcal{E}_i = (\mathcal{M}, \mathcal{S}_i, \lambda_i, \nu_i)$  be FESs for  $i \in \{1, 2\}$  with  $\text{events}(\mathcal{S}_1) \cap \text{events}(\mathcal{S}_2) = \emptyset$ , and let  $pr$  be one of their products. Then  $\text{behPr}(\mathcal{E}_1 \parallel \mathcal{E}_2, pr) = \text{behPr}(\mathcal{E}_1, pr) \parallel \text{behPr}(\mathcal{E}_2, pr)$ .*

**PROOF.** Let  $\mathcal{E}_1 \parallel \mathcal{E}_2 = (\mathcal{M}, \mathcal{S}, \lambda, \nu)$  with  $\mathcal{S} = (E, \#, \mapsto, A, l)$ , and let  $\mathcal{S}_i = (E_i, \#_i, \mapsto_i, A_i, l_i)$ . It is enough to show  $\text{events}(\text{behPr}(\mathcal{E}_1, pr) \parallel \text{behPr}(\mathcal{E}_2, pr)) = \text{events}(\text{behPr}(\mathcal{E}_1 \parallel \mathcal{E}_2, pr))$  (cf. Def. 4.31).

$$\begin{aligned}
 & \text{events}(\text{behPr}(\mathcal{E}_1, pr) \parallel \text{behPr}(\mathcal{E}_2, pr)) \\
 & \stackrel{\text{Def. 6.2}}{=} \{ (e, *) \in \text{events}(\text{behPr}(\mathcal{E}_1, pr)) \times \{*\} \mid l_1(e) \in A \setminus A_2 \} \\
 & \quad \cup \{ (*, f) \in \{*\} \times \text{events}(\text{behPr}(\mathcal{E}_2, pr)) \mid l_2(f) \in A \setminus A_1 \} \\
 & \quad \cup \{ (e, f) \in \text{events}(\text{behPr}(\mathcal{E}_1, pr)) \times \text{events}(\text{behPr}(\mathcal{E}_2, pr)) \mid l_1(e) = l_2(f) \} \\
 & \stackrel{\text{Def. 4.31}}{=} \{ (e, *) \in E_1 \times \{*\} \mid pr \models \nu_1(e) \wedge l_1(e) \in A \setminus A_2 \} \\
 & \quad \cup \{ (*, f) \in \{*\} \times E_2 \mid pr \models \nu_2(f) \wedge l_2(f) \in A \setminus A_1 \} \\
 & \quad \cup \{ (e, f) \in E_1 \times E_2 \mid pr \models \nu_1(e) \wedge pr \models \nu_2(f) \wedge l_1(e) = l_2(f) \} \\
 & \stackrel{\text{Def. 6.2, } (\dagger)}{=} \{ (e, *) \in E \mid pr \models \nu_1(e) \} \\
 & \quad \cup \{ (*, f) \in E \mid pr \models \nu_2(f) \} \\
 & \quad \cup \{ (e, f) \in E \mid e \neq * \neq f \wedge pr \models \lambda((e, f)) \wedge \nu_1(e) \wedge \nu_2(f) \} \\
 & \stackrel{\text{Def. 6.2}}{=} \text{events}(\text{behPr}(\mathcal{E}_1 \parallel \mathcal{E}_2, pr))
 \end{aligned}$$



For the third equality ( $\dagger$ ), note that we assume for every  $i \in \{1, 2\}$  and every  $e \in E_i$  that  $v_i(e) \Rightarrow \lambda_i(e)$  (see Def. 4.8). Thus,  $pr \models v_1(e)$  and  $pr \models v_2(f)$  implies  $pr \models \lambda_1(e)$  and  $pr \models \lambda_2(f)$ , which means  $\lambda_1(e) \in pr$  and  $\lambda_2(f) \in pr$ . By Rem. 6.12, this implies  $\lambda((e, f)) = \text{lca}(\mathcal{M}, \{\lambda_1(e), \lambda_2(f)\}) \in pr$ , which means  $pr \models \lambda((e, f))$ .  $\square$

**PROPOSITION 6.16 (PROJECTION OF FES TRACES).** *Let  $\mathcal{E}_1, \mathcal{E}_2$  be FESs with  $\mathcal{E}_i = (\mathcal{M}, \mathcal{S}_i, \lambda_i, v_i)$ ,  $i \in \{1, 2\}$ , and  $\text{events}(\mathcal{S}_1) \cap \text{events}(\mathcal{S}_2) = \emptyset$ , and let  $pr$  be a product of  $\mathcal{E}_1 \parallel \mathcal{E}_2$ .*

- (1) *Let  $tr$  be a sequence of events in  $\text{events}(\mathcal{E}_1 \parallel \mathcal{E}_2)$ . Then  $tr \in \text{Tr}^{\text{ev}}(\mathcal{E}_1 \parallel \mathcal{E}_2, pr) \Leftrightarrow \forall i \in \{1, 2\}. tr|_{\text{events}(\text{behPr}(\mathcal{E}_i, pr))} \in \text{Tr}^{\text{ev}}(\mathcal{E}_i, pr)$ .*
- (2) *Let  $tr$  be a sequence of actions in  $\text{actions}(\mathcal{E}_1 \parallel \mathcal{E}_2)$ . Then  $tr \in \text{Tr}(\mathcal{E}_1 \parallel \mathcal{E}_2, pr) \Leftrightarrow \forall i \in \{1, 2\}. tr|_{\text{actions}(\text{behPr}(\mathcal{E}_i, pr))} \in \text{Tr}(\mathcal{E}_i, pr)$ .*

**PROOF.** By Prop. 6.15 it holds  $\text{Tr}^{\text{ev}}(\mathcal{E}_1 \parallel \mathcal{E}_2, pr) = \text{Tr}^{\text{ev}}(\text{behPr}(\mathcal{E}_1, pr) \parallel \text{behPr}(\mathcal{E}_2, pr))$  and by Prop. 6.8 it holds  $tr \in \text{Tr}^{\text{ev}}(\text{behPr}(\mathcal{E}_1, pr) \parallel \text{behPr}(\mathcal{E}_2, pr)) \Leftrightarrow \forall i \in \{1, 2\}. tr|_{\text{events}(\text{behPr}(\mathcal{E}_i, pr))} \in \text{Tr}^{\text{ev}}(\text{behPr}(\mathcal{E}_i, pr))$ . Finally, by the definition of an event trace of an FES (Def. 4.36), it holds  $\text{Tr}^{\text{ev}}(\text{behPr}(\mathcal{E}_i, pr)) = \text{Tr}^{\text{ev}}(\mathcal{E}_i, pr)$  for  $i \in \{1, 2\}$ . The proof of (2) works in the same way.  $\square$

### 6.3 Parallel Composition of BFM

The parallel composition of two BFM is defined in terms of the FES-view of its components.

**Definition 6.17 (Parallel composition of BFM).** Let  $\mathcal{B}_1, \mathcal{B}_2$  be BFM with  $\text{fm}(\mathcal{B}_1) = \text{fm}(\mathcal{B}_2)$  and  $\text{events}(\mathcal{B}_1) \cap \text{events}(\mathcal{B}_2) = \emptyset$ . The parallel composition  $\mathcal{B}_1 \parallel \mathcal{B}_2$  of  $\mathcal{B}_1$  and  $\mathcal{B}_2$  is

$$\mathcal{B}_1 \parallel \mathcal{B}_2 \doteq \text{bfm}(\text{fes}(\mathcal{B}_1) \parallel \text{fes}(\mathcal{B}_2))$$

Note that, by definition, the parallel composition of BFM satisfies our key assumptions on BFM (cf. Rem. 4.4) and has the same products as its components.

**Example 6.18 (Parallel composition with a simplified BFM of the cleaning robot SPL).** Consider the BFM  $\mathcal{B}'_c$  in Fig. 9a (left), which is the BFM of the cleaning robot SPL with all behavior deleted apart from the behavior associated with the feature *cleaning*. That is, the BFM is isomorphic to  $\mathcal{E}_c$  from Example 6.14. Furthermore, consider the BFM  $\mathcal{B}'_{new}$  in Fig. 9a (right), which is isomorphic to  $\mathcal{E}_{new}$  from Example 6.14. Then  $\mathcal{B}'_c \parallel \mathcal{B}'_{new}$  is the BFM in Fig. 9b, which is isomorphic to  $\mathcal{E}_c \parallel \mathcal{E}_{new}$  from Example 6.14.

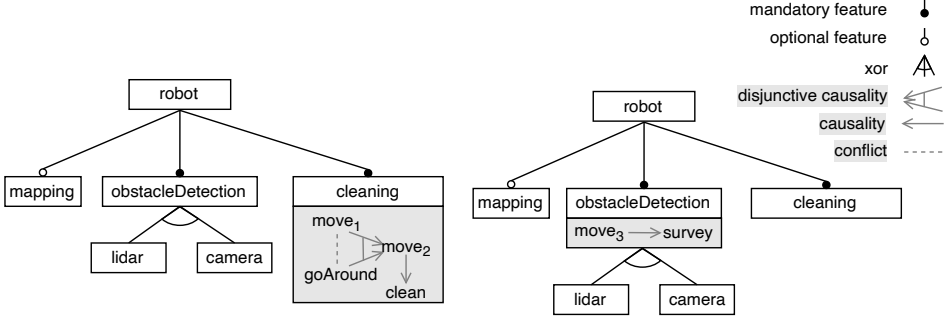
**PROPOSITION 6.19 (PROJECTION OF BFM TRACES).** *Let  $\mathcal{B}_1, \mathcal{B}_2$  be BFM with  $\text{fm}(\mathcal{B}_1) = \text{fm}(\mathcal{B}_2)$  and  $\text{events}(\mathcal{B}_1) \cap \text{events}(\mathcal{B}_2) = \emptyset$ , and let  $pr$  be a product of  $\mathcal{B}_1 \parallel \mathcal{B}_2$ .*

- (1) *Let  $tr$  be a sequence of events in  $\text{events}(\mathcal{B}_1 \parallel \mathcal{B}_2)$ . Then  $tr \in \text{Tr}^{\text{ev}}(\mathcal{B}_1 \parallel \mathcal{B}_2, pr) \Leftrightarrow \forall i \in \{1, 2\}. tr|_{\text{events}(\text{behPr}(\mathcal{B}_i, pr))} \in \text{Tr}^{\text{ev}}(\mathcal{B}_i, pr)$ .*
- (2) *Let  $tr$  be a sequence of actions in  $\text{actions}(\mathcal{B}_1 \parallel \mathcal{B}_2)$ . Then  $tr \in \text{Tr}(\mathcal{B}_1 \parallel \mathcal{B}_2, pr) \Leftrightarrow \forall i \in \{1, 2\}. tr|_{\text{actions}(\text{behPr}(\mathcal{B}_i, pr))} \in \text{Tr}(\mathcal{B}_i, pr)$ .*

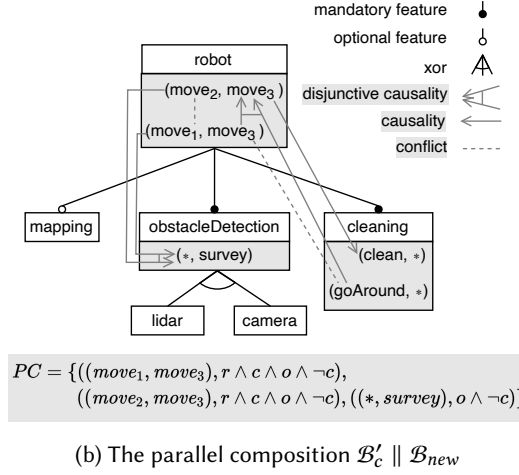
**PROOF.** Both results follow directly from the fact that *fes* and *bfm* are isomorphisms (Prop. 4.30):

$$\begin{array}{ll}
 & tr \in \text{Tr}^{\text{ev}}(\mathcal{B}_1 \parallel \mathcal{B}_2, pr) \\
 \text{Def. 6.17} & \Leftrightarrow \\
 & tr \in \text{Tr}^{\text{ev}}(\text{bfm}(\text{fes}(\mathcal{B}_1) \parallel \text{fes}(\mathcal{B}_2)), pr) \\
 \text{Prop. 4.30} & \Leftrightarrow \\
 & tr \in \text{Tr}^{\text{ev}}(\text{fes}(\mathcal{B}_1) \parallel \text{fes}(\mathcal{B}_2), pr) \\
 \text{Prop. 6.16} & \Leftrightarrow \\
 & \forall i \in \{1, 2\}. tr|_{\text{events}(\text{behPr}(\text{fes}(\mathcal{B}_i), pr))} \in \text{Tr}^{\text{ev}}(\text{fes}(\mathcal{B}_i), pr) \\
 \text{Prop. 4.30, Def. 4.31} & \Leftrightarrow \\
 & \forall i \in \{1, 2\}. tr|_{\text{events}(\text{behPr}(\mathcal{B}_i, pr))} \in \text{Tr}^{\text{ev}}(\mathcal{B}_i, pr)
 \end{array}$$

$\square$



(a) The BFM $s$   $\mathcal{B}'_c$  (left) and  $\mathcal{B}_{new}$  (right), isomorphic to  $\mathcal{E}_c$  and  $\mathcal{E}_{new}$  from Example 6.14, respectively.



(b) The parallel composition  $\mathcal{B}'_c \parallel \mathcal{B}_{new}$

Fig. 9. An example of a parallel composition of two BFM $s$

## 6.4 Parallel Composition of TSs

The parallel composition of two TSs synchronizes on common actions and interleaves for others.

*Definition 6.20 (Parallel composition of TSs).* Let  $\mathcal{T}_1, \mathcal{T}_2$  be TSs with  $\mathcal{T}_i = (S_i, A_i, s_{0,i}, \delta_i)$ ,  $i \in \{1, 2\}$ . The *parallel composition*  $\mathcal{T}_1 \parallel \mathcal{T}_2$  of  $\mathcal{T}_1$  and  $\mathcal{T}_2$  is the TS  $(S, A, S_0, \delta)$  with

- $S \doteq S_1 \times S_2$ ,
- $A \doteq A_1 \cup A_2$ ,
- $s_0 \doteq (s_{0,1}, s_{0,2})$ ,
- $\forall (s_1, s_2), (s'_1, s'_2) \in S, \forall a \in A. ((s_1, s_2), a, (s'_1, s'_2)) \in \delta$  if and only if both of the following conditions hold
  - (i)  $\forall i \in \{1, 2\}. a \in A_i \Rightarrow (s_i, a, s'_i) \in \delta_i$ ,
  - (ii)  $\forall i \in \{1, 2\}. a \notin A_i \Rightarrow s_i = s'_i$ .

The transition relation  $\delta$  reflects that the TSs  $\mathcal{T}_i$ ,  $i \in \{1, 2\}$ , synchronize on actions they have in common and interleave for the other actions: There exists a transition from a state  $(s_1, s_2) \in S$  to a state  $(s'_1, s'_2) \in S$  with action  $a \in A$  if and only if the following two conditions hold:

- (1) for every  $\mathcal{T}_i$  with  $a \in A_i$  it holds  $(s_i, a, s'_i) \in \delta_i$ , and
- (2) for every  $\mathcal{T}_i$  with  $a \notin A_i$ , the state is not changed, i.e.,  $s_i = s'_i$ .

*Example 6.21 (Parallel Composition of TSs).* Figure 10 (when ignoring the feature expressions associated with transitions) shows the TSs  $\mathcal{F}_1$  and  $\mathcal{F}_2$  as well as their parallel composition. Note that the trace  $\langle b \cdot a \cdot c \rangle$  of  $\mathcal{F}_2$  is not a trace of  $\mathcal{F}_1 \parallel \mathcal{F}_2$  because  $\mathcal{F}_1$  also contains the actions  $a$  and  $b$  but not the trace  $\langle b \cdot a \rangle$ .

**PROPOSITION 6.22 (PROJECTION OF TS TRACES).** *Let  $\mathcal{T}_1, \mathcal{T}_2$  be TSs, let  $\mathcal{T}_1 \parallel \mathcal{T}_2 = (S, A, s_0, \delta)$ , and let  $tr$  be a sequence of actions in  $A$ . Then  $tr \in \text{Tr}(\mathcal{T}_1 \parallel \mathcal{T}_2) \Leftrightarrow \forall i \in \{1, 2\}. tr|_{A_i} \in \text{Tr}(\mathcal{T}_i)$ .*

**PROOF.** The proof follows immediately from the definition of  $\delta$  (Def. 6.20).  $\square$

## 6.5 Parallel Composition of FTSs

We define the parallel composition of FTSs according to Classen [34].

*Definition 6.23 (Parallel composition of FTSs).* Let  $\mathcal{F}_1, \mathcal{F}_2$  be FTSs with  $\mathcal{F}_i = (\mathcal{M}, \mathcal{T}_i, \mu_i), i \in \{1, 2\}$ , where  $\mathcal{T}_i = (S_i, A_i, s_{0,i}, \delta_i)$ . The *parallel composition*  $\mathcal{F}_1 \parallel \mathcal{F}_2$  of  $\mathcal{F}_1$  and  $\mathcal{F}_2$  is the FTS  $\mathcal{F} = (\mathcal{M}, \mathcal{T}, \mu)$  with

- $\mathcal{T} \doteq \mathcal{T}_1 \parallel \mathcal{T}_2$ ,
- $\mu((s_1, s_2), a, (s'_1, s'_2)) \doteq \begin{cases} \mu_1((s_1, a, s'_1)) & \text{if } a \in A_1 \wedge a \notin A_2 \\ \mu_2((s_2, a, s'_2)) & \text{if } a \in A_2 \wedge a \notin A_1 \\ \mu_1((s_1, a, s'_1)) \wedge \mu_2((s_2, a, s'_2)) & \text{if } a \in A_1 \wedge a \in A_2 \end{cases}$

In the same way as the transition relation of the TS, the product knowledge  $\mu$  reflects that the FTSs  $\mathcal{F}_i, i \in \{1, 2\}$ , synchronize on actions they have in common and interleave for the other actions: For a transition  $((s_1, s_2), a, (s'_1, s'_2))$  in  $\mathcal{T}$ , the product knowledge  $\mu$  is defined as follows:

- if  $a \in A_1 \cap A_2$  and  $\mu_i((s_i, a, s'_i)) = \phi_i, i \in \{1, 2\}$ , then the feature expression associated with the transition  $((s_1, s_2), a, (s'_1, s'_2))$  is  $\phi_1 \wedge \phi_2$ ,
- if  $a \notin A_1 \cap A_2$ , i.e.,  $a \in A \setminus A_1$  or  $a \in A \setminus A_2$ , and  $\mu_i((s_i, a, s'_i)) = \phi_i$  for either  $i = 1$  or  $i = 2$ , then the feature expression associated with  $((s_1, s_2), a, (s'_1, s'_2))$  is  $\phi_i$ .

Note that by defining a composition operator on FMs as, e.g., in [3], it is also possible to compose FTSs with distinct FMs. With the definition in this paper (Def. 6.23), the parallel composition  $\mathcal{F}_1 \parallel \mathcal{F}_2$  has the same products as  $\mathcal{F}_1$  and  $\mathcal{F}_2$

*Example 6.24 (Parallel composition of FTSs).* Figure 10 shows the FTSs  $\mathcal{F}_1$  and  $\mathcal{F}_2$  as well as their parallel composition. As described in Example 6.21, the trace  $\langle b \cdot a \cdot c \rangle$  of  $\mathcal{F}_2$  is not a trace of  $\mathcal{F}_1 \parallel \mathcal{F}_2$ . Furthermore, if  $\langle a \cdot b \rangle$  is a trace of  $\mathcal{F}_1$  for a product  $pr$ , it might not be a trace of  $\mathcal{F}_1 \parallel \mathcal{F}_2$  for  $pr$  because  $pr$  might not satisfy  $\psi_1$  and  $\psi_2$ .

The following proposition is from Classen et al. [36, Thm. 10].

**PROPOSITION 6.25 (TRACES OF PARALLEL COMPOSITIONS OF FTSs).** *Let  $\mathcal{F}_1, \mathcal{F}_2$  be FTSs over the same feature model, and let  $pr$  be one of their products. Then*

$$\text{behPr}(\mathcal{F}_1 \parallel \mathcal{F}_2, pr) = \text{behPr}(\mathcal{F}_1, pr) \parallel \text{behPr}(\mathcal{F}_2, pr).$$

**PROPOSITION 6.26 (PROJECTION OF FTS TRACES).** *Let  $\mathcal{F}_1, \mathcal{F}_2$  be FTSs. Let  $pr$  be a product of  $\mathcal{F}_1 \parallel \mathcal{F}_2$ , and let  $tr$  be a sequence of actions in  $\text{actions}(\mathcal{F}_1 \parallel \mathcal{F}_2)$ . Then  $tr \in \text{Tr}(\mathcal{F}_1 \parallel \mathcal{F}_2, pr) \Leftrightarrow \forall i \in \{1, 2\}. tr|_{\text{actions}(\text{behPr}(\mathcal{F}_i, pr))} \in \text{Tr}(\mathcal{F}_i, pr)$ .*

**PROOF.** By Prop. 6.25 it holds  $\text{Tr}(\mathcal{F}_1 \parallel \mathcal{F}_2, pr) = \text{Tr}(\text{behPr}(\mathcal{F}_1, pr) \parallel \text{behPr}(\mathcal{F}_2, pr))$  and by Prop. 6.22 it holds  $tr \in \text{Tr}(\text{behPr}(\mathcal{F}_1, pr) \parallel \text{behPr}(\mathcal{F}_2, pr)) \Leftrightarrow \forall i \in \{1, 2\}. tr|_{\text{actions}(\text{behPr}(\mathcal{F}_i, pr))} \in \text{Tr}(\text{behPr}(\mathcal{F}_i, pr))$ . Finally, by the definition of a trace of an FTS (Def. 2.29), it holds  $\text{Tr}(\text{behPr}(\mathcal{F}_i, pr)) = \text{Tr}(\mathcal{F}_i, pr)$  for  $i \in \{1, 2\}$ .  $\square$

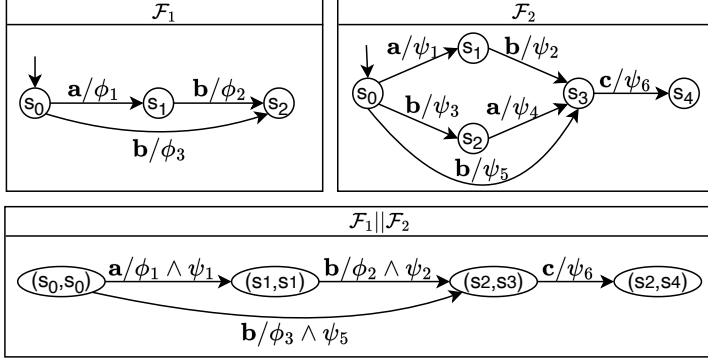


Fig. 10. An example of a parallel composition of FTSs

## 6.6 Translating Parallel Compositions

The following theorems show that our translations are “compositional encodings” in the sense of Gorla’s work on the comparative expressiveness of languages [64].

**THEOREM 6.27 (ACYFTS2BFM FOR PARALLEL COMPOSITIONS).** *Let  $\mathcal{F}_1$  and  $\mathcal{F}_2$  be acyclic FTSs over the sets of actions  $A_i, i \in \{1, 2\}$ , with the same feature model, and let  $pr$  be one of their products. Then  $\text{Tr}(\mathcal{F}_1 \parallel \mathcal{F}_2, pr) = \text{Tr}(\text{AcyFTS2BFM}(\mathcal{F}_1) \parallel \text{AcyFTS2BFM}(\mathcal{F}_2), pr)$ . In particular,  $\mathcal{F}_1 \parallel \mathcal{F}_2$  and  $\text{AcyFTS2BFM}(\mathcal{F}_1) \parallel \text{AcyFTS2BFM}(\mathcal{F}_2)$  are trace equivalent.*

**PROOF.** This is an immediate consequence of Props. 6.19 and 6.26 and Thm. 5.25:

$$\begin{aligned}
 & tr \in \text{Tr}(\mathcal{F}_1 \parallel \mathcal{F}_2, pr) \\
 \stackrel{\text{Prop. 6.26}}{\Leftrightarrow} & \forall i \in \{1, 2\}. tr|_{\text{actions}(\text{behPr}(\mathcal{F}_i, pr))} \in \text{Tr}(\mathcal{F}_i, pr) \\
 \stackrel{\text{Thm. 5.25}}{\Leftrightarrow} & \forall i \in \{1, 2\}. tr|_{\text{actions}(\text{behPr}(\text{AcyFTS2BFM}(\mathcal{F}_i), pr))} \in \text{Tr}(\text{AcyFTS2BFM}(\mathcal{F}_i), pr) \\
 \stackrel{\text{Prop. 6.19}}{\Leftrightarrow} & tr \in \text{Tr}(\text{AcyFTS2BFM}(\mathcal{F}_1) \parallel \text{AcyFTS2BFM}(\mathcal{F}_2), pr)
 \end{aligned}$$

□

**THEOREM 6.28 (BFM2ACYFTS FOR PARALLEL COMPOSITIONS).** *Let  $\mathcal{B}_1$  and  $\mathcal{B}_2$  be BFM over the sets of actions  $A_i, i \in \{1, 2\}$ , such that  $\text{fm}(\mathcal{B}_1) = \text{fm}(\mathcal{B}_2)$  and  $\text{events}(\mathcal{B}_1) \cap \text{events}(\mathcal{B}_2) = \emptyset$ , and let  $pr$  be one of their products. Then  $\text{Tr}(\mathcal{B}_1 \parallel \mathcal{B}_2, pr) = \text{Tr}(\text{BFM2AcyFTS}(\mathcal{B}_1) \parallel \text{BFM2AcyFTS}(\mathcal{B}_2), pr)$ . In particular,  $\mathcal{B}_1 \parallel \mathcal{B}_2$  and  $\text{BFM2AcyFTS}(\mathcal{B}_1) \parallel \text{BFM2AcyFTS}(\mathcal{B}_2)$  are trace equivalent.*

**PROOF.** This is an immediate consequence of Props. 6.19 and 6.26 and Thm. 5.27:

$$\begin{aligned}
 & tr \in \text{Tr}(\mathcal{B}_1 \parallel \mathcal{B}_2, pr) \\
 \stackrel{\text{Prop. 6.19}}{\Leftrightarrow} & \forall i \in \{1, 2\}. tr|_{\text{actions}(\text{behPr}(\mathcal{B}_i, pr))} \in \text{Tr}(\mathcal{B}_i, pr) \\
 \stackrel{\text{Thm. 5.27}}{\Leftrightarrow} & \forall i \in \{1, 2\}. tr|_{\text{actions}(\text{behPr}(\text{BFM2AcyFTS}(\mathcal{B}_i), pr))} \in \text{Tr}(\text{BFM2AcyFTS}(\mathcal{B}_i), pr) \\
 \stackrel{\text{Prop. 6.26}}{\Leftrightarrow} & tr \in \text{Tr}(\text{BFM2AcyFTS}(\mathcal{B}_1) \parallel \text{BFM2AcyFTS}(\mathcal{B}_2), pr)
 \end{aligned}$$

□

## 7 EVALUATION

In this section, we lay out the methodology, report on the carried out evaluation (both in terms of theoretical results and empirical evaluation), and reflect on the answers to the following overarching research question:

*Do BFM lead to a more concise representation of  
feature behavior than the state-of-the-art approaches?*

The conciseness of a representation expresses that irrelevant information is avoided [57, 91]. Thus, conciseness makes models simpler and easier to maintain [112], and is often seen as an indication of understandability [86]. We concretise the comparison of conciseness in terms of expressivity and succinctness (detailed in Def. 7.1). We then answer this overarching research question by means of the following sub-questions, through a series of theoretical results (propositions and proofs) and through experiments on a large benchmark set with models from the literature.

**RQ1** In theory, how do the expressiveness and succinctness of BFMs compare to the state-of-the-art modeling languages for behavioral variability?

**RQ1.1** Are BFMs as expressive as the state-of-the-art modeling languages for behavioral variability?

**RQ1.2** Are BFMs as succinct as the state-of-the-art modeling languages for behavioral variability?

**RQ2** In practice, how does the size of a BFM compare to the size of models of the state-of-the-art modeling languages for behavioral variability?

State-of-the-art behavioral variability modeling languages annotate standard behavioral models (e.g., finite-state machines or TSs) with expressions pertaining to structural variability (i.e., Boolean expressions over the features of an FM [8, 17, 36, 40, 61, 74]). The most developed formalism for specifying behavioral variability is that of an FTS, which comes equipped with tools for modeling and model checking. Moreover, it has been shown in earlier studies [25, 110] that FTSs are more expressive and exponentially more succinct than other formalisms for specifying the behavior of SPLs. This justifies the use of FTSs as the baseline to evaluate our proposal.

We do not consider modular variability modeling languages (e.g., OVL [88], CVL [67], and FORML [95]) that add variability in an orthogonal way to existing general purpose modeling languages (e.g., UML) instead of integrating variability into the language. We also do not compare BFMs to the recent *Universal Variability Language (UVL)* [20, 24, 98], a purely textual variability modeling language, since it ignores (feature) behavior.

### 7.1 Methodology

In this section, we describe the methodology used to answer our research questions. We start by defining the metrics we use to compare (parallel compositions of) FTSs and BFMs in Sect. 7.1.1. We then present the subject systems used in our evaluation in Sect. 7.1.2 and the details of our implementation in Sect. 7.1.3.

#### 7.1.1 Evaluation Metric.

In our research questions, we assess the expressiveness and succinctness of BFMs with respect to acyclic FTSs according to the following definition.

*Definition 7.1 (Expressiveness and succinctness).* Let  $M_1$  and  $M_2$  be modeling languages.

- $M_1$  is at least as expressive as  $M_2$ , if there exists a translation scheme  $||\_|| : M_1 \rightarrow M_2$  such that for every model  $m \in M_1$ , its translation  $||m|| \in M_2$  has the same semantics (set of traces).

- $M_1$  is *at least as succinct* as  $M_2$ , if the translation scheme  $||\_|| : M_1 \rightarrow M_2$  always maps a model  $m \in M_1$  to  $||m|| \in M_2$  such that  $||m||$  has a polynomial size with respect to  $m$ .
- $M_1$  is *less succinct* than  $M_2$  if there is no such polynomial translation scheme.

To assess the succinctness of BFM with respect to FTSs, we compare their respective sizes, defined by the following metric (further discussed in Sect. 7.4):

*Definition 7.2 (Size of FMs, TSs, FTSs, BESs, FESs, and BFMs).*

- The size  $|\mathcal{M}|$  of an FM  $\mathcal{M}$  is defined as  $|\mathcal{M}| = |\{f \mid f \in \text{features}(\mathcal{M})\}|$ .
- The size  $|\mathcal{T}|$  of a TS  $\mathcal{T} = (S, A, s_0, \delta)$  is defined as  $|\mathcal{T}| = |S| + |\delta|$ .
- The size  $|\mathcal{F}|$  of an FTS  $\mathcal{F} = (\mathcal{M}, \mathcal{T}, \mu)$  is defined as  $|\mathcal{F}| = |\mathcal{M}| + |\mathcal{T}|$ .
- The size  $|\mathcal{S}|$  of a BES  $\mathcal{S} = (E, \#, \mapsto, A, l)$  is defined as  $|\mathcal{S}| = |E| + |\#| + |\mapsto|$ .
- The size  $|\mathcal{E}|$  of an FES  $\mathcal{E} = (\mathcal{M}, \mathcal{S}, \lambda, \nu)$  is defined as  $|\mathcal{E}| = |\mathcal{M}| + |\mathcal{S}|$ .
- The size  $|\mathcal{B}|$  of a BFM  $\mathcal{B}$  is defined in terms of its FES-view:  $|\mathcal{B}| = |\text{fes}(\mathcal{B})|$ .

*Remark 7.3 (On sizes of FTSs and BFMs/FESs).* Without loss of generality, we may assume that trace-equivalent FTSs and BFMs/FESs have the same FM; this is the case for Algorithms 8 and 10. Therefore, to compare the sizes of an FTS and an BFM/FES, it is enough to compare the sizes of the underlying TS and BES, respectively.

### 7.1.2 Subject Systems.

To compare FTSs and BFMs in practice (RQ2), we build an evaluation benchmark with 46 subject systems, including acyclic versions of all well-known case studies of FTSs that are well-suited for an acyclic adaptation [16, 33, 47]. For systems where component FTSs were available, the benchmark set contains all possible combinations of their parallel compositions, leading to the largest evaluation benchmark of FTSs in the literature with a total of 46 subject systems, curated from four case studies. In the evaluation benchmark, we curated the FTS specifications as follows. In each case study, we first make the component FTSs acyclic by removing self-loops and transitions returning to the initial state. The composition of the component FTSs was then constructed both with and without synchronization of actions. Below, we describe the subject systems in our benchmark.

*Cleaning Robot.* The cleaning robot SPL, introduced in Example 2.1, with FTS presented in Example 2.25.

*Card Payment Terminal.* The Card Payment Terminal [47] models the Eurocard-Mastercard-Visa (EMV) protocol. It defines different card payment schemes for direct debit and/or credit transactions. The terminal supports optional connectivity and offers multiple ways of reading a card: via chip, magnetic stripe, or near-field communication (NFC). Before proceeding with a payment, the cardholder must always authenticate. Depending on the selected product variant, this is done either by entering a PIN or by providing a signature.

*Vending Machines.* We evaluate a collection of Vending Machines (VMs) and all their possible combinations through parallel composition. We use two versions of parallel composition: one with only interleaving, and one with synchronization over a subset of selected actions. Each machine type dispenses a different category of beverages. The *Coffee VM* [16] offers coffee, cappuccino, and tea. The *Soda VM* [33, 47] also provides tea, but replaces coffee-based drinks with sodas; it additionally includes an optional feature that allows dispensing free drinks. The *Soup VM* [16] offers three types of soup: chicken, pea, and tomato. All VMs support payments in euros or dollars and include an optional feature to cancel an order. Note that while the Soup and Coffee VMs have



previously been composed by ter Beek et al. [16], we additionally compose these two with the Soda VM.

*Mine Pump System.* The *Mine Pump* system [33, 47] models the regulation mechanism of a mine shaft and is composed of five components. At the heart of the system lies the *Controller* (*C*), which acts as the central orchestrator. It manages communications among components and determines the actions to take based on incoming signals. An alternative version of the Controller [16] includes its composition with the *State* (*S*) component which logs the system state at any given time. The remaining three components model actuators interacting with the environment. The *Water* (*W*) component monitors the water level in the shaft and signals the *Controller* when intervention is required (e.g., to raise or lower the water level). In response, the Controller activates the *Pump* (*P*), which adjusts the water level accordingly. The *Methane detector* (*M*) continuously reports the methane concentration to the Controller to ensure it remains within safe limits, thereby preventing potential explosions. In this case study, we systematically evaluate all possible compositions of the Controller with the other components. We enforce the presence of the Controller in every combination because it is the only component that exhibits internal variability. As before, we use two versions of parallel composition: one with only interleaving and one with synchronization over a subset of selected actions.

### 7.1.3 Implementation.

We implemented our six translation algorithms (Algorithms 2, 4, 7, 9, 11 and 13) in Java. The full implementation, along with all experimental data, is available in our replication package.<sup>3</sup> Our code builds upon VIBES,<sup>4</sup> a library for manipulating FTSS, and integrates the *Universal Variability Language* (UVL) [24, 98], recently developed by the SPL community, to ensure compatibility with modern feature modeling languages and tools.

To bridge the differences between UVL and VIBES, particularly in how they represent FMs, we introduced a unified structure that combines the advantages of both approaches. Specifically, we adopt a UVL-based FM restricted to Boolean features, and, using VIBES, couple it with a solver to evaluate feature constraints. In our implementation, we selected JavaBDD<sup>5</sup> as a solver to determine whether a given feature expression is valid with respect to the FM, or to compute the set of products that satisfy a set of feature constraints. As in VIBES, we encode feature expressions using Podgursky’s `jbool_expressions` library.<sup>6</sup> Our unified representation preserves the intuitive tree-like structure of FMs, lost in VIBES’s CNF-based encoding, while supporting efficient constraint reasoning. We support model import and export in XML format. For TSs and FTSS, we also support the *Graphviz* DOT format,<sup>7</sup> to facilitate visualization.

Our BFM is built on a new `BehavioralFeature` class, which extends the base `Feature` class. A `BehavioralFeatureModel` is implemented as a specialization of `FeatureModel<BehavioralFeature>`, conforming to the `FeaturedEventStructure` interface.

Because conflicts are persistent among alternative traces, we implement the representation of conflicts as relations between (maximal) subsets of events, rather than between individual events. In this more compact *biclique representation* [5, 30, 87], a conflict  $(A, B)$  denotes that every event in the set  $A$  is in conflict with every event in the set  $B$ . For example, a TS including the transitions  $s \xrightarrow{a} s_a \xrightarrow{b} s_b \xrightarrow{c} s_c$  and the transitions  $s' \xrightarrow{d} s_d \xrightarrow{e} s_e \xrightarrow{f} s_f$  would yield  $3^2 = 9$  conflict pairs in the

<sup>3</sup><https://github.com/sfortz/BehavioralFeatureModel>

<sup>4</sup><https://github.com/xdevroey/vibes>

<sup>5</sup><https://github.com/com-github-javabdd/com.github.javabdd>

<sup>6</sup>[https://github.com/bpodgursky/jbool\\_expressions](https://github.com/bpodgursky/jbool_expressions)

<sup>7</sup><https://graphviz.org/>

standard encoding. In our biclique encoding, they are captured by a single conflict:  $\{a, b, c\} \# \{d, e, f\}$ . In particular, the set  $\#$  is encoded as a set  $bcq_{\#} \doteq \bigcup_{i \in \{1, \dots, n\}} \{ (A_i, B_i) \mid \forall a \in A_i. \forall b \in B_i. a \# b \}$  with  $n \geq 0$ . This representation is semantically equivalent yet can be significantly more compact; in particular, the bicliques of any given model have a representation with fixed upper bound. To reflect this change, Tbl. 1 in Sect. 7.3 reports the total number of conflicts, defined as the number of bicliques  $|bcq_{\#}|$ .

---

**Algorithm 20** Greedy Biclique Construction with Edge covering
 

---

**Input:** Conflict graph  $(E, \#)$  ▷ Nodes are events and edges are conflicts  
**Output:** Set of bicliques  $bcq_{\#}$  covering all edges in  $\#$

```

1:  $bcq_{\#} := \emptyset$ 
2:  $U := \#$  ▷ Uncovered edges
3: while  $U \neq \emptyset$  do ▷ Until all edges have been covered
4:   pick  $(e_1, e_2) \in U$  ▷ Pick a random edge from the remaining uncovered edges
5:    $A := \{e_1\}, B := \{e_2\}$  ▷ Initialize biclique sets
6:    $changed := \mathbf{true}$  ▷ Variable for indicating expansion of A or B
7:   while  $changed = \mathbf{true}$  do ▷ As long as an event has been added to A or B
8:      $changed := \mathbf{false}$  ▷ Nothing has been changed yet
9:     for all  $e \in E \setminus A$  do ▷ Expand A
10:      if  $\forall b \in B. (e, b) \in \#$  then
11:         $A := A \cup \{e\}$ 
12:         $changed := \mathbf{true}$ 
13:      for all  $e \in E \setminus B$  do ▷ Expand B
14:        if  $\forall a \in A. (a, e) \in \#$  then
15:           $B := B \cup \{e\}$ 
16:           $changed := \mathbf{true}$ 
17:    $bcq_{\#} := bcq_{\#} \cup \{(A, B)\}$  ▷ Include new biclique
18:   for all  $(a, b) \in A \times B$ 
19:      $U := U \setminus \{(a, b)\}$  ▷ Delete covered edges from U
20: return  $bcq_{\#}$ 

```

---

Transforming the pairwise conflict representation into a biclique-based representation involves solving the *minimum biclique edge cover problem* [87], which is NP-complete. To this end, we first construct a conflict graph capturing all conflicts present in the BFM by representing all events as nodes and all conflicts as edges between nodes. This graph can quickly scale to over 100 nodes and hundreds or even thousands of edges. We then partition this graph into bicliques, adopting a greedy heuristic that iteratively covers all edges (cf. Alg. 20, explained below). Given the computational hardness of the problem and the size of our problem instances, such a heuristic provides a practical and scalable solution.

Algorithm 20 for constructing a biclique representation works as follows. Given a conflict graph with set of conflicts  $\#$ , initialize the set of bicliques  $bcq_{\#}$  with  $\emptyset$  (Line 1) and the set of uncovered edges  $U$  with  $\#$  (Line 2). As long as there are still uncovered edges, i.e.,  $U \neq \emptyset$ , (Line 3), randomly pick an edge from the remaining uncovered edges (Line 4). Initialize the sets  $A$  and  $B$ , which will be combined into a biclique, with the randomly picked events (Line 5) and the variable  $changed$ , which indicates that an event has been added to  $A$  or  $B$ , with  $\mathbf{true}$  (Line 6). As long as an event has been added to  $A$  or  $B$  (Line 7), expand  $A$  with all events that are in conflict with all events in  $B$  (Lines 9 to 11) and  $B$  with all events that are in conflict with all events in  $A$  (Lines 13 to 15). Once

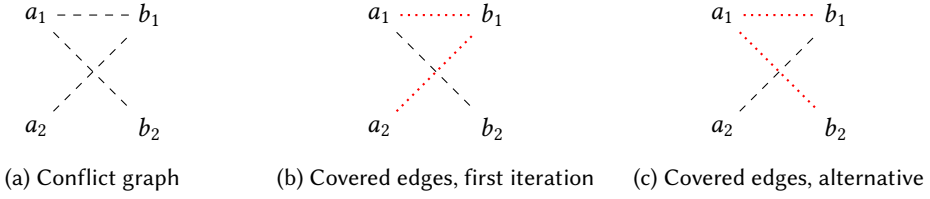


Fig. 11. Conflict graph for the events and conflicts in Example 7.4 and covered edges after the first iteration of the while loop of Alg. 20, displayed as red, dotted lines

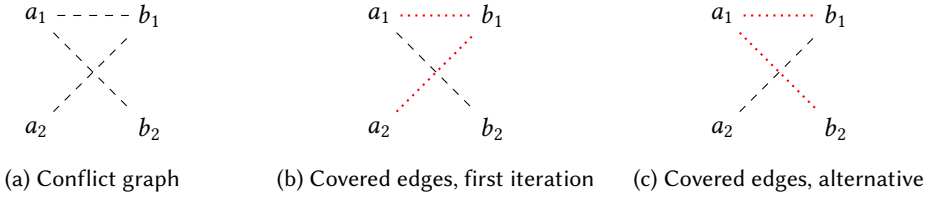


Fig. 12. Conflict graph for the events and conflicts in Example 7.4 and covered edges after the first iteration of the while loop of Alg. 20, displayed as red, dotted lines

nothing can be added to  $A$  or  $B$  anymore, include the biclique  $(A, B)$  in the set of bicliques  $bcq_{\#}$  (Line 17) and delete all added conflicts from  $U$  (Lines 18 and 19).

*Example 7.4 (Biclique Construction).* Consider the set of events  $\{a_1, a_2, b_1, b_2\}$  and the set of conflicts  $\# \doteq \{(a_1, b_1), (a_1, b_2), (a_2, b_1)\}$ .

Then the conflict graph is displayed in Fig. 12a. The algorithm for constructing bicliques (Alg. 20) for this conflict graph works as follows:

**Initialize**  $bcq_{\#} \doteq \emptyset$

**First iteration of the while loop:**

- Pick the edge  $(a_1 \# b_1)$  (This step is non-deterministic, another edge could have been picked)
- Initialize  $A \doteq \{a_1\}$  and  $B \doteq \{b_1\}$
- Expand:  $A = \{a_1, a_2\}, B = \{b_1\}$
- No further expansion is possible:  $bcq_{\#} \doteq \{(\{a_1, a_2\}, \{b_1\})\}$  (see Fig. 12b for a graphical representation of the covered edges of the conflict graph)

**Second iteration of the while loop:**

- Remaining edge:  $(a_1 \# b_2)$
- Initialize  $A \doteq \{a_1\}$  and  $B \doteq \{b_2\}$
- No further expansion is possible:  $bcq_{\#} = bcq_{\#} \cup \{(\{a_1\}, \{b_2\})\}$

**No remaining edges.**

**Final result:**  $bcq_{\#} = \{(\{a_1, a_2\}, \{b_1\}), (\{a_1\}, \{b_2\})\}$ .

Note that this process is sensitive to which edge of the conflict graph you randomly pick. For example, had we picked the edge  $(a_1 \# b_2)$  first, then the first iteration of the while loop would have given  $bcq'_{\#} \doteq \{(\{a_1\}, \{b_1, b_2\})\}$  (see Fig. 12c for a graphical representation of the covered edges of the conflict graph). Thus, it would lead to a final result of  $bcq'_{\#} = \{(\{a_1\}, \{b_1, b_2\}), (\{a_2\}, \{b_1\})\}$ .

## 7.2 RQ1: In theory, how do the expressiveness and succinctness of BFM compare to the state-of-the-art modeling languages for behavioral variability?

We answer this research question using results from Sect. 5.

### 7.2.1 RQ1.1: Are BFMs as expressive as the state-of-the-art modeling languages for behavioral variability?

To show that BFMs are at least as expressive as acyclic FTSs, we prove in Thm. 5.25 that Alg. 11 translates every acyclic FTS to a trace equivalent BFM.

**Answer to RQ1.1:** Are BFMs as expressive as the state-of-the-art modeling languages for behavioral variability?

BFMs are as expressive as acyclic FTSs because we can translate every acyclic FTS to a trace-equivalent BFM.

### 7.2.2 RQ1.2: Are BFMs as succinct as the state-of-the-art modeling languages for behavioral variability?

To show that BFMs are at least as succinct as acyclic FTSs, we show in the following proposition that Alg. 3 translates an acyclic TS to a BES that has at most quadratic size with respect to the source TS.

**PROPOSITION 7.5 (UPPER BOUND FOR THE OUTPUT OF ACYTS2BES).** *Let  $\mathcal{T} = (S, A, s_0, \delta)$  be an acyclic TS, and let  $\mathcal{S} = (E, \#, \mapsto, A, l)$  be its translation via Alg. 3. Then the size of  $\mathcal{S}$  is in  $O(|\mathcal{T}|^2)$ .*

**PROOF.** Let  $\mathcal{T}^{lin} = (S, E, s_0, \delta^{lin})$  be the translation of  $\mathcal{T}$  via Alg. 1 (called in Line 1 of Alg. 3). That is,  $\mathcal{T}^{lin}$  is a linear TS. By the definition of  $\mathcal{T}^{lin}$  in Alg. 1, the states and the number of transitions in  $\delta^{lin}$  and in  $\delta$  are the same. Thus, it holds that  $|\mathcal{T}^{lin}| = |\mathcal{T}|$  and it is enough to show  $\mathcal{S} \in O(|\mathcal{T}^{lin}|^2)$ .

The fact that  $\mathcal{T}^{lin}$  is linear and its construction in Alg. 1 imply that  $|\delta| = |\text{usedAct}(\mathcal{T}^{lin})| = |E|$ . Furthermore, it must hold  $1 \leq |S| \leq |\delta| + 1 = |E| + 1$ . Therefore,  $|\mathcal{T}^{lin}| = |S| + |\delta|$  is bounded by  $1 + |E|$  from below and by  $2|E| + 1$  from above. In particular,  $\mathcal{T}^{lin} \in O(|E|)$ . We want to find upper bounds for  $|\#|$  and  $|\mapsto|$ .

The conflict relation  $\#$  is a symmetric relation containing pairs of events. If  $E = \{e_1, \dots, e_n\}$ , then  $e_1$  can be combined with  $e_2, \dots, e_n$ , i.e., with  $|E| - 1$  elements,  $e_2$  can be combined with  $e_3, \dots, e_n$ , i.e.,  $|E| - 2$  elements, and so on. Thus, an upper bound for  $|\#|$  is  $\sum_{i=1}^{|E|-1} i$ . This upper bound can be reached if  $\mathcal{T}^{lin}$  is a “star”, where each transition in  $\delta^{lin}$  is of the form  $(s_0, e, s)$  for  $e \in E$  and  $s \in S$ . Then Alg. 2 (called in Line 2 of Alg. 3) gives that  $\#$  contains all pairs of events because no event is reachable from another event.

The causality relation  $\mapsto$  can contain at most  $|E|$  elements, one for each event in  $E$ . This upper bound can almost be reached if  $\mathcal{T}^{lin}$  is a “row” of transitions, i.e.,  $\mathcal{T}^{lin}$  has exactly one trace. Then Alg. 2 (called in Line 2 of Alg. 3) gives that  $\mapsto$  contains a pair  $(X_e, e)$  for every  $e \in E$  apart from the event associated with the first transition, where  $X_e$  contains only the event associated with the transition directly before the one  $e$  is associated with.

Considering both upper bounds, we have  $|S| = |E| + |\#| + |\mapsto| = |E| + \sum_{i=1}^{|E|-1} i + |E| = |E| + \sum_{i=1}^{|E|} i$ , implying  $|S| \in O(|\mathcal{T}^{lin}|^2)$ .  $\square$

To show that acyclic FTSs are less succinct than BFMs, we show in the following propositions that Alg. 5 translates a BES to an acyclic TS that has at most exponential size with respect to the source BES, and that this exponential size can be reached, independent of the translation scheme.

**PROPOSITION 7.6 (UPPER BOUND FOR THE OUTPUT OF BES2TS).** *Let  $\mathcal{S} = (E, \#, \mapsto, A, l)$  be a BES, and let  $\mathcal{T} = (S, A, s_0, \delta)$  be its translation via Alg. 5. Then the size of  $\mathcal{T}$  is in  $O(2^{|S|})$ .*

**PROOF.** We want to find upper bounds for  $S$  and  $|\delta|$ .

The set of states  $S$  is defined as the set of all configurations of  $(E, \#, \mapsto, E, \text{id}_E)$  in Line 1 of Alg. 4 (called in Line 1 of Alg. 5), which is exactly the set of configurations of  $\mathcal{S}$ . The set of configurations of  $\mathcal{S}$  is a subset of  $\mathcal{P}(E)$ . Thus, an upper bound for the size of  $S$  is  $|\mathcal{P}(E)| = 2^{|E|}$ . If this upper bound is reached, there exist no conflicts and causalities between the events of  $\mathcal{S}$ . Thus, in this case we have  $|S| = |E|$ .

The upper bound for the number of transitions can be reached with the maximum number of states because there is only a transition between two states, if they differ by exactly one event (see Lines 2 and 3 of Alg. 4). If  $S = \mathcal{P}(E)$ , then for every  $s \in S$  there exist  $|E| - |s|$  outgoing transitions from  $s$ . Namely, for every  $e \in E \setminus \{e' \in s\}$  there exists a transition  $(s, e, s \cup \{e\})$ . There are  $|E| - i + 1$  states with size  $i \neq 0$  and one state with size 0. Therefore, an upper bound for  $|\delta|$  is  $|E| + \sum_{i=1}^{|E|} (|E| - i)(|E| - i + 1) = 1/3 * |E|(|E|^2 - 1) + |E|$ .

Considering both upper bounds, we have  $|\mathcal{T}| = |S| + |\delta| = 2^{|E|} + 1/3 * |E|(|E|^2 - 1) + |E|$ . This implies  $|\mathcal{T}| \in O(2^{|S|})$ .  $\square$

**PROPOSITION 7.7 (TIGHT UPPER BOUND FOR THE OUTPUT OF BES2TS).** *For every natural number  $n$  there exists a BES  $\mathcal{S}$  with  $|S| = n$  such that for all TSs  $\mathcal{T}$  with  $\text{Tr}(\mathcal{T}) = \text{Tr}(\mathcal{S})$  it holds that  $|\mathcal{T}| \in \Theta(2^{|S|})$ .*

**PROOF.** Let  $n$  be a natural number, and let  $\mathcal{S} = (E, \emptyset, \emptyset, E, \text{id}_E)$  with  $E = \{e_1, \dots, e_n\}$ , i.e.,  $|S| = n = |E|$ . Then  $\text{Tr}(\mathcal{S})$  contains for every subset  $E'$  of  $E$  all permutations of events in  $E'$ . Let  $\mathcal{T} = (S, E, s_0, \delta)$  be a TS with  $\text{Tr}(\mathcal{T}) = \text{Tr}(\mathcal{S})$ . Then  $\mathcal{T}$  has to contain at least  $2^{|S|}$  states: To ensure that the traces of  $\mathcal{T}$  contain each event at most once, each state of the TS must distinguish which events have led to this state. Thus, for every subset of  $E$ , there exists at least one state in  $\mathcal{T}$ . This implies  $|S| \geq |\mathcal{P}(E)| = 2^{|E|}$  and thus  $|\mathcal{T}| \in \Omega(2^{|S|})$ . In combination with Prop. 7.6 this implies  $|\mathcal{T}| \in \Theta(2^{|S|})$ .  $\square$

In conclusion, given an acyclic TS  $\mathcal{T}$ , we can construct a BES  $\mathcal{S}$  with  $|S| \in O(|\mathcal{T}|^2)$ , see Prop. 7.5. On the other hand, for every  $n$  there exists a BES such that for all trace-equivalent TSs  $\mathcal{T}$  it holds  $|\mathcal{T}| \in \Theta(2^{|S|})$ . By Rem. 7.3, these results can be translated to acyclic FTSs and BFM: Given an acyclic FTS  $\mathcal{F} = (\mathcal{M}, \mathcal{T}, \mu)$ , we can construct a BFM/FES  $\mathcal{X}$  with  $|\mathcal{X}| \in O(|\mathcal{M}| + |\mathcal{T}|^2)$ . Thus, BFM/FESs are at least as succinct as acyclic FTSs. On the other hand, for every  $n$  there exists an FES  $\mathcal{E} = (\mathcal{M}, \mathcal{S}, \lambda, \nu)$  with  $|\mathcal{E}| = |\mathcal{M}| + n$  (and equivalently a BFM  $\mathcal{B}$  with  $\text{fes}(\mathcal{B}) = (\mathcal{M}, \mathcal{S}, \lambda, \nu)$  and  $|\mathcal{B}| = |\mathcal{M}| + n$ ) such that for all trace-equivalent FTSs  $\mathcal{F}$  it holds  $|\mathcal{F}| \in \Theta(|\mathcal{M}| + 2^{|S|})$ . This shows that FTSs are less succinct than BFM.

**Answer to RQ1.2:** Are BFM as succinct as the state-of-the-art modeling languages for behavioral variability?

BFMs are at least as succinct as acyclic FTSs. FTSs are less succinct than BFM, i.e., for certain BFM any translation scheme leads to an exponential blow-up in size.

### 7.3 RQ2: In practice, how does the size of a BFM compare to the size of models of the state-of-the-art modeling languages for behavioral variability?

We address this research question through an empirical evaluation where we apply our implementation of Alg. 11 to a large evaluation benchmark consisting of 46 subject systems (cf. Sect. 7.1.2).

Table 1. Evaluation results of applying Alg. 11 to various case studies. For each system, we report metrics from the input FTS  $\mathcal{F}$ —number of states ( $|S|$ ) and transitions ( $|\delta|$ ), along with their total ( $|\mathcal{F}|$ ) and the sum of the size of each of their components ( $\sum |\mathcal{F}_{\text{comp}}|$ )—and the resulting BFM  $\mathcal{B}$ —number of events ( $|E|$ ), conflicts ( $|\#|$ ), and causalities ( $|\mapsto|$ ), along with their total ( $|\mathcal{B}|$ ) and the sum of the size of each of their components ( $\sum |\mathcal{B}_{\text{comp}}|$ ).

| System                                                                                        | FTS Metrics |            |                 |                                    | BFM Metrics |              |             |                 |                                    |
|-----------------------------------------------------------------------------------------------|-------------|------------|-----------------|------------------------------------|-------------|--------------|-------------|-----------------|------------------------------------|
|                                                                                               | $ S $       | $ \delta $ | $ \mathcal{F} $ | $\sum  \mathcal{F}_{\text{comp}} $ | $ E $       | $ bcq_{\#} $ | $ \mapsto $ | $ \mathcal{B} $ | $\sum  \mathcal{B}_{\text{comp}} $ |
| <b>Systems without composition</b>                                                            |             |            |                 |                                    |             |              |             |                 |                                    |
| Cleaning Robot                                                                                | 6           | 10         | 16              | —                                  | 10          | 6            | 6           | 22              | —                                  |
| Card Payment Terminal                                                                         | 13          | 17         | 30              | —                                  | 17          | 7            | 16          | 40              | —                                  |
| Coffee Vending Machine                                                                        | 14          | 22         | 36              | —                                  | 22          | 11           | 20          | 53              | —                                  |
| Soda Vending Machine                                                                          | 10          | 13         | 23              | —                                  | 13          | 4            | 11          | 28              | —                                  |
| Soup Vending Machine                                                                          | 12          | 23         | 35              | —                                  | 23          | 13           | 21          | 57              | —                                  |
| Mine Pump Controller (C)                                                                      | 26          | 41         | 67              | —                                  | 41          | 18           | 40          | 99              | —                                  |
| Mine Pump Controller with State (C    S)                                                      | 47          | 70         | 117             | —                                  | 70          | 26           | 66          | 162             | —                                  |
| Methane (M)                                                                                   | 3           | 2          | 5               | —                                  | 2           | 0            | 1           | 3               | —                                  |
| Pump (P)                                                                                      | 3           | 2          | 5               | —                                  | 2           | 0            | 1           | 3               | —                                  |
| Water (W)                                                                                     | 4           | 4          | 8               | —                                  | 4           | 1            | 2           | 7               | —                                  |
| <b>Vending Machine system in parallel composition (no synchronization)</b>                    |             |            |                 |                                    |             |              |             |                 |                                    |
| Coffee    Soda                                                                                | 140         | 402        | 542             | 59                                 | 35          | 13           | 31          | 79              | 81                                 |
| Coffee    Soup                                                                                | 168         | 586        | 754             | 71                                 | 45          | 28           | 41          | 114             | 112                                |
| Soda    Soup                                                                                  | 120         | 386        | 506             | 58                                 | 36          | 24           | 32          | 92              | 87                                 |
| Coffee    Soda    Soup                                                                        | 1680        | 8044       | 9724            | 94                                 | 58          | 35           | 52          | 145             | 140                                |
| <b>Vending Machine system in parallel composition with synchronization over common events</b> |             |            |                 |                                    |             |              |             |                 |                                    |
| Coffee    Soda                                                                                | 71          | 171        | 242             | 59                                 | 31          | 14           | 33          | 78              | 81                                 |
| Coffee    Soup                                                                                | 107         | 290        | 397             | 71                                 | 39          | 24           | 44          | 107             | 112                                |
| Soda    Soup                                                                                  | 87          | 242        | 329             | 58                                 | 34          | 22           | 34          | 90              | 87                                 |
| Coffee    Soda    Soup                                                                        | 549         | 1956       | 2505            | 94                                 | 48          | 27           | 59          | 134             | 140                                |
| <b>Mine Pump system in parallel composition (no synchronization)</b>                          |             |            |                 |                                    |             |              |             |                 |                                    |
| C    M                                                                                        | 78          | 175        | 253             | 72                                 | 43          | 20           | 41          | 104             | 103                                |
| C    P                                                                                        | 78          | 175        | 253             | 72                                 | 43          | 20           | 41          | 104             | 103                                |
| C    W                                                                                        | 104         | 268        | 372             | 75                                 | 45          | 21           | 42          | 108             | 107                                |
| C    M    W                                                                                   | 312         | 1012       | 1324            | 80                                 | 47          | 18           | 43          | 108             | 110                                |
| C    P    M                                                                                   | 234         | 681        | 915             | 77                                 | 45          | 17           | 42          | 104             | 106                                |
| C    P    W                                                                                   | 312         | 1012       | 1324            | 80                                 | 47          | 18           | 43          | 108             | 110                                |
| C    S    M                                                                                   | 141         | 304        | 445             | 122                                | 72          | 26           | 67          | 165             | 170                                |
| C    S    P                                                                                   | 141         | 304        | 445             | 122                                | 72          | 26           | 67          | 165             | 170                                |
| C    S    W                                                                                   | 188         | 468        | 656             | 125                                | 74          | 27           | 68          | 169             | 174                                |
| C    P    M    W                                                                              | 936         | 3660       | 4596            | 85                                 | 49          | 19           | 44          | 112             | 113                                |
| C    S    M    W                                                                              | 564         | 1780       | 2344            | 130                                | 76          | 36           | 69          | 181             | 177                                |
| C    S    P    M                                                                              | 423         | 1194       | 1617            | 127                                | 74          | 35           | 68          | 177             | 173                                |
| C    S    P    W                                                                              | 564         | 1780       | 2344            | 130                                | 76          | 36           | 69          | 181             | 177                                |
| C    S    P    M    W                                                                         | 1692        | 6468       | 8160            | 135                                | 78          | 22           | 70          | 170             | 180                                |
| <b>Mine Pump system in parallel composition with synchronization over common events</b>       |             |            |                 |                                    |             |              |             |                 |                                    |
| C    M                                                                                        | 78          | 175        | 253             | 72                                 | 43          | 20           | 41          | 104             | 103                                |
| C    P                                                                                        | 27          | 38         | 65              | 72                                 | 41          | 21           | 43          | 105             | 103                                |
| C    W                                                                                        | 104         | 268        | 372             | 75                                 | 45          | 21           | 42          | 108             | 107                                |
| C    M    W                                                                                   | 312         | 1012       | 1324            | 80                                 | 47          | 18           | 43          | 108             | 110                                |
| C    P    M                                                                                   | 81          | 168        | 249             | 77                                 | 43          | 17           | 44          | 104             | 106                                |
| C    P    W                                                                                   | 108         | 260        | 368             | 80                                 | 45          | 18           | 45          | 108             | 110                                |
| C    S    M                                                                                   | 141         | 304        | 445             | 122                                | 72          | 26           | 67          | 165             | 170                                |
| C    S    P                                                                                   | 42          | 58         | 100             | 122                                | 70          | 24           | 69          | 163             | 170                                |
| C    S    W                                                                                   | 188         | 468        | 656             | 125                                | 74          | 27           | 68          | 169             | 174                                |
| C    P    M    W                                                                              | 324         | 996        | 1320            | 85                                 | 47          | 18           | 46          | 111             | 113                                |
| C    S    M    W                                                                              | 564         | 1780       | 2344            | 130                                | 76          | 36           | 69          | 181             | 177                                |
| C    S    P    M                                                                              | 126         | 258        | 384             | 127                                | 72          | 36           | 70          | 178             | 173                                |
| C    S    P    W                                                                              | 168         | 400        | 568             | 130                                | 74          | 37           | 71          | 182             | 177                                |
| C    S    P    M    W                                                                         | 504         | 1536       | 2040            | 135                                | 76          | 24           | 72          | 172             | 180                                |



Table 2. Estimation of the minimum number of bits required to encode each model. For each system FTS, we report the number of bits required to encode one transition ( $|Bits|_{\delta}$ ) and the total number of bits ( $|Bits|_{\mathcal{T}}$ ). For each system BFM, we report the number of bits required to encode one conflict ( $|Bits|_{\#}$ ), one causality ( $|Bits|_{\mapsto}$ ) and the total number of bits ( $|Bits|_{\mathcal{G}}$ ).

| System                                                                                        | FTS Metrics       |                        | BFM Metrics   |                    |                        |
|-----------------------------------------------------------------------------------------------|-------------------|------------------------|---------------|--------------------|------------------------|
|                                                                                               | $ Bits _{\delta}$ | $ Bits _{\mathcal{T}}$ | $ Bits _{\#}$ | $ Bits _{\mapsto}$ | $ Bits _{\mathcal{G}}$ |
| <b>Systems without composition</b>                                                            |                   |                        |               |                    |                        |
| Cleaning Robot                                                                                | 9                 | 90                     | 20            | 14                 | 204                    |
| Card Payment Terminal                                                                         | 12                | 204                    | 34            | 22                 | 590                    |
| Coffee Vending Machine                                                                        | 12                | 264                    | 44            | 27                 | 1024                   |
| Soda Vending Machine                                                                          | 12                | 156                    | 26            | 17                 | 291                    |
| Soup Vending Machine                                                                          | 12                | 276                    | 46            | 28                 | 1186                   |
| Mine Pump Controller (C)                                                                      | 15                | 615                    | 82            | 47                 | 3356                   |
| Mine Pump Controller with State (C    S)                                                      | 17                | 1190                   | 140           | 77                 | 8722                   |
| Methane (M)                                                                                   | 5                 | 10                     | 4             | 3                  | 3                      |
| Pump (P)                                                                                      | 5                 | 10                     | 4             | 3                  | 3                      |
| Water (W)                                                                                     | 5                 | 20                     | 8             | 6                  | 20                     |
| <b>Vending Machine system in parallel composition (no synchronization)</b>                    |                   |                        |               |                    |                        |
| Coffee    Soda                                                                                | 21                | 8442                   | 70            | 41                 | 2181                   |
| Coffee    Soup                                                                                | 21                | 12306                  | 90            | 51                 | 4611                   |
| Soda    Soup                                                                                  | 19                | 7334                   | 72            | 42                 | 3072                   |
| Coffee    Soda    Soup                                                                        | 27                | 217188                 | 116           | 64                 | 7388                   |
| <b>Vending Machine system in parallel composition with synchronization over common events</b> |                   |                        |               |                    |                        |
| Coffee    Soda                                                                                | 19                | 3249                   | 62            | 36                 | 2056                   |
| Coffee    Soup                                                                                | 19                | 5510                   | 78            | 45                 | 3852                   |
| Soda    Soup                                                                                  | 19                | 4598                   | 68            | 40                 | 2856                   |
| Coffee    Soda    Soup                                                                        | 25                | 48900                  | 96            | 54                 | 5778                   |
| <b>Mine Pump system in parallel composition (no synchronization)</b>                          |                   |                        |               |                    |                        |
| C    M                                                                                        | 19                | 3325                   | 86            | 49                 | 3643                   |
| C    P                                                                                        | 19                | 3325                   | 86            | 49                 | 3643                   |
| C    W                                                                                        | 19                | 5092                   | 90            | 51                 | 3942                   |
| C    M    W                                                                                   | 23                | 23276                  | 94            | 53                 | 4159                   |
| C    P    M                                                                                   | 21                | 14301                  | 90            | 51                 | 3852                   |
| C    P    W                                                                                   | 23                | 23276                  | 94            | 53                 | 4159                   |
| C    S    M                                                                                   | 21                | 6384                   | 144           | 79                 | 8893                   |
| C    S    P                                                                                   | 21                | 6384                   | 144           | 79                 | 8893                   |
| C    S    W                                                                                   | 21                | 9828                   | 148           | 81                 | 9356                   |
| C    P    M    W                                                                              | 25                | 91500                  | 98            | 55                 | 4184                   |
| C    S    M    W                                                                              | 25                | 44500                  | 152           | 83                 | 10135                  |
| C    S    P    M                                                                              | 23                | 27462                  | 148           | 81                 | 9652                   |
| C    S    P    W                                                                              | 25                | 44500                  | 152           | 83                 | 10135                  |
| C    S    P    M    W                                                                         | 27                | 174636                 | 156           | 85                 | 10162                  |
| <b>Mine Pump system in parallel composition with synchronization over common events</b>       |                   |                        |               |                    |                        |
| C    M                                                                                        | 19                | 3325                   | 86            | 49                 | 3643                   |
| C    P                                                                                        | 15                | 570                    | 82            | 47                 | 3579                   |
| C    W                                                                                        | 19                | 5092                   | 90            | 51                 | 3942                   |
| C    M    W                                                                                   | 23                | 23276                  | 94            | 53                 | 4159                   |
| C    P    M                                                                                   | 19                | 3192                   | 86            | 49                 | 3618                   |
| C    P    W                                                                                   | 19                | 4940                   | 90            | 51                 | 3915                   |
| C    S    M                                                                                   | 21                | 6384                   | 144           | 79                 | 8893                   |
| C    S    P                                                                                   | 17                | 986                    | 140           | 77                 | 8813                   |
| C    S    W                                                                                   | 21                | 9828                   | 148           | 81                 | 9356                   |
| C    P    M    W                                                                              | 23                | 22908                  | 94            | 53                 | 4130                   |
| C    S    M    W                                                                              | 25                | 44500                  | 152           | 83                 | 10135                  |
| C    S    P    M                                                                              | 19                | 4902                   | 144           | 79                 | 9706                   |
| C    S    P    W                                                                              | 21                | 8400                   | 148           | 81                 | 9895                   |
| C    S    P    M    W                                                                         | 23                | 35328                  | 152           | 83                 | 9624                   |

We consider single FTSs and parallel compositions of FTSs, distinguishing between compositions with and without synchronization. In the absence of synchronization, all actions interleave. For

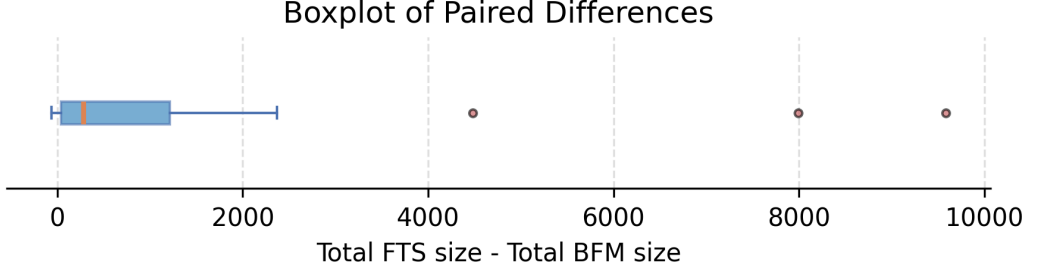


Fig. 13. Distribution of per-instance size differences between FTS and BFM; Positive values indicate larger FTS models.

parallel compositions, we first translate each component FTS to a BFM using Alg. 11, before taking the parallel composition of the resulting BFMs. This procedure preserves trace equivalence (cf. Thm. 6.27).

For each system, we report the size-related metrics, on which we expand below, for both the input FTS and the resulting BFM in Tbl. 1. Our size-related metrics are based on Def. 7.2. Concretely, for each FTS, we include the number of states and the number of transitions along with their sum. Additionally, we report on the sum of the sizes of all component FTSs ( $\sum |\mathcal{F}_{comp}|$ ). For each BFM, we include the number of events, the number of conflicts (each represented as a biclique  $(A, B)$ ), the number of causality relations, and their total count. Furthermore, we report on the sum of the sizes of all component BFMs ( $\sum |\mathcal{B}_{comp}|$ ). To ensure a fair comparison between the sizes of the two models, we have additionally calculated the total memory requirements for each of the models (cf. Tbl. 2), which corroborate the results reported in Tbl. 1.

Table 1 shows that FTSs are smaller in size for most non-compositional systems, whereas for large compositional systems, BFMs offer a far smaller representation than FTSs. This is reflected in the size reduction from the FTS ( $|\mathcal{F}|$ ) to the corresponding BFM ( $|\mathcal{B}|$ ). The BFM size never exceeds 200, whereas the corresponding FTS typically involves hundreds or even thousands of elements, even in medium-sized systems. The size reduction is particularly apparent in large systems; e.g., in the largest case study,  $C||S||P||M||W$  (non-synchronized), we observe a size reduction of up to 98.55%. Overall, we observe a median size reduction of 70.81%, with a standard deviation of 52.15%.

To statistically assess the size reduction between the FTS and BFM models, we performed a Wilcoxon signed-rank test [113] that compares the distributions of model sizes of (compositional) FTSs and their translations to BFMs.

- **Null hypothesis ( $H_0$ ):** Algorithm 11 returns models of equal size.
- **Alternative hypothesis ( $H_1$ ):** Algorithm 11 returns models of different sizes.

A  $w$ -statistic close to 0 indicates no difference in model sizes, a negative value indicates smaller FTS models, while a positive value indicates smaller BFM models.

We considered  $n = 46$  paired observations and analyzed the differences in model size (FTS minus BFM). The distribution is summarized as follows: minimum =  $-63$ , first quartile =  $37.75$ , median =  $280$ , third quartile =  $1214.25$ , and maximum =  $9583$  (mean =  $992.33$ , standard deviation =  $1911.51$ ). Figure 13 shows a boxplot of the per-instance size differences between FTS and BFM. The Wilcoxon signed-rank test yielded a  $w$ -statistic of  $1009$  and a  $p$ -value of  $0.00000015$ . Since the  $p$ -value is less than the significance threshold  $\alpha = 0.05$ , we can safely reject  $H_0$ . The positive  $w$ -statistic confirms that Alg. 11 produces BFMs with statistically significantly smaller sizes than the source FTSs.

The only FTSs where Alg. 11 returned BFMs of larger size were non-compositional models and two compositional models of the Mine Pump system with synchronization but minimal interleaving. For the non-compositional models, this result was expected since BFMs were designed to model systems compositionally. In the absence of composition, their main strength cannot be exploited, leading to slightly larger sizes compared to FTSs (cf. “Systems without composition” in Tbl. 1). The same holds for systems with a lot of synchronization, where composition only allows few interleavings (cf.  $C||P$  and  $C||S||P$  in Tbl. 1). Concerning the sizes of all component FTSs and all component BFMs, we have  $\sum |\mathcal{B}_{comp}| < 2 \cdot \sum |\mathcal{F}_{comp}|$ .

The results reported in Tbl. 1 concern the size metrics based on Def. 7.2, where conflicts are represented as bicliques (cf. Sect. 7.1.3). However, the elements of the different components reported in Tbl. 1 have different types and thus, may require different amounts of memory. To ensure that the comparison reported in Tbl. 1 is fair, we calculated the required memory for each component and reported the results both for individual components and for the full models ( $|Bits_{\mathcal{F}}|$  and  $|Bits_{\mathcal{B}}|$ ) in Tbl. 2. For states and events, the reported sizes in Tbl. 1 directly correspond to the amount of memory needed to store them and no further calculations are needed. Thus, we focus on the size of transitions ( $|Bits_{\delta}|$ ), conflict- ( $|Bits_{\#}|$ ), and causal sets ( $|Bits_{\rightarrow}|$ ).

- $|Bits_{\delta}| = \lceil \log_2(|S|) \rceil + \lceil \log_2(|E|) \rceil + \lceil \log_2(|A|) \rceil$ : Transitions comprise source- and target state, and action labels. For storing  $n$  numbers, one needs  $\lceil \log_2(n) \rceil$  bits. Thus, the number of bits needed to store a transition is the sum of the logarithm of the source- and target state and the action label.
- $|Bits_{\#}| = |E| + |E|$ : For conflict sets, the components are subsets of events. To represent a subset of  $|E|$  events, one needs  $|E|$  bits.
- $|Bits_{\rightarrow}| = |E| + \lceil \log_2(|E|) \rceil$ : A causality relation consists of a set of events, for which we need  $|E|$  to store it, and an event, for which we need  $\lceil \log_2(|E|) \rceil$  to store it.

Although the bit size for individual transitions is smaller than the corresponding conflict and causal pairs, because of the exponential growth in the number of transitions in FTS, the total size of the FTS models typically grows beyond the total size of BFM models as reported in Tbl. 2, especially for the larger systems. This corroborates the fairness of the comparison reported in Tbl. 1.

In conclusion, we observe that in nearly all cases in the benchmark, BFMs are strictly smaller in size than the flattened FTSs.

**Answer to RQ2:** In practice, how does the size of a BFM compare to the size of models of the state-of-the-art modeling languages for behavioral variability?

In practice, we observe that BFMs are significantly smaller in size than acyclic FTSs in nearly all cases (37 out of 46). The size reductions are particularly notable in large, highly interleaved systems, where BFMs compress the representation by up to 98.55%. A Wilcoxon signed-rank test further confirmed that our acyclic FTS to BFM translation yields models that are on average significantly smaller in size.

## 7.4 Threats to Validity

In this section, we discuss threats to validity concerning the two research questions and the measures we have taken (or plan to take in future research) to address them.

*Theoretical Evaluation (RQ1).* BFMs, as formalized in this paper, can only model acyclic behavior. Thus, even though BFMs and acyclic FTSs are trace equivalent, FTSs in general are more expressive than BFMs. To the best of our knowledge, there currently exists no work on modeling cyclic

behavior using event structures with a finite representation which we need for BFM s since their aim is to be used in practice. Finding a finite symbolic representation requires some foundational work on event structures which we plan to address in future work, e.g., through tagging the events with an inductive symbolic index.

*Empirical Evaluation (RQ2).* First, there is a threat regarding the representativeness of the evaluation benchmark. Some of the FTSs used in our evaluation are constructed by researchers to showcase the benefits of FTSs [33] and therefore do not pose any threat with respect to bias towards our proposed approach. We selected these systems because they are widely used and recognized in the literature (see, e.g., [16]). However, they do not necessarily represent diverse industrial models. This issue is symptomatic of a broader challenge for the community: the lack of diverse and representative datasets [101]. Despite this limitation, our study makes a step forward by evaluating on larger FTSs than those typically used for evaluations.

Another risk is the presence of faults in algorithms and code. For the algorithms, we mitigated this risk by producing formal proofs of their correctness (cf. Sects. 5 and 6). For the implementation, we mitigated the risk of bugs by applying good object-oriented practices (e.g., design patterns, modularity) and performing individual integration tests for each translation and composition algorithm.

As FTSs and BFM s are non-isomorphic representations, there is also a risk that our choice of metric for comparing them is biased towards either one or the other. To make this transparent, we report all structural elements of the FTSs and BFM s in Tbl. 1. The aggregated metrics  $|\mathcal{F}|$  and  $|\mathcal{B}|$  are computed by summing up all these elements. One could argue that certain elements do not have direct counterparts in the two representations such as states in FTSs and conflicts in BFM s. However, excluding these from the total metric would be unjustified, as they remain essential to the structure and semantics of each model.

## 8 RELATED WORK

In this section, we first relate our work to the compositional semantics of concurrency in Sect. 8.1, after which we review related work regarding compositionality of structural and behavioral models of SPLs in Sects. 8.2 and 8.3, respectively. Subsequently, we focus on unifying structural and behavioral modeling of SPLs in Sect. 8.4. Finally, we provide an overview of the available body of work concerning the analysis of behavioral models of SPLs in Sect. 8.5.

### 8.1 Compositional Semantics of Concurrency

Compositionality [46] (also called congruence for behavioral equivalences [80]) is a central notion to semantics of concurrent programming and specification languages, as well as in formal verification. Prominent examples of proposed semantic models in this domain are event structures [83, 111] and partial order semantic models [89]. These were mainly proposed to model true concurrency; they disentangle genuine orders of events that are semantically sequential (also called causality) from orders that are an artifact of scheduling concurrent events. Event structure models evolved over time and can currently capture other semantic models defined by a total order (such as TSs) [63]. In our case, explicit causality and conflict in event structures are essential to disentangle the behavior of different features in BFM s. Moreover, the extension of event structures to BESs [73] is essential to specify conflicts among groups of events. Using BESs in BFM s allows us to model concurrent behavior.

## 8.2 Compositionality of Structural Models of SPLs

The composition of FMs has been studied in the literature [2–4, 45], with the aim of scaling up the analysis by separating different concerns into different FMs. In this approach, composition is realized through an insert and a merge operator. The insert operator either inserts newly created elements, or (a part of) a so-called aspect, into a base FM. The merge operator merges two FMs that share several features. In our approach, we also take a compositional view of FMs: we construct an FM incrementally in a bottom-up fashion, and we also allow to insert single features or new FMs into an existing FM. We improve upon the state of the art, since previous work on structural models of SPLs considers neither feature constraints nor behavior in the compositional approach.

## 8.3 Compositionality of Behavioral Models of SPLs

There is a plethora of behavioral models originating from foundational models of computation (e.g., finite automata [96]) and concurrency theory (e.g., labeled transition systems, process algebras [1], and event structures [83, 111]). Some models have been extended to capture behavioral variability (cf. a formal comparison of a number of these extensions [15, 25]); notable examples are FTSS [36, 40], featured finite state machines (FFSMs) [61], delta statecharts [77], and variants of modal transition systems [8, 17, 74, 76] and of featured process calculi [58, 62, 65, 78, 108]. In our research, we started by considering composition operators of FMs on FTSSs and FFSMs, both of which are equipped with a notion of parallel composition [9]. However, their composition mechanisms do not allow for associating behavior to features and to capture the intricate interactions among different features in a composition (e.g., reminiscent of conjunctive or disjunctive operators as used in FMs). In other words, one has to repeat substantial pieces of behavior across different features. Particularly, when there are cross-tree causal dependencies (e.g., a part of a feature behavior happening before and another part appearing after the behavior of another feature), the behavior of each feature has to be copied in both specifications to allow for synchronization between them. The composition operators provided by SPL process calculi [58, 62, 65, 78, 108] offer more flexibility. However, we improve upon the state of the art, since previous work on behavioral models of SPLs does not unify structural and behavioral variability and there is no known technique to date to provide a behavioral specification of individual features.

## 8.4 Unifying Structural and Behavioral SPL Modeling

We are aware of only few approaches aimed at unifying structural and behavioral variability. We discuss two of them here to argue that we improve upon the state of the art, since previous work on unifying models of SPLs does not cater for modeling behavior by feature and compositionally.

Supervisory Control Theory (SCT) [90] is a model-based approach to automatically synthesize a controller from specification. It produces a correct-by-construction controller, typically in the form of automata. SCT has been applied to SPLs [19]; there, it was shown how the CIF 3 toolset [109] can deal with structural as well as behavioral variability. The approach manages both product structural configuration, and product behavior synthesis. To achieve this it uses behavioral requirements (in the form of admissible scenarios given by state invariants, event orderings, and guards on events akin to FTSSs). Also, the SCT approach [11] unifies structural and behavioral variability of service product lines. It guarantees that the synthesized orchestrations respect both an FM and contract-based service product line behavior (modeled as featured modal contract automata). The approach has also been implemented in the FMCAT tool [13]. Both approaches were extended to cater for dynamic feature reconfigurations [12, 104]. These approaches use the flattened representation of FMs in propositional logic, whereas our approach is inherently compositional and paves the way for compositional synthesis of components.

Clafer with behavior [70] is a general-purpose modeling language for both structural and behavioral variability. It unifies FMs with classes, meta-models, and behavior by providing feature modeling by means of a special-purpose constraint language akin to Alloy [69]. Behavioral variability is introduced by means of hierarchical UML state diagrams and automata (in a superimposed fashion akin to FTSs). Additional behavioral constraints have the form of temporal logic patterns enabling (bounded) model checking. Clafer thus provides an integrated approach to link behavior to structure through constraints as well as alphabet variability akin to color-blind I/O-alternating transition systems [75] and transition variability akin to FTSs. Clafer offers first-class support for architectural modeling, but no first-class constructs for mapping features to behavior or for compositional behavior, and no support for controller synthesis. Our proposed formalism offers dedicated conflict and disjunctive causality operators with a structural event-level composition discipline that matches FM composition operators, enabling a compositional approach to behavior analysis and synthesis.

### 8.5 Analysis of Behavioral Models of SPLs

SPLs are concerned with massively reused critical software components. Thus, it is important to demonstrate their correct structural and behavioral (re)configuration. The automated analysis of structural variability models has more than three decades of history. Examples of such analyses include detecting dead or false optional *features* or edits in FMs [23, 106, 107]. However, the automated analysis of behavioral variability models has a shorter history. There are only few available results, mostly concerning family-based model checking [17, 18, 21, 22, 32, 39, 56, 66, 76]. More recently, some attention has been paid to ambiguity detection in behavioral models, including the identification of dead or false optional *transitions* and deadlocks in FTSs [16]. In addition, a number of family-based (model-based) testing approaches have been proposed. These techniques address automated test case generation in the context of variability. Some of these works [49–52, 55] explore techniques for test case selection and generation based on coverage criteria, sampling strategies, or similarity and dissimilarity metrics. Others focus on mutation analysis, aiming to support test generation or evaluate the fault-detection ability of existing test suites [48, 54].

Other analysis approaches operate directly on the source code of SPLs, often obtained by adapting existing tools to deal with variability. In this case, structural analysis is related to static (program) analysis [43, 84], which includes the detection of bugs in the code (e.g., using a variable before initialization) but also the identification of code that is redundant or unreachable, whereas behavioral analysis includes software model-checking tools such as ProMoVer [97] with variability annotations [93], the SPLverifier [7] tool chain for Java and CPAchecker [26] for C code.

We are not aware of any other related work that unifies the structural and behavioral analysis in a compositional manner (since the underlying models for such analyses has not been available). Our integrated proposal of BFM enables the analysis of both structural and behavioral anomalies.

## 9 DISCUSSION

In the following, we include a short discussion on the relative (dis)advantages of FTSs in Sect. 9.1, on the scalability of translating FTSs in Sect. 9.2 and on composing and decomposing them in Sect. 9.3. Furthermore, we discuss the location of events in BFMs in Sect. 9.4.

### 9.1 On the Relative (Dis)Advantages of FTSs

Through our running example and empirical evaluations, it becomes clear that FTSs lack sufficient composition operators to specify the behavior of individual features without repetition and duplication involving the actions of other features. This leads to less succinct specifications that are more difficult to design, modify, or reuse. However, FTSs also offer a relative advantage: they build



upon the familiar abstraction of TSs (and state machines). While the structure of BFM is inherited from the widely known structure of FMs, gaining familiarity with the level of abstraction offered by events, actions, causality, and conflict relations requires some training. Furthermore, unlike FTSs, BFMs are only able to express acyclic behavior until now.

Regarding analysis tasks on FTSs (such as model checking [16, 22, 35, 39], testing [49, 53, 54], or model translation as performed in this work), the dominant computational cost typically stems from graph traversal, and thus depends on the number of states and transitions. For BFMs, similar tasks are expected to depend primarily on the number of causalities, with conflicts playing a comparatively minor role (i.e., graph traversal versus table lookup). As shown in Tbl. 1, the number of causalities in BFMs is consistently lower than the number of transitions in the corresponding FTSs, even when the overall BFM size exceeds that of the FTS, suggesting a potential practical performance advantage of the BFM structure.

## 9.2 On the Scalability of Translating FTSs

The implementation for translating parallel compositions of acyclic FTSs to BFMs (cf. Sect. 7.1.3) iterates over all pairs of FTS transitions, resulting in an overall complexity of  $O(t^2)$ , where  $t$  denotes the number of transitions. In practice, however, our approach primarily relies on composing results obtained from individual components. This allows us to apply the translation algorithm at the component level, thereby mitigating scalability concerns. Owing to the relatively small size of these components, the translation consistently executes in under one second in our experiments. Nevertheless, we acknowledge that scalability may become a concern for larger systems.

There are several ways to optimize the translation algorithms, which was not the goal of this paper. For Alg. 11, efficiency could be improved by pre-selecting candidate transition pairs instead of considering all pairs, thereby avoiding unnecessary iterations. Additionally, conflict detection could be reformulated as a reachability query on the product graph, enabling the use of established techniques such as partition refinement [85] or on-the-fly strongly connected component (SCC) removal [99] to speed up detection in sparse graphs. In Alg. 13, the naive configuration generator is the most computationally expensive step and the primary target for optimization. Another direction would be to parallelize suitable parts of the algorithms.

Model translation—and, more broadly, model construction—is inherently expensive. Although there is growing work on automatically learning FTSs [44, 60, 100, 102], most existing benchmarks, such as [101], focus on relatively small FTSs. In [16], FTS benchmarks with up to 11236 states are analyzed. Our work provides further empirical evidence of both the feasibility and the computational challenges involved in handling such large models.

## 9.3 On Composing and Decomposing FTSs

To the best of our knowledge, only one composition operator for FTSs has been presented in the literature. This is exactly the parallel composition of FTSs presented in Def. 6.23. In the literature, the parallel composition is usually defined between two FTSs, where the FTSs additionally contain atomic propositions and a labeling function for labeling states with a set of atomic propositions [36, Def. 9].

However, to the best of our knowledge, there is no work on decomposing an FTS into FTSs for each feature, reflecting the behavior of each particular feature. We provide evidence, using the cleaning robot SPL, that this can lead to a significant blowup in the sizes of the feature FTSs because behavior has to be shared between FTSs.

*Example 9.1 (Decomposing the FTS of the cleaning robot SPL).* Consider the FTS  $\mathcal{F}_r$  of the cleaning robot as defined in Example 2.25 and shown in Fig. 3. Then we assign actions (and thus behavior)

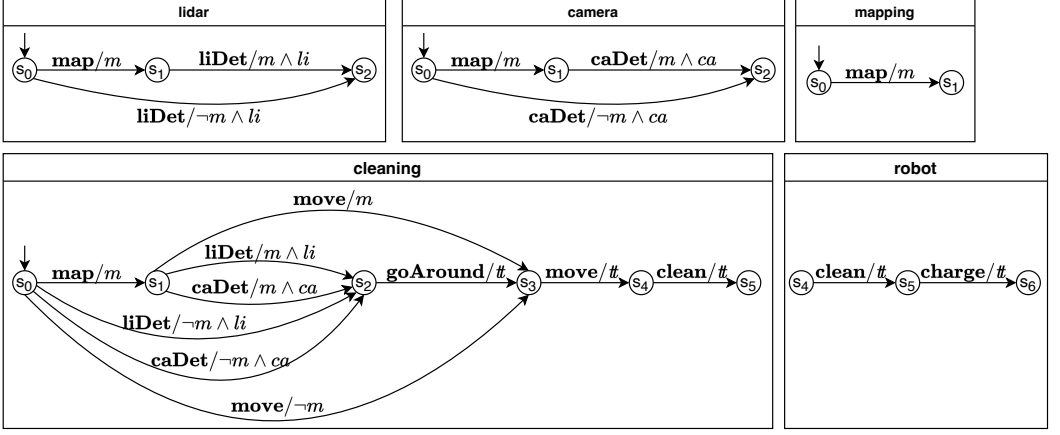


Fig. 14. A decomposed version of the FTS for the cleaning robot SPL (see Example 2.25) into an FTS for each feature.

to the features of the FM as in the BFM of the cleaning robot, see Example 4.6. That is, the FTSS  $\mathcal{F}_i$ ,  $i \in \{r, m, c, o, li, ca\}$ , for the features *robot*, *mapping*, *cleaning*, *obstacleDetection*, *lidar*, and *camera*, respectively, have to contain at least the sets of actions  $\{\text{charge}\}$ ,  $\{\text{map}\}$ ,  $\{\text{move}, \text{goAround}, \text{clean}\}$ ,  $\emptyset$ ,  $\{\text{liDet}\}$ , and  $\{\text{caDet}\}$ , respectively.

However, the parallel composition of the  $\mathcal{F}_i$ ,  $i \in \{r, m, c, o, li, ca\}$ , has to be  $\mathcal{F}_r$ . This means that some  $\mathcal{F}_i$  have to share behavior, i.e., include the same transitions, for a correct synchronization: Assume that  $\mathcal{F}_m$  contains only a transition labeled with *map*. Then the FTSS  $\mathcal{F}_{ca}$  and  $\mathcal{F}_{li}$  have to include the action *map* in addition to *caDet* and *liDet*, respectively, because the occurrence of these actions depends on the action *map*. Now the FTS  $\mathcal{F}_c$  has to ensure that the action *move* has to happen after the action *map* if the feature *mapping* is included, and that either *liDet* or *caDet* happens before the action *goAround*. Lastly, the FTS  $\mathcal{F}_r$  has to ensure that the action *clean* happens before the action *charge*. A graphical representation of the resulting FTSSs can be seen in Fig. 14.

Note that other decompositions into FTSSs related to the features of the FM are of course possible, but they are likely to require shared behavior as well.

As in Sect. 7.3, we can compare the size of the “flattened” FTS of the cleaning robot SPL (cf. Example 2.25), as used throughout the paper, and its BFM (cf. Example 4.6). The result is reported in the last row of Tbl. 3; the FTS contains less elements than the BFM. However, we can also compare the sizes of the models modeling the behavior of single features, i.e., the sizes of the FTSSs in Fig. 14 and sub-BFMs in Example 4.6. The results are shown in Tbl. 3, where  $|\mathcal{F}|$  and  $|\mathcal{B}|$  are the sums of the considered elements (states and transitions for FTSSs and events, conflicts and causalities for BFMs), and the total  $\Sigma$  is the sum of all FTS/BFM models per feature. It can be seen that, even though for the “flattened” models the FTS is smaller, the sizes of the models per feature are mostly bigger for FTSSs. Thus, when comparing the sum of the sizes of the models per feature, shown in  $\Sigma$ , the sum of all FTS models is 1.84 times higher than the sum of all BFM models.

The decomposition of the FTS of the cleaning robot SPL illustrated in Example 9.1 shows that, since each action can be associated with multiple features, decomposing an FTS intrinsically requires to duplicate transitions among the component FTSSs to ensure a correct synchronization. This redundancy, which entangles the behavior of different features, makes it harder to reuse

Table 3. Sizes of “flattened” and compositional FTSs  $\mathcal{F}$  and BFM/FESs  $\mathcal{B}$  for the cleaning robot SPL

| System      | FTS |        |             |        | BFM |    |         |             |        |
|-------------|-----|--------|-------------|--------|-----|----|---------|-------------|--------|
|             | S   | \delta | \mathcal{F} | \Sigma | E   | \# | \mapsto | \mathcal{B} | \Sigma |
| lidar       | 3   | 3      | 6           | 35     | 1   | 0  | 0       | 1           | 19     |
| camera      | 3   | 3      | 6           |        | 1   | 0  | 0       | 1           |        |
| mapping     | 2   | 1      | 3           |        | 1   | 0  | 0       | 1           |        |
| cleaning    | 5   | 10     | 15          |        | 4   | 1  | 2       | 7           |        |
| robot       | 3   | 2      | 5           |        | 1   | 2  | 6       | 9           |        |
| “flattened” | 6   | 10     | 16          |        | 8   | 3  | 7       | 25          |        |

the decomposed FTSs for other SPLs and exacerbates the state explosion problem related to the analyses of behavioral models.

#### 9.4 On the Location of Events in BFMs

BFMs enable us to determine which behavior is associated with which feature, and how the behavior of different features interact. This is not possible in FTSs, where the behavior of all features is modeled at the same time. This also has an effect on our translation algorithm from an FTS to a BFM (cf. Alg. 11): Every event is associated with the root feature of the FM, even if it might be possible to use domain knowledge about which feature exhibits which behavior to associate events to other features. Since FTSs do not contain information about which behavior is associated with which feature, it is impossible to automatically determine which feature an event should be associated with. The translation from FTS to BFM will therefore lead to a BFM where all behavior is associated with the root feature. This also means we lose information about the location of behavior when going from an BFM to an FTS.

## 10 CONCLUSION AND FUTURE WORK

To conclude the paper, we provide a summary and reflections as well as ideas for future work.

### 10.1 Summary and Reflections

In this paper, we propose BFMs as a compositional, unifying model for both structural and behavioral variability in SPLs. Our proposal fills two significant gaps in the literature: (i) the lack of compositionality in behavioral variability; and (ii) the isolated development of structural and behavioral variability. We fill this gap by introducing BFMs, where we extend an FM by attaching a BES to every feature and introducing constraints among the events of BESs associated to different features. In fact, we show that this is the same as extending BESs to cater for variability by mapping each event to a feature and a feature expression over features of an FM (which we call a FES). Thus, BFMs introduce behavioral variability in a structural model, while FESSs introduce structural variability in a behavioral model, but we show that the result is isomorphic.

We provide algorithms to translate (parallel compositions of) acyclic FTSs to BFMs and BFMs to acyclic FTSs, and show their trace equivalence. Furthermore, we evaluate our approach both theoretically and empirically; the empirical evaluation, conducted on a large set of FTSs, shows that BFMs can reduce the size of FTSs by up to 98.55%. Our evaluation shows that BFMs are as expressive as acyclic FTSs, yet they are much more succinct. These results demonstrate that BFMs lead to a more concise representation of feature behavior in SPLs, compared to state-of-the-art approaches. Ultimately, we believe the strength of BFMs lies in the ability to hierarchically structure, reuse, and

adapt behavioral models of features. Additionally, BFM models directly profit from previous work on SPL model checking of FTSS via the trace preserving translation.

## 10.2 Future Work

BFBMs enable a wealth of future developments which we plan to address in our ongoing research. We would like to investigate extensions to event structures (and thus BFBMs) that allow expressing cyclic behavior with a finite symbolic representation (cf. Sect. 7.4). Then we would like to extend our translation algorithms to translations between (cyclic) FTSS and BFBMs.

To extend BFBMs with more composition operators [9] currently not available in FBMs, we would like to identify and characterize typical composition patterns in SPLs. Ideally, these composition patterns, translated into composition operators in BFBMs, would compose both structural and behavioral aspects of SPLs.

We would like to explore how the compositional structure of BFBMs can be leveraged for compositional analysis [79, 81]. This would allow (i) analysis results to be reused among different SPLs, and (ii) focusing the analysis effort on the effect of changes in the case of dynamic SPLs. Synthesizing feature behavior from global or local properties of the system [27, 31, 59] as well as compositional model learning [68, 72, 82, 92] are other promising areas of future research.

Finally, we plan to study how to verify the correctness of feature behavior modeled with BFBMs, based on some of the well-established family-based analysis techniques mentioned in Sects. 2 and 8.

## ACKNOWLEDGMENTS

This work was supported by the European Union’s Horizon 2020 Framework Programme through the MSCA network REMARO (Grant Agreement No 956200), by the Italian project NODES (which has received funding from the MUR – M4C2 1.5 of PNRR with grant agreement no. ECS00000036) and by the CNR project “Formal Methods in Software Engineering 2.0”, CUP B53C24000720005; by the EPSRC project on Verified Simulation for Large Quantum Systems (VSL-Q), grant reference EP/Y005244/1; by the EPSRC project on Robust and Reliable Quantum Computing (RoaRQ), Investigation 009 Model-based monitoring and calibration of quantum computations (ModeMCQ) grant reference EP/W032635/1; by InnovateUK QAssure; and by the ITEA/InnovateUK projects GENIUS and ITEA/InnovateUK GreenCode.

During the preparation of this work, the authors used ChatGPT-4 (OpenAI) to provide ideas for how to prove Prop. 7.7, suggest algorithms and design patterns from the literature for addressing specific problems in the implementation, provide guidance on structuring the project’s README, adopt standard software engineering practices for testing, and assist in formatting output data. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

## REFERENCES

- [1] Luca Aceto, Anna Ingólfssdóttir, Kim Guldstrand Larsen, and Jiri Srba. 2007. *Reactive Systems: Modelling, Specification and Verification*. Cambridge University Press, UK. <https://doi.org/10.1017/CBO9780511814105>
- [2] Mathieu Acher, Philippe Collet, Philippe Lahire, and Robert B. France. 2010. Comparing Approaches to Implement Feature Model Composition. In *Proceedings of the 6th European Conference on Modelling Foundations and Applications (ECMFA’10) (Lecture Notes in Computer Science, Vol. 6138)*, Thomas Kühne, Bran Selic, Marie-Pierre Gervais, and François Terrier (Eds.). Springer, Germany, 3–19. [https://doi.org/10.1007/978-3-642-13595-8\\_3](https://doi.org/10.1007/978-3-642-13595-8_3)
- [3] Mathieu Acher, Philippe Collet, Philippe Lahire, and Robert B. France. 2010. Composing Feature Models. In *Proceedings of the 2nd International Conference on Software Language Engineering (SLE’09) (Lecture Notes in Computer Science, Vol. 5969)*, Mark van den Brand, Dragan Gasevic, and Jeff Gray (Eds.). Springer, Germany, 62–81. [https://doi.org/10.1007/978-3-642-12107-4\\_6](https://doi.org/10.1007/978-3-642-12107-4_6)

- [4] Mathieu Acher, Benoît Combemale, Philippe Collet, Olivier Barais, Philippe Lahire, and Robert B. France. 2013. Composing Your Compositions of Variability Models. In *Proceedings of the 16th International Conference on Model-Driven Engineering Languages and Systems (MODELS'13) (Lecture Notes in Computer Science, Vol. 8107)*, Ana Moreira, Bernhard Schätz, Jeff Gray, Antonio Vallecillo, and Peter J. Clarke (Eds.). Springer, Germany, 352–369. [https://doi.org/10.1007/978-3-642-41533-3\\_22](https://doi.org/10.1007/978-3-642-41533-3_22)
- [5] Jérôme Amilhastre, Marie-Catherine Vilarem, and Philippe Janssen. 1998. Complexity of minimum biclique cover and minimum biclique decomposition for bipartite domino-free graphs. *Discrete Appl. Math.* 86, 2–3 (1998), 125–144. [https://doi.org/10.1016/S0166-218X\(98\)00039-0](https://doi.org/10.1016/S0166-218X(98)00039-0)
- [6] Sven Apel, Don S. Batory, Christian Kästner, and Gunter Saake. 2013. *Feature-Oriented Software Product Lines: Concepts and Implementation*. Springer, Germany. <https://doi.org/10.1007/978-3-642-37521-7>
- [7] Sven Apel, Hendrik Speidel, Philipp Wendler, Alexander von Rhein, and Dirk Beyer. 2011. Detection of feature interactions using feature-aware verification. In *Proceedings of the 26th International Conference on Automated Software Engineering (ASE'11)*. IEEE, USA, 372–375. <https://doi.org/10.1109/ASE.2011.6100075>
- [8] Patrizia Asirelli, Maurice H. ter Beek, Alessandro Fantechi, and Stefania Gnesi. 2011. Formal Description of Variability in Product Families. In *Proceedings of the 15th International Software Product Lines Conference (SPLC'11)*. IEEE, USA, 130–139. <https://doi.org/10.1109/SPLC.2011.34>
- [9] J. C. M. Baeten, T. Basten, and M. A. Reniers. 2010. *Process Algebra: Equational Theories of Communicating Processes*. Cambridge Tracts in Theor. Comput. Sci., Vol. 50. Cambridge University Press, UK.
- [10] Christel Baier and Joost-Pieter Katoen. 2008. *Principles of Model Checking*. MIT Press, USA.
- [11] Davide Basile, Maurice H. ter Beek, Pierpaolo Degano, Axel Legay, Gian Luigi Ferrari, Stefania Gnesi, and Felicita Di Giandomenico. 2020. Controller synthesis of service contracts with variability. *Sci. Comput. Program.* 187 (2020), 102344. <https://doi.org/10.1016/j.scico.2019.102344>
- [12] Davide Basile, Maurice H. ter Beek, Felicita Di Giandomenico, and Stefania Gnesi. 2017. Orchestration of Dynamic Service Product Lines with Featured Modal Contract Automata. In *Proceedings of the 21st International Systems and Software Product Line Conference (SPLC'17)*, Vol. 2. ACM, USA, 117–122. <https://doi.org/10.1145/3109729.3109741>
- [13] Davide Basile, Maurice H. ter Beek, and Stefania Gnesi. 2018. Modelling and Analysis with Featured Modal Contract Automata. In *Proceedings of the 22nd International Systems and Software Product Line Conference (SPLC'18)*, Vol. 2. ACM, USA, 11–16. <https://doi.org/10.1145/3236405.3236408>
- [14] Don Batory. 2026. *Science of Software Product Lines*. ACM Books, Vol. 72. ACM, USA. <https://doi.org/10.1145/3731302>
- [15] Maurice H. ter Beek, Ferruccio Damiani, Stefania Gnesi, Franco Mazzanti, and Luca Paolini. 2019. On the Expressiveness of Modal Transition Systems with Variability Constraints. *Sci. Comput. Program.* 169 (2019), 1–17. <https://doi.org/10.1016/j.scico.2018.09.006>
- [16] Maurice H. ter Beek, Ferruccio Damiani, Michael Lienhardt, Franco Mazzanti, and Luca Paolini. 2022. Efficient static analysis and verification of featured transition systems. *Empir. Softw. Eng.* 22, 1 (2022), 10:1–10:43. <https://doi.org/10.1007/s10664-020-09930-8>
- [17] Maurice H. ter Beek, Alessandro Fantechi, Stefania Gnesi, and Franco Mazzanti. 2016. Modelling and analysing variability in product families: Model checking of modal transition systems with variability constraints. *J. Log. Algebr. Methods Program.* 85, 2 (2016), 287–315. <https://doi.org/10.1016/j.jlamp.2015.11.006>
- [18] Maurice H. ter Beek, Axel Legay, Alberto Lluch Lafuente, and Andrea Vandin. 2020. A Framework for Quantitative Modeling and Analysis of Highly (Re)configurable Systems. *IEEE Trans. Softw. Eng.* 46, 3 (2020), 321–345. <https://doi.org/10.1109/TSE.2018.2853726>
- [19] Maurice H. ter Beek, Michel A. Reniers, and Erik P. de Vink. 2016. Supervisory Controller Synthesis for Product Lines Using CIF 3. In *Proceedings of the 7th International Symposium on Leveraging Applications of Formal Methods, Verification and Validation: Foundational Techniques (ISoLA'16) (Lecture Notes in Computer Science, Vol. 9952)*, Tiziana Margaria and Bernhard Steffen (Eds.). Springer, Germany, 856–873. [https://doi.org/10.1007/978-3-319-47166-2\\_59](https://doi.org/10.1007/978-3-319-47166-2_59)
- [20] Maurice H. ter Beek, Klaus Schmid, and Holger Eichelberger. 2025. Comparing the Universal Variability Language with other Textual Variability Modelling Languages. In *Proceedings of the 29th ACM International Systems and Software Product Line Conference (SPLC'25)*, Vol. 2. ACM, USA, 71–75. <https://doi.org/10.1145/3748269.3749381>
- [21] Maurice H. ter Beek and Erik P. de Vink. 2014. Towards Modular Verification of Software Product Lines with mCRL2. In *Proceedings of the 6th International Symposium on Leveraging Applications of Formal Methods, Verification and Validation (ISoLA'14) (Lecture Notes in Computer Science, Vol. 8802)*, T. Margaria and B. Steffen (Eds.). Springer, Germany, 368–385. [https://doi.org/10.1007/978-3-662-45234-9\\_26](https://doi.org/10.1007/978-3-662-45234-9_26)
- [22] Maurice H. ter Beek, Erik P. de Vink, and Tim A. C. Willmse. 2017. Family-Based Model Checking with mCRL2. In *Proceedings of the 20th International Conference on Fundamental Approaches to Software Engineering (FASE'17) (Lecture Notes in Computer Science, Vol. 10202)*, Marieke Huisman and Julia Rubin (Eds.). Springer, Germany, 387–405. [https://doi.org/10.1007/978-3-662-54494-5\\_23](https://doi.org/10.1007/978-3-662-54494-5_23)



- [23] David Benavides, Sergio Segura, and Antonio Ruiz-Cortés. 2010. Automated Analysis of Feature Models 20 Years Later: a Literature Review. *Inf. Syst.* 35, 6 (2010), 615–636. <https://doi.org/10.1016/j.is.2010.01.001>
- [24] David Benavides, Chico Sundermann, Kevin Feichtinger, José A Galindo, Rick Rabiser, and Thomas Thüm. 2025. UVL: Feature modelling with the Universal Variability Language. *J. Syst. Softw.* 225 (2025), 112326. <https://doi.org/10.1016/j.jss.2024.112326>
- [25] Harsh Beohar, Mahsa Varshosaz, and Mohammad Reza Mousavi. 2016. Basic behavioral models for software product lines: Expressiveness and testing pre-orders. *Sci. Comput. Program.* 123 (2016), 42–60. <https://doi.org/10.1016/J.SCICO.2015.06.005>
- [26] Dirk Beyer and M. Erkan Keremoglu. 2011. CPAchecker: A Tool for Configurable Software Verification. In *Proceedings of the 23rd International Conference on Computer Aided Verification (CAV'11) (Lecture Notes in Computer Science, Vol. 6806)*, Ganesh Gopalakrishnan and Shaz Qadeer (Eds.). Springer, Germany, 184–190. [https://doi.org/10.1007/978-3-642-22110-1\\_16](https://doi.org/10.1007/978-3-642-22110-1_16)
- [27] Roderick Bloem, Barbara Jobstmann, Nir Piterman, Amir Pnueli, and Yaniv Sa'ar. 2012. Synthesis of Reactive(1) designs. *J. Comput. System Sci.* 78, 3 (2012), 911–938. <https://doi.org/10.1016/j.jcss.2011.08.007>
- [28] Gérard Boudol and Ilaria Castellani. 1991. *Flow models of distributed computations: event structures and nets*. Technical Report RR-1482. INRIA. <https://inria.hal.science/inria-00075080>
- [29] Rafael Capilla, Jan Bosch, and Kyo Chul Kang (Eds.). 2013. *Systems and Software Variability Management: Concepts, Tools and Experiences*. Springer, Germany. <https://doi.org/10.1007/978-3-642-36583-6>
- [30] Sunil Chandran, Davis Issac, and Andreas Karrenbauer. 2017. On the Parameterized Complexity of Biclique Cover and Partition. In *Proceedings of the 11th International Symposium on Parameterized and Exact Computation (IPEC'16) (LIPIcs, Vol. 63)*, Jiong Guo and Danny Hermelin (Eds.). Schloss Dagstuhl - Leibniz-Zentrum für Informatik, Germany, 11:1–11:13. <https://doi.org/10.4230/LIPICS.IPEC.2016.11>
- [31] Wonhyuk Choi, Bernd Finkbeiner, Ruzica Piskac, and Mark Santolucito. 2022. Can Reactive Synthesis and Syntax-Guided Synthesis Be Friends?. In *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI'22)*. ACM, USA, 229–243. <https://doi.org/10.1145/3519939.3523429>
- [32] Philipp Chrszon, Clemens Dubsclaff, Sascha Klüppelholz, and Christel Baier. 2018. ProFeat: feature-oriented engineering for family-based probabilistic model checking. *Form. Asp. Comput.* 30, 1 (2018), 45–75. <https://doi.org/10.1007/s00165-017-0432-4>
- [33] Andreas Classen. 2010. *Modelling with FTS: a Collection of Illustrative Examples*. Technical Report P-CS-TR SPLMC-00000001. University of Namur. <https://researchportal.unamur.be/files/1051983/69416.pdf>
- [34] Andreas Classen. 2011. *Modelling and Model Checking Variability-Intensive Systems*. Ph. D. Dissertation. University of Namur. <https://researchportal.unamur.be/en/studentTheses/modelling-and-model-checking-variability-intensive-systems/>
- [35] Andreas Classen, Maxime Cordy, Patrick Heymans, Axel Legay, and Pierre-Yves Schobbens. 2012. Model checking software product lines with SNIP. *Int. J. Softw. Tools Technol. Transfer* 14, 5 (2012), 589–612. <https://doi.org/10.1007/s10009-012-0234-1>
- [36] Andreas Classen, Maxime Cordy, Pierre-Yves Schobbens, Patrick Heymans, Axel Legay, and Jean-François Raskin. 2013. Featured Transition Systems: Foundations for Verifying Variability-Intensive Systems and Their Application to LTL Model Checking. *IEEE Trans. Softw. Eng.* 39, 8 (2013), 1069–1089. <https://doi.org/10.1109/TSE.2012.86>
- [37] Andreas Classen, Patrick Heymans, and Pierre-Yves Schobbens. 2008. What's in a Feature: A Requirements Engineering Perspective. In *Proceedings of the 11th International Conference on Fundamental Approaches to Software Engineering (FASE 2008)*, José Luiz Fiadeiro and Paola Inverardi (Eds.). Lecture Notes in Computer Science, Vol. 4961. Springer, Berlin, Heidelberg, 16–30. [https://doi.org/10.1007/978-3-540-78743-3\\_2](https://doi.org/10.1007/978-3-540-78743-3_2)
- [38] Paul C. Clements and Linda M. Northrop. 2002. *Software Product Lines: Practices and Patterns*. Addison-Wesley, USA.
- [39] Maxime Cordy, Andreas Classen, Patrick Heymans, Pierre-Yves Schobbens, and Axel Legay. 2013. ProVeLines: a product line of verifiers for software product lines. In *Proceedings of the 17th International Software Product Line Conference co-located workshops (SPLC'13)*, Vol. 2. ACM, USA, 141–146. <https://doi.org/10.1145/2499777.2499781>
- [40] Maxime Cordy, Xavier Devroey, Axel Legay, Gilles Perrouin, Andreas Classen, Patrick Heymans, Pierre-Yves Schobbens, and Jean-François Raskin. 2019. A Decade of Featured Transition Systems. In *From Software Engineering to Formal Methods and Tools, and Back*, Maurice H. ter Beek, Alessandro Fantechi, and Laura Semini (Eds.). Lecture Notes in Computer Science, Vol. 11865. Springer, Germany, 285–312. [https://doi.org/10.1007/978-3-030-30985-5\\_18](https://doi.org/10.1007/978-3-030-30985-5_18)
- [41] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. 2022. *Introduction to Algorithms* (4th ed.). MIT Press, USA.
- [42] Krzysztof Czarnecki, Paul Grünbacher, Rick Rabiser, Klaus Schmid, and Andrzej Wąsowski. 2012. Cool Features and Tough Decisions: A Comparison of Variability Modeling Approaches. In *Proceedings of the 6th International Workshop on Variability Modelling of Software-Intensive Systems (VaMoS'12)*. ACM, USA, 173–182. <https://doi.org/10.1145/2110147.2110167>



- [43] Krzysztof Czarnecki and Andrzej Wąsowski. 2007. Feature Diagrams and Logics: There and Back Again. In *Proceedings of the 11th International Conference on Software Product Lines (SPLC'07)*. IEEE, USA, 23–34. <https://doi.org/10.1109/SPLINE.2007.24>
- [44] Carlos Diego Nascimento Damasceno, Mohammad Reza Mousavi, and Adenilso da Silva Simao. 2021. Learning by sampling: learning behavioral family models from software product lines. *Empir. Softw. Eng.* 26, 4 (2021), 1–46. <https://doi.org/10.1007/s10664-020-09912-w>
- [45] Ferruccio Damiani, Michael Lienhardt, and Luca Paolini. 2022. On logical and extensional characterizations of attributed feature models. *Theor. Comput. Sci.* 912 (2022), 56–80. <https://doi.org/10.1016/j.tcs.2022.01.016>
- [46] Willem-Paul de Roever, Frank de Boer, Ulrich Hanneman, Jozef Hooman, Yassine Lakhnech, Mannes Poel, and Job Zwiers. 2001. *Concurrency Verification: Introduction to Compositional and Noncompositional Methods*. Cambridge Tracts in Theor. Comput. Sci., Vol. 54. Cambridge University Press, UK.
- [47] Xavier Devroey. 2020. VIBeS Case Studies: Featured Transition Systems and Feature Models. Dataset, Zenodo. <https://doi.org/10.5281/zenodo.4105900>
- [48] Xavier Devroey, Gilles Perrouin, Maxime Cordy, Mike Papadakis, Axel Legay, and Pierre-Yves Schobbens. 2014. A Variability Perspective of Mutation Analysis. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE'14)*. ACM, USA, 841–844. <https://doi.org/10.1145/2635868.2666610>
- [49] Xavier Devroey, Gilles Perrouin, Maxime Cordy, Hamza Samih, Axel Legay, Pierre-Yves Schobbens, and Patrick Heymans. 2017. Statistical prioritization for software product line testing: an experience report. *Softw. Syst. Model.* 16 (2017), 153–171. <https://doi.org/10.1007/s10270-015-0479-8>
- [50] Xavier Devroey, Gilles Perrouin, Axel Legay, Maxime Cordy, Pierre-Yves Schobbens, and Patrick Heymans. 2014. Coverage Criteria for Behavioural Testing of Software Product Lines. In *Proceedings of the 6th International Symposium on Leveraging Applications of Formal Methods, Verification and Validation: Technologies for Mastering Change (ISoLA'14) (Lecture Notes in Computer Science, Vol. 8802)*, Tiziana Margaria and Bernhard Steffen (Eds.). Springer, Germany, 336–350. [https://doi.org/10.1007/978-3-662-45234-9\\_24](https://doi.org/10.1007/978-3-662-45234-9_24)
- [51] Xavier Devroey, Gilles Perrouin, Axel Legay, Pierre-Yves Schobbens, and Patrick Heymans. 2015. Covering SPL Behaviour with Sampled Configurations: An Initial Assessment. In *Proceedings of the 9th International Workshop on Variability Modelling of Software-intensive Systems (VaMoS'15)*, Klaus Schmid, Øystein Haugen, and Johannes Müller (Eds.). ACM, USA, 59–66. <https://doi.org/10.1145/2701319.2701325>
- [52] Xavier Devroey, Gilles Perrouin, Axel Legay, Pierre-Yves Schobbens, and Patrick Heymans. 2016. Search-based Similarity-driven Behavioural SPL Testing. In *Proceedings of the 10th International Workshop on Variability Modelling of Software-Intensive Systems (VaMoS'16)*. ACM, USA, 89–96. <https://doi.org/10.1145/2866614.2866627>
- [53] Xavier Devroey, Gilles Perrouin, Axel Legay, Pierre-Yves Schobbens, and Patrick Heymans. 2016. Search-based Similarity-driven Behavioural SPL Testing. In *Proceedings of the 10th International Workshop on Variability Modelling of Software-Intensive Systems (VaMoS'16)*. ACM, USA, 89–96. <https://doi.org/10.1145/2866614.2866627>
- [54] Xavier Devroey, Gilles Perrouin, Mike Papadakis, Axel Legay, Pierre-Yves Schobbens, and Patrick Heymans. 2016. Featured Model-based Mutation Analysis. In *Proceedings of the 38th International Conference on Software Engineering (ICSE'16)*. ACM, USA, 655–666. <https://doi.org/10.1145/2884781.2884821>
- [55] Xavier Devroey, Gilles Perrouin, and Pierre-Yves Schobbens. 2014. Abstract Test Case Generation for Behavioural Testing of Software Product Lines. In *Proceedings of the 18th International Software Product Line Conference (SPLC'14)*, Vol. 2. ACM, USA, 86–93. <https://doi.org/10.1145/2647908.2655971>
- [56] Aleksandar S. Dimovski, Ahmad S. Al-Sibahi, Claus Brabrand, and Andrzej Wąsowski. 2015. Family-Based Model Checking using Off-the-Shelf Model Checkers. In *Proceedings of the 19th International Software Product Line Conference (SPLC'15)*. ACM, USA, 397. <https://doi.org/10.1145/2791060.2791119>
- [57] Natanael Djajadi, Amirhossein Deljouyi, and Andy Zaidman. 2025. Using Large Language Models to Generate Concise and Understandable Test Case Summaries. In *Proc. 33rd Intl. Conf. on Program Comprehension (ICPC'25)*. IEEE, USA, 322–326. <https://doi.org/10.1109/ICPC66645.2025.00040>
- [58] Martin Erwig and Eric Walkingshaw. 2011. The Choice Calculus: A Representation for Software Variation. *ACM Trans. Softw. Eng. Methodol.* 21, 1 (2011), 6:1–6:27. <https://doi.org/10.1145/2063239.2063245>
- [59] Bernd Finkbeiner, Gideon Geier, and Noemi Passing. 2023. Specification decomposition for reactive synthesis. *Innov. Syst. Softw. Eng.* 19, 4 (2023), 339–357. <https://doi.org/10.1007/s11334-022-00462-6>
- [60] Sophie Fortz. 2023. Variability-aware Behavioural Learning. In *Proceedings of the 27th ACM International Systems and Software Product Line Conference (SPLC'23)*, Vol. 2. ACM, USA, 11–15. <https://doi.org/10.1145/3579028.3609007>
- [61] Vanderson Hafemann Fragal, Adenilso Simão, and Mohammad Reza Mousavi. 2019. Hierarchical featured state machines. *Sci. Comput. Program.* 171 (2019), 67–88. <https://doi.org/10.1016/J.SCICO.2018.10.001>
- [62] Fatemeh Ghassemi and Mohammad Reza Mousavi. 2016. Product line process theory. *J. Log. Algebr. Methods Program.* 85, 1 (2016), 200–226. <https://doi.org/10.1016/J.JLAMP.2015.09.008>

- [63] Rob J. van Glabbeek and Frits W. Vaandrager. 2003. Bundle Event Structures and CCSP. In *Proceedings of the 14th International Conference on Concurrency Theory (CONCUR'03) (Lecture Notes in Computer Science, Vol. 2761)*, Roberto M. Amadio and Denis Lugiez (Eds.). Springer, Germany, 57–71. [https://doi.org/10.1007/978-3-540-45187-7\\_4](https://doi.org/10.1007/978-3-540-45187-7_4)
- [64] Daniele Gorla. 2010. Towards a unified approach to encodability and separation results for process calculi. *Inf. Comput.* 208, 9 (2010), 1031–1053. <https://doi.org/10.1016/J.IC.2010.05.002>
- [65] Alexander Gruler, Martin Leucker, and Kathrin D. Scheideemann. 2008. Calculating and Modeling Common Parts of Software Product Lines. In *Proceedings of the 12th International Software Product Lines Conference (SPLC'08)*. IEEE, USA, 203–212. <https://doi.org/10.1109/SPLC.2008.22>
- [66] Alexander Gruler, Martin Leucker, and Kathrin D. Scheideemann. 2008. Modeling and Model Checking Software Product Lines. In *Proceedings of the 10th International Conference on Formal Methods for Open Object-Based Distributed Systems (FMOODS'08) (Lecture Notes in Computer Science, Vol. 5051)*, Gilles Barthe and Frank S. de Boer (Eds.). Springer, Germany, 113–131. [https://doi.org/10.1007/978-3-540-68863-1\\_8](https://doi.org/10.1007/978-3-540-68863-1_8)
- [67] Øystein Haugen, Andrzej Wąsowski, and Krzysztof Czarnecki. 2012. CVL – Common Variability Language. In *Proc. 16th Intl. Software Product Line Conf. (SPLC'12)*. ACM, USA, 266–267. <https://doi.org/10.1145/2364412.2364462>
- [68] Leo Henry, Thomas Neele, Mohammad Reza Mousavi, and Matteo Sammartino. 2025. A Detailed Account of Compositional Automata Learning through Alphabet Refinement. arXiv:2504.16624 [cs.LG]
- [69] Daniel Jackson. 2006. *Software Abstractions: Logic, Language, and Analysis*. MIT Press, USA.
- [70] Paulius Juodisius, Atrisha Sarkar, Raghava Rao Mukkamala, Michał Antkiewicz, Krzysztof Czarnecki, and Andrzej Wąsowski. 2019. Clafer: Lightweight Modeling of Structure, Behaviour, and Variability. *Art. Sci. Eng. Program.* 3, 1 (2019), 657–681. <https://doi.org/10.22152/programming-journal.org/2019/3/2>
- [71] Kyo C. Kang, Sholom G. Cohen, James A. Hess, William E. Novak, and A. Spencer Peterson. 1990. *Feature-Oriented Domain Analysis (FODA) Feasibility Study*. Technical Report CMU/SEI-90-TR-021. Carnegie Mellon University. <https://www.sei.cmu.edu/library/feature-oriented-domain-analysis-foda-feasibility-study/>
- [72] Faezeh Labbaf, Jan Friso Groote, Hossein Hojjat, and Mohammad Reza Mousavi. 2023. Compositional Learning for Interleaving Parallel Automata. In *Proceedings of the 26th International Conference on Foundations of Software Science and Computation Structures (FoSSaCS'23) (Lecture Notes in Computer Science, Vol. 13992)*, Orna Kupferman and Pawel Sobocinski (Eds.). Springer, Germany, 413–435. [https://doi.org/10.1007/978-3-031-30829-1\\_20](https://doi.org/10.1007/978-3-031-30829-1_20)
- [73] Rom Langerak. 1992. Bundle event structures: a non-interleaving semantics for LOTOS. In *Proceedings of the 5th IFIP TC6/WG6.1 International Conference on Formal Description Techniques for Distributed Systems and Communication Protocols (FORTE'92) (IFIP Transactions, Vol. C-10)*, Michel Diaz and Roland Groz (Eds.). North-Holland, The Netherlands, 331–346.
- [74] Kim G. Larsen, Ulrik Nyman, and Andrzej Wąsowski. 2007. Modal I/O Automata for Interface and Product Line Theories. In *Proceedings of the 16th European Symposium on Programming (ESOP'07) (Lecture Notes in Computer Science, Vol. 4421)*, Rocco De Nicola (Ed.). Springer, Germany, 64–79. [https://doi.org/10.1007/978-3-540-71316-6\\_6](https://doi.org/10.1007/978-3-540-71316-6_6)
- [75] Kim Guldstrand Larsen, Ulrik Nyman, and Andrzej Wąsowski. 2007. Modeling software product lines using color-blind transition systems. *Int. J. Softw. Tools Technol. Transf.* 9, 5-6 (2007), 471–487. <https://doi.org/10.1007/S10009-007-0046-X>
- [76] Kim Lauenroth, Klaus Pohl, and Simon Töhning. 2009. Model Checking of Domain Artifacts in Product Line Engineering. In *Proceedings of the 24th International Conference on Automated Software Engineering (ASE'09)*. IEEE, USA, 269–280. <https://doi.org/10.1109/ASE.2009.16>
- [77] Michael Lienhardt, Ferruccio Damiani, Lorenzo Testa, and Gianluca Turin. 2018. On checking delta-oriented product lines of statecharts. *Science of Computer Programming* 166 (2018), 3–34. <https://doi.org/10.1016/j.scico.2018.05.007>
- [78] Malte Lochau, Stephan Mennicke, Hauke Baller, and Lars Ribbeck. 2014. DeltaCCS: A Core Calculus for Behavioral Change. In *Proceedings of the 6th International Symposium on Leveraging Applications of Formal Methods, Verification and Validation: Technologies for Mastering Change (ISoLA'14) (Lecture Notes in Computer Science, Vol. 8802)*, Tiziana Margaria and Bernhard Steffen (Eds.). Springer, Germany, 320–335. [https://doi.org/10.1007/978-3-662-45234-9\\_23](https://doi.org/10.1007/978-3-662-45234-9_23)
- [79] Ken L. McMillan. 2000. A methodology for hardware verification using compositional model checking. *Sci. Comput. Program.* 37, 1 (2000), 279–309. [https://doi.org/10.1016/S0167-6423\(99\)00030-1](https://doi.org/10.1016/S0167-6423(99)00030-1)
- [80] MohammadReza Mousavi, Michel A. Reniers, and Jan Friso Groote. 2007. SOS formats and meta-theory: 20 years after. *Theor. Comput. Sci.* 373, 3 (2007), 238–272. <https://doi.org/10.1016/j.tcs.2006.12.019>
- [81] Kedar S. Namjoshi and Richard J. Treller. 2016. Parameterized Compositional Model Checking. In *Proceedings of the 22nd International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS'16) (Lecture Notes in Computer Science, Vol. 9636)*, Marsha Chechik and Jean-François Raskin (Eds.). Springer, Germany, 589–606. [https://doi.org/10.1007/978-3-662-49674-9\\_39](https://doi.org/10.1007/978-3-662-49674-9_39)
- [82] Thomas Neele and Matteo Sammartino. 2023. Compositional Automata Learning of Synchronous Systems. In *Proceedings of the 26th International Conference on Fundamental Approaches to Software Engineering (FASE'23) (Lecture Notes in Computer Science, Vol. 13991)*, Leen Lambers and Sebastián Uchitel (Eds.). Springer, Germany, 47–66. [https://doi.org/10.1007/978-3-031-30826-0\\_3](https://doi.org/10.1007/978-3-031-30826-0_3)

- [83] Mogens Nielsen, Gordon D. Plotkin, and Glynn Winskel. 1981. Petri Nets, Event Structures and Domains, Part I. *Theor. Comput. Sci.* 13 (1981), 85–108. [https://doi.org/10.1016/0304-3975\(81\)90112-2](https://doi.org/10.1016/0304-3975(81)90112-2)
- [84] Flemming Nielson, Hanne Riis Nielson, and Chris Hankin. 2005. *Principles of Program Analysis*. Springer, Germany. <https://doi.org/10.1007/978-3-662-03811-6>
- [85] Robert Paige and Robert E. Tarjan. 1987. Three Partition Refinement Algorithms. *SIAM J. Comput.* 16, 6 (1987), 973–989. <https://doi.org/10.1137/0216062>
- [86] Richard F. Paige, Jonathan S. Ostroff, and Phillip J. Brooke. 2000. Principles for modeling language design. *Inf. Softw. Technol.* 42, 10 (2000), 665–675. [https://doi.org/10.1016/S0950-5849\(00\)00109-9](https://doi.org/10.1016/S0950-5849(00)00109-9)
- [87] René Peeters. 2003. The maximum edge biclique problem is NP-complete. *Discrete Appl. Math.* 131, 3 (2003), 651–654. [https://doi.org/10.1016/S0166-218X\(03\)00333-0](https://doi.org/10.1016/S0166-218X(03)00333-0)
- [88] Klaus Pohl, Günter Böckle, and Frank J. van der Linden. 2005. *Software Product Line Engineering: Foundations, Principles, and Techniques*. Springer, Germany. <https://doi.org/10.1007/3-540-28901-1>
- [89] Vaughan Pratt. 1986. Modeling concurrency with partial orders. *Int. J. Parallel Program.* 15, 1 (1986), 33–71. <https://doi.org/10.1007/BF01379149>
- [90] Peter J. Ramadge and Walter M. Wonham. 1987. Supervisory Control of a Class of Discrete Event Processes. *SIAM J. Control. Optim.* 25, 1 (1987), 206–230. <https://doi.org/10.1137/0325013>
- [91] Devjeet Roy, Ziyi Zhang, Maggie Ma, Venera Arnaoudova, Annibale Panichella, Sebastiano Panichella, Danielle Gonzalez, and Mehdi Mirakhorli. 2020. DeepTC-Enhancer: Improving the Readability of Automatically Generated Tests. In *Proc. 35th Intl. Conf. on Automated Software Engineering (ASE'20)*. IEEE, USA, 287–298. <https://doi.org/10.1145/3324884.3416622>
- [92] Mahboubeh Samadi, Aryan Bastany, and Hossein Hojjat. 2025. Compositional Learning for Synchronous Parallel Automata. In *Proceedings of the 28th International Conference on Fundamental Approaches to Software Engineering (FASE'25) (Lecture Notes in Computer Science, Vol. 15693)*, Artur Boronat and Gordon Fraser (Eds.). Springer, Germany, 101–121. [https://doi.org/10.1007/978-3-031-90900-9\\_6](https://doi.org/10.1007/978-3-031-90900-9_6)
- [93] Ina Schaefer, Dilian Gurov, and Siavash Soleimanifard. 2012. Compositional Algorithmic Verification of Software Product Lines. In *Revised Papers of the 9th International Symposium on Formal Methods for Components and Objects (FMCO'10) (Lecture Notes in Computer Science, Vol. 6957)*, Bernhard K. Aichernig, Frank S. de Boer, and Marcello M. Bonsangue (Eds.). Springer, Germany, 184–203. [https://doi.org/10.1007/978-3-642-25271-6\\_10](https://doi.org/10.1007/978-3-642-25271-6_10)
- [94] Pierre-Yves Schobbens, Patrick Heymans, and Jean-Christophe Trigaux. 2006. Feature Diagrams: A Survey and a Formal Semantics. In *Proceedings of the 14th IEEE International Conference on Requirements Engineering (RE'06)*. IEEE, USA, 136–145. <https://doi.org/10.1109/RE.2006.23>
- [95] Pourya Shaker, Joanne M. Atlee, and Shige Wang. 2012. A Feature-Oriented Requirements Modelling Language. In *Proc. 20th IEEE Intl. Conf. on Requirements Engineering (RE'12)*. IEEE, USA, 151–160. <https://doi.org/10.1109/RE.2012.6345799>
- [96] Michael Sipser. 2012. *Introduction to the Theory of Computation*. Cengage, USA.
- [97] Siavash Soleimanifard, Dilian Gurov, and Marieke Huisman. 2011. ProMoVer: Modular Verification of Temporal Safety Properties. In *Proceedings of the 9th International Conference on Software Engineering and Formal Methods (SEFM'11) (Lecture Notes in Computer Science, Vol. 7041)*, Gilles Barthe, Alberto Pardo, and Gerardo Schneider (Eds.). Springer, Germany, 366–381. [https://doi.org/10.1007/978-3-642-24690-6\\_25](https://doi.org/10.1007/978-3-642-24690-6_25)
- [98] Chico Sundermann, Kevin Feichtinger, Dominik Engelhardt, Rick Rabiser, and Thomas Thüm. 2021. Yet Another Textual Variability Language? A Community Effort Towards a Unified Language. In *Proceedings of the 25th ACM International Systems and Software Product Line Conference (SPLC'21)*. ACM, USA, 136–147. <https://doi.org/10.1145/3461001.3471145>
- [99] Robert Tarjan. 1972. Depth-First Search and Linear Graph Algorithms. *SIAM J. Comput.* 1, 2 (1972), 146–160. <https://doi.org/10.1137/0201010>
- [100] Shaghayegh Tavassoli, Carlos Diego N. Damasceno, Ramtin Khosravi, and Mohammad Reza Mousavi. 2022. Adaptive Behavioral Model Learning for Software Product Lines. In *Proceedings of the 26th ACM International Systems and Software Product Line Conference (SPLC'22)*. ACM, USA, 142–153. <https://doi.org/10.1145/3546932.3546991>
- [101] Shaghayegh Tavassoli, Carlos Diego N. Damasceno, Mohammad Reza Mousavi, and Ramtin Khosravi. 2022. A Benchmark for Active Learning of Variability-Intensive Systems. In *Proceedings of the 26th ACM International Systems and Software Product Line Conference (SPLC'22)*. ACM, USA, 245–249. <https://doi.org/10.1145/3546932.3547014>
- [102] Shaghayegh Tavassoli and Ramtin Khosravi. 2025. Efficient construction of family-based behavioral models from adaptively learned models. *Softw. Syst. Model.* 24, 1 (2025), 225–251. <https://doi.org/10.1007/s10270-024-01199-5>
- [103] Paul Temple and Gilles Perrouin. 2023. Explicit or Implicit? On Feature Engineering for ML-based Variability-Intensive Systems. In *Proceedings of the 17th International Working Conference on Variability Modelling of Software-Intensive Systems (VaMoS'23)*. ACM, USA, 91–93. <https://doi.org/10.1145/3571788.3571804>
- [104] Sander Thuijsman and Michel Reniers. 2024. Supervisory Control for Dynamic Feature Configuration in Product Lines. *ACM Trans. Embed. Comput. Syst.* 23, 5 (2024), 71:1–71:25. <https://doi.org/10.1145/3579644>

- [105] Thomas Thüm, Sven Apel, Christian Kästner, Ina Schaefer, and Gunter Saake. 2014. A Classification and Survey of Analysis Strategies for Software Product Lines. *ACM Comput. Surv.* 47, 1 (2014), 6:1–6:45. <https://doi.org/10.1145/2580950>
- [106] Thomas Thüm, Don S. Batory, and Christian Kästner. 2009. Reasoning about Edits to Feature Models. In *Proceedings of the 31st International Conference on Software Engineering (ICSE'09)*. IEEE, USA, 254–264. <https://doi.org/10.1109/ICSE.2009.5070526>
- [107] Thomas Thüm, Christian Kästner, Fabian Benduhn, Jens Meinicke, Gunter Saake, and Thomas Leich. 2014. FeatureIDE: An extensible framework for feature-oriented software development. *Sci. Comput. Program.* 79 (2014), 70–85. <https://doi.org/10.1016/j.scico.2012.06.002>
- [108] Mirco Tribastone. 2014. Behavioral Relations in a Process Algebra for Variants. In *Proceedings of the 18th International Software Product Line Conference (SPLC'14)*. ACM, USA, 82–91. <https://doi.org/10.1145/2648511.2648520>
- [109] Dirk A. van Beek, Wan J. Fokkink, Dennis Hendriks, Albert T. Hofkamp, Jasen Markovski, Joanna M. van de Mortel-Fronczak, and Michel A. Reniers. 2014. CIF 3: Model-Based Engineering of Supervisory Controllers. In *Proceedings of the 20th International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS'14) (Lecture Notes in Computer Science, Vol. 8413)*, Erika Ábrahám and Klaus Havelund (Eds.). Springer, Germany, 575–580. [https://doi.org/10.1007/978-3-642-54862-8\\_48](https://doi.org/10.1007/978-3-642-54862-8_48)
- [110] Mahsa Varshosaz, Harsh Beohar, and Mohammad Reza Mousavi. 2018. Basic behavioral models for software product lines: Revisited. *Sci. Comput. Program.* 168 (2018), 171–185. <https://doi.org/10.1016/J.SCICO.2018.09.001>
- [111] Glynn Winskel. 1986. Event Structures. In *Proceedings of an Advanced Course on Petri Nets: Central Models and Their Properties (ACPN'86) (Lecture Notes in Computer Science, Vol. 255)*, Wilfried Brauer, Wolfgang Reisig, and Grzegorz Rozenberg (Eds.). Springer, Germany, 325–392. [https://doi.org/10.1007/3-540-17906-2\\_31](https://doi.org/10.1007/3-540-17906-2_31)
- [112] Andrzej Wąsowski and Thorsten Berger. 2023. *Domain-Specific Languages: Effective Modeling, Automation, and Reuse*. Springer, Germany. <https://doi.org/10.1007/978-3-031-23669-3>
- [113] R. F. Woolson. 2008. *Wilcoxon Signed-Rank Test*. Wiley Online Library, 1–3. <https://doi.org/10.1002/9780471462422.eoct979>